Lecture Notes on Model Order Reduction Techniques

Lorenzo Vergata

July 12, 2026

Contents

1 Introduction to Model Order Reduction **19**

1.1 Course introduction . . . 19
1.2 Course material . . . 19
1.3 Practical organization . . . 20
1.4 Prerequisites and review of background material . . . 20
1.5 Bibliography and course notes . . . 20
1.6 Exam . . . 20
1.7 Exam dates and flexibility . . . 21
1.8 The big picture . . . 21
1.8.1 Parametric problems . . . 22
1.8.2 Reduced-order modeling . . . 22
1.8.3 When is this useful? . . . 23
1.8.4 How parameters affect the solution . . . 23
1.8.5 Learning from previous simulations . . . 23
1.8.6 Another example: Navier–Stokes flows . . . 23
1.8.7 Why do we care? . . . 23
1.9 Dimensionality reduction and computational complexity . . . 24
1.9.1 Reduced-order modeling . . . 24
1.9.2 Reduced basis methods . . . 24
1.9.3 Connection with finite elements . . . 25
1.9.4 Snapshots and reduced spaces . . . 25
1.9.5 A roadmap for the course . . . 25
1.9.6 Dimensionality reduction techniques . . . 25
1.9.7 Projection-based reduced-order models . . . 26
1.9.8 Linear, nonlinear, and hyper-reduced problems . . . 26
1.9.9 Non-intrusive reduced-order modeling . . . 26
1.9.10 Neural networks and deep learning . . . 26
1.10 Dynamical systems and data-driven modeling . . . 27
1.10.1 Dynamic Mode Decomposition . . . 27
1.10.2 Physics-informed and sensor-based learning . . . 27
1.11 Course perspective . . . 27
1.12 Why do we need reduction? . . . 27
1.12.1 From differential equations to algebraic systems . . . 28
1.12.2 Parameter dependence . . . 28
1.12.3 Computational cost . . . 28
1.12.4 Finite elements as a first reduction . . . 29
1.13 From finite elements to reduced bases . . . 29
1.13.1 Weak formulation and Galerkin method . . . 29
1.13.2 Analogy with reduced-order modeling . . . 30
1.14 From high-fidelity solutions to reduced bases . . . 30
1.14.1 Snapshot matrix . . . 30
1.14.2 Reduced representation . . . 30

1.15 Why not simply use a coarser mesh? . . . 31
1.16 How is the reduced basis constructed? . . . 31
1.17 Online evaluation . . . 31
1.18 From the reduced basis to the reduced problem . . . 31
1.18.1 The residual . . . 32
1.18.2 Why the residual cannot be set to zero directly . . . 32
1.18.3 Galerkin projection . . . 32
1.18.4 The reduced system . . . 32
1.18.5 Offline and online stages . . . 32
1.18.6 A crucial subtlety . . . 33
1.18.7 Summary of the idea . . . 33

2 Recalls on High-Fidelity, Full-Order Models (Finite Elements) 35
2.1 Construction of a reduced space . . . 35
2.2 Projection onto the reduced space . . . 35
2.3 Computational issue . . . 36
2.4 Affine parameter dependence . . . 36
2.5 Why reduction still preserves the physics . . . 37
2.6 Geometric interpretation . . . 37
2.7 Offline–online workflow . . . 38
2.8 Open questions . . . 38
2.9 Online stage and error control . . . 38
2.10 A numerical example: an affine case . . . 38
2.11 A more difficult case: non-affine parametrization . . . 39
2.12 Transition to the weak formulation . . . 39
2.13 A crash course on the weak formulation . . . 39
2.14 From the strong form to the weak form . . . 40
2.15 Why the weak formulation is useful . . . 40
2.16 Variable and discontinuous coefficients . . . 40
2.17 An example with affine parameter dependence . . . 41
2.18 Right-hand side . . . 41
2.19 Non-affine parameter dependence . . . 41
2.20 Weak formulation as a bilinear variational problem . . . 42
2.21 Galerkin approximation . . . 42
2.22 From the weak form to a linear system . . . 43
2.23 Summary . . . 43
2.24 From the weak formulation to the finite element discretization . . . 44
2.25 Discretization of the domain . . . 44
2.26 Finite element spaces . . . 44
2.27 Boundary conditions . . . 45
2.28 Galerkin approximation . . . 45
2.29 Basis expansion and algebraic system . . . 45
2.30 Main idea . . . 45
2.31 From the weak formulation to the algebraic system . . . 46
2.32 Galerkin orthogonality . . . 47

3 The Reduced Basis Method 49
3.1 From the full-order model to the reduced-order model . . . 49
3.2 Projection viewpoint . . . 49
3.3 Why this is useful . . . 50
3.4 Connection with the finite element method . . . 50
3.5 Geometric interpretation . . . 50
3.6 Parameterized high-fidelity models and the solution manifold . . . 50
3.6.1 The role of the parameter . . . 51

3.6.2 The solution manifold . . . 51
3.6.3 Why reduction is needed . . . 51
3.6.4 A geometric picture . . . 51
3.6.5 Relation with finite elements . . . 52
3.7 The Reduced Basis Method . . . 52
3.7.1 A smaller space for parametrized problems . . . 52
3.7.2 Snapshot-based construction of the reduced space . . . 52
3.7.3 Why this is a reduction . . . 53
3.7.4 Galerkin reduced basis approximation . . . 53
3.7.5 Projection interpretation . . . 53
3.7.6 Algebraic meaning . . . 53
3.7.7 What remains to be done . . . 54

4 Lab 2 - Finite element method in FEniCS 55
4.1 Introduction . . . 55
4.1.1 The `PINIX` and `DRROMS` Packages . . . 55
4.2 From Functions to Finite Dimensional Vectors . . . 55
4.2.1 Setting Up the Finite Element Environment . . . 56
4.2.2 Interpolating a Function . . . 56
4.3 Conclusion . . . 57
4.4 What is the point? . . . 57
4.4.1 Plotting a finite element function . . . 57
4.5 Function–vector correspondence . . . 57
4.5.1 From function to vector . . . 57
4.5.2 From vector to function . . . 57
4.6 Exercises . . . 57
4.6.1 Exercise 1: Circle domain . . . 58
4.6.2 Exercise 2: Domain with a hole (annulus) . . . 58
4.7 Solutions and Discussion . . . 58
4.7.1 Solution to Exercise 1 . . . 58
4.7.2 Solution to Exercise 2 . . . 59
4.8 Next steps . . . 59
4.9 Continuous Galerkin of Degree 0 (Piecewise Constant) . . . 59
4.10 Stationary Heat Equation with Variable Conductivity . . . 59
4.10.1 Weak Formulation . . . 60
4.10.2 Galerkin Projection . . . 60
4.10.3 Discretization into a Linear System . . . 60
4.11 Implementation in Python (using Phoenix/DRROMS) . . . 61
4.11.1 Defining the bilinear form $a(u, v)$ . . . 61
4.11.2 Defining the linear form $L(v)$ . . . 61
4.11.3 Handling Dirichlet boundary conditions . . . 61
4.12 Summary of the Workflow . . . 61
4.13 Remarks on the Test Space and Boundary Conditions . . . 61
4.14 Boundary Conditions: Dirichlet and Neumann . . . 62
4.15 Assembly of the Linear System . . . 62
4.16 Applying Dirichlet Boundary Conditions . . . 62
4.17 Solving the Linear System . . . 62
4.18 Verification of the Solution . . . 62
4.19 Incorporating Neumann Conditions (General Case) . . . 63
4.20 Exercises . . . 63
4.21 Next Steps . . . 63

5 Proper Orthogonal Decomposition **65**
5.1 From the full order model to the reduced basis method . . . 65
5.2 The reduced order model . . . 65
5.3 Global basis and parameter dependence . . . 66
5.4 The algebraic connection between reduced and full order models . . . 66
5.4.1 The transformation matrix V . . . 66
5.5 The missing piece: constructing V . . . 67
5.6 The Singular Value Decomposition (SVD) . . . 67
5.6.1 Relation with eigenvalues . . . 68
5.6.2 Low-rank approximations . . . 68
5.6.3 Why low-rank approximation matters . . . 69
5.7 Proper Orthogonal Decomposition (POD) . . . 69
5.7.1 The snapshot matrix . . . 69
5.7.2 POD basis from SVD . . . 70
5.7.3 Optimality of the POD basis . . . 70
5.7.4 From reconstruction to prediction . . . 70
5.7.5 Choosing the dimension of the POD basis . . . 71
5.8 Practical considerations: Sampling and redundancy . . . 71
5.9 Summary and outlook . . . 71
5.10 What you need for the Friday exercise . . . 72
5.11 Motivation and Recap . . . 72
5.12 The Snapshot Matrix and the SVD . . . 72
5.13 The POD Basis . . . 73
5.13.1 Optimality of the POD Basis (Eckart–Young–Mirsky Theorem) . . . 73
5.14 Projection onto the POD Space . . . 73
5.15 Reduced-Order Models (ROMs) . . . 73
5.16 Error Quantification . . . 74
5.17 Summary of the POD Procedure . . . 74
5.18 Building the Reduced Problem: Galerkin Projection . . . 74
5.18.1 Combining POD with Galerkin Projection . . . 75
5.19 Choosing the Dimension of the POD Basis . . . 75
5.20 Example: Advection–Diffusion Problem with Multiple Parameters . . . 75
5.20.1 Sampling the Parameter Space . . . 76
5.21 Summary of the ROM Workflow . . . 76
5.22 Sampling the Parameter Space: Strategies and Convergence . . . 76
5.22.1 The Curse of Dimensionality . . . 76
5.22.2 Comparison of Sampling Strategies . . . 77
5.22.3 Convergence with Respect to the Number of Snapshots . . . 77
5.22.4 Error Estimation and Basis Size Selection . . . 78
5.23 Computational Cost: Offline vs. Online . . . 78
5.24 When Does POD Work? Reducibility and Singular Value Decay . . . 79
5.24.1 Example 1: A Diffusive Problem (Low Effective Dimension) . . . 79
5.24.2 Example 2: Pure Transport (Wave Propagation) . . . 79
5.25 Practical Assessment of Reducibility . . . 80
5.26 Assessing Reducibility: The Role of Physics and Sampling . . . 80
5.26.1 Influence of Physics: Diffusion vs. Transport . . . 81
5.26.2 The Role of Sampling Frequency . . . 81
5.26.3 When Linear Reduction Fails: Outlook to Nonlinear Methods . . . 81
5.27 Handling Boundary Conditions in POD–Galerkin . . . 82
5.27.1 Homogeneous Dirichlet Conditions (Lucky Case) . . . 82
5.27.2 Non-Homogeneous Dirichlet Conditions (Lifting Function) . . . 82
5.28 Summary and Practical Takeaways . . . 82

6 Lab 3 - Reduced basis (POD-Galerkin) for stationary problems 85
6.1 Introduction . . . 85
6.1.1 Problem Description: Stationary Heat Equation on a Plate with Disks . . . 85
6.1.2 Finite Element Discretization . . . 86
6.2 Snapshots Collection for the Reduced Basis Method . . . 86
6.2.1 Random Parameter Sampling . . . 86
6.2.2 Inspecting the Snapshots . . . 87
6.2.3 Plotting Multiple Solutions . . . 87
6.3 Next Steps: Building the Reduced Basis Model . . . 87
6.4 Closing Remarks . . . 87
6.5 Motivation . . . 87
6.6 Snapshot Matrix (Data Matrix) . . . 88
6.6.1 Train / Test Split . . . 88
6.7 POD via Singular Value Decomposition (SVD) . . . 88
6.7.1 Choosing the Number of Modes n . . . 89
6.8 The POD Basis Matrix . . . 89
6.8.1 Visualising the POD Modes . . . 89
6.9 Projection onto the Reduced Basis . . . 90
6.9.1 Example: Reconstructing a Solution Using Increasing Numbers of Modes . . . 90
6.10 Next Steps: Galerkin Projection of the PDE . . . 90
6.11 Important Notation Reminder . . . 90
6.12 Projection onto the POD Basis . . . 91
6.12.1 Example: Projection of a Single Test Solution . . . 91
6.12.2 Batch Computation on the Entire Test Set . . . 91
6.13 From Projection to Galerkin Reduced Order Model (ROM) . . . 91
6.13.1 Affine Parametrization: Key to Efficiency . . . 92
6.13.2 Affine Decomposition for the Cookie Problem . . . 92
6.14 Offline Preassembly of Reduced Matrices . . . 92
6.15 Online Assembler and Solver . . . 92
6.16 Validation: Compare ROM with Full FE Solution . . . 93
6.17 Computational Speed Comparison . . . 93
6.18 Projection Coefficients vs. Galerkin Coefficients . . . 93
6.19 Conclusion . . . 94

7 The Reduced Basis Method for Nonstationary Problems and Some Hints on Nonaffine Problems 95
7.1 Proper Orthogonal Decomposition (POD) . . . 95
7.2 Galerkin Projection . . . 95
7.3 Efficient Assembly: The Affine Case . . . 96
7.4 Challenges Beyond the Stationary Linear Affine Setting . . . 96
7.4.1 Non-Stationary Problems . . . 96
7.4.2 Non-Affine Problems . . . 96
7.4.3 Nonlinear Problems . . . 97
7.4.4 Combined Difficulties . . . 97
7.5 Hints on Nonaffine Problems . . . 97
7.6 Recap of the Stationary Linear Affine Case . . . 97
7.7 Extension to Non-Stationary (Time-Dependent) Problems . . . 98
7.7.1 Full Order Discretization . . . 98
7.7.2 Recycling the Two Ingredients . . . 98
7.7.3 POD for Time-Dependent Data . . . 99
7.7.4 Reduced Dynamical System via Galerkin Projection . . . 99
7.8 Outlook: Nonaffine and Nonlinear Problems . . . 100
7.8.1 Two Strategies for Building a Global POD Basis from Time-Dependent Data . . . 100
7.8.2 Reduced Dynamical System via Galerkin Projection . . . 101

7.9 Greedy Algorithms and Error Estimation (A Cultural Note) . . . 101
7.10 Local Bases and the Bottleneck of Global Linear Techniques . . . 102
7.11 Hints on Non-Affine Problems . . . 102
7.11.1 Idea: Approximate the Non-Affine Object by an Affine Expansion . . . 103
7.12 Hyper-Reduction for Non-Affine Problems: Empirical Interpolation and Its Challenges . . . 103
7.12.1 Restoring Affinity by Approximating the Non-Affine Term . . . 103
7.12.2 Determining the Coefficients via Interpolation (Empirical Interpolation) . . . 104
7.12.3 Hyper-Reduction: Two Levels of Reduction . . . 104
7.12.4 The Cost–Accuracy Trade-Off and Why Hyper-Reduction Is Often Impractical . . . 104
7.13 Conclusion and Outlook . . . 105

8 Some Hints on Nonaffine Problems and Anover View on Projection-based Reduced Order Models 107
8.1 Overview and Scope of Today's Lecture . . . 107
8.2 Full-Order Models and the Starting Point . . . 107
8.3 The Reduced-Basis Idea . . . 107
8.4 The Two Pillars of the Reduced-Basis Method . . . 108
8.5 Galerkin Projection and the Reduced Linear System . . . 108
8.6 Extension to Time-Dependent Problems . . . 109
8.7 Construction of the Reduced Basis: Proper Orthogonal Decomposition (POD) . . . 109
8.8 Hyper-Reduction: Handling Non-Affine and Nonlinear Problems . . . 109
8.8.1 Additional Challenges for Nonlinear Problems . . . 109
8.9 Offline–Online Decomposition for Linear Stationary Affine Problems . . . 110
8.10 Mathematical Foundations: Singular Value Decomposition (SVD) and POD . . . 110
8.11 The Affine Assumption: The Third Pillar of Efficiency . . . 111
8.12 A Non-Affine Example and Hyper-Reduction for the Right-Hand Side . . . 111
8.13 Classical Polynomial Interpolation as Motivation for Hyper-Reduction . . . 112
8.14 The Discrete Empirical Interpolation Method (DEIM) . . . 112
8.15 Greedy Selection of the Magic Points . . . 113
8.16 Practical Considerations: Balancing Interpolation and Reduction Errors . . . 113
8.17 Conclusion and Outlook: Where Do We Go from Here? . . . 114

9 Lab 4 - Reduced basis (POD-Galerkin) for time-dependent problems 115
9.1 Introduction . . . 115
9.2 Time-Dependent Parametric PDEs . . . 115
9.2.1 Full-Order Discretization . . . 115
9.2.2 Reduced Basis (POD-Galerkin) Formulation . . . 116
9.3 Example: Heat Diffusion in a Three-Block Material . . . 116
9.3.1 Problem Setup . . . 116
9.3.2 Full-Order Simulation . . . 116
9.3.3 Precomputed Dataset . . . 117
9.4 Exercise: POD Basis Construction and Projection Error . . . 117
9.5 Next Steps . . . 117
9.6 Data Exploration . . . 118
9.7 Train / Test Split . . . 118
9.8 Constructing the Snapshot Matrix for POD . . . 118
9.9 Singular Value Decomposition (SVD) . . . 118
9.10 Choosing the Reduced Dimension n . . . 119
9.10.1 Visualising the First Modes . . . 119
9.11 Projection Error on the Test Set . . . 119
9.11.1 Build the test snapshot matrix . . . 119
9.11.2 Project and reconstruct . . . 119
9.11.3 Compute relative errors . . . 119
9.11.4 Aggregate over time and trajectories . . . 120

9.12 Building the Reduced Solver . . . 120
9.12.1 Precompute parameter-independent reduced matrices . . . 120
9.12.2 Time stepping (Backward Euler) . . . 120
9.12.3 Rollout . . . 121
9.13 Summary of the Workflow . . . 121
9.14 Time Stepping in the Reduced Space . . . 121
9.14.1 Full Reduced Solver . . . 121
9.15 Exploiting Affine Parametrization . . . 122
9.15.1 Important Note on Efficiency . . . 122
9.16 Verification and Debugging . . . 122
9.17 Assignments for the Exam . . . 122
9.17.1 Assignment 1: Vector Field Problem (Stationary) . . . 123
9.17.2 Assignment 2: Chemical Mixer (Stationary, Non-homogeneous Dirichlet) . . . 123
9.17.3 Assignment 3: Time-Dependent Schrödinger Equation (Time-Dependent) . . . 123
9.17.4 General Advice . . . 123
9.18 Final Remarks . . . 123

10 An Introduction to Neural Networks from the Point of View of Functions' Approximation 125
10.1 Introduction: Why Data-Driven ROMs? . . . 125
10.2 Beyond Linear Dimensionality Reduction: DL-ROMs . . . 125
10.3 From Polynomial Fitting to Neural Networks . . . 126
10.3.1 Polynomial Interpolation . . . 126
10.3.2 Linear Least Squares Regression . . . 126
10.3.3 Towards Nonlinear Dependence on Parameters . . . 127
10.3.4 The Core Idea of Neural Networks . . . 127
10.3.5 Training a Neural Network . . . 127
10.4 Autoencoders: Nonlinear Generalization of POD . . . 128
10.4.1 The Autoencoder Framework . . . 128

11 Lab 5 - Fundamentals of neural networks 129
11.1 Why Neural Networks? . . . 129
11.2 Basic Structure of a Neural Network . . . 129
11.2.1 Choosing Layer Dimensions . . . 130
11.2.2 Activation Functions . . . 130
11.3 Training a Neural Network . . . 130
11.4 Implementing Neural Networks with PyTorch . . . 130
11.4.1 Creating a Layer . . . 130
11.4.2 Weights and Biases . . . 130
11.4.3 Forward Pass . . . 131
11.4.4 Batch Processing . . . 131
11.4.5 Automatic Differentiation (Gradients) . . . 131
11.5 Device Management (CPU / GPU) . . . 131
11.6 Next Steps . . . 132
11.7 From Reduced Basis to Neural Networks . . . 132
11.7.1 Layer Definition . . . 132
11.7.2 Choosing Layer Dimensions . . . 132
11.7.3 Activation Functions . . . 132
11.8 Data Shape Conventions . . . 133
11.9 Implementing a Network in PyTorch . . . 133
11.9.1 Creating a Layer . . . 133
11.9.2 Device Management (CPU/GPU) . . . 133
11.9.3 Forward Pass with Batching . . . 133
11.9.4 Accessing Weights and Biases . . . 133

11.9.5 Automatic Differentiation . . . 133
11.10 Building a Network via Composition . . . 134
11.11 Training a Neural Network . . . 134
11.11.1 Data Preparation . . . 134
11.11.2 Network Construction for 1D Problem . . . 134
11.11.3 Loss Function . . . 134
11.11.4 Training the Network . . . 135
11.11.5 Initialization . . . 135
11.12 Optimizers and Batching . . . 135
11.12.1 Full Batch vs. Mini-Batch . . . 135
11.12.2 Choice of Optimizer . . . 135
11.12.3 Learning Rate and Epochs . . . 135
11.12.4 Monitoring Training . . . 136
11.13 Practical Recommendations . . . 136
11.14 Conclusion . . . 136
11.15 Finite Elements in Python . . . 136
11.15.1 Introduction . . . 136
11.15.2 Mesh and Finite Element Space . . . 136
11.15.3 Function – Vector Correspondence . . . 137
11.15.4 PDE: Stationary Heat Equation . . . 137
11.16 Reduced Basis Method (RBM) . . . 137
11.16.1 Motivation for the Cookie Problem . . . 137
11.16.2 Snapshot Collection . . . 137
11.16.3 POD Basis . . . 137
11.16.4 Galerkin Projection . . . 138
11.16.5 Time-Dependent Problems . . . 138
11.17 Neural Networks for Surrogate Modeling . . . 138
11.17.1 Basic Layer . . . 138
11.17.2 PyTorch Implementation . . . 138
11.17.3 Training . . . 138
11.17.4 Activation Functions . . . 139
11.17.5 Saving and Loading . . . 139
11.17.6 High-Dimensional Inputs: Convolutional Layers . . . 139
11.18 Assignments (Exam Preparation) . . . 139
11.18.1 Assignment 1: Vector Field (Stationary) . . . 139
11.18.2 Assignment 2: Chemical Mixer (Stationary, Non-homogeneous Dirichlet) . . . 139
11.18.3 Assignment 3: Time-Dependent Schrödinger Equation . . . 139
11.18.4 Exam Procedure . . . 139
11.19 Concluding Remarks . . . 140

12 Non-intrusive ROMs for Parametrized Problems through Proper Orthogonal Decomposition & Neural Networks (POD+NN) 141
12.1 From Neural Networks to PoDNN . . . 141
12.1.1 Neural networks: a formal viewpoint . . . 141
12.1.2 Why training a neural network is difficult . . . 142
12.1.3 Gradient-based training . . . 142
12.1.4 Transition to PoDNN . . . 143
12.1.5 Stochastic gradient descent . . . 143
12.1.6 Underfitting, overfitting, and validation . . . 143
12.1.7 How should one choose a neural network architecture? . . . 144
12.1.8 Classical approximation results . . . 144
12.1.9 Quantitative approximation results . . . 144
12.1.10 Transition to the next part . . . 145
12.1.11 The PoDNN idea . . . 145

12.1.12 Learning the reduced coordinates . . . 145
12.1.13 Why this is useful . . . 146
12.1.14 Time-dependent problems . . . 146
12.1.15 Summary . . . 146
12.1.16 Next step . . . 146

13 Lab 6 - The POD-NN method 147
13.1 Introduction . . . 147
13.1.1 Recap of Standard Reduced Basis (RB) Method . . . 147
13.1.2 Motivation for POD-NN . . . 147
13.2 Test Case: A Nonlinear Stationary Heat Problem . . . 148
13.3 Dataset and POD Decomposition . . . 148
13.3.1 Step 1: Compute POD Basis . . . 148
13.3.2 Step 2: Choose the Reduced Dimension n . . . 148
13.3.3 Step 3: Validate the Basis via Projection Error . . . 149
13.4 Step 4: Train the Neural Network (POD-NN) . . . 149
13.4.1 Prepare Training Data . . . 149
13.4.2 Network Architecture . . . 149
13.4.3 Loss Function and Training . . . 149
13.4.4 Online Prediction . . . 149
13.5 Comparison with Standard RB (Where Applicable) . . . 150
13.6 Exercises for the Lab . . . 150
13.7 Conclusion . . . 150
13.8 Introduction . . . 150
13.9 Recap: Standard Reduced Basis (RB) Method . . . 150
13.9.1 Motivation for POD-NN . . . 151
13.10 Test Case: Nonlinear Stationary Heat Equation on a Rod . . . 151
13.11 Dataset and POD Decomposition . . . 151
13.11.1 Split into Training and Test . . . 151
13.11.2 Compute the POD Basis . . . 152
13.11.3 Choosing the Reduced Dimension n . . . 152
13.11.4 Projection Error on Training and Test . . . 152
13.12 Training the Neural Network (POD-NN) . . . 152
13.12.1 Prepare the Coefficient Dataset . . . 152
13.12.2 Design the Network Architecture . . . 153
13.12.3 Training . . . 153
13.13 Evaluating the POD-NN Model . . . 153
13.14 Discussion . . . 153
13.14.1 When to Use POD-NN vs. Standard RB . . . 153
13.14.2 Advantages of POD-NN . . . 153
13.14.3 Practical Tips . . . 154
13.15 Exercises . . . 154
13.16 Conclusion . . . 154
13.17 Finite Elements in Python . . . 154
13.17.1 Introduction . . . 154
13.17.2 Mesh and Finite Element Space . . . 155
13.17.3 Function–Vector Correspondence . . . 155
13.17.4 PDE: Stationary Heat Equation . . . 155
13.18 Reduced Basis Method (RBM) . . . 155
13.18.1 The Cookie Problem . . . 155
13.18.2 POD via SVD . . . 156
13.18.3 Galerkin Projection . . . 156
13.18.4 Time-Dependent Problems . . . 156
13.19 Neural Networks . . . 156

13.19.1 Dense Layer . . . 156
13.19.2 PyTorch Implementation . . . 156
13.19.3 Training a Network . . . 156
13.20 POD-NN: Hybrid Approach . . . 156
13.20.1 Motivation . . . 156
13.20.2 Steps . . . 157
13.20.3 Shape Considerations . . . 157
13.20.4 Error Decomposition . . . 157
13.20.5 Practical Training . . . 157
13.20.6 Hyperparameter Tuning . . . 157
13.21 Assignments (Exam Preparation) . . . 158
13.22 Conclusion . . . 158

14 Deep learning-based reduced order models (DL-ROM & POD-DL-ROM) 159
14.1 Deep-Learning Reduced-Order Models (DL-ROMs) . . . 159
14.1.1 From POD to DL-ROMs . . . 159
14.1.2 POD as a Linear Compression Method . . . 159
14.1.3 POD-NN and Its Limitation . . . 160
14.1.4 Autoencoders as a Nonlinear Generalization of POD . . . 160
14.1.5 The DL-ROM Idea . . . 160
14.1.6 Training and Inference . . . 161
14.1.7 Main Message . . . 161
14.1.8 Training the DL-ROM . . . 161
14.1.9 Joint Learning Strategy . . . 161
14.1.10 Training Stage and Testing Stage . . . 162
14.1.11 Advantages and Limitations . . . 162
14.1.12 Physics-Informed Extensions . . . 162
14.1.13 Choice of the Latent Dimension . . . 162
14.1.14 Convolutional Architectures . . . 163
14.1.15 Motivation for the POD-DL-ROM Architecture . . . 163
14.1.16 A Hybrid Idea: POD Followed by a DL-ROM . . . 163
14.1.17 The POD-DL-ROM Pipeline . . . 163
14.1.18 Loss Function . . . 164
14.1.19 Why This Helps . . . 164
14.1.20 Application: Cardiac Electrophysiology . . . 164
14.1.21 Why Classical Reduction Becomes Difficult . . . 164
14.1.22 Why an Autoencoder? . . . 165
14.1.23 Training Strategy . . . 165
14.1.24 A Simple One-Dimensional Example . . . 165
14.1.25 A Two-Parameter Benchmark . . . 165
14.1.26 Computational Performance . . . 166
14.1.27 Application to Cardiac Electrophysiology . . . 166
14.1.28 Application to Fluid Dynamics . . . 166
14.1.29 Fluid-Structure Interaction . . . 166
14.1.30 MEMS Applications . . . 167
14.1.31 Main Message . . . 167

15 Autoencoder-based DL-ROMs 169
15.1 From POD-NN to Deep Learning ROMs for Time-Dependent Problems . . . 169
15.2 Example: 2D Navier–Stokes Flow Past a Cylinder . . . 169
15.2.1 Physical Problem . . . 169
15.2.2 Available Dataset . . . 169
15.3 Why Not an Intrusive Reduced Basis Method? . . . 170
15.4 Data Preparation: Mapping (μ, t) to Snapshots . . . 170

15.4.1 Conceptual Framework . . . 170
15.4.2 Constructing the Input Tensor $\tilde{\boldsymbol{\mu}}$ . . . 170
15.4.3 Verification . . . 171
15.5 Data Preparation (Continued) . . . 171
15.5.1 Verification of the Input Tensor . . . 171
15.5.2 Reshaping to Standard Machine-Learning Format . . . 171
15.6 Toward a Surrogate: Singular Value Analysis . . . 171
15.7 Autoencoder for Nonlinear Dimensionality Reduction . . . 171
15.7.1 Architecture . . . 171
15.7.2 Training Objective . . . 172
15.7.3 Implementation Details . . . 172
15.8 Choice of Activation Functions . . . 172
15.9 Autoencoder Design: Practical Guidelines . . . 173
15.9.1 Choice of LeakyReLU Slopes for Encoder and Decoder . . . 173
15.9.2 Network Depth and Symmetry . . . 173
15.10 On the Minimal Latent Dimension . . . 173
15.10.1 Theoretical Bounds . . . 173
15.10.2 Comparison with Linear POD . . . 173
15.11 Geometric Intuition for the Upper Bound . . . 174
15.12 Next Steps . . . 174

16 Data-driven dynamical systems (DMD & SINDy) 175
16.0.1 Driven Dynamical Systems . . . 175
16.0.2 From State Observations to Dynamics Discovery . . . 175
16.0.3 Latent Dynamics Discovery . . . 175
16.0.4 Why the Latent Dynamics Matter . . . 176
16.0.5 Reduced-Time Integration in Latent Space . . . 176
16.0.6 Connection with POD . . . 176
16.0.7 Main Message . . . 176
16.0.8 Learning the Reduced Dynamics . . . 177
16.0.9 Option 1: Black-Box Neural Dynamics . . . 177
16.0.10 Reduced Dynamics as a Forecasting Problem . . . 177
16.0.11 Option 2: Linear Reduced Dynamics and DMD . . . 177
16.0.12 Discrete Formulation of DMD . . . 178
16.0.13 Computation via the SVD . . . 178
16.0.14 Why DMD is Useful . . . 178
16.0.15 Sparse Identification of Nonlinear Dynamics . . . 178
16.0.16 Interpretation of the Sparse Model . . . 179
16.0.17 Connection with Reduced-Order Modeling . . . 179
16.0.18 Main Message . . . 179
16.0.19 Practical Implementation of DMD . . . 179
16.0.20 Connection with POD-DMD . . . 180
16.0.21 Beyond Linear Dynamics . . . 180
16.0.22 Sparse Identification of Nonlinear Dynamics . . . 180
16.0.23 Sparse Regression . . . 181
16.0.24 Interpretation . . . 181
16.0.25 Example: The Lorenz System . . . 181
16.0.26 Main Message . . . 181
16.0.27 Sparse Identification of Nonlinear Dynamics: Interpretation of the Regression . . . 181
16.0.28 Sparse Regression and Coefficient Selection . . . 182
16.0.29 Autoencoder + SINDy . . . 182
16.0.30 Joint Loss Function . . . 182
16.0.31 Why This Matters . . . 183
16.0.32 Examples . . . 183

16.0.33 Main Message . . . 183

17 High-dimensional parameter spaces (PCA-net & mesh-informed layers) 185
17.1 DL-ROM: Learning the Parameter-to-Latent Map . . . 185
17.1.1 Recap of the Current State . . . 185
17.1.2 Constructing the Training Set for ϕ . . . 185
17.1.3 Network Architecture and Training . . . 185
17.1.4 Assembling the Final DROM Surrogate . . . 186
17.2 Evaluation and Performance . . . 186
17.2.1 Choice of Error Metric . . . 186
17.2.2 Test Results . . . 186
17.3 Improving Accuracy: Fourier Embedding for Time . . . 186
17.3.1 Motivation . . . 186
17.3.2 Implementation Idea . . . 186
17.3.3 Implementation Sketch . . . 187
17.4 High-Dimensional Parameter Field: Permeability Mapping . . . 187
17.4.1 Problem Statement . . . 187
17.4.2 Discretisation and High Dimensionality . . . 187
17.4.3 Dataset . . . 187
17.4.4 Why Reduced Basis Fails . . . 188
17.4.5 Singular Value Analysis of the Solution Manifold . . . 188
17.4.6 Next Step: POD-NN with High-Dimensional Input . . . 188
17.5 Addressing High-Dimensional Inputs: The PCINet Architecture . . . 188
17.5.1 Failure of Plain PODNN . . . 188
17.5.2 Possible Remedies . . . 189
17.5.3 PCINet: Principal Component Interaction Network . . . 189
17.5.4 Implementation and Comparison . . . 189
17.6 PCINet Performance and Further Refinements . . . 190
17.6.1 Evaluating PCINet on Test Data . . . 190
17.6.2 Why POD Compression of the Input May Be Suboptimal . . . 190
17.7 Strategy A: Sparse Mesh-Informed Layers (Geometric Deep Learning) . . . 190
17.7.1 Intuition: Limiting the Receptive Field . . . 190
17.7.2 Constructing a Mesh-Informed Layer . . . 191
17.7.3 Parameter Count and Sparsity . . . 191
17.7.4 Full Model and Results . . . 191
17.8 Visualising the Distribution of Errors: Box Plots . . . 191

18 Time extrapolation and latent dynamics for data-driven ROMs via DMD & SINDy 193
18.1 Extrapolation in Time: System Identification from Surrogate Data . . . 193
18.1.1 Motivation: Limitations of Data-Driven Surrogates . . . 193
18.1.2 Distillation: Using DMD/SINDy on ROM Output . . . 193
18.1.3 Comparison with Reduced Basis Methods . . . 194
18.2 Learning Dynamics from Data: A Simple Harmonic Oscillator . . . 194
18.2.1 True System . . . 194
18.2.2 Assumed Partial Data . . . 194
18.2.3 Linear Dynamics Assumption (DMD Ansatz) . . . 194
18.2.4 Least-Squares Estimation of A . . . 194
18.2.5 Eigenvalue Decomposition of A . . . 194
18.2.6 From Simple Example to DMD on PDE Data . . . 195
18.3 Distillation: Extracting Dynamics from a Trained Surrogate . . . 195
18.3.1 Motivation and Problem Setup . . . 195
18.4 Dynamic Mode Decomposition: Linear Dynamics from Data . . . 195
18.4.1 A Toy Example: Damped Harmonic Oscillator . . . 195
18.4.2 Linear Discrete-Time Model . . . 195

18.4.3 Least-Squares Estimation of **A** . . . 195
18.4.4 Prediction via Eigenvalue Decomposition . . . 196
18.4.5 SVD-Based Formulation and Truncation . . . 196
18.5 DMD on the DL-ROM Surrogate for the Navier–Stokes Problem . . . 196
18.5.1 Problem Setup . . . 196
18.5.2 Implementation Steps . . . 197
18.5.3 Expected Outcome . . . 197
18.6 Truncated SVD and DMD Evolution . . . 197
18.6.1 Truncation of the SVD . . . 197
18.6.2 DMD-based Evolution . . . 198
18.7 Issue of Growing Modes: Non-Autonomous Bias . . . 198
18.8 Motivation for Latent Space Dynamics . . . 198
18.9 Nonlinear Dynamics Example: DMD Failure . . . 198
18.10SINDy: Sparse Identification of Nonlinear Dynamics . . . 199
18.10.1 Mathematical Formulation . . . 199
18.10.2 PySINDy Package . . . 199
18.10.3 Application to Latent Space Dynamics . . . 199
18.11SINDy: Sparse Identification of Nonlinear Dynamics . . . 199
18.11.1 Extending Linear DMD to Nonlinear Dynamics . . . 199
18.11.2 Application to a Nonlinear Oscillator . . . 200
18.12SINDy on the Latent Space of a DL-ROM . . . 200
18.12.1 Motivation . . . 200
18.12.2 Data Preparation . . . 200
18.12.3 Choice of the Library . . . 201
18.12.4 Training and Results . . . 201
18.12.5 Discussion . . . 201
18.13Outlook . . . 201

19 Shallow recurrent decoder networks for reduced order modeling of parametrized systems from sensor data (SHRED-ROMs) 203
19.1 Motivation: learning solutions from sensor data . . . 203
19.1.1 From parameters to measurements . . . 203
19.2 Classical motivation: separation of variables . . . 203
19.2.1 Reconstruction from the initial condition . . . 204
19.2.2 Observing the solution at sensor locations . . . 204
19.3 From separation of variables to SHRED . . . 205
19.3.1 The learning problem . . . 205
19.3.2 Why the encoder is no longer needed . . . 205
19.3.3 Role of the recurrent decoder . . . 205
19.4 Summary of the main intuition . . . 205
19.5 Time-windowed sensor data and SHRED . . . 206
19.5.1 Chunking the time series . . . 206
19.5.2 From forecasting to state reconstruction . . . 206
19.5.3 The role of the recurrent network . . . 206
19.5.4 Decoder-based reconstruction . . . 206
19.6 Training objective . . . 206
19.7 Remarks on dynamical constraints . . . 207
19.8 POD-based variants . . . 207
19.8.1 Interpretation . . . 207
19.9 Examples and intuition . . . 207
19.10Extension to multiple scenarios . . . 207
19.10.1 Learning from parametric sensor data . . . 207
19.11A pedagogical example: a three-dimensional dynamical system . . . 208
19.11.1 Classical solution via eigen-decomposition . . . 208

19.11.2 Recovering the coefficients from the initial condition . . . 208
19.11.3 Recovering the coefficients from sensor data . . . 208
19.12 Parametric setting and time-windowed sensor data . . . 209
19.12.1 Training data . . . 209
19.12.2 Offline and online phases . . . 209
19.13 Training objective . . . 209
19.14 Why this is easier than autoencoder-based approaches . . . 210
19.14.1 Shallow recurrent decoder . . . 210
19.15 Numerical examples . . . 210
19.15.1 Unsteady cylinder flow . . . 210
19.15.2 A one-dimensional parametric problem . . . 210
19.16 Robustness with respect to sensors and chunk size . . . 210
19.16.1 Main takeaway . . . 211
19.17 Numerical experiments: fixed and moving sensors . . . 211
19.17.1 Sensor placement and robustness to noise . . . 211
19.17.2 Fixed versus moving sensors . . . 211
19.17.3 Parametric setting . . . 211
19.18 Why this is useful . . . 211
19.19 Need for a time-prediction network . . . 212
19.20 Recurrent neural networks for time forecasting . . . 212
19.20.1 Motivation . . . 212
19.20.2 Basic recurrent architecture . . . 212
19.20.3 Prediction . . . 212
19.21 Training of recurrent models . . . 212
19.22 Variants of recurrent architectures . . . 213
19.23 Connection with the global picture . . . 213
19.24 Closing remarks . . . 213

20 Recovering from sensors with SHRED based ROMs 215
20.1 Assignments for the Second Part of the Course . . . 215
20.1.1 General Guidelines . . . 215
20.1.2 The Three Test Cases . . . 215
20.2 SHRED: Sensor-Based Surrogate Modelling . . . 216
20.2.1 Motivation and Core Idea . . . 216
20.2.2 Architecture . . . 216
20.2.3 Training and Generalisation . . . 216
20.2.4 Examples . . . 216
20.2.5 Connection with Separation of Variables . . . 217
20.3 SHRED Tutorial: Drug Diffusion Model . . . 217
20.3.1 Problem Description . . . 217
20.3.2 Observability and Sensor Choice . . . 217
20.3.3 Data Preprocessing for LSTM . . . 217
20.3.4 Model Architecture and Training . . . 218
20.4 SHRED Model Implementation . . . 218
20.4.1 LSTM and Decoder Initialization . . . 218
20.4.2 Forward Pass . . . 218
20.4.3 Understanding the LSTM Recurrence . . . 219
20.4.4 Training and Regularization . . . 219
20.5 SHRED Model: LSTM–Decoder Forward Pass Details . . . 219
20.5.1 Role of LSTM Hidden States . . . 219
20.5.2 Computational Details in PyTorch . . . 219
20.6 Training and Evaluation . . . 220
20.6.1 Training Loop . . . 220
20.6.2 Extracting Synthetic Sensor Data . . . 220

20.6.3 Applying Padding and Windowing . . . 220
20.6.4 Model Initialisation and Training . . . 220
20.7 Extension to a High-Dimensional Problem: Double-Gyre Flow . . . 221
20.7.1 Problem Description . . . 221
20.7.2 Output Compression via POD . . . 221
20.7.3 Sensors and Training . . . 221
20.8 Fourier DROM: Alternative Compression Strategy . . . 221
20.9 Concluding Remarks . . . 221

21 Our Path Review and Conclusion **223**
21.1 The Broader Role of Reduced Order Modeling in Engineering . . . 223
21.2 Recap: Classical Reduced Basis Methods and Their Limits . . . 223
21.3 Entering the World of Neural Networks: POD-NN . . . 224
21.4 An Industrial Application: Centrifugal Pump Efficiency . . . 224
21.5 The Broader Role of Reduced Order Modeling in Engineering . . . 224
21.6 Recap: Classical Reduced Basis Methods and Their Limits . . . 225
21.7 Data-Driven Surrogates: From POD-NN to DL-ROM . . . 225
21.7.1 POD-NN . . . 225
21.7.2 DL-ROM and POD-DL-ROM . . . 225
21.7.3 Industrial Example: Centrifugal Pump Efficiency . . . 225
21.8 Injecting Physics: Physics-Informed ROMs . . . 226
21.9 Extension to Optimal Control . . . 226
21.9.1 Learning Controlled Solutions . . . 226
21.9.2 Closing the Loop in Latent Space . . . 226
21.10 Concluding Remarks . . . 226
21.11 The Broader Role of Reduced Order Modeling in Engineering . . . 227
21.12 Recap: Classical Reduced Basis Methods and Their Limitations . . . 227
21.13 Data-Driven Surrogates: From POD-NN to DL-ROM . . . 227
21.13.1 POD-NN . . . 227
21.13.2 DL-ROM and POD-DL-ROM . . . 228
21.13.3 Industrial Example: Centrifugal Pump Efficiency . . . 228
21.14 Injecting Physics: Physics-Informed ROMs . . . 228
21.15 Extension to Optimal Control . . . 228
21.16 Time Extrapolation via Latent Dynamics Discovery . . . 229
21.17 Handling Uncertainty: When Parameters Are Unknown . . . 229
21.18 Geometry Adaptation . . . 229
21.19 Concluding Reflections: Blending Simplicity and Power . . . 229
21.20 Stochastic and Generative Reduced Order Models . . . 230
21.20.1 Motivation: Uncertainty in Data and Models . . . 230
21.20.2 Latent Dynamics with Random Coefficients . . . 230
21.21 Multifidelity Reduced Order Modeling . . . 230
21.21.1 The Multifidelity Paradigm . . . 230
21.21.2 POD-Based Multifidelity Correction . . . 230
21.21.3 Example: Navier–Stokes Flow Past a Cylinder . . . 231
21.21.4 Progressive / Multi-Level Fidelity . . . 231
21.22 Conclusions and Outlook . . . 231

Chapter 1

Introduction to Model Order Reduction

1.1 Course introduction

Welcome to the third edition of the *Modern Model Order Reduction Techniques* course.

My name is Andrea Manzoni. I work in numerical analysis in the Department of Mathematics, and I am a mathematical engineer. Over the last 10–15 years, I have been working extensively with these kinds of tools.

About three years ago, we decided to offer this course in the Master of Mechanical Engineering in order to introduce a bit more mathematics than in previous courses, provide a new perspective on numerical simulation, and tackle some of the challenges that naturally arise in practical, real-world computations.

The course will therefore combine:

- organizational and technical information;
- the main theoretical content;
- practical aspects related to numerical simulation and implementation.

1.2 Course material

Two books will be used as reference material.

The first one, which I know very well, is already available on WeBuild. There is a folder containing it, and I can share my book for sure. The other one is freely downloadable; I have already downloaded it legally and uploaded it to the folder as well.

What you find there is the first edition; this course will also refer to the second edition. This does not mean that you are expected to read both books entirely. I will select relevant parts here and there, present them in class, and also provide slides. In addition, I will explain some topics at the blackboard.

If you are interested in new tools related to dynamical systems, especially from the point of view of control and machine learning, the companion book can be very useful. It will be used more in the second part of the course. For the first half, we will follow the other book more closely.

At the moment, there is no single book that fully covers the content of the course. I will also point out some papers that may be interesting to read. This is a rapidly evolving research field, so the material is distributed across several sources. My goal is to put everything together into a coherent story, hopefully an enjoyable one, with a more engineering-oriented perspective. Of course, a more mathematical and theoretical presentation would also be possible, but this course will be more practical.

I will be in charge of the lectures.

1.3 Practical organization

Nicola Ranessfranco, a young postdoctoral research fellow, will be in charge of the laboratory sessions. We will use Python throughout the labs.

The practical activities will be organized during the semester in order to:

- learn how to implement the techniques presented in class;
- become familiar with Python;
- develop practical skills in numerical simulation.

We believe that being able to use Python is now a very useful outcome of this course.

1.4 Prerequisites and review of background material

If you have never heard about finite elements or mathematical modeling, do not worry. Perhaps this is not your case, since you may already have attended a course in numerical and mathematical methods during the bachelor degree in Mechanical Engineering, or a similar course in IRO.

In any case, I will assume that you have forgotten almost everything.

On Friday, I will provide a kind of crash course on mathematical modeling, starting from simple problems. The idea is to recover the basic concepts that should still be somewhere in your memory and build on top of them.

We will not redo everything from scratch, because that would be both boring and useless. Instead, we will use the knowledge you already have as a foundation.

Model order reduction, and reduced-order modeling more generally, must be built on top of strategies that we already use to solve and simulate problems. We do not want to discard our favorite solvers. Rather, we want to use them to generate essential data in many different scenarios, so as to leverage what we know about the solutions of the problem.

If you have never heard about numerical models for differential problems, there is a very good book on the subject. In the slides, you will find a selected list of chapters, especially Chapters 3 and 4. If you already have notes on classical numerical methods, that will also be perfectly fine.

Do not worry about studying everything immediately. We will first go back to the basic notions and notation that we need.

1.5 Bibliography and course notes

The bibliography is broad. You can search for papers and books on any aspect you want to deepen.

The slides will always be made public. We are also preparing course notes, which are still in progress. If time permits, we will provide the notes for at least the first part of the course in due time.

There are many books and additional materials available, so there is no need to worry about every detail at this stage.

1.6 Exam

The exam is oral. It covers both:

- the theoretical lectures;
- the practical lab sessions.

Students will be required to:

- present a selection of assigned exercises proposed during the practical sessions;
- answer theoretical questions related to the course topics.

The exercises will involve the use, adaptation, or extension of the methods presented during the course to some cases of interest. During the labs, you will be asked to solve exercises in order to learn how to implement each technique and verify that it works correctly.

Around the middle of the course and again near the end, we will publish two sets of exercises. These exercises may be slightly longer than those done in class and may concern situations that are somewhat different from the examples already seen.

You will have time to solve them, and you will select:

- one exercise from the first set;
- one exercise from the second set.

At the oral exam, you will present the solutions of the two chosen exercises, essentially reproducing and explaining what you have done during the labs.

You will prepare a notebook for each exercise. If you have never used notebooks before, you will learn how to code with them. In the notebook, you should explain:

- which technique you decided to use;
- why you chose it;
- what results you obtained.

Of course, we will also ask questions and expect you to justify your choices. In the same oral exam, you will also be asked some theoretical questions, such as:

- how a given technique works;
- what the underlying principle is;
- how two techniques compare;
- when one method should be preferred over another.

The labs are essential because they allow you to implement methods directly, work on the code, and obtain concrete results. Everything will be done in Python.

The exercises from the labs will also serve as inspiration for the assigned exercises that you will solve on your own, possibly also with your colleagues. That is not forbidden. Of course, you will take the oral exam individually.

1.7 Exam dates and flexibility

Since the exam is oral, the official dates may already be published. You can keep those dates, but I am also flexible enough to take into account other constraints that you may have.

For example, if you have written exams during the session and need to shift the model order reduction exam slightly before or after the published date, we can try to find a solution. Once enough students with similar needs collect, we can arrange oral exams on dates different from the published ones.

Please do not ask for a separate date every day, otherwise it becomes difficult to organize everything. The organizational part is therefore open, and all relevant information will be provided. If there are questions, feel free to ask at any time.

1.8 The big picture

The goal of these first two hours, together with the additional hour on Friday, is to introduce the big picture of model order reduction, together with the notation and the main ideas that will be used throughout the course. Then, point by point, we will come back to the individual techniques and strategies.

At least once in your life, you have probably simulated the solution of a problem in linear elasticity, fluid dynamics, or another area of applied mechanics. Often, when facing a new problem in the same class, one ends up writing the code from scratch again and again.

A further difficulty appears when the solution depends on parameters. By changing some parameters, boundary conditions, or forcing terms, the solution may change significantly. For instance, if you are interested in changing the angle of attack in a fluid-flow problem around an obstacle, such as a cylinder, you may want to compute the velocity pattern for several values of the angle of attack. In that case, although the code can be reused, every new value of the parameter may still require a new simulation from scratch.

This becomes problematic when simulations take a long time. If a computation takes only a few minutes, repeating it may be acceptable. But if it takes days, it is not practical to rerun everything every time a parameter changes. In many situations, we would like to obtain answers in seconds rather than in days.

1.8.1 Parametric problems

The main idea is that we are dealing with problems that depend on parameters. These parameters may be:

- **physical**, for instance material properties, boundary conditions, or source terms;
- **geometrical**, for instance geometric changes in the domain.

In this course, we will mostly focus on physical parameters. Geometrical parameters are also important, but they usually lead to a much heavier computational burden, because changing the geometry affects the discretization itself.

A typical workflow is the following:

1. start from a physical system, such as linear elasticity or fluid flow;
2. discretize the problem;
3. obtain a *high-fidelity* or *full-order* approximation, for instance with a finite element code or a finite volume code;
4. solve the resulting large algebraic system.

This is what you already know how to do. The difficulty is that each time the parameters change, the large system must be solved again. Since we want accurate solutions, the resulting discrete problem often has a very large number of degrees of freedom.

1.8.2 Reduced-order modeling

The objective of model order reduction is to replace the high-fidelity model with a *reduced-order* approximation that is:

- much smaller;
- much cheaper to solve;
- still accurate enough for the task at hand.

At first sight, this raises a natural question: if the original model was large because it was accurate, how can a smaller model still provide reliable results?

The answer is that the reduced model must be built intelligently, using information extracted from the high-fidelity simulations. In this way, one can construct a reduced representation that preserves the essential behavior of the system.

1.8.3 When is this useful?

Reduced-order modeling is useful whenever we need:

- repeated simulations for different scenarios;
- optimal design;
- control of a system;
- fast prediction of the solution for new parameter values.

In control problems, for example, one changes the input or the control action in order to steer the system toward a desired target configuration. In optimization, one repeatedly modifies the parameters and compares the resulting outputs in order to improve a design.

1.8.4 How parameters affect the solution

A key issue in the whole story is how the parameters influence the solution. The more strongly the solution changes when the parameters vary, the more difficult the problem becomes.

For example, consider a heat-transfer problem in a heat exchanger. Suppose that the problem depends on four parameters:

- three temperatures imposed on different portions of the boundary;
- the Peclet number, which plays a role similar to the Reynolds number in fluid mechanics and measures the relative strength of advection with respect to diffusion.

When the parameters change, the temperature field may change globally. In other words, varying the parameters can affect the whole solution, not just a small portion of it. Sometimes the effect is localized, but in general one should expect a global impact.

1.8.5 Learning from previous simulations

Now imagine that the problem has been simulated many times for different parameter values. If a new parameter value is given, one may hope to reconstruct the corresponding solution by combining information from previously computed solutions.

This is not just a matter of naive interpolation. Even though interpolation is familiar for scalar functions, here we are trying to predict an entire field, that is, a vector with potentially thousands or millions of components. Interpolating in such a high-dimensional space is not straightforward.

Nevertheless, the underlying idea is similar: use previous data to infer the new solution. Reduced-order methods can be seen as advanced ways of doing exactly this.

1.8.6 Another example: Navier–Stokes flows

A classical example comes from Navier–Stokes simulations. Suppose we change the Reynolds number in a backward-facing step problem, or vary the aspect ratio of the geometry. The overall flow may remain qualitatively similar, but the details of the vortices and the local structures can change.

In this case, the solution depends on multiple parameters, and the reduced-order model must be able to capture these variations efficiently.

1.8.7 Why do we care?

Very often, we want to go beyond direct simulation. This is especially true in:

- control;
- optimization;

- parameter estimation;
- real-time prediction;
- uncertainty quantification.

The difficulty comes from two sources:

1. the *data complexity*, since simulation outputs are often very rich;
2. the *computational complexity*, since the original high-fidelity model may be too expensive to use repeatedly.

The aim of model order reduction is precisely to cope with these difficulties while preserving the essential features of the original system.

1.9 Dimensionality reduction and computational complexity

We have to reduce the dimensionality of the data, and we also have to reduce the complexity of the problem. These are two key aspects of model order reduction.

For instance, if solving a Stokes or Navier–Stokes problem requires solving a nonlinear problem at every time step, we also want to reduce the computational cost associated with this repeated procedure. At the same time, we must fight the curse of dimensionality, because the data themselves must also be compressed.

1.9.1 Reduced-order modeling

Reduced-order modeling, or model order reduction, is a broad family of techniques aimed at replacing the original numerical solver with a new one having much lower computational complexity.

The objective is to solve problems not only on large computing clusters, but also on much smaller devices, such as laptops, tablets, or even smartphones in some settings.

A reduced-order model should be able to evaluate the solution for a new parameter value at a cost that does not depend on the dimension of the original problem. Otherwise, we are back to the original computational bottleneck.

This raises several challenges:

- we must learn from previously computed data;
- we must reduce the dimensionality of the data;
- we must preserve sufficient accuracy;
- we must keep the online computational cost low.

The underlying hope is that, if the parameter changes only slightly, then the solution will not change dramatically. In that case, we should not need to solve a large numerical problem again from scratch. Instead, we want to solve a much smaller one.

1.9.2 Reduced basis methods

Reduced basis methods are a particular class of reduced-order models. In this course, we will focus on them in the first part of the semester, since historically they represent one of the earliest and most influential developments in reduced-order modeling.

Interestingly, the first reduced basis ideas were introduced not by mathematicians, but by engineers, in particular structural engineers, at the end of the 1970s and the beginning of the 1980s. The basic idea was already there: learn from solutions that have already been computed.

This is similar to what happened with finite elements. First the engineers developed the method, and later the mathematicians provided a rigorous theoretical framework. A similar path was followed for reduced basis methods.

1.9.3 Connection with finite elements

Finite element methods already perform a first reduction of dimensionality. Indeed, they replace an infinite-dimensional problem with a finite-dimensional one by restricting the solution to a discrete set of functions.

However, to achieve sufficient accuracy, a finite element discretization often requires a very large number of degrees of freedom.

Thus, reduced-order modeling can be seen as a second level of reduction:

- finite elements reduce the problem from infinite to finite dimension;
- reduced-order models reduce it further from very large dimension to a much smaller one.

In finite elements, the basis functions are usually very general and universal. For example, one may use:

- continuous piecewise linear functions;
- piecewise quadratic functions;
- higher-order polynomial functions.

These functions are local and problem-independent. The drawback is that many of them are needed in order to represent the solution accurately.

Reduced basis methods instead rely on a small number of global basis functions, chosen in a way that is tailored to the problem at hand. In this sense, the idea is closer to modal analysis, where one studies the vibration modes of a structure.

1.9.4 Snapshots and reduced spaces

The key observation is that all the solutions we are interested in are solutions of the same problem for different values of the parameters.

If we can compute some of these solutions in advance, we can use them to build a reduced space. The high-fidelity solutions used to build this space are called *snapshots*.

The reduced basis method then constructs a low-dimensional space spanned by a few carefully selected snapshots, so that new solutions can be approximated efficiently by combining them.

The result is that, instead of solving a large algebraic system, we solve a much smaller one.

1.9.5 A roadmap for the course

The main topics of the semester are the following:

1. dimensionality reduction algorithms;
2. projection-based reduced-order models;
3. reduced basis methods;
4. nonlinear and hyper-reduced models;
5. non-intrusive reduced-order models;
6. neural-network-based approaches.

1.9.6 Dimensionality reduction techniques

We will begin with dimensionality reduction algorithms. You may already have seen the singular value decomposition of a matrix. If not, at least eigenvalues and eigenvectors should already be familiar tools.

These methods provide a different representation of the data. In particular, we will rely on:

- singular value decomposition;

- proper orthogonal decomposition;
- in the second part of the course, autoencoders.

Autoencoders are neural networks that can reduce the dimensionality of data in a general way. We will use them later in the course.

1.9.7 Projection-based reduced-order models

Once a reduced representation of the data has been constructed, we need a new formulation of the problem to be solved. This leads to projection-based reduced-order models.

In particular, we will study:

- reduced basis methods;
- Galerkin projection.

Galerkin projection is the mathematical mechanism underlying finite element formulations as well. In the reduced-order setting, it allows us to project the high-dimensional problem onto a low-dimensional reduced space.

1.9.8 Linear, nonlinear, and hyper-reduced problems

The basic framework works well for linear problems, either stationary or time-dependent, and for problems in which the parameters enter in a relatively mild way.

The situation becomes more difficult for nonlinear problems, or when the parameters affect the system in a more complex manner. In those cases, a further reduction step is often needed.

This leads to *hyper-reduction* techniques, which were developed to make reduced-order methods applicable to more challenging problems. Extending reduced-order modeling to nonlinear and highly complex settings is difficult from a computational point of view, and this has been a major research topic in the community over the last decade.

1.9.9 Non-intrusive reduced-order modeling

Later in the course, we will also study non-intrusive reduced-order models. This is the point where neural networks become relevant.

The idea is to exploit the predictive power of neural networks while using the principles introduced earlier in the course. This is especially useful when one can query a solver only through its outputs, but cannot access its internal algebraic structures such as matrices or vectors.

This is common in industrial software: one may have access to a code that produces solutions, but not to the underlying linear algebra. In such a situation, non-intrusive approaches are particularly attractive.

1.9.10 Neural networks and deep learning

We will also use this course to become familiar with neural networks. In particular, we will discuss:

- autoencoders;
- basic deep learning ideas;
- the use of neural networks in reduced-order modeling.

The goal is not to study neural networks in full generality, but rather to understand how they can be combined with reduced-order modeling strategies.

1.10 Dynamical systems and data-driven modeling

Last but not least, we will also study dynamical systems. For mechanical and aerospace engineers, dynamical systems are essential.

Up to now, you have mainly used numerical tools to solve dynamical systems, such as Euler schemes, Runge–Kutta schemes, and related methods. In those cases, the dynamical system was given, and the task was to approximate its solution.

A different question is the following: suppose that we observe the state of a system, but we do not know the dynamical law that generated the data. In other words, we see the evolution of the system in time, but the right-hand side of the model is unknown.

For instance, if we write

$$y'(t) = f(y(t)),$$

the function f is usually known in classical modeling. But can we infer f from data? If we can, then we may be able to reconstruct the system from observations and predict its future behavior.

This is the motivation for data-driven dynamical systems. This topic is also closely related to the construction of reduced-order models for time-dependent problems in an intrusive setting.

1.10.1 Dynamic Mode Decomposition

Since we like linearity, dynamic mode decomposition (DMD) is a way to find the best linear representation of a dynamical system. Linearity is attractive because it allows us to study stability through the eigenvalues of the corresponding operator.

1.10.2 Physics-informed and sensor-based learning

We will also discuss how to combine neural networks with time-dependent modeling. This is not exactly the same as physics-informed neural networks, but rather a broader idea of combining data, dynamics, and learning.

Finally, we will consider the case in which only sensor measurements are available. In many applications, one has access only to data such as:

- pressure measurements at selected locations;
- strain or stress measurements at sensors;
- other partial observations of the system.

There are ways to build reduced-order models using neural networks that exploit this type of sensor data.

1.11 Course perspective

Do not be afraid of this part: the course is designed for a five-credit class in Mechanical Engineering, so the level will remain accessible.

The plan is to alternate slides and blackboard exposition. At this point, we will start by motivating the need for reduction.

1.12 Why do we need reduction?

Let us return to the example of the heat exchanger. The model may be an advection–diffusion equation, which is a second-order linear elliptic PDE. A typical stationary problem has the form

$$-\nabla \cdot (\kappa \nabla u) + \beta \cdot \nabla u = f \quad \text{in } \Omega,$$

with suitable boundary conditions on $\partial\Omega$.

The exact form of the equation is not the main point here. What matters is the computational workflow required to solve it numerically.

1.12.1 From differential equations to algebraic systems

If we want to solve the problem by finite elements, finite differences, or any other numerical strategy, we must first discretize the domain.

This means that we replace the continuous domain with a computational mesh or grid. A computer cannot handle infinitely many degrees of freedom, so the unknown function must be approximated in a finite-dimensional space.

For example, in finite elements we may look for a piecewise linear, globally continuous approximation u_h. Here h denotes the mesh size. In general:

- small h means a finer mesh and a more accurate approximation;
- large h means a coarser mesh and a less accurate approximation.

After discretization, the differential problem becomes a linear algebraic system:

$$A_h(\mu)\,\mathbf{u}_h(\mu) = \mathbf{f}_h(\mu),$$

where:

- $A_h(\mu)$ is the system matrix;
- $\mathbf{f}_h(\mu)$ is the right-hand side;
- $\mathbf{u}_h(\mu)$ is the vector of unknown coefficients;
- $\mu \in \mathbb{R}^p$ is a vector of parameters.

The dimension of the system is denoted by N_h.

1.12.2 Parameter dependence

If the problem depends on parameters, then the matrix and the right-hand side may change when the parameters change. This means that the solution also depends on the parameter vector μ.

For instance, if only the Reynolds number changes, then $p = 1$. More generally, the parameters may influence:

- material properties;
- boundary conditions;
- source terms;
- geometric features.

Even a small variation in the parameters may require reassembling the matrix and the right-hand side, and then solving the system again.

1.12.3 Computational cost

Using the high-fidelity or full-order model means that for each parameter value μ, we must:

1. assemble $A_h(\mu)$ and $\mathbf{f}_h(\mu)$;
2. solve the linear system;
3. recover the solution.

This has a cost, both for assembly and for solving. Since the system can be very large, the cost may scale at least quadratically with the number of degrees of freedom, depending on the solver used.

If we need to explore many scenarios, this becomes very expensive. For example, if the solution has to be computed for 100 different parameter values, repeating the full procedure 100 times may be impractical.

1.12.4 Finite elements as a first reduction

Finite element methods already express the solution as a linear combination of basis functions:

$$u_h(x) = \sum_{i=1}^{N_h} u_i \, \varphi_i(x).$$

The functions φ_i are the building blocks of the approximation.

This already represents a first reduction from an infinite-dimensional problem to a finite-dimensional one. However, the resulting dimension N_h may still be very large.

Model order reduction aims at performing a second reduction, replacing the large system by a much smaller one while preserving accuracy as much as possible.

1.13 From finite elements to reduced bases

In the finite element setting, the unknown solution is expanded in a basis of simple functions. In one spatial dimension, and for linear finite elements, these are the familiar "hat" functions.

If x_i is a node of the mesh, then the associated basis function φ_i satisfies:

$$\varphi_i(x_j) = \delta_{ij},$$

that is, it is equal to 1 at the node x_i and equal to 0 at all the other nodes.

By taking linear combinations of such basis functions, we can describe the candidate solutions of the problem. In other words, the finite element approximation has the form

$$u_h(x) = \sum_{i=1}^{N_h} u_i \, \varphi_i(x),$$

where the coefficients u_i are the degrees of freedom.

Thus, the vector

$$\mathbf{u}_h = (u_1, \ldots, u_{N_h})^T$$

contains exactly the information needed to represent the discrete solution in that basis.

1.13.1 Weak formulation and Galerkin method

To compute these coefficients, we do not solve the differential equation directly. Instead, we first write its weak formulation and then restrict it to the discrete finite element space.

This procedure leads to the Galerkin formulation, which can be written in matrix form as a linear system:

$$A_h \mathbf{u}_h = \mathbf{f}_h.$$

Here:

- A_h is the stiffness matrix;
- $\mathbf{f}_h$ is the load vector;
- $\mathbf{u}_h$ is the vector of unknown coefficients.

The matrix and the vector are obtained from the bilinear and linear forms associated with the weak formulation.

1.13.2 Analogy with reduced-order modeling

The finite element approximation already uses a basis expansion. However, the basis functions φ_i are completely generic: they are not designed for one specific problem, but rather for a large class of problems.

This is the key point. In finite elements, the basis is universal; in reduced-order modeling, the basis is problem-dependent.

The reduced basis method follows the same general philosophy:

- represent the solution in terms of basis functions;
- compute the corresponding coefficients;
- reduce the number of basis functions dramatically.

The goal is to replace the large finite element basis by a much smaller reduced basis that is tailored to the problem and to the parameter range of interest.

1.14 From high-fidelity solutions to reduced bases

Suppose now that we can solve the high-fidelity problem for selected parameter values

$$\mu_1, \mu_2, \ldots, \mu_n.$$

For each of these values, we compute a full finite element solution:

$$\mathbf{u}_h(\mu_1),\ \mathbf{u}_h(\mu_2),\ \ldots,\ \mathbf{u}_h(\mu_n).$$

These solutions are called *snapshots*. They are high-fidelity solutions obtained from the original solver.

The idea is to extract from these snapshots a basis that captures the dominant features of the solution manifold.

1.14.1 Snapshot matrix

We collect the snapshots in a matrix:

$$S = \big[\mathbf{u}_h(\mu_1)\ \mathbf{u}_h(\mu_2)\ \cdots\ \mathbf{u}_h(\mu_n)\big].$$

From this matrix, we extract a reduced basis matrix

$$V \in \mathbb{R}^{N_h \times N},$$

where:

- N_h is the dimension of the high-fidelity space;
- N is the dimension of the reduced space;
- $N \ll N_h$.

The columns of V span the reduced space. They are not arbitrary: they are constructed from the information contained in the snapshots.

1.14.2 Reduced representation

For a new parameter value μ, the high-fidelity solution $\mathbf{u}_h(\mu)$ is approximated as

$$\mathbf{u}_h(\mu) \approx V\mathbf{u}_N(\mu),$$

where $\mathbf{u}_N(\mu) \in \mathbb{R}^N$ is a vector of reduced coefficients.

This is the central idea of reduced basis methods:

- the full solution lives in a large space of dimension N_h;
- the reduced solution lives in a much smaller space of dimension N;
- the reduced coefficients are enough to reconstruct an accurate approximation.

The resulting approximation is not exact, but it is much cheaper to compute and, if the basis is well chosen, sufficiently accurate.

1.15 Why not simply use a coarser mesh?

A natural question is why we do not simply increase the mesh size h and reduce N_h directly.

The answer is that this is indeed possible, but it is not the same thing.

If we use a coarser mesh, we reduce the number of degrees of freedom, but we also change the discretization error of the numerical method. The approximation becomes cheaper, but generally less accurate.

Reduced-order modeling follows a different strategy:

- the high-fidelity solver is kept as the reference model;
- the reduced model is built from high-fidelity solutions;
- the reduced model aims to preserve accuracy while drastically lowering the online cost.

In practice, mesh coarsening and model reduction can also be combined, but they play different roles.

1.16 How is the reduced basis constructed?

At this stage, the basis matrix V is still abstract. In the next lectures, we will explain how to compute it systematically.

For those of you who know singular value decomposition, the reduced basis can be obtained from the dominant singular vectors of the snapshot matrix. This idea is closely related to proper orthogonal decomposition.

The key principle is that the reduced basis should capture the most relevant directions of variation among the snapshots.

1.17 Online evaluation

Once the reduced basis has been built, the solution for a new parameter value μ can be approximated by solving a much smaller algebraic system.

This is the main advantage of reduced-order modeling:

- an expensive *offline* stage is used to build the reduced basis;
- a fast *online* stage is used to evaluate new parameter values.

The online cost is much smaller than that of the original high-fidelity solver, and ideally it should depend only weakly on N_h.

1.18 From the reduced basis to the reduced problem

Now we need to determine the reduced coordinates $\mathbf{u}_N$ for any new value of the parameter μ.

The idea is to find a recipe that, given a new parameter value, allows us to compute the new reduced vector $\mathbf{u}_N(\mu)$. If we are able to do this, then the approximation

$$\mathbf{u}_h(\mu) \approx V\mathbf{u}_N(\mu)$$

becomes very useful, because the matrix V has already been built once and for all in the offline stage.

In other words, the expensive part of the computation is performed only once. Then, for each new parameter value, we only need to solve a much smaller problem.

1.18.1 The residual

Let us go back to the high-fidelity problem:

$$A_h(\mu)\mathbf{u}_h(\mu) = \mathbf{f}_h(\mu).$$

If we insert the reduced approximation $\mathbf{u}_h(\mu) \approx V\mathbf{u}_N(\mu)$ into this equation, we do not obtain an exact equality. Instead, we get

$$A_h(\mu)V\mathbf{u}_N(\mu) \neq \mathbf{f}_h(\mu).$$

This motivates the definition of the *residual*:

$$\mathbf{r}_h(\mu; \mathbf{u}_N) := \mathbf{f}_h(\mu) - A_h(\mu)V\mathbf{u}_N(\mu).$$

If the reduced approximation is good, then the residual should be small.

1.18.2 Why the residual cannot be set to zero directly

The residual lives in the large high-fidelity space, while the unknown $\mathbf{u}_N$ has only N components. Since $N \ll N_h$, we do not have enough degrees of freedom to impose

$$\mathbf{r}_h(\mu; \mathbf{u}_N) = \mathbf{0}$$

directly.

Indeed, this would produce an overdetermined system: many equations, but too few unknowns.

What we can do instead is to require that the residual vanishes only in the directions spanned by the reduced basis.

1.18.3 Galerkin projection

The reduced basis method is built by imposing that the residual be orthogonal to the reduced space:

$$V^T\mathbf{r}_h(\mu; \mathbf{u}_N) = \mathbf{0}.$$

This means that the residual must be orthogonal to every column of V. If the columns of V are denoted by

$$V = [\mathbf{v}_1, \ldots, \mathbf{v}_N],$$

then the orthogonality conditions are

$$\mathbf{v}_i^T\mathbf{r}_h(\mu; \mathbf{u}_N) = 0, \qquad i = 1, \ldots, N.$$

These N conditions determine the N unknown reduced coefficients.

1.18.4 The reduced system

Substituting the residual definition into the orthogonality condition gives

$$V^T\big(\mathbf{f}_h(\mu) - A_h(\mu)V\mathbf{u}_N(\mu)\big) = \mathbf{0}.$$

Therefore,

$$V^T A_h(\mu)V\mathbf{u}_N(\mu) = V^T\mathbf{f}_h(\mu).$$

This is the reduced system.

Its dimension is $N \times N$, which is much smaller than the original $N_h \times N_h$ system. If N is small, for instance ten or twenty, then the reduced problem is extremely cheap to solve.

1.18.5 Offline and online stages

The reduced basis method naturally separates the computation into two stages.

Offline stage. In the offline stage, we:

- solve the full-order problem for selected parameter values;
- collect the snapshots;
- build the reduced basis matrix V.

This stage may be expensive, but it is performed only once.

Online stage. In the online stage, for each new parameter value μ, we:

- assemble the reduced matrix $V^T A_h(\mu) V$;
- assemble the reduced right-hand side $V^T \mathbf{f}_h(\mu)$;
- solve the reduced system;
- reconstruct the approximate solution $\mathbf{u}_h(\mu) \approx V \mathbf{u}_N(\mu)$.

Since the reduced system is small, the online cost is much lower than in the full-order setting.

1.18.6 A crucial subtlety

There is however an important issue. Even if the reduced system is small, we do not want its assembly to be expensive.

If assembling the reduced matrix requires manipulating very large full-order objects at every parameter value, then much of the gain would be lost.

For this reason, one needs efficient strategies to precompute as much as possible offline and to exploit the parameter dependence cleverly. This issue will be addressed later in the course.

1.18.7 Summary of the idea

The basic reduced basis strategy is therefore:

1. compute several high-fidelity solutions;
2. use them to build a reduced basis;
3. approximate new solutions in the reduced space;
4. determine the reduced coefficients by Galerkin projection;
5. solve a small reduced system instead of the original large one.

This is the core mechanism behind projection-based reduced-order modeling.

Chapter 2

Recalls on High-Fidelity, Full-Order Models (Finite Elements)

On Wednesday, during the first class, we motivated why modeling and model-order reduction are essential from a computational point of view. We now return to the need for reduction.

The starting point is the *high-fidelity model* (or *full-order model*, FOM), namely the large problem obtained by discretizing a parametrized differential equation on a domain. In the finite element setting, this typically leads to a large linear system. For instance, in the case of advection–diffusion, one obtains a system of the form

$$A_h(\mu)\, u_h(\mu) = f_h(\mu),$$

where $u_h(\mu) \in \mathbb{R}^H$ is the high-dimensional solution vector, $A_h(\mu) \in \mathbb{R}^{H\times H}$ is the full-order matrix, $f_h(\mu) \in \mathbb{R}^H$ is the right-hand side, and H is very large.

The objective of model-order reduction is to replace this large problem with a much smaller one. The reduced problem has the same algebraic structure, but it is defined in a low-dimensional space. In particular, we want to replace the large vector $u_h(\mu)$ by an approximation of the form

$$u_h(\mu) \approx V\, u_N(\mu),$$

where $V \in \mathbb{R}^{H\times N}$ is the reduced basis matrix, with $N \ll H$, and $u_N(\mu) \in \mathbb{R}^N$ is the reduced solution vector.

This approach involves two main steps.

2.1 Construction of a reduced space

The first step consists in identifying a good reduced space, that is, a low-dimensional subspace capable of approximating well the set of solutions of the original problem. This is typically done by computing a collection of high-fidelity solutions, called *snapshots*,

$$u_h(\mu_1),\ u_h(\mu_2),\ \ldots,\ u_h(\mu_M),$$

and extracting from them the dominant information. The resulting basis vectors form the matrix V, which we call the *reduced basis*. Once this basis is available, any new solution is approximated by a small vector of coefficients $u_N(\mu)$.

In other words, we are compressing information: instead of representing a solution with H degrees of freedom, we represent it using only N coordinates, with $N \ll H$.

2.2 Projection onto the reduced space

The second step is to determine the reduced system satisfied by the reduced coefficients $u_N(\mu)$. We substitute the approximation

$$u_h(\mu) \approx V\, u_N(\mu)$$

into the full-order equations and impose a Galerkin orthogonality condition. This means that the residual must be orthogonal to the reduced space:

$$V^T\big(A_h(\mu)V\,u_N(\mu) - f_h(\mu)\big) = 0.$$

Hence, the reduced system is

$$A_N(\mu)\,u_N(\mu) = f_N(\mu),$$

with

$$A_N(\mu) = V^T A_h(\mu) V, \qquad f_N(\mu) = V^T f_h(\mu).$$

This is the reduced-order model (ROM).

2.3 Computational issue

Even if the reduced system is much smaller than the original one, we still need to build $A_N(\mu)$ and $f_N(\mu)$. If these quantities are assembled naively, the computation may still be expensive, because it involves large matrix–matrix and matrix–vector products.

For this reason, we study how the dependence on the parameter μ can be exploited. In particular, if the parameter enters the problem in a simple way, then it may be possible to precompute some parameter-independent quantities offline and combine them online at very low cost.

2.4 Affine parameter dependence

A very favorable situation occurs when the full-order matrix and right-hand side admit an affine decomposition:

$$A_h(\mu) = \sum_{q=1}^{Q_A} \theta_q^A(\mu)\,A_h^q, \qquad f_h(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\,f_h^q,$$

where the matrices A_h^q and vectors f_h^q do not depend on μ, while the scalar functions $\theta_q^A(\mu)$ and $\theta_q^f(\mu)$ are easy to evaluate.

In this case, the reduced matrix becomes

$$A_N(\mu) = V^T A_h(\mu) V = \sum_{q=1}^{Q_A} \theta_q^A(\mu)\,\big(V^T A_h^q V\big),$$

and similarly

$$f_N(\mu) = V^T f_h(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\,\big(V^T f_h^q\big).$$

The reduced operators

$$V^T A_h^q V, \qquad V^T f_h^q$$

can be computed once and for all in an offline stage, since they do not depend on the parameters. Then, for each new value of μ, the online stage only requires a cheap linear combination of these precomputed quantities.

This is particularly useful when parameters appear as coefficients that can be factored out of the variational formulation. For example, this happens when a thermal diffusivity is constant over the domain, or when a Reynolds number multiplies the operator in the Navier–Stokes equations. In such cases, whenever a parameter can be taken outside the integrals in the weak formulation, the offline/online decomposition becomes especially efficient.

In the favorable case, the parameter dependence is *affine*. This means that the full-order operators can be written as linear combinations of parameter-independent matrices and vectors:

$$A_h(\mu) = \sum_{q=1}^{Q_A} \theta_q^A(\mu)\,A_h^q, \qquad f_h(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\,f_h^q,$$

where the scalar functions $\theta_q^A(\mu)$ and $\theta_q^f(\mu)$ depend on the parameter, while the matrices A_h^q and vectors f_h^q do not.

In this case, the reduced operators inherit the same structure. Indeed, the reduced matrix becomes

$$A_N(\mu) = V^T A_h(\mu) V = \sum_{q=1}^{Q_A} \theta_q^A(\mu) \left(V^T A_h^q V\right),$$

and the reduced right-hand side is

$$f_N(\mu) = V^T f_h(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu) \left(V^T f_h^q\right).$$

We may then define

$$A_N^q := V^T A_h^q V, \qquad f_N^q := V^T f_h^q,$$

so that

$$A_N(\mu) = \sum_{q=1}^{Q_A} \theta_q^A(\mu)\, A_N^q, \qquad f_N(\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\, f_N^q.$$

Since the quantities A_N^q and f_N^q do not depend on μ, they can be computed once and for all in an offline stage and stored in memory. Then, for each new parameter value, the online stage only requires the evaluation of the scalar functions and a cheap linear combination of precomputed reduced operators.

This is the *lucky case*: the problem depends on the parameters in a simple way, and the resulting online cost is very low. Unfortunately, this is not always the case. In general, the parameter dependence may be more complicated, and then additional techniques are needed to recover efficiency.

2.5 Why reduction still preserves the physics

The reduced-order model is not arbitrary. It retains the information contained in the full-order model in two ways.

First, the reduced basis matrix V is built from snapshots of the high-fidelity solutions, so the reduced space is extracted directly from the solutions of the original problem. Second, the reduced operators are obtained by projecting the full-order equations onto this reduced space. Therefore, the reduced model still encodes the physics of the original problem, only in a compressed form.

For this reason, reduced-order modeling is often described as *physics-informed*: the governing equations, the matrices, and the right-hand side are not discarded, but rather compressed and projected.

2.6 Geometric interpretation

It is helpful to think of the set of all high-fidelity solutions as a manifold in a very high-dimensional space. As the parameters vary, the corresponding solutions move on this manifold. Each snapshot is one point on it:

$$u_h(\mu_1),\ u_h(\mu_2),\ u_h(\mu_3), \ldots$$

The reduced basis seeks a low-dimensional subspace that approximates this manifold well. Instead of searching for solutions in a space of dimension H, we search in a smaller space of dimension $N \ll H$. This is analogous to representing vectors in $\mathbb{R}^3$ using a plane: rather than using three coordinates, we use only two coordinates in a suitable subspace.

The trade-off is clear: a smaller reduced space gives a faster model, but if it is too small, the approximation error increases. Hence there is always a balance between accuracy and efficiency.

2.7 Offline–online workflow

The overall workflow is the following:

1. start from a high-fidelity discretization of the full-order model;
2. solve the original problem for selected parameter values and collect snapshots;
3. build the reduced basis matrix V from these snapshots;
4. project the full-order operators onto the reduced space;
5. solve the reduced problem online for new parameter values.

The first steps are performed offline and can be computationally expensive, but they are done only once. The online stage is then very cheap, since it only involves operations in the reduced dimension.

2.8 Open questions

Two main issues remain:

- how to construct the reduced basis in an effective way;
- how to extend the strategy to more complex settings, such as time-dependent problems or non-affine parameter dependence.

These points will be addressed in the next lectures.

2.9 Online stage and error control

Once the reduced basis has been constructed offline, every new parameter value μ only requires the solution of a small reduced system. In practice, one assembles a reduced-order matrix and a reduced right-hand side from the precomputed reduced blocks, solves the reduced system, and then reconstructs an approximation in the full space.

More precisely, if $u_N(\mu)$ denotes the reduced solution, then the corresponding high-dimensional approximation is

$$u_h(\mu) \approx V\, u_N(\mu).$$

If one wants to visualize or postprocess the result in the original finite element space, the reduced coefficients are mapped back through the basis matrix V.

One is often also interested in estimating the error between the reduced approximation and the high-fidelity solution. This is essential to certify the quality of the reduced model and to decide whether the reduced dimension N is sufficiently large.

2.10 A numerical example: an affine case

Consider now an advection–diffusion problem with four parameters: three parameters controlling temperature boundary conditions and one parameter corresponding to the Péclet number. In this setting, the parameter dependence is affine.

The corresponding PDE can be written in the form

$$-\nabla \cdot (\kappa \nabla u) + \mathbf{b} \cdot \nabla u = f \quad \text{in } \Omega,$$

with suitable boundary conditions. Here κ is constant, so it can be taken out of the differential operator, while $\mathbf{b}$ is a given velocity field. The three temperature values appear as parameters in the boundary data, and the Péclet number enters as a multiplicative coefficient.

This is a case where reduced basis methods work very well. In the numerical experiment discussed in class, the finite element model had about

$$H \approx 44\,000$$

degrees of freedom, and the corresponding computation took about 3.5 seconds on a laptop. After reduction, the reduced problem had dimension

$$N = 31.$$

This represents a dramatic reduction in size: the reduced problem is roughly three orders of magnitude smaller than the full-order one. As a consequence, the online solve becomes extremely cheap.

Of course, there is a price to pay: the offline stage is expensive. One must compute the snapshots, build the reduced basis, and precompute the reduced operators. However, this cost is amortized if the reduced model is queried many times. If only a few simulations are needed, then the overhead of the offline stage may not be worthwhile. If many queries are required, the gain is substantial.

The error decay observed in this example is very fast. The approximation error decreases exponentially with the reduced dimension N, and already for moderate values such as $N = 31$, the reduced solution is extremely accurate, with an error on the order of 10^{-5} compared with the finite element solution.

2.11 A more difficult case: non-affine parametrization

The situation becomes more challenging when the parameters enter the problem in a non-affine way. Consider, for example, an advection–diffusion problem in which the source term is localized and depends on several parameters. In such a case, the source may resemble a Gaussian-shaped function whose center and width are controlled by parameters.

In this example, five parameters are involved:

- μ_1, related to the thermal diffusivity;
- μ_2, related to the direction of the advection field;
- μ_3, μ_4, μ_5, which control the position and shape of the source term.

Here the variability induced by the parameters is much stronger. The solution manifold is more complex, so more snapshot information is typically needed to represent it accurately. This makes the reduced basis construction harder.

This is an example of a *non-affine parametrization*. In such cases, the offline–online decomposition is not straightforward, and additional techniques are required to recover efficiency.

2.12 Transition to the weak formulation

We now move to the second part of the course. Starting from the advection–diffusion problem

$$-\Delta u + \mathbf{b} \cdot \nabla u = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega,$$

we want to understand why this leads to a linear system after finite element discretization.

The key step is the weak formulation. By multiplying the equation by a test function and integrating over the domain, one obtains a variational problem. After choosing a finite-dimensional approximation space, this weak problem is discretized, and the resulting algebraic system is exactly the full-order linear system that we use as the starting point for model reduction.

2.13 A crash course on the weak formulation

To go from a differential equation to a linear algebraic system, we first need to reformulate the problem in a weaker sense. This is important because the strong formulation may require too much regularity. For instance, if the source term is discontinuous, or if the coefficients vary abruptly in space, then the classical pointwise formulation may no longer be the right setting.

Consider, for example, a simple second-order problem such as

$$-\frac{d^2u}{dx^2} = f.$$

If f is discontinuous, then the second derivative of u must also be discontinuous. In a classical setting, this is problematic, because one would typically require u to be sufficiently smooth, for instance C^2. This motivates the introduction of a *weak formulation*.

2.14 From the strong form to the weak form

The basic idea is to multiply the equation by a test function v and integrate over the domain. After integration by parts, derivatives are transferred from the unknown u to the test function v, so that fewer regularity requirements are imposed on the solution.

As a model problem, consider

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega.$$

We multiply by a test function v and integrate over Ω:

$$\int_\Omega (-\Delta u)\, v\, dx = \int_\Omega f\, v\, dx.$$

Using integration by parts, we obtain

$$\int_\Omega \nabla u \cdot \nabla v\, dx = \int_\Omega f\, v\, dx,$$

provided that $v = 0$ on the boundary, so that the boundary term vanishes. Thus the weak formulation is:

Find $u \in H_0^1(\Omega)$ **such that**

$$\int_\Omega \nabla u \cdot \nabla v\, dx = \int_\Omega f\, v\, dx \qquad \forall v \in H_0^1(\Omega).$$

Here $H_0^1(\Omega)$ is the Sobolev space of functions that, together with their first derivatives, are square-integrable and vanish on the boundary.

2.15 Why the weak formulation is useful

This formulation is weaker in two senses:

- it only requires one derivative of u, rather than two;
- it is expressed in an integral sense, which is better suited to discontinuous data and coefficients.

This is the reason why weak formulations are the natural starting point for the finite element method.

2.16 Variable and discontinuous coefficients

The same approach works when the coefficients are not constant. For instance, if the thermal diffusivity depends on the space variable, say $\mu = \mu(x)$, then one may write

$$-\nabla \cdot \big(\mu(x)\nabla u\big) = f.$$

After multiplying by a test function and integrating by parts, one obtains

$$\int_\Omega \mu(x)\, \nabla u \cdot \nabla v\, dx = \int_\Omega f\, v\, dx.$$

If μ is piecewise constant, for example if the domain is split into subdomains with different materials, then the integral can simply be written as a sum over the subdomains:

$$\int_\Omega \mu(x)\,\nabla u \cdot \nabla v\,dx = \int_{\Omega_1} \mu_1\,\nabla u \cdot \nabla v\,dx + \int_{\Omega_2} \mu_2\,\nabla u \cdot \nabla v\,dx.$$

So discontinuities do not create any difficulty at the level of the weak formulation.

2.17 An example with affine parameter dependence

Let us now consider an advection–diffusion problem with parameters. Suppose that the diffusivity is piecewise constant and depends on two parameters μ_1 and μ_2, and that the advection field depends on an angle parameter μ_3. Then a model problem may be written as

$$-\nabla \cdot \big(\mu(x;\mu)\nabla u\big) + \beta(\mu_3)\cdot \nabla u = f,$$

where

$$\mu(x;\mu) = \begin{cases} \mu_1, & x \in \Omega_1, \\ \mu_2, & x \in \Omega_2, \end{cases} \qquad \beta(\mu_3) = \begin{bmatrix} \cos(\mu_3) \\ \sin(\mu_3) \end{bmatrix}.$$

The weak formulation is obtained by multiplying by a test function v and integrating by parts:

$$\int_{\Omega_1} \mu_1\,\nabla u \cdot \nabla v\,dx + \int_{\Omega_2} \mu_2\,\nabla u \cdot \nabla v\,dx + \int_\Omega \cos(\mu_3)\,\partial_{x_1} u\,v\,dx + \int_\Omega \sin(\mu_3)\,\partial_{x_2} u\,v\,dx = \int_\Omega f\,v\,dx.$$

This is the key point: the parameter-dependent quantities appear as scalar coefficients multiplying parameter-independent integrals. In other words, the dependence on the parameters is *affine*. The terms involving μ_1, μ_2, $\cos(\mu_3)$, and $\sin(\mu_3)$ can be separated from the spatial operators.

This is exactly the structure that is favorable for reduced-order modeling, because it allows one to precompute the parameter-independent parts and combine them cheaply online.

2.18 Right-hand side

The same idea applies to the right-hand side. If the source term admits a decomposition of the form

$$f(x;\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\,f^q(x),$$

then after testing with v we obtain

$$\int_\Omega f(x;\mu)\,v(x)\,dx = \sum_{q=1}^{Q_f} \theta_q^f(\mu) \int_\Omega f^q(x)\,v(x)\,dx.$$

Again, the dependence on the parameter is separated from the spatial integrals. This is the same affine structure we want for efficient model reduction.

2.19 Non-affine parameter dependence

Consider now a source term whose shape depends on the parameters in a genuinely nonlinear way. For example, the source may be localized in space and its center and width may vary with parameters μ_4, μ_5, and μ_6. A typical model is a rapidly decaying Gaussian-like function of the spatial variables (x, y) whose position and spread change with the parameters.

If such a source is inserted into the weak formulation, the right-hand side may take the form

$$\int_\Omega f(x;\mu)\, v(x)\, dx,$$

but in general it is no longer possible to factor out the parameter dependence as

$$f(x;\mu) = \sum_{q=1}^{Q_f} \theta_q^f(\mu)\, f^q(x).$$

Indeed, the dependence on x and on μ may be coupled in a nonlinear way, as happens for an exponential source term whose center and decay rate depend on the parameters.

This is an example of a *non-affine parametrization*. The underlying physics is unchanged, but the way the parameters enter the problem is more complicated. As a consequence, the reduced basis machinery becomes less straightforward, and additional techniques are needed to recover an efficient offline–online decomposition.

2.20 Weak formulation as a bilinear variational problem

Let us return to the weak formulation. After multiplying by a test function v and integrating by parts, the model problem can be written in the abstract form:

$$\text{Find } u \in V \text{ such that } a(u,v) = \ell(v) \qquad \forall v \in V,$$

where V is an infinite-dimensional Hilbert space, $a(\cdot,\cdot)$ is a bilinear form, and $\ell(\cdot)$ is a linear form.

For the prototypical diffusion problem,

$$-\Delta u = f \quad \text{in } \Omega, \qquad u = 0 \quad \text{on } \partial\Omega,$$

the weak formulation is

$$\text{Find } u \in H_0^1(\Omega) \text{ such that } \int_\Omega \nabla u \cdot \nabla v\, dx = \int_\Omega f\, v\, dx \qquad \forall v \in H_0^1(\Omega).$$

Here the left-hand side depends on both u and v, while the right-hand side depends only on v. The left-hand side is therefore a bilinear form, and the right-hand side is a linear functional.

This abstract formulation is useful because it places the PDE in a functional-analytic setting where general results can be applied. It also prepares the passage to finite-dimensional approximation.

2.21 Galerkin approximation

We now replace the infinite-dimensional space V by a finite-dimensional subspace $V_h \subset V$. The Galerkin approximation reads:

Find $u_h \in V_h$ **such that**

$$a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h.$$

This is the finite element formulation of the problem. The space V_h is chosen so that it is rich enough to approximate functions in V, while still being finite-dimensional.

For simplicity, consider the one-dimensional domain $\Omega = (0,1)$. We partition it into N_h subintervals,

$$0 = x_0 < x_1 < \cdots < x_{N_h} = 1,$$

with mesh size

$$h = x_{j+1} - x_j.$$

A typical finite element space of degree one is the space of continuous, piecewise linear functions that vanish on the boundary:

$$V_h = \{v_h \in C^0([0,1]) : v_h|_{[x_j,x_{j+1}]} \in \mathbb{P}_1,\ v_h(0) = v_h(1) = 0\}.$$

More generally, one may define spaces of degree r, where the functions are piecewise polynomials of degree r.

The basis functions $\{\varphi_i\}_{i=1}^{N_h}$ are usually chosen so that

$$\varphi_i(x_j) = \delta_{ij},$$

that is, each basis function is equal to 1 at one node and 0 at the others. These are the familiar hat functions in the piecewise linear case.

Any function $u_h \in V_h$ can then be written as

$$u_h(x) = \sum_{j=1}^{N_h} U_j \, \varphi_j(x),$$

where the coefficients U_j are the degrees of freedom of the approximation. In vector form,

$$u_h \leftrightarrow U = (U_1, \ldots, U_{N_h})^T.$$

This is why, in a finite-dimensional space, a function can be represented by a vector.

2.22 From the weak form to a linear system

We now insert the expansion of u_h into the Galerkin formulation and test with each basis function φ_i:

$$a\left(\sum_{j=1}^{N_h} U_j \varphi_j, \varphi_i\right) = \ell(\varphi_i), \qquad i = 1, \ldots, N_h.$$

Using the linearity of $a(\cdot,\cdot)$ in its first argument, we obtain

$$\sum_{j=1}^{N_h} a(\varphi_j, \varphi_i)\, U_j = \ell(\varphi_i), \qquad i = 1, \ldots, N_h.$$

This has the matrix form

$$A_h \, U = F_h,$$

where

$$(A_h)_{ij} = a(\varphi_j, \varphi_i), \qquad (F_h)_i = \ell(\varphi_i).$$

Thus, discretizing a PDE by finite elements leads naturally to a linear system. The matrix A_h is the stiffness matrix, and the vector F_h is the load vector. Solving the PDE therefore amounts to solving a linear algebraic system for the vector of degrees of freedom U.

2.23 Summary

The key steps are:

1. rewrite the PDE in weak form;
2. choose a finite-dimensional approximation space V_h;
3. expand the unknown in a finite element basis;
4. test against the basis functions;
5. obtain a linear system $A_h U = F_h$.

This is the bridge between the differential model and the algebraic system used in computation.

2.24 From the weak formulation to the finite element discretization

We are still discussing high-fidelity approximations, also called the *full-order model* (FOM). This is the model used to generate data and to benchmark the reduced-order model (ROM).

Let us recall the simplest prototype problem, namely the Poisson equation

$$-\Delta u = f \qquad \text{in } \Omega, \qquad u = 0 \qquad \text{on } \partial\Omega.$$

Its strong formulation requires a fairly high level of regularity. To weaken these requirements, we multiply the equation by a test function v, integrate over Ω, and integrate by parts. This yields the weak formulation:

$$\int_\Omega \nabla u \cdot \nabla v \, dx = \int_\Omega f \, v \, dx \qquad \forall v \in V,$$

where the test space is

$$V = H_0^1(\Omega).$$

In particular, functions in $H_0^1(\Omega)$ are square-integrable, have square-integrable gradients, and vanish on the boundary. This is why the boundary term produced by integration by parts disappears.

It is useful to rewrite the weak formulation in abstract form. Define

$$a(u, v) := \int_\Omega \nabla u \cdot \nabla v \, dx, \qquad \ell(v) := \int_\Omega f \, v \, dx.$$

Then the weak problem reads:

Find $u \in V$ **such that**

$$a(u, v) = \ell(v) \qquad \forall v \in V.$$

Here $a(\cdot, \cdot)$ is a bilinear form, since it depends linearly on both arguments, while $\ell(\cdot)$ is a linear form. This is the functional setting behind the finite element method.

2.25 Discretization of the domain

The finite element method begins by discretizing the domain Ω. We replace Ω with a mesh

$$\mathcal{T}_h = \{K\},$$

consisting of a finite number of elements. The parameter h denotes the mesh size, typically the maximum diameter of the elements. As h decreases, the mesh becomes finer and the approximation improves.

Since the domain is now represented by a finite mesh, we also replace the infinite-dimensional space V by a finite-dimensional subspace

$$V_h \subset V.$$

This is the discrete space where we seek an approximate solution.

2.26 Finite element spaces

In the simplest case, one uses continuous piecewise linear functions. More generally, one may use continuous piecewise polynomials of degree r.

For example, in one space dimension, if $\Omega = (0, 1)$ is partitioned into intervals

$$0 = x_0 < x_1 < \cdots < x_{N_h} = 1,$$

then the finite element space of degree one is

$$V_h = \left\{ v_h \in C^0([0,1]) : v_h|_{[x_j, x_{j+1}]} \in \mathbb{P}_1, \ v_h(0) = v_h(1) = 0 \right\}.$$

Thus, functions in V_h are globally continuous, vanish on the boundary, and are affine on each element.

More generally, for degree r, the restriction of a function to each element belongs to $\mathbb{P}_r$, the space of polynomials of degree at most r.

2.27 Boundary conditions

The essential boundary conditions must be enforced in the discrete space as well. In this example, since the exact solution satisfies

$$u = 0 \quad \text{on } \partial\Omega,$$

we require the same for every discrete function in V_h:

$$V_h \subset H_0^1(\Omega).$$

This is important because the boundary conditions are built directly into the approximation space.

2.28 Galerkin approximation

The finite element approximation is then:

Find $u_h \in V_h$ **such that**

$$a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h.$$

This is the Galerkin formulation. It is the discrete counterpart of the weak problem.

2.29 Basis expansion and algebraic system

Let $\{\varphi_1, \ldots, \varphi_{N_h}\}$ be a basis of V_h. Then the discrete solution can be written as

$$u_h(x) = \sum_{j=1}^{N_h} U_j \, \varphi_j(x),$$

where the coefficients $U_1, \ldots, U_{N_h}$ are the degrees of freedom of the approximation.

By testing the discrete problem against each basis function φ_i, we obtain

$$a(u_h, \varphi_i) = \ell(\varphi_i), \qquad i = 1, \ldots, N_h.$$

Substituting the expansion of u_h gives

$$\sum_{j=1}^{N_h} a(\varphi_j, \varphi_i) \, U_j = \ell(\varphi_i), \qquad i = 1, \ldots, N_h.$$

This can be written in matrix form as

$$A_h U = F_h,$$

where

$$(A_h)_{ij} = a(\varphi_j, \varphi_i), \qquad (F_h)_i = \ell(\varphi_i).$$

The matrix A_h is the stiffness matrix, and F_h is the load vector. Therefore, solving the PDE numerically amounts to solving a linear system for the vector of degrees of freedom U.

2.30 Main idea

The finite element method follows a clear sequence:

1. start from the weak formulation;
2. discretize the domain with a mesh;
3. choose a finite-dimensional approximation space V_h;
4. expand the unknown on a basis of V_h;
5. obtain a linear algebraic system.

This is the bridge between the differential problem and the computational model used in practice.

2.31 From the weak formulation to the algebraic system

We are still discussing high-fidelity approximations, or full-order models (FOMs). The finite element method is the standard way to discretize such models.

Starting from the weak formulation, the problem reads:

Find $u \in V$ **such that**

$$a(u, v) = \ell(v) \qquad \forall v \in V,$$

where V is an infinite-dimensional space, $a(\cdot, \cdot)$ is a bilinear form, and $\ell(\cdot)$ is a linear form.

The first step is to discretize the domain Ω by a mesh $\mathcal{T}_h$, and then to replace V with a finite-dimensional subspace

$$V_h \subset V.$$

This space is generated by a finite element basis

$$\{\varphi_1, \dots, \varphi_{N_h}\}.$$

Since V_h is finite-dimensional, every function $u_h \in V_h$ can be written as

$$u_h(x) = \sum_{j=1}^{N_h} U_j \, \varphi_j(x),$$

where the coefficients U_j are numbers. In this sense, a function in V_h is represented by the vector

$$U = (U_1, \dots, U_{N_h})^T \in \mathbb{R}^{N_h}.$$

We now impose that the weak formulation holds not for every test function in V, but only for every test function in V_h. This gives the Galerkin finite element approximation:

Find $u_h \in V_h$ **such that**

$$a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h.$$

Since every test function $v_h \in V_h$ can be written as a linear combination of the basis functions, it is enough to enforce the equation on each basis function φ_i, $i = 1, \dots, N_h$:

$$a(u_h, \varphi_i) = \ell(\varphi_i), \qquad i = 1, \dots, N_h.$$

Using the expansion of u_h, and the bilinearity of $a(\cdot, \cdot)$, we obtain

$$a\left(\sum_{j=1}^{N_h} U_j \varphi_j, \varphi_i\right) = \sum_{j=1}^{N_h} U_j \, a(\varphi_j, \varphi_i) = \ell(\varphi_i), \qquad i = 1, \dots, N_h.$$

This is a linear system of the form

$$A_h U = F_h,$$

where

$$(A_h)_{ij} = a(\varphi_j, \varphi_i), \qquad (F_h)_i = \ell(\varphi_i).$$

The matrix A_h is the stiffness matrix, and F_h is the load vector.

Therefore, solving the PDE numerically reduces to solving a linear system for the vector of degrees of freedom U. Once these coefficients are known, the finite element solution is reconstructed as

$$u_h(x) = \sum_{j=1}^{N_h} U_j \, \varphi_j(x).$$

2.32 Galerkin orthogonality

The Galerkin formulation also has a geometric interpretation. Since the exact solution u satisfies

$$a(u, v_h) = \ell(v_h) \qquad \forall v_h \in V_h,$$

and the discrete solution satisfies

$$a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h,$$

subtracting the two relations gives

$$a(u - u_h, v_h) = 0 \qquad \forall v_h \in V_h.$$

This is called *Galerkin orthogonality*. It says that the error $u - u_h$ is orthogonal to the discrete space V_h with respect to the bilinear form $a(\cdot, \cdot)$.

In many important cases, $a(\cdot, \cdot)$ behaves like an inner product. Then the finite element solution u_h can be interpreted as the best approximation of u in the space V_h, measured in the energy norm associated with $a(\cdot, \cdot)$.

This is the key link between the weak formulation, the finite element discretization, and the resulting linear algebraic system.

Chapter 3

The Reduced Basis Method

3.1 From the full-order model to the reduced-order model

We now move from the high-fidelity approximation, namely the full-order model (FOM), to the reduced basis approximation. The idea is to keep the same variational structure, but to replace the finite element space by a much smaller reduced space.

Recall the weak formulation of the problem: find $u \in V$ such that

$$a(u, v) = \ell(v) \qquad \forall v \in V,$$

where V is the infinite-dimensional solution space, $a(\cdot, \cdot)$ is a bilinear form, and $\ell(\cdot)$ is a linear form.

In the finite element setting, we replace V by a finite-dimensional subspace $V_h \subset V$ and obtain the Galerkin finite element approximation:

$$\text{Find } u_h \in V_h \text{ such that } a(u_h, v_h) = \ell(v_h) \qquad \forall v_h \in V_h.$$

This is the high-fidelity approximation, or full-order model, used to generate data and to serve as a reference solution.

The reduced basis method follows exactly the same philosophy, but with a different space. We introduce a much smaller reduced space

$$V_N \subset V_h, \qquad \dim(V_N) = N \ll N_h,$$

and we seek the reduced approximation in that space:

$$\text{Find } u_N \in V_N \text{ such that } a(u_N, v_N) = \ell(v_N) \qquad \forall v_N \in V_N.$$

This is the Galerkin reduced basis problem.

3.2 Projection viewpoint

The reduced basis method can be interpreted in two equivalent ways.

First, it is a Galerkin approximation in the reduced space V_N. Second, it is a projection of the high-fidelity solution onto that reduced space. In particular, the reduced solution is the best approximation of the full-order solution within the reduced space, measured in the norm induced by the problem.

This leads to the Galerkin orthogonality property:

$$a(u_h - u_N, v_N) = 0 \qquad \forall v_N \in V_N.$$

If the bilinear form $a(\cdot, \cdot)$ is symmetric and coercive, then u_N can be interpreted as the orthogonal projection of u_h onto V_N with respect to the energy inner product.

This is why reduced basis methods are called *projection-based reduced-order models*.

3.3 Why this is useful

The finite element approximation already replaces an infinite-dimensional problem by a finite-dimensional one. However, the dimension N_h of the finite element space may still be very large. The reduced basis method aims at a further reduction:

$$N \ll N_h.$$

As a result, the reduced system is much smaller and therefore much cheaper to solve.

The reduced basis method is especially effective when one needs to solve the same parametrized problem for many different values of the parameter μ. In that case, the reduced model provides a fast online solver after an expensive offline stage.

3.4 Connection with the finite element method

The reduced basis method uses the finite element solution as its reference. In practice, the finite element approximation u_h is the full-order model against which we compare the reduced approximation u_N.

The overall workflow is therefore:

1. discretize the PDE with finite elements and obtain the full-order model;
2. compute several high-fidelity solutions, called snapshots;
3. build a reduced basis space from those snapshots;
4. solve the reduced Galerkin problem in that space;
5. reconstruct an approximation of the full solution from the reduced coefficients.

3.5 Geometric interpretation

Geometrically, the reduced basis method looks for a low-dimensional subspace that captures the essential behavior of the solution manifold. The full-order solutions live in a large space, but the set of all relevant solutions often lies close to a much smaller subspace.

The reduced solution u_N is then the projection of the full-order solution onto that subspace. This geometric picture is the key to understanding why the method works so well.

3.6 Parameterized high-fidelity models and the solution manifold

So far, we have discussed the high-fidelity approximation, or full-order model (FOM), in the case of a fixed problem. We now move to the parameter-dependent setting, which is the one of real interest for model reduction.

When the problem depends on a parameter μ, the weak formulation becomes parameter dependent as well. In abstract form, we write:

Find $u_h(\mu) \in V_h$ **such that**

$$a(u_h(\mu), v_h; \mu) = \ell(v_h; \mu) \qquad \forall v_h \in V_h.$$

Here, for each value of the parameter μ, the bilinear form

$$a(\cdot, \cdot; \mu) : V_h \times V_h \to \mathbb{R}$$

and the linear form

$$\ell(\cdot; \mu) : V_h \to \mathbb{R}$$

may change with μ. Since the finite element space V_h is finite-dimensional, this means that the stiffness matrix and the right-hand side vector also depend on the parameter:

$$A_h(\mu)\, U_h(\mu) = F_h(\mu).$$

3.6.1 The role of the parameter

The parameter μ may represent a material property, a boundary condition, a source location, a transport direction, or any other quantity that changes from one simulation to another. The main motivation for reduced basis methods is precisely the need to solve the same PDE repeatedly for many values of μ.

From the functional point of view, the map

$$\mu \mapsto u_h(\mu)$$

associates to each parameter value the corresponding high-fidelity solution. From the algebraic point of view, the same information is encoded by the map

$$\mu \mapsto U_h(\mu),$$

where $U_h(\mu)$ is the vector of degrees of freedom of the finite element approximation.

These two viewpoints are equivalent, since the finite element basis gives a one-to-one correspondence between functions in V_h and vectors in $\mathbb{R}^{N_h}$.

3.6.2 The solution manifold

As μ varies over the admissible parameter set, the collection of all high-fidelity solutions forms what is usually called the *solution manifold*:

$$\mathcal{M}_h := \{u_h(\mu) \ : \ \mu \in \mathcal{P}\}.$$

This set is generally not a linear space, but it is the geometric object that the reduced basis method tries to approximate.

The more the solution changes when μ varies, the more difficult it is to approximate the solution manifold with a small reduced space. Conversely, if the dependence on μ is mild, then a low-dimensional approximation may work extremely well.

3.6.3 Why reduction is needed

Suppose that the full-order problem has N_h degrees of freedom, where N_h may be very large. Then solving the FOM for many values of μ can be computationally expensive. The goal of reduced basis methods is therefore to find a surrogate of the map

$$\mu \mapsto u_h(\mu)$$

that is much cheaper to evaluate online.

This surrogate will not replace the full-order model completely. Instead, it will exploit the high-fidelity model, using it to generate data and build a reduced space. In this sense, the reduced model is trained on information extracted from the FOM.

3.6.4 A geometric picture

One may visualize the set of high-fidelity solutions as a curved manifold in a very large space. Each new parameter value selects one point on that manifold. The reduced basis method aims to find a low-dimensional space that captures this manifold well.

This is analogous to approximating a complicated set of points in a high-dimensional space by a much smaller subspace. The reduced model then searches for approximations inside that subspace rather than in the full finite element space.

In many situations, this idea can be interpreted as a supervised learning problem: one collects pairs of parameter values and corresponding high-fidelity solutions, and then uses that information to predict the solution for new parameter values.

3.6.5 Relation with finite elements

It is important to stress that reduced basis methods do not start from the PDE directly. They start from the finite element approximation. The finite element method provides the high-fidelity reference, and the reduced basis method constructs a much smaller approximation space on top of it.

So the overall picture is:

- the PDE is first written in weak form;
- the weak problem is discretized with finite elements;
- the parameter-dependent high-fidelity solution is computed;
- a reduced space is built from selected high-fidelity solutions;
- the reduced problem is solved in that low-dimensional space.

3.7 The Reduced Basis Method

3.7.1 A smaller space for parametrized problems

The goal of the reduced basis method is to build a new approximation space that is much smaller than the finite element space, but still able to capture the essential behavior of the solution for new parameter values.

Given a parameter-dependent high-fidelity solution $u_h(\mu) \in V_h$, we want to approximate it in a reduced space

$$V_N \subset V_h, \qquad \dim(V_N) = N \ll N_h.$$

The idea is that, for every new value of the parameter μ, we only need to solve a very small problem in V_N.

The reduced approximation should satisfy two fundamental requirements:

- it should reproduce, at least approximately, the solutions that we already know;
- it should provide a low-dimensional representation for new parameter values.

Geometrically, one may think of the set of all high-fidelity solutions as a manifold. The reduced basis method seeks a low-dimensional subspace that approximates this manifold well.

3.7.2 Snapshot-based construction of the reduced space

The reduced space is built from high-fidelity solutions computed for selected parameter values. These solutions are called *snapshots*:

$$u_h(\mu_1),\ u_h(\mu_2),\ \ldots,\ u_h(\mu_M).$$

From this collection, we extract a basis of the reduced space. The vectors of that basis are called *reduced basis functions*.

Thus, the reduced basis method requires three ingredients:

1. a reduced basis space V_N;
2. a basis $\{\zeta_1, \ldots, \zeta_N\}$ of that space;
3. a recipe to compute the reduced coefficients for each new parameter value.

Once the basis is known, any reduced solution can be written as

$$u_N(\mu) = \sum_{j=1}^{N} \alpha_j(\mu)\, \zeta_j,$$

where the coefficients $\alpha_j(\mu)$ are the unknowns of the reduced problem.

3.7.3 Why this is a reduction

In finite element methods, the basis functions are usually local: each one is almost everywhere zero and is associated with a small portion of the mesh. This is useful because it produces sparse matrices.

In reduced basis methods, the basis functions are instead global. They are typically nonzero over the whole domain. The reduced matrices are therefore usually dense, but this is not a problem because their dimension is very small.

So the key point is not sparsity, but dimensionality reduction:

$$N \ll N_h.$$

The smaller N is, the cheaper the reduced problem becomes.

3.7.4 Galerkin reduced basis approximation

The reduced basis approximation is obtained by imposing the same weak formulation as before, but now only on the reduced space.

Recall the full-order weak problem:

$$a(u_h, v_h; \mu) = \ell(v_h; \mu) \qquad \forall v_h \in V_h.$$

The reduced basis Galerkin problem is:

Find $u_N(\mu) \in V_N$ **such that**

$$a(u_N(\mu), v_N; \mu) = \ell(v_N; \mu) \qquad \forall v_N \in V_N.$$

This is the *Galerkin reduced basis method*. It is the exact analogue of the finite element Galerkin method, except that the trial and test spaces are now much smaller.

3.7.5 Projection interpretation

The reduced basis method can also be understood as a projection method. If $u_h(\mu)$ is the high-fidelity solution and $u_N(\mu)$ is the reduced one, then the error satisfies the Galerkin orthogonality property

$$a\big(u_h(\mu) - u_N(\mu), v_N; \mu\big) = 0 \qquad \forall v_N \in V_N.$$

This means that the residual is orthogonal to the reduced space.

If the bilinear form is symmetric and coercive, then $u_N(\mu)$ can be interpreted as the best approximation of $u_h(\mu)$ in V_N, measured in the energy norm associated with the problem. In this sense, the reduced basis solution is the projection of the high-fidelity solution onto the reduced space.

This is why reduced basis methods are also called *projection-based reduced-order models*.

3.7.6 Algebraic meaning

The functional derivation is completely equivalent to the algebraic one. Since V_N is finite-dimensional, one can choose a basis $\{\zeta_1, \dots, \zeta_N\}$ and write

$$u_N(\mu) = \sum_{j=1}^{N} \alpha_j(\mu)\, \zeta_j.$$

Testing the reduced problem against each basis function gives a linear system of size $N \times N$:

$$A_N(\mu)\, \alpha(\mu) = F_N(\mu),$$

where $\alpha(\mu) \in \mathbb{R}^N$ collects the reduced coefficients.

This is the same structure we already saw for the finite element method, but now the dimension is much smaller. Therefore, the reduced basis model is much cheaper to solve online.

3.7.7 What remains to be done

The remaining open question is how to construct the reduced space V_N in practice. This is the main topic of the next class.

The general strategy is:

1. compute several high-fidelity snapshots;
2. extract a reduced basis from those snapshots;
3. formulate the reduced Galerkin problem;
4. solve the reduced system for each new parameter value.

In this way, the reduced basis method closes the circle: it starts from the high-fidelity model, compresses the information into a low-dimensional space, and then provides a fast surrogate for repeated simulations.

Chapter 4

Lab 2 - Finite element method in FEniCS

4.1 Introduction

In the previous session we discussed Python; today we move to finite elements and begin the actual topics of the course. Some of you have already encountered finite elements in other courses, but most likely not in a Python context. Therefore, we need to go through the syntax together. I will recall the fundamentals that link the theory to the code.

4.1.1 The `PINIX` and `DRROMS` Packages

The finite element implementation we will use is based on a package called `PINIX`. However, we will employ a slightly more user-friendly version that relies on a GitHub package developed during my PhD. This package will also be used for neural networks and reduced order modeling in general.

Before we start, we need to run a cell that installs the package on Google Colab. Colab already comes with several pre-installed packages (e.g., PyTorch, NumPy), but this custom package is not included. You must reinstall it every time you reopen the notebook (e.g., tomorrow). The installation takes less than a minute. Keep in mind:

- Restarting the session does *not* uninstall packages.
- If you choose "disconnect and delete", all packages are removed and you need to reinstall.

4.2 From Functions to Finite Dimensional Vectors

Consider a domain Ω and a function $u : \Omega \to \mathbb{R}$. For example, in the heat equation, $u(x)$ represents the temperature at point x. A function is an infinite-dimensional object (it assigns a value to every point). To make the problem computable, we discretize.

Figure 4.1: A simple mesh with 9 nodes.

Instead of the full function u, we only store its values at a finite set of points (e.g., the nine nodes of a mesh). This gives a vector $\mathbf{u} = (u(x_1), u(x_2), \ldots, u(x_9))$. Of course, information is lost in this process. However, we can reconstruct an approximation of u from these nodal values using, for instance, linear interpolation. This reconstruction produces a function u_h defined on the whole domain, which belongs to a finite-dimensional subspace $V_h \subset V$ (where V is a suitable function space like $L^2(\Omega)$). V_h is called the **finite element space**.

Key property: There is a bijection between any vector of nodal values and a function in V_h (piecewise linear and continuous). In formulas:

$$\mathbf{u} \in \mathbb{R}^N \quad \longleftrightarrow \quad u_h \in V_h.$$

4.2.1 Setting Up the Finite Element Environment

After installing the package, we import the necessary submodule:

```
import DRROMS.bases as FE
```

The `DRROMS` package (Deep Learning based Reduced Order Models) contains a submodule for finite element spaces, which we alias as `FE`.

Building the Mesh

First, define a geometric domain. For example, a rectangle with opposite vertices $(0, 0)$ and $(1, 1)$:

```
domain = FE.Rectangle((0,0), (1,1))
```

Then generate a mesh by specifying a step size (e.g., 0.5):

```
mesh = FE.mesh_domain(domain, step_size=0.5)
```

You can visualize the mesh with:

```
FE.plot(mesh)
```

Creating the Finite Element Space

On top of the mesh we build the space V_h of continuous, piecewise linear functions (Continuous Galerkin of degree 1):

```
Vh = FE.finite_element_space(mesh, continuous_galerkin=1)
```

(The output from `PINIX` in the background can be cleared with `clc`.)

4.2.2 Interpolating a Function

Consider the function $u(x, y) = x^2 + y^2$. We can represent it in the finite element space by interpolation. A convenient way to define such a function is using a lambda expression:

```
u = lambda x: x[0]**2 + x[1]**2
```

We then interpolate:

```
uh = FE.interpolate(u, Vh)
```

At a node of the mesh, u_h coincides with u. At other points, they differ because u_h is the piecewise linear interpolant.

Example:

```
x0 = np.array([0.5, 0.5])
print(u(x0))   # 0.5
print(uh(x0))  # 0.5 (if (0.5,0.5) is a node)
```

For a point in the interior of an element, e.g., $(0.25, 0.25)$, the values will be different:

```
x1 = np.array([0.25, 0.25])
print(u(x1))   # 0.125
print(uh(x1))  # approximate value from linear interpolation
```

4.3 Conclusion

This framework provides the basic tools for finite element discretization. In the next sessions we will apply it to partial differential equations, starting with the heat equation.

4.4 What is the point?

When solving a partial differential equation (PDE), our unknown is the continuous function u. However, in practice we only ever compute a discrete approximation u_h that lives on a mesh. The discretized version is easy to plot: we can directly call `FE.plot(u_h)` because the finite element space object contains the mesh information. Plotting a generic function u is not possible directly; we must first interpolate it onto a finite element space.

4.4.1 Plotting a finite element function

To plot u_h we need both the grid (the finite element space) and the values. This is analogous to using `plt.plot(x, y)`. The `FE` module handles this internally:

```
FE.plot(uh)          # uh is a finite element function
```

4.5 Function–vector correspondence

Let V_h be a finite element space (e.g., continuous piecewise linear functions on a mesh). Every function $u_h \in V_h$ can be represented uniquely by a vector $\mathbf{u}$ of its values at the degrees of freedom (DoFs). Conversely, any vector $\mathbf{u} \in \mathbb{R}^{N_h}$ (where $N_h = \dim V_h$) defines a unique function $u_h \in V_h$.

4.5.1 From function to vector

Given u_h, its vector representation is obtained by evaluating at all DoFs:

```
u_vec = uh.vector()        # or FE.dofs(uh)
```

The ordering of the entries is not obvious, but you can inspect it with:

```
dofs = FE.dofs(Vh)         # returns coordinates of all DoFs
```

4.5.2 From vector to function

Given a vector $\mathbf{u} \in \mathbb{R}^{N_h}$, we can reconstruct the corresponding finite element function:

```
u_vec = np.array([ ... ])    # N_h entries
uh = FE.function(u_vec, Vh)
```

This is the most useful direction: when solving a PDE, the solver returns a vector; we then convert it back to a function for visualization or post-processing.

Remark: In many codes, passing the vector directly to plotting routines works because the conversion to a function is done implicitly. So you can often write `FE.plot(u_vec, Vh)` and obtain the same plot.

4.6 Exercises

Before moving to PDEs, complete these two short exercises to become familiar with the finite element environment.

4.6.1 Exercise 1: Circle domain

- Build a domain that is a disk of radius 1 centered at $(0, 0)$.
- Create a finite element space of continuous piecewise linear functions on a mesh of this disk.
- Define a function $v(x, y) = \sin\big(4\pi(x + y^2)\big)$.
- Compute its finite element interpolant v_h.
- Plot v_h.
- Compare the true value $v(0.5, 0.3)$ with $v_h(0.5, 0.3)$.

4.6.2 Exercise 2: Domain with a hole (annulus)

- Use a domain that is a disk with a circular hole (an annulus). The mesh is coarse.
- Build a continuous Galerkin finite element space of degree 1 (CG1).
- Plot the mesh and mark the positions of the degrees of freedom.
- Repeat with a space of piecewise quadratic polynomials (CG2).
- Observe how the number and location of DoFs change.

4.7 Solutions and Discussion

4.7.1 Solution to Exercise 1

First, create the circular domain and mesh:

```
import DRROMS.bases as FE
import numpy as np
import matplotlib.pyplot as plt

domain = FE.Circle(center=(0,0), radius=1)
mesh = FE.mesh_domain(domain, step_size=0.1)   # fine mesh
Vh = FE.finite_element_space(mesh, continuous_galerkin=1)
```

The number of degrees of freedom N_h can be obtained in two ways:

```
Nh = Vh.dim
# or
dofs = FE.dofs(Vh)          # array of shape (Nh, 2)
Nh = dofs.shape[0]
```

For a CG1 space on a triangle mesh, N_h equals the number of vertices (approximately 1000 for step size 0.1).

Now define v and its interpolant:

```
v = lambda x: np.sin(4 * np.pi * (x[0] + x[1]**2))
vh = FE.interpolate(v, Vh)
FE.plot(vh)
```

Compare at a point:

```
x0 = np.array([0.5, 0.3])
true_val = v(x0)
int_val = vh(x0)
print(f"True: {true_val:.6f}, Interpolant: {int_val:.6f}")
```

Because the mesh is fine, the difference is small. If we use a coarser mesh (e.g., step size 0.5), the error becomes large. The interpolant approximates the true function well only when the mesh resolves the variations of v.

4.7.2 Solution to Exercise 2

For an annulus domain (disk with a hole):

```
domain = FE.Annulus(inner_radius=0.3, outer_radius=1.0, center=(0,0))
mesh = FE.mesh_domain(domain, step_size=0.2)
Vh_cg1 = FE.finite_element_space(mesh, continuous_galerkin=1)
```

Plot the mesh and overlay the degrees of freedom (red dots):

```
FE.plot(mesh)
dofs = FE.dofs(Vh_cg1).T   # transpose to shape (2, Nh)
plt.scatter(dofs[0], dofs[1], color='red', s=10)
plt.show()
```

For CG1, DoFs are exactly the vertices.

Now for quadratic elements (CG2):

```
Vh_cg2 = FE.finite_element_space(mesh, continuous_galerkin=2)
FE.plot(mesh)
dofs2 = FE.dofs(Vh_cg2).T
plt.scatter(dofs2[0], dofs2[1], color='red', s=10)
plt.show()
```

Now we see additional DoFs: each edge contributes a midpoint because a quadratic polynomial on a triangle requires three nodes per edge to be uniquely determined. Consequently $\dim V_h^{\mathrm{CG2}} > \dim V_h^{\mathrm{CG1}}$. The extra DoFs allow the function to be quadratic inside each element while remaining continuous across edges.

Why three points per edge? On a single edge, a linear function is determined by two values (at the endpoints). A quadratic (parabolic) function requires three values: the two endpoints and the midpoint. Therefore, the finite element space of degree 2 includes degrees of freedom at vertices and at edge midpoints.

4.8 Next steps

With these tools we are ready to solve PDEs. The unknown solution will be represented as a vector (the degrees of freedom); after solving we will convert it to a finite element function for plotting and analysis.

4.9 Continuous Galerkin of Degree 0 (Piecewise Constant)

Before moving to PDEs, we examine the finite element space of degree 0:

- Functions are piecewise constant on each triangle.
- One degree of freedom (DoF) per triangle (the constant value on that element).
- Useful to represent coefficients that are piecewise constant, e.g., material conductivity.

This space will be used in the PDE example.

4.10 Stationary Heat Equation with Variable Conductivity

We consider a two-dimensional domain Ω (a plate) composed of two materials with different thermal conductivities. The conductivity $\sigma(x)$ is a function of space, piecewise constant:

- $\sigma = 0.1$ inside a certain region,
- $\sigma = 10$ outside.

We solve the stationary heat equation (a Poisson-type problem with variable coefficients). The strong form is:

$$\begin{cases} -\nabla \cdot (\sigma \nabla u) = f & \text{in } \Omega, \\ u = g & \text{on } \partial\Omega, \end{cases}$$

where f is a source term, g is the Dirichlet boundary condition, and u is the unknown temperature.

4.10.1 Weak Formulation

Derivation steps:

1. Multiply the PDE by a test function v (which will vanish on the boundary) and integrate over Ω:

$$\int_\Omega (-\nabla \cdot (\sigma \nabla u))\, v \, dx = \int_\Omega f v \, dx.$$

2. Integrate by parts (Green's formula):

$$\int_\Omega \sigma \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} \sigma \frac{\partial u}{\partial n} v \, ds = \int_\Omega f v \, dx.$$

3. Since we require $v = 0$ on $\partial\Omega$, the boundary term vanishes. The weak problem becomes:

$$\text{Find } u \in H^1(\Omega) \text{ with } u = g \text{ on } \partial\Omega \text{ such that } \int_\Omega \sigma \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx \quad \forall v \in V_{\text{test}},$$

where $V_{\text{test}} = \{v \in H^1(\Omega) : v = 0 \text{ on } \partial\Omega\}$.

Here $H^1(\Omega)$ denotes the Sobolev space of functions that are square-integrable together with their first derivatives.

4.10.2 Galerkin Projection

We replace the infinite-dimensional space V_{test} by a finite-dimensional subspace $V_h \subset H^1(\Omega)$ constructed via finite elements. Also, we seek $u_h \in V_h$ that satisfies the same weak equation for all test functions $v_h \in V_h$ (with the boundary condition $u_h = g_h$, where g_h is an approximation of g on the boundary).

The Galerkin problem:

$$\text{Find } u_h \in V_h \text{ such that } \int_\Omega \sigma \nabla u_h \cdot \nabla v_h \, dx = \int_\Omega f v_h \, dx \quad \forall v_h \in V_h.$$

4.10.3 Discretization into a Linear System

Every function $u_h \in V_h$ corresponds to a vector $\mathbf{u} \in \mathbb{R}^{N_h}$ (its values at the degrees of freedom). Similarly, any test function v_h corresponds to a vector $\mathbf{v}$. Using the linearity of the integrals, we obtain:

- The bilinear form $a(u_h, v_h) = \int_\Omega \sigma \nabla u_h \cdot \nabla v_h \, dx$ becomes a matrix-vector product $\mathbf{v}^T \mathbf{A} \mathbf{u}$.
- The linear form $L(v_h) = \int_\Omega f v_h \, dx$ becomes $\mathbf{v}^T \mathbf{F}$.

Thus the weak equation $\forall v_h$ is equivalent to the linear system:

$$\mathbf{A}\mathbf{u} = \mathbf{F},$$

where $\mathbf{A}$ is the stiffness matrix and $\mathbf{F}$ is the load vector. The Dirichlet boundary conditions are then imposed by modifying the system (eliminating rows/columns corresponding to boundary DoFs).

4.11 Implementation in Python (using Phoenix/DRROMS)

The following code snippets illustrate the assembly of the weak form.

4.11.1 Defining the bilinear form $a(u, v)$

```
# sigma_h is a piecewise constant function (DG0 space)
a = sigma_h * inner(grad(u), grad(v)) * dx
```

Here:

- `inner(grad(u), grad(v))` computes the scalar product $\nabla u \cdot \nabla v$.
- `dx` indicates integration over the domain.
- The expression `a` is symbolic; when assembled, it produces the matrix $\mathbf{A}$.

4.11.2 Defining the linear form $L(v)$

```
# f is a given source term
L = f * v * dx
```

This assembles the vector $\mathbf{F}$.

4.11.3 Handling Dirichlet boundary conditions

The condition $u = g$ on $\partial\Omega$ is applied after assembling $\mathbf{A}$ and $\mathbf{F}$ (typically by modifying the system). In the library, this is done with a few lines that identify boundary DoFs and set the corresponding equations.

4.12 Summary of the Workflow

1. Create the mesh and finite element spaces V_h (for the solution) and $DG0$ (for piecewise constant coefficients).
2. Define the coefficient σ_h as a function in the $DG0$ space.
3. Define the symbolic bilinear form $a(u, v) = \int \sigma_h \nabla u \cdot \nabla v \, dx$.
4. Define the linear form $L(v) = \int f v \, dx$.
5. Assemble the matrix $\mathbf{A}$ and vector $\mathbf{F}$.
6. Apply Dirichlet boundary conditions $u = g$ on $\partial\Omega$.
7. Solve the linear system $\mathbf{Au} = \mathbf{F}$.
8. Convert the solution vector $\mathbf{u}$ back to a finite element function u_h and plot.

4.13 Remarks on the Test Space and Boundary Conditions

In the derivation we required $v = 0$ on the boundary. This is automatically enforced in the finite element assembly if we only consider test functions that vanish on the Dirichlet boundary. The library manages this by restricting the space of test functions accordingly.
Important: The weak formulation with $v = 0$ on the boundary eliminates the boundary term from integration by parts, leading to a symmetric positive definite system (for diffusion problems). Non-homogeneous Dirichlet conditions are imposed after assembly by lifting the boundary values.

4.14 Boundary Conditions: Dirichlet and Neumann

The boundary $\partial\Omega$ is split into two parts:

- Γ_D (Dirichlet boundary): left edge, where $u = 0$.
- Γ_N (Neumann boundary): remaining edges, where a no-flux condition applies: $\sigma \frac{\partial u}{\partial n} = 0$.

The source term is $f \equiv 1$ (uniform heating). We interpolate this constant function into the finite element space V_h:

```
fh = FE.interpolate(lambda x: 1.0, Vh)
```

4.15 Assembly of the Linear System

The weak formulation yields a bilinear form $a(u, v) = \int_\Omega \sigma \nabla u \cdot \nabla v \, dx$ and a linear form $L(v) = \int_\Omega f v \, dx$. After Galerkin projection, we assemble the matrix $\mathbf{A}$ and vector $\mathbf{F}$:

```
A = FE.assemble(a, Vh)    # stiffness matrix
F = FE.assemble(L, Vh)    # load vector
```

For a problem with $N_h \approx 10\,000$ degrees of freedom, $\mathbf{A}$ is a $10\,000 \times 10\,000$ sparse matrix, and $\mathbf{F}$ is a vector of length 10 000.

4.16 Applying Dirichlet Boundary Conditions

We must enforce $u = 0$ on the left edge (Γ_D). First, define a boolean function that identifies points on the left boundary:

```
def left_edge(x):
    return x[0] < 1e-10   # first coordinate is near zero
```

Then, create a Dirichlet boundary condition object and apply it to the assembled system:

```
bc = FE.DirichletBC(Vh, 0.0, left_edge)
A, F = FE.apply_boundary_conditions(A, F, bc)
```

4.17 Solving the Linear System

We solve $\mathbf{Au} = \mathbf{F}$ using a sparse solver (e.g., from SciPy):

```
from scipy.sparse.linalg import spsolve
u_vec = spsolve(A, F)   # solution vector
```

The solution u_h is then obtained as a finite element function:

```
uh = FE.function(u_vec, Vh)
```

4.18 Verification of the Solution

Plot the solution and check the boundary conditions:

```
FE.plot(uh)
```

Expected observations:

- On the left edge (Γ_D), the solution is zero (cold boundary).

- On the remaining edges (Γ_N), the no-flux condition implies $\partial u/\partial n = 0$. This is visible as vertical color contours near the top, bottom, and right boundaries (no change in the normal direction).
- Inside the domain, a region of low conductivity ($\sigma = 0.1$) causes heat to accumulate, forming a local hot spot. The left cold boundary is "screened" by this insulating region.

4.19 Incorporating Neumann Conditions (General Case)

Recall the integration by parts:

$$\int_\Omega \sigma \nabla u \cdot \nabla v \, dx - \int_{\partial\Omega} \sigma \frac{\partial u}{\partial n} v \, ds = \int_\Omega f v \, dx.$$

Split the boundary integral into Dirichlet and Neumann parts:

$$\int_{\partial\Omega} = \int_{\Gamma_D} + \int_{\Gamma_N}.$$

- On Γ_D, we set $v = 0$ (test functions vanish), so that integral disappears.
- On Γ_N, we prescribe $\sigma \frac{\partial u}{\partial n} = \phi$ (given). For the no-flux case, $\phi = 0$.

Thus the boundary term becomes $-\int_{\Gamma_N} \phi v \, ds$, which we move to the right-hand side:

$$\int_\Omega \sigma \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx - \int_{\Gamma_N} \phi v \, ds.$$

In the code, if $\phi \neq 0$, we add a boundary integral term to the linear form:

```
L = f * v * dx + phi * v * ds(domain, subdomain="Neumann_boundary")
```

where `ds` denotes integration over the boundary. The detailed implementation for marking Γ_N is given in the exercise hints.

4.20 Exercises

1. Modify the code to solve a pure diffusion problem with different source terms f.
2. Implement a non-zero Neumann condition (e.g., $\phi = 1$ on the right edge) and observe the effect.
3. Change the conductivity σ to a different piecewise constant distribution.
4. **Exercise 5 (preparatory for next lab):** Set up a parametrized problem (e.g., varying conductivity or source) as an introduction to reduced basis methods.

4.21 Next Steps

The next session will focus on parametrized PDEs and the reduced basis method. The assembly and boundary condition handling presented here form the foundation for those topics.

Chapter 5

Proper Orthogonal Decomposition

5.1 From the full order model to the reduced basis method

We begin by recalling the weak formulation of a boundary value problem. We seek a solution in a suitable function space such that

$$a(u, v; \mu) = f(v; \mu) \qquad \forall v \in V,$$

where parameters appear in both the bilinear form $a(\cdot, \cdot; \mu)$ and the linear form $f(\cdot; \mu)$. This is an infinite-dimensional problem, which we refer to as the *original problem*.

To obtain a computable approximation, we introduce a finite-dimensional subspace $V_h \subset V$ (e.g., the space of piecewise linear finite elements over a triangulation of the domain). The *full order model* (or high-fidelity approximation) reads: find $u_h \in V_h$ such that

$$a(u_h, v_h; \mu) = f(v_h; \mu) \qquad \forall v_h \in V_h.$$

In practice, this leads to a linear system of size $N_h = \dim(V_h)$:

$$A_h(\mu)\mathbf{u}_h(\mu) = \mathbf{f}_h(\mu).$$

The dimension N_h can be very large, as it is directly related to the number of degrees of freedom in the mesh.

5.2 The reduced order model

The goal of the reduced basis method is to replace the large space V_h by a much smaller space V_n of dimension $n \ll N_h$. Using a Galerkin projection, we look for an approximation $u_n \in V_n$ such that

$$a(u_n, v_n; \mu) = f(v_n; \mu) \qquad \forall v_n \in V_n.$$

If we choose a basis $\{\zeta_1, \ldots, \zeta_n\}$ for V_n, then we can write

$$u_n(x; \mu) = \sum_{j=1}^{n} \mathbf{u}_n(\mu)_j \, \zeta_j(x),$$

where $\mathbf{u}_n(\mu) \in \mathbb{R}^n$ is the vector of reduced coefficients. Substituting this expansion and testing with ζ_i yields the reduced linear system

$$A_n(\mu)\, \mathbf{u}_n(\mu) = \mathbf{f}_n(\mu),$$

with

$$(A_n(\mu))_{ij} = a(\zeta_j, \zeta_i; \mu), \qquad (\mathbf{f}_n(\mu))_i = f(\zeta_i; \mu).$$

The size of this system is n, which is much smaller than N_h. The reduced basis space V_n is global: its basis functions are defined over the entire domain and are obtained by post-processing a set of *snapshots*. Snapshots are solutions of the full order model computed for selected parameter values.

5.3 Global basis and parameter dependence

It is important to note that the basis $\{\zeta_i\}$ is the same for all parameter values. Only the coefficients $\mathbf{u}_n(\mu)$ depend on μ. This is analogous to the finite element method, where the basis functions are fixed and the nodal values vary with the parameters. The key difference is that the reduced basis is specifically adapted to the problem at hand, and its construction relies on the snapshot set. The question of how to build these basis functions from the snapshots is the central topic of this chapter.

5.4 The algebraic connection between reduced and full order models

Let us now formalize the relationship between the reduced basis matrices and the finite element matrices. This is what we have just written at the blackboard.

We insert the expansion of u_n into the weak form and test against ζ_i. Exploiting the linearity of $a(\cdot,\cdot;\mu)$, we obtain

$$\sum_{j=1}^{n} a(\zeta_j, \zeta_i; \mu)\, \mathbf{u}_n(\mu)_j = f(\zeta_i; \mu), \qquad i = 1, \dots, n.$$

This is exactly the linear system $A_n(\mu)\mathbf{u}_n(\mu) = \mathbf{f}_n(\mu)$, where $(A_n(\mu))_{ij} = a(\zeta_j, \zeta_i; \mu)$ and $(\mathbf{f}_n(\mu))_i = f(\zeta_i; \mu)$.

Remark 5.1 (Affine parameter dependence)**.** *If the bilinear form depends affinely on the parameters, i.e.,*

$$a(w, v; \mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu)\, a^q(w, v),$$

and similarly for the linear form, then the reduced matrices can be assembled once and for all in the offline stage:

$$A_n(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu)\, A_n^q, \qquad A_n^q = \left(a^q(\zeta_j, \zeta_i)\right)_{i,j=1}^{n}.$$

This avoids rebuilding the system for each new μ.

5.4.1 The transformation matrix V

Since each reduced basis function ζ_m (for $m = 1, \dots, n$) belongs to the finite element space V_h, it can be expressed as a linear combination of the finite element basis functions $\{\phi_1, \dots, \phi_{N_h}\}$:

$$\zeta_m(x) = \sum_{k=1}^{N_h} V_{k,m}\, \phi_k(x).$$

The coefficients $V_{k,m}$ form the entries of a matrix $V \in \mathbb{R}^{N_h \times n}$. This matrix is often called the *transformation matrix* or the *reduced basis coefficient matrix*. Column m of V contains the finite element coordinates of the reduced basis function ζ_m.

Using this representation, we can relate the reduced matrices to the finite element ones. For any two reduced basis functions ζ_i, ζ_j, we have

$$a(\zeta_j, \zeta_i; \mu) = a\left(\sum_{k=1}^{N_h} V_{k,j}\phi_k,\ \sum_{\ell=1}^{N_h} V_{\ell,i}\phi_\ell; \mu\right) = \sum_{k=1}^{N_h}\sum_{\ell=1}^{N_h} V_{k,j}\, a(\phi_k, \phi_\ell; \mu)\, V_{\ell,i}.$$

In matrix form, this reads

$$A_n(\mu) = V^{\mathsf{T}} A_h(\mu) V,$$

where $(A_h(\mu))_{k\ell} = a(\phi_\ell, \phi_k; \mu)$ is the standard finite element stiffness matrix. Similarly, the right-hand side becomes

$$\mathbf{f}_n(\mu) = V^{\mathsf{T}} \mathbf{f}_h(\mu).$$

Thus, once the transformation matrix V is known, assembling the reduced system is simply a matter of matrix multiplications involving the (large) finite element matrices and the (tall, skinny) matrix V. This is exactly the algebraic viewpoint we introduced in the very first lectures: the reduced basis solution is obtained by projecting the full order system onto the column space of V.

5.5 The missing piece: constructing V

The matrix V is the key ingredient. Its columns are the coefficients of the reduced basis functions expressed in the finite element basis. But how do we obtain these basis functions? The answer lies in the *Proper Orthogonal Decomposition (POD)*.

We assume that we can sample the parameter space and compute the corresponding *snapshots*, i.e., solutions of the full order model for selected parameter values:

$$\mathbf{u}_h(\mu_1),\ \mathbf{u}_h(\mu_2),\ \ldots,\ \mathbf{u}_h(\mu_M) \in \mathbb{R}^{N_h}.$$

These snapshots are the only data we need. They are expensive to compute, but this cost is paid only once, in the so-called *offline stage*.

Question 5.1. *1. What is the relationship between the snapshots and the reduced basis functions?*

2. How many snapshots do we need to compute? (We want to minimize this number while retaining accuracy.)

3. How do we actually construct the reduced basis space from the snapshots?

The tool that answers all these questions is the *Proper Orthogonal Decomposition*. POD is the Swiss army knife of reduced order modeling: it provides a systematic way to extract an optimal low-dimensional basis from a set of snapshots. The optimality is measured in the sense that the POD basis minimizes the average projection error of the snapshots onto the reduced space. This leads to a singular value decomposition (SVD) of the snapshot matrix.

Remark 5.2 (Singular values)**.** *You may have already heard about singular values (or perhaps not yet). In the context of POD, the singular values of the snapshot matrix tell us how much energy is captured by each mode. They guide the choice of the reduced basis size n. This is a fundamental concept that we will now explore in detail.*

In the following, we will start from the reduced basis problem and show how POD constructs the space V_n from the snapshots, and how the affine parameter dependence can be exploited to achieve efficiency. The Swiss knife metaphor is indeed the best part: POD does everything for us.

5.6 The Singular Value Decomposition (SVD)

We now introduce a fundamental tool for constructing optimal reduced bases: the singular value decomposition. The SVD can be seen as a generalization of eigenvalues and eigenvectors to rectangular matrices.

Definition 5.1 (Singular Value Decomposition)**.** *Let $A \in \mathbb{R}^{m\times n}$ be an arbitrary real matrix. There exist orthogonal matrices $U \in \mathbb{R}^{m\times m}$ and $Z \in \mathbb{R}^{n\times n}$ (often denoted V) and a rectangular diagonal matrix $\Sigma \in \mathbb{R}^{m\times n}$ such that*

$$A = U\Sigma Z^{\mathsf{T}}.$$

The diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$ of Σ are called the singular values *of A. The columns of U are the* left singular vectors*, and the columns of Z are the* right singular vectors*.*

Remark 5.3. *If A is square and symmetric, its singular values are the absolute values of its eigenvalues. In general, the SVD exists for any matrix, regardless of shape or rank.*

5.6.1 Relation with eigenvalues

From the SVD we obtain two important relationships:

$$AA^{\mathsf{T}} = U\Sigma\Sigma^{\mathsf{T}}U^{\mathsf{T}}, \qquad A^{\mathsf{T}}A = Z\Sigma^{\mathsf{T}}\Sigma Z^{\mathsf{T}}.$$

Thus:

- The left singular vectors (columns of U) are the eigenvectors of AA^{T}.
- The right singular vectors (columns of Z) are the eigenvectors of $A^{\mathsf{T}}A$.
- The non-zero singular values σ_i satisfy $\sigma_i^2 = \lambda_i(AA^{\mathsf{T}}) = \lambda_i(A^{\mathsf{T}}A)$.

Theorem 5.1 (Frobenius norm and singular values)**.** *The Frobenius norm of a matrix is defined as*

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2}.$$

It satisfies

$$\|A\|_F^2 = \sum_{i=1}^{\min(m,n)} \sigma_i^2.$$

5.6.2 Low-rank approximations

A key property of the SVD is that it provides the best low-rank approximation to a matrix in the Frobenius norm (Eckart–Young theorem).

Theorem 5.2 (Eckart–Young)**.** *Let A have rank r. For any $k < r$, define*

$$A_k = \sum_{i=1}^{k} \sigma_i \, u_i z_i^{\mathsf{T}},$$

where u_i and z_i are the i-th left and right singular vectors. Then A_k minimizes $\|A - B\|_F$ over all matrices B of rank at most k. Moreover,

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2.$$

The proof relies on the fact that the Frobenius norm is unitarily invariant, and the optimal approximation is obtained by truncating the SVD. The error is exactly the sum of squares of the discarded singular values.

Example 5.1 (Image compression)**.** *Consider a grayscale image represented as a matrix $A \in \mathbb{R}^{m\times n}$, where each entry is a pixel intensity (0 to 255). Compute its SVD and keep only the first k singular values and corresponding singular vectors. The approximation A_k uses only $k(m+n)$ numbers instead of mn, achieving compression. For example:*

- *$k = 10$: very blurry but recognizable.*
- *$k = 15, 20$: improved quality.*
- *$k = 400$: virtually indistinguishable from the original.*

This is the principle behind many compression algorithms (e.g., JPEG). The singular values typically decay, so a small k captures most of the energy.

5.6.3 Why low-rank approximation matters

Why is this important? Because if A has rank r, we can write

$$A = \sum_{i=1}^{r} \sigma_i \, u_i z_i^{\mathsf{T}}.$$

If some singular values are very small, we can truncate the sum at $k < r$ and obtain an approximation A_k of rank k. The discarded modes contribute little to the total energy (measured by the Frobenius norm). This is exactly the principle of low-rank approximation: we keep only the most significant modes.

Remark 5.4 (Energy interpretation)**.** *The squared Frobenius norm $\|A\|_F^2 = \sum_i \sigma_i^2$ can be interpreted as the total energy or information content of the matrix. By retaining the first k singular values, we capture the fraction*

$$\frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}$$

of the total energy. If the singular values decay rapidly, a small k suffices.

The Eckart–Young theorem tells us two things:

1. Among all matrices of rank at most k, the SVD truncation A_k is the best approximation to A (in the Frobenius norm). Any other rank-k matrix is worse.

2. The error is exactly the sum of squares of the neglected singular values: $\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$.

Thus, we can decide the reduced rank k based on the decay of the singular values. If the decay is fast, we are happy because we can discard many modes with little loss. If the decay is slow, we need many modes to achieve a good approximation – this depends on the physical problem and the variability of the solution with respect to parameters.

Remark 5.5 (Ordering of singular values)**.** *The singular values are ordered: $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)}$. This ordering is essential because it tells us which modes are most important. The plot of singular values versus index typically shows a decay; we want the decay to be as steep as possible.*

5.7 Proper Orthogonal Decomposition (POD)

The POD is the application of the SVD to a collection of solutions (snapshots) of the full order model. This provides an optimal basis for representing the snapshot set and, by extension, for constructing reduced order models.

5.7.1 The snapshot matrix

We collect N_s snapshots by solving the full order model for selected parameter values $\mu_1, \ldots, \mu_{N_s}$:

$$\mathbf{u}_h(\mu_i) \in \mathbb{R}^{N_h}, \qquad i = 1, \ldots, N_s.$$

These vectors are the finite element coefficients of the high-fidelity solutions. We assemble the *snapshot matrix*

$$S = \begin{bmatrix} \mathbf{u}_h(\mu_1) & \mathbf{u}_h(\mu_2) & \cdots & \mathbf{u}_h(\mu_{N_s}) \end{bmatrix} \in \mathbb{R}^{N_h \times N_s}.$$

Typically, $N_s \ll N_h$ but still potentially large enough to capture the parametric variability.

5.7.2 POD basis from SVD

We compute the SVD of S:

$$S = U\Sigma Z^{\mathsf{T}},$$

where $U \in \mathbb{R}^{N_h \times N_h}$ contains the left singular vectors, $\Sigma \in \mathbb{R}^{N_h \times N_s}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{N_s} \geq 0$, and $Z \in \mathbb{R}^{N_s \times N_s}$ contains the right singular vectors.

The *POD basis* of dimension $n \leq N_s$ is defined as the first n left singular vectors:

$$V = \begin{bmatrix} \mathbf{u}_1 & \mathbf{u}_2 & \cdots & \mathbf{u}_n \end{bmatrix} \in \mathbb{R}^{N_h \times n}.$$

These vectors are orthonormal (by construction) and serve as the global basis functions for the reduced space:

$$V_n = \operatorname{span}\{\mathbf{u}_1, \ldots, \mathbf{u}_n\}.$$

Remark 5.6. *The columns of U are the discrete representation of the reduced basis functions $\zeta_i(x)$ we introduced earlier. Each $\mathbf{u}_i$ corresponds to a function defined over the entire domain, obtained by post-processing the snapshots.*

5.7.3 Optimality of the POD basis

The Eckart–Young theorem translates directly into an optimality property for the POD basis. Consider the projection of each snapshot onto a subspace of dimension n. For any orthonormal basis $W \in \mathbb{R}^{N_h \times n}$ (i.e., $W^{\mathsf{T}}W = I_n$), the projection of a snapshot $\mathbf{u}_h(\mu)$ onto $\operatorname{span}(W)$ is given by $WW^{\mathsf{T}}\mathbf{u}_h(\mu)$. The total projection error over all snapshots is

$$\sum_{i=1}^{N_s} \left\| \mathbf{u}_h(\mu_i) - WW^{\mathsf{T}}\mathbf{u}_h(\mu_i) \right\|_2^2 = \|S - WW^{\mathsf{T}}S\|_F^2.$$

Minimizing this over all orthonormal W is equivalent to finding the best rank-n approximation of S. By the Eckart–Young theorem, the minimizer is given by the first n left singular vectors of S. Hence:

Theorem 5.3 (Optimality of POD)**.** *For any $n \leq N_s$, the POD basis V (first n left singular vectors of the snapshot matrix) minimizes the projection error*

$$\sum_{i=1}^{N_s} \|\mathbf{u}_h(\mu_i) - \Pi_{V_n}\mathbf{u}_h(\mu_i)\|_2^2$$

over all n-dimensional subspaces $V_n \subset \mathbb{R}^{N_h}$. The minimal error is given by

$$E(n) = \sum_{i=n+1}^{N_s} \sigma_i^2.$$

Thus, the POD basis is optimal for representing the collected snapshots. The decay of the singular values tells us how many modes are needed to achieve a desired accuracy in this reconstruction.

5.7.4 From reconstruction to prediction

A crucial question remains: The POD basis is optimal for reconstructing the snapshots we have already computed. However, in reduced order modeling, we are not interested in merely compressing existing data. We want to use the reduced basis to predict solutions for new parameter values μ that were not in the snapshot set. The POD basis provides a space that is rich enough to represent the entire solution manifold (if the snapshots are sufficiently representative). Then, for a new μ, we solve the reduced Galerkin system

$$V^{\mathsf{T}}A_h(\mu)V\,\mathbf{u}_n(\mu) = V^{\mathsf{T}}\mathbf{f}_h(\mu),$$

and obtain the approximation $u_n(x;\mu) = \sum_{j=1}^{n}(\mathbf{u}_n(\mu))_j\,\zeta_j(x)$. The optimality property does not directly guarantee that the error for new parameters is minimal; it only guarantees that the projection of the snapshots is optimal. Nevertheless, if the solution manifold is low-dimensional and the snapshots sample it well, the POD basis yields excellent approximations for new parameters.

5.7.5 Choosing the dimension of the POD basis

The projection error for the snapshots is given by the sum of the squares of the neglected singular values. Therefore, we can select the reduced dimension n by requiring that the relative error is below a tolerance ε_{POD}:

$$\frac{\sum_{i=n+1}^{N_s} \sigma_i^2}{\sum_{i=1}^{N_s} \sigma_i^2} \leq \varepsilon_{\text{POD}}^2.$$

Equivalently, we require that the retained energy fraction

$$\frac{\sum_{i=1}^{n} \sigma_i^2}{\sum_{i=1}^{N_s} \sigma_i^2} \geq 1 - \varepsilon_{\text{POD}}^2.$$

Typical choices for ε_{POD} are 10^{-3} or 10^{-6}, depending on the required accuracy. In practice, we plot the singular values (or the cumulative energy) and choose n where the curve flattens.

Remark 5.7. *The decay of the singular values is problem-dependent. If the parameter-to-solution map exhibits mild variability, the singular values decay rapidly, and a small n suffices (lucky case). If the solution changes dramatically with parameters (e.g., the sunflower image, which is rich in details), the singular values decay slowly, and many modes are needed. In the latter case, the reduction may be less effective, but if the original problem has millions of degrees of freedom, a few hundred modes may still be acceptable.*

5.8 Practical considerations: Sampling and redundancy

All the machinery depends on the collected snapshots. Two important questions arise:

1. How many snapshots N_s should we compute? (Given a computational budget.)
2. How should we choose the parameter values μ_i to sample?

These questions are related to the rank of the snapshot matrix. Ideally, we want to avoid redundant information (e.g., duplicate or nearly duplicate simulations). If the parameterization is clever and the sampling strategy avoids repetition, the number of non-zero singular values (the rank of S) will reflect the intrinsic dimensionality of the solution manifold. However, if we sample poorly (e.g., many points close together in parameter space), the snapshot matrix may be rank-deficient or have very small singular values that can be truncated without loss.

Remark 5.8. *If the singular values are zero (or numerically zero), those modes can be neglected naturally. The practical criterion based on energy retention automatically discards unimportant directions. The challenge is to sample the parameter space sufficiently well to capture all relevant behaviors without exceeding the computational budget. We will discuss sampling strategies (e.g., greedy algorithms, random sampling, Latin hypercube) in the next lecture.*

5.9 Summary and outlook

The Proper Orthogonal Decomposition provides:

- An optimal low-dimensional basis (in the least-squares sense) for a given set of snapshots.
- A rigorous error bound in terms of the neglected singular values.
- A practical tool for constructing reduced order models: the POD basis is used in a Galerkin projection to obtain a small linear system.

The SVD is the Swiss army knife that makes all this possible. Understanding its properties is essential for successful implementation of reduced basis methods.

Remark 5.9 (Limitations and future directions)**.** *So far we have assumed that a global, linear combination of basis functions suffices. For problems with very high parametric variability or non-linear dependencies, a single global basis may not be efficient. In the second half of the course, we will explore methods that remove the assumptions of globality and linear superposition (e.g., local reduced bases, dictionary methods, and non-linear model reduction). Nevertheless, POD remains a cornerstone technique that works remarkably well in many applications.*

5.10 What you need for the Friday exercise

For the practical session, you will need to:

1. Run your finite element code multiple times (for different parameter values) to generate snapshots.
2. Assemble the snapshot matrix S (each column is a solution vector).
3. Compute the SVD of S (using a library function).
4. Extract the first n left singular vectors as the POD basis V.
5. Use V to project the full order system and obtain the reduced matrices $A_n(\mu) = V^\mathsf{T} A_h(\mu) V$ and $\mathbf{f}_n(\mu) = V^\mathsf{T} \mathbf{f}_h(\mu)$.
6. Solve the small reduced system for new parameters and compare with full order solutions.

The singular values will tell you how many modes are needed. If the decay is fast, you are happy; if slow, you may need many snapshots or a different sampling strategy.

We will continue the discussion on sampling strategies and diagnostics next week. For now, you have all the mathematical and computational tools to implement the first part of the reduced basis approximation using POD.

5.11 Motivation and Recap

In the previous lecture and the accompanying laboratory session, you have already gained practical experience with the Proper Orthogonal Decomposition (POD). We will not repeat the entire theoretical derivation, but rather recall the key ideas and then elaborate on aspects that are essential for building reduced-order models.

The central idea is to construct a low-dimensional space for our reduced basis problem using the singular value decomposition (SVD) of a matrix. During the lab, you built a *snapshot matrix* whose columns are individual solutions of your problem. By performing the SVD of this matrix and extracting the first few left singular vectors, you obtained a set of basis vectors that define the reduced basis space.

5.12 The Snapshot Matrix and the SVD

Let $\mathbf{S} \in \mathbb{R}^{N \times N_s}$ be the snapshot matrix, where N is the number of degrees of freedom (size of the full-order model) and N_s is the number of collected solutions. Each column $\mathbf{s}_i$ corresponds to a solution computed for a particular parameter value or time instant.

The singular value decomposition of $\mathbf{S}$ is

$$\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where:

- $\mathbf{U} \in \mathbb{R}^{N \times N}$ contains the left singular vectors,
- $\mathbf{\Sigma} \in \mathbb{R}^{N \times N_s}$ is a diagonal matrix with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ ($r = \text{rank}(\mathbf{S})$),
- $\mathbf{V} \in \mathbb{R}^{N_s \times N_s}$ contains the right singular vectors.

5.13 The POD Basis

For any given $n \leq r$, the POD basis of dimension n is defined as the set of the first n left singular vectors:

$$\mathbf{W}_n = [\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_n] \in \mathbb{R}^{N \times n},$$

where $\mathbf{w}_i$ are the first n columns of $\mathbf{U}$. By construction, the POD basis is orthonormal: $\mathbf{W}_n^T \mathbf{W}_n = \mathbf{I}_n$.

5.13.1 Optimality of the POD Basis (Eckart–Young–Mirsky Theorem)

The POD basis is optimal in the sense that it provides the best low-rank approximation to the snapshot matrix. For any orthonormal matrix $\mathbf{Q} \in \mathbb{R}^{N \times n}$, the approximation error measured in the Frobenius norm satisfies

$$\|\mathbf{S} - \mathbf{W}_n \mathbf{W}_n^T \mathbf{S}\|_F \leq \|\mathbf{S} - \mathbf{Q}\mathbf{Q}^T \mathbf{S}\|_F.$$

Moreover, the error is directly linked to the neglected singular values:

$$\|\mathbf{S} - \mathbf{W}_n \mathbf{W}_n^T \mathbf{S}\|_F^2 = \sum_{i=n+1}^{r} \sigma_i^2.$$

Thus, if the singular values decay rapidly, a small n suffices to represent the data accurately. If the decay is slow (e.g., the "sunflower" example), many modes are needed.

Remark 5.10. *A quick visual check of the singular value decay (a plot of σ_i vs. i) gives immediate insight into the reducibility of the problem. Fast decay indicates that a low-dimensional POD basis can capture most of the solution variability.*

5.14 Projection onto the POD Space

Given any vector $\mathbf{x} \in \mathbb{R}^N$ (e.g., a new solution), its orthogonal projection onto the subspace spanned by $\mathbf{W}_n$ is

$$\mathbf{x}_{\text{proj}} = \mathbf{W}_n \mathbf{W}_n^T \mathbf{x}.$$

The coefficients (or modal amplitudes) are given by $\mathbf{c} = \mathbf{W}_n^T \mathbf{x} \in \mathbb{R}^n$, so that

$$\mathbf{x}_{\text{proj}} = \sum_{j=1}^{n} c_j \mathbf{w}_j.$$

Because the basis is orthonormal, each coefficient $c_j = \mathbf{w}_j^T \mathbf{x}$ measures the contribution of the j-th POD mode.

5.15 Reduced-Order Models (ROMs)

A reduced-order model consists of two independent ingredients:

1. A reduced basis (e.g., obtained via POD) that spans a low-dimensional subspace where the solution is approximated.
2. A reduced problem (e.g., Galerkin projection of the full-order equations onto the reduced subspace).

POD is a versatile tool because it does not assume any specific structure of the underlying problem. The only requirement is a solver capable of producing snapshots (solutions) for different parameters or time instants. These snapshots can be collected from:

- Parameter variations (e.g., different material properties, boundary conditions),
- Time evolution (each time step yields a column),
- A combination of both (then the snapshot matrix contains solutions at different parameter–time pairs).

Hence, POD is often referred to as the "Swiss army knife" of model reduction.

5.16 Error Quantification

The optimality property also provides an *a posteriori* error estimate for the representation of the snapshots:

$$\frac{\sum_{i=1}^{N_s} \|\mathbf{s}_i - \mathbf{W}_n \mathbf{W}_n^T \mathbf{s}_i\|_2^2}{\sum_{i=1}^{N_s} \|\mathbf{s}_i\|_2^2} = \frac{\sum_{i=n+1}^{r} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

Thus, the fraction of "energy" lost by projecting onto the n-dimensional POD space equals the ratio of the sum of squares of the omitted singular values to the total sum of squares. If this quantity is small, the reduced space provides an excellent representation of the original data.

5.17 Summary of the POD Procedure

1. Collect snapshots (solutions) into a matrix $\mathbf{S}$.
2. Compute the SVD: $\mathbf{S} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$.
3. Choose a dimension n based on the decay of the singular values.
4. Set the POD basis $\mathbf{W}_n$ as the first n columns of $\mathbf{U}$.
5. Use $\mathbf{W}_n$ to build a reduced-order model (e.g., via Galerkin projection).

POD is simple to implement, optimal for representing given data, and widely applicable. The next sections will illustrate its use in parameter-dependent and time-dependent problems.

5.18 Building the Reduced Problem: Galerkin Projection

Once a reduced basis $\mathbf{W}_n \in \mathbb{R}^{N\times n}$ (e.g., obtained via POD) is available, we need to construct a reduced-order model (ROM) for the parametric or time-dependent problem. The standard approach is *Galerkin projection*, which replaces the high-fidelity finite element space with the reduced subspace spanned by the columns of $\mathbf{W}_n$.

Consider the high-fidelity (full-order) linear system arising from, say, a finite element discretization of a stationary linear PDE:

$$\mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h(\boldsymbol{\mu}) = \mathbf{f}_h(\boldsymbol{\mu}),$$

where $\mathbf{A}_h \in \mathbb{R}^{N\times N}$ is the stiffness matrix, $\mathbf{f}_h \in \mathbb{R}^N$ is the load vector, and $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p$ denotes parameters. The high-fidelity solution $\mathbf{u}_h(\boldsymbol{\mu}) \in \mathbb{R}^N$ is expensive to compute for many parameter queries.

In the reduced basis method, we seek an approximation $\mathbf{u}_n(\boldsymbol{\mu}) \in \mathbb{R}^N$ of the form

$$\mathbf{u}_n(\boldsymbol{\mu}) = \mathbf{W}_n\, \mathbf{u}_n(\boldsymbol{\mu}),$$

where $\mathbf{u}_n(\boldsymbol{\mu}) \in \mathbb{R}^n$ is the vector of coefficients (generalized coordinates). Galerkin projection imposes that the residual be orthogonal to the reduced subspace:

$$\mathbf{W}_n^T\big(\mathbf{A}_h(\boldsymbol{\mu})\mathbf{W}_n\mathbf{u}_n(\boldsymbol{\mu}) - \mathbf{f}_h(\boldsymbol{\mu})\big) = \mathbf{0}.$$

This yields the reduced linear system:

$$\mathbf{A}_n(\boldsymbol{\mu})\, \mathbf{u}_n(\boldsymbol{\mu}) = \mathbf{f}_n(\boldsymbol{\mu}),$$

with

$$\mathbf{A}_n(\boldsymbol{\mu}) = \mathbf{W}_n^T\mathbf{A}_h(\boldsymbol{\mu})\mathbf{W}_n \in \mathbb{R}^{n\times n}, \qquad \mathbf{f}_n(\boldsymbol{\mu}) = \mathbf{W}_n^T\mathbf{f}_h(\boldsymbol{\mu}) \in \mathbb{R}^n.$$

Remark 5.11 (Galerkin optimality)**.** *The Galerkin projection satisfies the* Galerkin orthogonality *property: the error* $\mathbf{e} = \mathbf{u}_h - \mathbf{u}_n$ *is orthogonal to the reduced subspace. For coercive problems, this also guarantees that the reduced solution is the best approximation in the energy norm (Céa's lemma). Thus, Galerkin projection is both physics-informed (it uses the original equations) and optimal in a certain sense.*

5.18.1 Combining POD with Galerkin Projection

The POD basis is optimal for representing a given set of snapshots, while Galerkin projection is optimal for solving the reduced system (given a fixed subspace). When we use the POD basis as $\mathbf{W}_n$ in the Galerkin procedure, we obtain a ROM that:

- Leverages the best possible low-dimensional subspace (from the data),
- Respects the physics via projection of the full-order operators.

The final approximation is

$$\mathbf{u}_{\text{rom}}(\boldsymbol{\mu}) = \mathbf{W}_n\, \mathbf{u}_n(\boldsymbol{\mu}),$$

where $\mathbf{u}_n(\boldsymbol{\mu})$ solves the small $n \times n$ linear system.

Remark 5.12 (Non-intrusive vs. intrusive)**.** *Building the POD basis is* non-intrusive*: it only requires snapshots (solutions) and no access to the internal structure of the solver. However, constructing the reduced matrices $\mathbf{A}_n$ and $\mathbf{f}_n$ is* intrusive*, because it requires the full-order matrices $\mathbf{A}_h$ and $\mathbf{f}_h$. For nonlinear problems, the projection becomes even more cumbersome. In later chapters, we will explore neural network based surrogates that bypass the explicit projection step while retaining the optimal POD basis.*

5.19 Choosing the Dimension of the POD Basis

Given the singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ of the snapshot matrix $\mathbf{S}$, the error incurred by projecting the snapshots onto the n-dimensional POD subspace is

$$\frac{\|\mathbf{S} - \mathbf{W}_n \mathbf{W}_n^T \mathbf{S}\|_F^2}{\|\mathbf{S}\|_F^2} = \frac{\sum_{i=n+1}^{r} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

Thus, the fraction of "information" (or energy) retained by the first n POD modes is

$$\mathcal{E}(n) = \frac{\sum_{i=1}^{n} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

A common criterion for selecting n is to set a tolerance $\varepsilon_{\text{POD}} \in (0, 1)$ (e.g., 0.99 or 0.999) and choose the smallest n such that

$$\mathcal{E}(n) \geq 1 - \varepsilon_{\text{POD}}^2,$$

or equivalently,

$$\sqrt{\frac{\sum_{i=n+1}^{r} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}} \leq \varepsilon_{\text{POD}}.$$

Remark 5.13. *In practice, one plots the singular values σ_i versus i (on a semilog scale). A rapid decay indicates that a small n suffices; a slow decay suggests the problem is not easily reducible. This empirical check is routinely performed before building a ROM.*

5.20 Example: Advection–Diffusion Problem with Multiple Parameters

We revisit a parameterized advection–diffusion problem introduced earlier:

$$-\Delta u + \mathbf{b} \cdot \nabla u = 0 \quad \text{in } \Omega,$$

with Dirichlet boundary conditions. The Peclet number Pe (ratio of advection to diffusion) and three boundary temperatures T_1, T_2, T_3 are parameters:

$$\boldsymbol{\mu} = (\text{Pe}, T_1, T_2, T_3) \in \mathcal{P} \subset \mathbb{R}^4.$$

Here $\text{Pe} \in [1, 600]$ and each $T_i \in [0, 12]$. The goal is to build a ROM that approximates the temperature field for any $\boldsymbol{\mu}$ in this four-dimensional domain.

5.20.1 Sampling the Parameter Space

To construct the snapshot matrix, we must choose a set of parameter points $\{\boldsymbol{\mu}^{(k)}\}_{k=1}^{N_s}$ and compute the corresponding high-fidelity solutions. Several sampling strategies exist; the choice strongly affects the quality of the POD basis. Common approaches include:

- **Uniform sampling**: parameter ranges are scaled to $[-1, 1]^p$ and a Cartesian grid (possibly with different numbers of points per dimension) is used.
- **Random sampling**: points are drawn independently from a uniform distribution over $[-1, 1]^p$.
- **Latin Hypercube Sampling (LHS)**: a stratified sampling technique that ensures a more even coverage of the parameter space than pure random sampling. For a given number of points, LHS often yields better space-filling properties.
- **Vertex/center sampling**: points are placed at vertices, center, and along diagonals of the parameter hypercube.

Remark 5.14. *The theory of experimental design and computational statistics provides many advanced sampling methods. For reduced basis methods, greedy algorithms (not discussed here) adaptively select snapshots to minimize the ROM error. The key message is that the sampling strategy is a critical design choice; random sampling is rarely optimal.*

The following figure (conceptual) illustrates different two-parameter sampling patterns:

5.21 Summary of the ROM Workflow

Combining all pieces, the complete reduced-order modeling procedure consists of:

1. Choose a sampling strategy and generate N_s high-fidelity snapshots.
2. Build the snapshot matrix $\mathbf{S}$ and compute its SVD.
3. Select n using the energy criterion (e.g., retain 99.9% of the snapshot energy).
4. Set the POD basis $\mathbf{W}_n$ as the first n left singular vectors.
5. For any new parameter $\boldsymbol{\mu}$, assemble the reduced matrices $\mathbf{A}_n(\boldsymbol{\mu}) = \mathbf{W}_n^T \mathbf{A}_h(\boldsymbol{\mu}) \mathbf{W}_n$ and $\mathbf{f}_n(\boldsymbol{\mu}) = \mathbf{W}_n^T \mathbf{f}_h(\boldsymbol{\mu})$.
6. Solve the small linear system $\mathbf{A}_n(\boldsymbol{\mu}) \mathbf{u}_n(\boldsymbol{\mu}) = \mathbf{f}_n(\boldsymbol{\mu})$.
7. Reconstruct the approximate solution $\mathbf{u}_{\text{rom}}(\boldsymbol{\mu}) = \mathbf{W}_n \mathbf{u}_n(\boldsymbol{\mu})$.

This framework is modular: the basis can be built non-intrusively from any data, and the projection step (if feasible) yields a fast, physics-based surrogate. Future chapters will address the case where the projection step is too expensive (e.g., nonlinear or non-affine parameter dependence) and will introduce neural network surrogates that learn the map $\boldsymbol{\mu} \mapsto \mathbf{u}_n(\boldsymbol{\mu})$ directly.

5.22 Sampling the Parameter Space: Strategies and Convergence

5.22.1 The Curse of Dimensionality

In many applications, the parameter vector $\boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p$ can have dimension $p = 8, 10$ or even higher. When we sample such a high-dimensional hypercube with a limited budget of, say, $N_s = 100$ simulations, the points are extremely sparse. In other words, the hypercube becomes essentially empty, and we cannot hope to explore all relevant scenarios. Therefore, a good sampling strategy is not a luxury but a necessity.

5.22.2 Comparison of Sampling Strategies

We illustrate the impact of the sampling strategy on the POD basis using the four-parameter advection–diffusion problem introduced earlier. We fix the total number of snapshots and compare different ways of choosing the parameter points:

- **Uniform grid** (Cartesian product of one-dimensional grids),
- **Random sampling** (independent uniform draws),
- **Latin Hypercube Sampling (LHS)** (stratified sampling ensuring each dimension is evenly covered),
- Other space-filling designs (e.g., Sobol sequences).

For each sampling method, we build the snapshot matrix, compute its SVD, and plot the normalized singular values σ_i/σ_1. Figure 5.1 shows typical results.

Figure 5.1: Decay of normalized singular values for different sampling strategies (same number of snapshots). Faster decay might misleadingly suggest a low-dimensional solution manifold if the sampling is poor.

At first glance, the uniform grid may produce a very rapid decay of the singular values, tempting one to believe that only a few POD modes are needed. However, this is an artifact: the uniform grid fails to capture the full variability of the solution because it samples the parameter space in a rigid, low-dimensional pattern. A more thorough sampling (random or LHS) reveals that the singular values decay more slowly, indicating that more basis functions are required to represent the solution with the same accuracy.

For example, to achieve a given tolerance in the snapshot reconstruction error, the uniform grid might suggest $n = 20$ modes, whereas LHS shows that $n = 35$ modes are actually needed. Hence, the choice of sampling strategy directly affects the perceived complexity of the problem.

Remark 5.15. *In high dimensions ($p \geq 5$), uniform Cartesian grids become prohibitively expensive and miss most of the volume. Random sampling and, even better, Latin Hypercube Sampling are the recommended default choices. LHS guarantees that each parameter range is partitioned into intervals, and exactly one point falls into each interval per dimension, leading to a much more uniform coverage than pure random sampling.*

5.22.3 Convergence with Respect to the Number of Snapshots

Once we have fixed a good sampling strategy (e.g., LHS), we must decide how many snapshots N_s to compute. Intuitively, more snapshots provide a richer picture of the solution manifold. However, computational resources are finite. We therefore perform a convergence study: increase N_s progressively and examine how the singular value decay stabilizes.

Figure 5.2 shows the normalized singular values for $N_s = 25, 50, 100, 200, 5000$ (where 5000 is considered the "infinite" reference). With only 25 snapshots, the decay appears very fast, but this is because we have not yet sampled enough points to see the full variability. As we increase to 50, 100, and finally 200, the singular value decay curve converges. Beyond 200 snapshots, adding more (e.g., 5000) does not significantly change the curve, indicating that 200 snapshots are sufficient to capture the essential variability of the solution set.

Figure 5.2: Convergence of the normalized singular values with increasing number of snapshots (using Latin Hypercube Sampling). After a certain point (here $N_s = 200$), the decay stabilizes, indicating that the snapshot set is representative.

This behaviour is analogous to mesh refinement in finite element methods: we start with a coarse sampling, observe the singular values, then add more snapshots and recompute the SVD. Because the new snapshots are appended to the matrix, we do not discard previous simulations; we simply enlarge the snapshot matrix and perform a new SVD. After two or three such refinements, if the singular value decay no longer changes appreciably, we can trust that our snapshot set is sufficiently rich.

Remark 5.16 (Practical heuristics)**.** *There is no universal rule for the required number of snapshots. A rough rule of thumb is $N_s \approx 100 \times p$ (e.g., $p = 3 \to 300$ snapshots, $p = 4 \to 400$ snapshots), but this depends strongly on the problem's complexity. The convergence test described above is the only rigorous way to determine a sufficient N_s.*

5.22.4 Error Estimation and Basis Size Selection

Recall that the error in representing the snapshots with the first n POD modes is

$$\frac{\|\mathbf{S} - \mathbf{W}_n \mathbf{W}_n^T \mathbf{S}\|_F^2}{\|\mathbf{S}\|_F^2} = \frac{\sum_{i=n+1}^{r} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

Given a desired tolerance ε_{POD} (e.g., 10^{-6}), we choose the smallest n such that

$$\sqrt{\frac{\sum_{i=n+1}^{r} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}} \leq \varepsilon_{\text{POD}}.$$

The convergence study tells us which singular values are reliable. If we had stopped at $N_s = 25$, the apparent error for $n = 20$ might be 10^{-6}, but with $N_s = 200$ the same n might give an error of only 10^{-2}. Thus, the sampling strategy and the number of snapshots directly influence the error estimate and the required basis size.

5.23 Computational Cost: Offline vs. Online

When building a reduced-order model (ROM) using POD–Galerkin, we must consider the computational effort. Let N be the number of degrees of freedom in the high-fidelity (full-order) model, and let $n \ll N$ be the dimension of the POD basis. Solving the full-order system (e.g., a linear system of size $N \times N$) costs T_{FOM} per parameter query. Solving the reduced system (size $n \times n$) costs T_{ROM} per query, where typically $T_{\text{ROM}} \ll T_{\text{FOM}}$.

However, the ROM requires an *offline* stage:

- Computing N_s snapshots (each costing T_{FOM}),
- Performing the SVD of the snapshot matrix,
- Assembling the reduced matrices $\mathbf{A}_n(\boldsymbol{\mu})$ and $\mathbf{f}_n(\boldsymbol{\mu})$ for each parameter of interest (if not done online).

Let T_{offline} be the total offline time. Then, for N_{online} online queries, the total time is

$$T_{\text{total}} = T_{\text{offline}} + N_{\text{online}} \cdot T_{\text{ROM}}.$$

The ROM is advantageous only if $T_{\text{total}} < N_{\text{online}} \cdot T_{\text{FOM}}$, i.e.,

$$T_{\text{offline}} < N_{\text{online}} \cdot (T_{\text{FOM}} - T_{\text{ROM}}).$$

Example 5.2 (Concrete numbers)**.** *Consider a problem with $N = 44\,000$ degrees of freedom. Suppose:*

- *Full simulation time: $T_{\text{FOM}} = 3$ seconds.*
- *Reduced simulation time (with $n = 31$): $T_{\text{ROM}} = 1$ millisecond $= 0.001$ seconds.*
- *Offline cost: $T_{\text{offline}} = 95$ seconds (includes snapshots, SVD, and projection).*

Then:

- *For $N_{\text{online}} = 10$: $T_{\text{total}} = 95 + 10 \times 0.001 = 95.01$ seconds, while full order would take $10 \times 3 = 30$ seconds $\to$ ROM is worse.*

- *For* $N_{\text{online}} = 1000$*:* $T_{\text{total}} = 95 + 1 = 96$ *seconds, while full order would take* $1000 \times 3 = 3000$ *seconds* $\rightarrow$ *ROM is far superior.*

Thus, the offline cost must be amortized over a sufficiently large number of online queries.

Remark 5.17. *The offline cost includes all the expensive operations that are performed once. In many industrial contexts (e.g., real-time control, many-query optimization, uncertainty quantification), the number of online simulations is huge, making ROMs indispensable. For a handful of simulations, however, the full-order model may be more efficient.*

5.24 When Does POD Work? Reducibility and Singular Value Decay

POD is not a universal remedy. Its success depends on the *reducibility* of the problem, which is empirically assessed by the decay of the singular values of the snapshot matrix. If the singular values decay rapidly, a low-dimensional POD basis captures most of the solution variability; if they decay slowly, many modes are needed, and the ROM may not offer significant compression.

The rate of decay is intimately related to the physical nature of the problem. We illustrate this with two extreme, one-dimensional examples.

5.24.1 Example 1: A Diffusive Problem (Low Effective Dimension)

Consider the one-dimensional parameterized boundary value problem:

$$-\mu\, u''(x) = 1, \quad x \in (0,1), \qquad u(0) = u(1) = 0,$$

where $\mu > 0$ is a parameter. The exact solution is a parabola:

$$u(x;\mu) = \frac{1}{2\mu} x(1-x).$$

For any μ, the solution is a scalar multiple of the fixed shape $x(1-x)$. Thus, the solution manifold is one-dimensional: all solutions lie on the line spanned by the single function $x(1-x)$.

If we collect snapshots for many μ values, the snapshot matrix will have rank at most 2 (the constant factor and the shape). The singular values will show a very rapid decay: the first singular value captures almost all the energy, the second is tiny, and the rest are zero (up to numerical noise). This is the ideal case, analogous to the "black-and-white squares" image mentioned earlier.

Remark 5.18. *This problem is purely diffusive. Diffusion tends to smooth out variations and make the solution manifold low-dimensional. More generally, the stronger the diffusion (or, more precisely, the more the physics is dominated by smoothing operators), the faster the singular values decay.*

5.24.2 Example 2: Pure Transport (Wave Propagation)

Now consider the one-dimensional linear transport equation:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \quad x \in \mathbb{R}, \quad t > 0,$$

with initial condition $u(x,0) = u_0(x)$. The solution is a rigid translation:

$$u(x,t) = u_0(x - ct).$$

We treat time t as a parameter. Snapshots are taken at different times $t_1, t_2, \ldots$, each being the same initial profile shifted by ct_i.

Now, can a linear combination of a few snapshots represent a snapshot at an intermediate time not in the training set? In general, no: the set $\{u_0(x - ct_i)\}$ is not a low-dimensional linear subspace; it is a

nonlinear manifold (a translation of a function). POD, being a linear method, will try to approximate it by a subspace, but the required dimension can be very high, especially if the initial profile is localized (e.g., a narrow Gaussian).

Consider a Gaussian initial condition:

$$u_0(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right).$$

If σ is large (broad Gaussian), the function is smooth and its translates overlap significantly; the singular values decay reasonably fast. If σ is very small (narrow spike), translates of a narrow spike are almost orthogonal when the shifts are larger than the width. In that case, the singular values decay very slowly, and many modes are needed to approximate even a small set of snapshots. This is the "sunflower" case: the problem is not reducible by linear methods.

Remark 5.19. *The key message: POD works well when the solution manifold has a low linear dimension, i.e., when the set of all possible solutions (for varying parameters) lies close to a low-dimensional subspace. This often happens for diffusive problems. For transport-dominated problems (e.g., advection, wave propagation), the solution manifold is typically nonlinear and high-dimensional, so POD alone may not be sufficient. In such cases, more advanced techniques (e.g., nonlinear model reduction, neural networks, or moving frames) are needed.*

5.25 Practical Assessment of Reducibility

Given any problem, we can always:

1. Choose a sampling strategy (preferably Latin Hypercube or random, with a sufficient number of snapshots),
2. Compute the snapshots,
3. Perform the SVD,
4. Plot the normalized singular values σ_i/σ_1.

If the decay is fast (e.g., the first 10–20 modes capture 99.9% of the energy), the problem is reducible and POD will be effective. If the decay is slow (many modes needed), then either:

- The problem is intrinsically high-dimensional (e.g., pure transport),
- Or the sampling is insufficient (more snapshots might reveal a faster decay, but this is rare for transport-dominated problems).

Remark 5.20. *The cost of this assessment is the cost of computing the snapshots and the SVD, which is exactly the offline cost of building the ROM. Even if the problem turns out not to be reducible, this assessment is valuable: it saves us from investing further effort into an ineffective ROM.*

5.26 Assessing Reducibility: The Role of Physics and Sampling

We have seen that the decay of the singular values of the snapshot matrix is a practical indicator of whether a problem is amenable to linear model reduction (POD–Galerkin). A fast decay means that a low-dimensional subspace captures most of the solution variability; a slow decay suggests that many modes are needed, and linear reduction may be ineffective.

The rate of decay depends on two intertwined factors:

1. The **underlying physics** (diffusion vs. transport),
2. The **sampling strategy** (how we choose the parameter points or time instants).

5.26.1 Influence of Physics: Diffusion vs. Transport

Consider again the one-dimensional linear transport equation:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \qquad u(x,0) = u_0(x),$$

whose solution is a rigid translation: $u(x,t) = u_0(x-ct)$. We treat time t as a parameter and take snapshots at different times.

Let the initial profile be a Gaussian:

$$u_0(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right).$$

The parameter σ controls the width of the pulse.

- **Large σ (broad pulse)**: The function is smooth, and its translates overlap significantly. The singular values of the snapshot matrix (for a set of times) decay relatively fast. This case resembles the diffusive behavior, where the solution manifold has a low linear dimension.
- **Small σ (narrow spike)**: The pulse is very concentrated. Translates of a narrow spike are almost orthogonal when the time shift is larger than the width. Consequently, the singular values decay very slowly (see Figure 5.3). This is the "sunflower" case: the problem is not reducible by linear methods.

Figure 5.3: Normalized singular values for the transport equation with Gaussian initial condition. Blue: small σ (narrow spike) – slow decay. Green: large σ (broad pulse) – faster decay.

5.26.2 The Role of Sampling Frequency

Even for a given physical problem, the sampling of the parameter or time domain matters. If the solution changes rapidly (e.g., a narrow pulse moving quickly) and we sample too coarsely, the snapshot matrix will miss important intermediate states. In the extreme case where the support of the initial data is small and the time step Δt is too large, the correlation matrix (or the Gram matrix of snapshots) may become nearly diagonal, leading to singular values that do not decay at all. Thus, insufficient sampling can falsely suggest that the problem is irreducible, even when a finer sampling might yield a faster decay.

Remark 5.21. *The lesson is twofold:*

- ***Physics matters**: Diffusion-dominated problems are generally more reducible than transport-dominated ones.*
- ***Sampling matters**: Even for a reducible problem, a poor sampling strategy can produce a misleading singular value decay. Conversely, for a transport-dominated problem, no amount of sampling will make the singular values decay fast if the underlying solution manifold is truly high-dimensional.*

In practice, we always perform a convergence study with respect to the number of snapshots (as discussed in the previous section) to ensure that the observed decay is not an artifact of undersampling.

5.26.3 When Linear Reduction Fails: Outlook to Nonlinear Methods

POD is a linear method: it approximates new solutions as linear combinations of a fixed set of basis functions. For problems where the solution manifold is intrinsically nonlinear (e.g., traveling waves, shocks, advection-dominated phenomena), linear methods require a very large number of modes. In such cases, we must resort to *nonlinear model reduction* techniques. Among these, neural networks (in particular, autoencoders or deep learning surrogates) have shown great promise. We will explore these advanced tools later in the course. For now, it is sufficient to remember that POD–Galerkin is most effective when the singular values decay rapidly, which typically occurs for diffusive or elliptic problems with smooth parameter dependence.

5.27 Handling Boundary Conditions in POD–Galerkin

So far, we have implicitly assumed homogeneous Dirichlet boundary conditions. In many applications, boundary conditions may be non-homogeneous (nonzero) and possibly parameter-dependent. We must adapt the POD procedure accordingly.

5.27.1 Homogeneous Dirichlet Conditions (Lucky Case)

Suppose the problem is

$$-\Delta u = f(\boldsymbol{\mu}) \quad \text{in } \Omega, \qquad u = 0 \text{ on } \partial\Omega.$$

All snapshots satisfy $u = 0$ on the boundary. When we assemble the snapshot matrix $\mathbf{S}$, the rows corresponding to boundary degrees of freedom are identically zero. The SVD then produces left singular vectors $\mathbf{w}_i$ that also have zero entries on those rows. Consequently, any linear combination $\sum_i c_i \mathbf{w}_i$ will also be zero on the boundary. Thus, the POD basis automatically respects the homogeneous boundary condition, and no special treatment is required.

5.27.2 Non-Homogeneous Dirichlet Conditions (Lifting Function)

Now consider

$$-\Delta u = f(\boldsymbol{\mu}) \quad \text{in } \Omega, \qquad u = g(\boldsymbol{\mu}) \text{ on } \partial\Omega,$$

where $g(\boldsymbol{\mu})$ may be nonzero and possibly parameter-dependent. A direct application of POD to snapshots of u would yield basis functions that are not guaranteed to satisfy the boundary condition for arbitrary coefficients. The standard remedy is the *method of lifting functions*.

We decompose the solution as

$$u(\boldsymbol{\mu}) = u_0(\boldsymbol{\mu}) + R(\boldsymbol{\mu}),$$

where $R(\boldsymbol{\mu})$ is a *lift function* chosen such that

$$R(\boldsymbol{\mu}) = g(\boldsymbol{\mu}) \quad \text{on } \partial\Omega,$$

and $u_0(\boldsymbol{\mu})$ satisfies the homogeneous boundary condition $u_0 = 0$ on $\partial\Omega$. Substituting into the PDE gives a modified problem for u_0:

$$-\Delta u_0 = f(\boldsymbol{\mu}) + \Delta R(\boldsymbol{\mu}) \quad \text{in } \Omega, \qquad u_0 = 0 \text{ on } \partial\Omega.$$

Now, we apply POD to snapshots of u_0 (the homogeneous part). All these snapshots are zero on the boundary, so the resulting POD basis $\mathbf{W}_n$ will also be zero on the boundary. We then build a reduced-order model for u_0 using Galerkin projection. After solving for the reduced coefficients, we reconstruct the full solution as

$$u_{\text{rom}}(\boldsymbol{\mu}) = \mathbf{W}_n \mathbf{u}_{0,n}(\boldsymbol{\mu}) + R(\boldsymbol{\mu}).$$

Remark 5.22. *If the boundary condition depends on parameters, $R(\boldsymbol{\mu})$ must also depend on $\boldsymbol{\mu}$. The lift function is usually chosen as a simple extension (e.g., the solution of a Laplace problem with the same boundary data, or a linear interpolation). The term $\Delta R(\boldsymbol{\mu})$ then appears as an additional source in the modified equation. This is handled naturally during the offline stage: we compute snapshots of u_0 by solving the modified problem, and the parameter-dependent lift is evaluated online. The procedure is standard and will be illustrated in the exercises.*

5.28 Summary and Practical Takeaways

- The decay of singular values is the primary tool to assess reducibility. Fast decay → good candidate for POD–Galerkin.
- Diffusive problems tend to be reducible; transport-dominated problems are challenging.

- Sampling must be sufficiently fine to capture the solution's variability; always perform a convergence study in the number of snapshots.
- Non-homogeneous Dirichlet conditions require a lifting function to transform the problem into a homogeneous one. The POD basis is then built for the homogeneous part.
- When linear reduction fails, nonlinear techniques (e.g., neural networks) are needed.

In the next chapter, we will apply these concepts to time-dependent problems and parameter-time mixed scenarios, and we will begin the transition to non-intrusive methods.

Chapter 6

Lab 3 - Reduced basis (POD-Galerkin) for stationary problems

6.1 Introduction

Today we begin model order reduction (MOR), specifically the reduced basis method. We will implement a pure Galerkin reduced basis method—the only MOR approach we will code in this course. The example is stationary (since you haven't seen time-dependent PDEs yet) and is taken from the last exercise of the previous lab, known in the literature as the *cookie problem*.

6.1.1 Problem Description: Stationary Heat Equation on a Plate with Disks

We consider a two-dimensional domain Ω (the whole plate) containing four circular subdomains (disks). Let:

$$\Omega_0, \Omega_1, \Omega_2, \Omega_3 \subset \Omega$$

be the four disks. The thermal conductivity σ is piecewise constant:

$$\sigma(x; \boldsymbol{\mu}) = \sigma_{\text{out}} + \sum_{j=0}^{3} \mu_j^2 \, \mathbf{1}_{\Omega_j}(x),$$

where:

- $\sigma_{\text{out}} = 0.001$ (fixed conductivity outside the disks),
- $\mathbf{1}_{\Omega_j}$ is the indicator function of disk j,
- μ_j^2 is the additional conductivity inside disk j (so μ_j is the square root of the conductivity contrast),
- $\boldsymbol{\mu} = (\mu_0, \mu_1, \mu_2, \mu_3) \in [0, 3]^4$ are the parameters.

Thus the conductivity inside disk j is $\sigma_{\text{out}} + \mu_j^2$. The boundary condition is $u = 0$ on $\partial\Omega$ (the entire boundary is kept at zero temperature). The source term is $f = 1$ (uniform heating). The PDE is the stationary heat equation:

$$-\nabla \cdot (\sigma \nabla u) = f \quad \text{in } \Omega, \qquad u = 0 \text{ on } \partial\Omega.$$

6.1.2 Finite Element Discretization

The problem is discretized using continuous piecewise linear finite elements ($\mathbb{P}_1$) on a mesh with about 10 000 degrees of freedom. The implementation is available in the library `DRROMS`. A single simulation for a given parameter vector $\boldsymbol{\mu}$ can be run with:

```
from DRROMS import FEMFormSolver
# ... setup ...
solver = FEMFormSolver(Vh, a, L)
uh = solver.solve(mu)
```

We will look at a few parameter configurations to understand the solution behavior.

Example 1: $\boldsymbol{\mu} = (1, 0.1, 1, 1)$

The disk with $\mu_1 = 0.1$ has very low additional conductivity, so it is almost invisible. The other three disks stand out.

Example 2: $\boldsymbol{\mu} = (0.1, 0.1, 0.1, 0.1)$

All disks have the same conductivity ($\sigma_{\text{out}} + 0.01$). They are only slightly more conductive than the outside, so they are barely distinguishable.

6.2 Snapshots Collection for the Reduced Basis Method

We want to build a surrogate model (reduced order model) that can solve the problem much faster than the full finite element solver. The first step is to collect *snapshots*: solutions of the full FE model for many different parameter values.

6.2.1 Random Parameter Sampling

We will sample N_{samples} parameter vectors uniformly at random in $[0, 3]^4$. For each sampled $\boldsymbol{\mu}_i$, we compute the corresponding FE solution $\mathbf{u}_i \in \mathbb{R}^{N_h}$ (where $N_h \approx 10\,000$). The code:

```
import numpy as np

n_samples = 200
param_list = []
sol_list = []

for i in range(n_samples):
    # Sample 4 parameters uniformly in [0,3]
    mu = 3.0 * np.random.rand(4)
    # Solve the FE problem for this mu
    uh = solver.solve(mu)   # returns a vector of size N_h
    param_list.append(mu)
    sol_list.append(uh)

# Convert lists to 2D arrays
mu_samples = np.array(param_list)          # shape (n_samples, 4)
U = np.stack(sol_list, axis=0)             # shape (n_samples, N_h)
```

The array `U` contains the snapshot matrix: each row is a solution vector corresponding to a parameter sample.

6.2.2 Inspecting the Snapshots

After running the loop, we can check the dimensions:

```
print(mu_samples.shape)    # (200, 4)
print(U.shape)             # (200, 10000)
```

We can plot individual solutions using `FE.plot`:

```
FE.plot(U[103], Vh)   # solution for sample 103
print(mu_samples[103]) # corresponding parameters
```

6.2.3 Plotting Multiple Solutions

To compare several solutions side by side, use `FE.multiplot`. By default, each subplot uses its own colorbar scale, which can be misleading. To enforce a common colorbar range:

```
# First five solutions
solutions = U[:5]

# Determine global min and max over these five
vmin = np.min(solutions)
vmax = np.max(solutions)

# Multiplot with a fixed colorbar range
FE.multiplot(solutions, Vh, nrows=1, ncols=5, vmin=vmin, vmax=vmax)
```

Alternatively, you can set the range manually (e.g., `vmin=0, vmax=5`) if you know appropriate bounds from the entire dataset.

6.3 Next Steps: Building the Reduced Basis Model

With the snapshot matrix $\mathbf{U} \in \mathbb{R}^{N_{\text{samples}} \times N_h}$, we will:

1. Compute a reduced basis via proper orthogonal decomposition (POD).
2. Project the finite element system onto the reduced space to obtain a small linear system.
3. Use the reduced model for fast parameter sweeps (e.g., for solving inverse problems).

This will be the focus of the next lab, where we will also discuss the application to parameter estimation (e.g., detecting material degradation by comparing temperature measurements to reduced model predictions).

6.4 Closing Remarks

The full FE solver takes about one second per simulation on this toy problem. With a reduced basis of dimension $k \ll N_h$, we can achieve speedups of several orders of magnitude, making real-time parameter estimation feasible.

6.5 Motivation

The finite element (FE) discretization of the cookie problem leads to a linear system of size $N_h \approx 10\,000$. In principle, we need to solve for 10 000 unknowns every time we change the parameters $\boldsymbol{\mu}$. However, the solution manifold (the set of all possible temperature fields) is very low-dimensional in practice: the solutions always look like smooth variations around the disks. Therefore we can try to find a low-dimensional subspace $V_n \subset V_h$ of dimension $n \ll N_h$ that contains all relevant solutions (or very accurate approximations thereof). This is the idea behind the **reduced basis method** (RB).

Today we construct such a subspace using **Proper Orthogonal Decomposition (POD)** applied to a set of precomputed FE solutions (snapshots).

6.6 Snapshot Matrix (Data Matrix)

We first collect N_{samples} solutions $\mathbf{u}_i \in \mathbb{R}^{N_h}$ for different parameter values $\boldsymbol{\mu}_i$. In the lab we store them in a matrix where **each row is one sample**:

$$\mathbf{U} \in \mathbb{R}^{N_{\text{samples}} \times N_h}, \qquad \mathbf{U}_{i,:} = \mathbf{u}_i^\top \quad \text{(row vector)}.$$

This is standard in data science and machine learning (first axis = samples). In some lectures the snapshot matrix is defined as the transpose, but the underlying mathematics is the same; we just have to be careful about dimensions.

6.6.1 Train / Test Split

We split the N_{samples} simulations into a **training set** (to build the basis) and a **test set** (to evaluate the reduced model). For example, with $N_{\text{samples}} = 200$:

```
n_train = N_samples // 2   # 100
U_train = U[:n_train, :]    # shape (100, N_h)
U_test  = U[n_train:, :]    # shape (100, N_h)
```

6.7 POD via Singular Value Decomposition (SVD)

The POD basis is obtained by performing a singular value decomposition of the (transposed) training snapshot matrix. Let

$$\mathbf{X} = \mathbf{U}_{\text{train}}^\top \in \mathbb{R}^{N_h \times n_{\text{train}}}$$

so that each column of $\mathbf{X}$ is a snapshot (consistent with the usual mathematical convention). Then we compute the **thin SVD**:

$$\mathbf{X} = \mathbf{V}\boldsymbol{\Sigma}\mathbf{W}^\top,$$

where:

- $\mathbf{V} \in \mathbb{R}^{N_h \times n_{\text{train}}}$ has orthonormal columns (the left singular vectors),
- $\boldsymbol{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \ldots, \sigma_{n_{\text{train}}})$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{n_{\text{train}}} \geq 0$,
- $\mathbf{W} \in \mathbb{R}^{n_{\text{train}} \times n_{\text{train}}}$ is orthogonal.

In Python using `scipy.linalg.svd`:

```
from scipy.linalg import svd

X = U_train.T   # shape (N_h, n_train)
V, s, Wt = svd(X, full_matrices=False)   # s = array of singular values
```

6.7.1 Choosing the Number of Modes n

The columns of $\mathbf{V}$ are the POD modes, ordered by decreasing importance (singular values). The singular values decay quickly, as seen in a logarithmic plot:

```
import matplotlib.pyplot as plt
plt.semilogy(s, 'o-')
plt.xlabel('Mode index')
plt.ylabel('Singular value')
plt.title('Decay of singular values')
plt.grid(True)
```

Because we have not centered the data, the first singular value is often much larger than the others. It is common to skip it when looking for an "elbow".

Energy Criterion

The fraction of energy captured by the first n modes is:

$$E(n) = \frac{\sum_{i=1}^{n} \sigma_i^2}{\sum_{i=1}^{n_{\text{train}}} \sigma_i^2}.$$

We choose n such that $E(n)$ is close to 1 (e.g., 99.9%). In our example with $n = 10$ we capture 99.999% of the energy:

```
total_energy = np.sum(s**2)
energy_captured = np.sum(s[:n]**2) / total_energy
print(f"Energy captured by first {n} modes: {energy_captured*100:.5f}%")
```

6.8 The POD Basis Matrix

We take the first n left singular vectors as our reduced basis:

$$\mathbf{V}_n = \mathbf{V}(:, 1:n) \in \mathbb{R}^{N_h \times n}.$$

These vectors are orthonormal by construction:

$$\mathbf{V}_n^\top \mathbf{V}_n = \mathbf{I}_n.$$

We can verify this numerically:

```
Vn = V[:, :n]
print(Vn.T @ Vn)    # should be approximately identity
```

6.8.1 Visualising the POD Modes

We can plot the first few modes using the finite element plotting function:

```
# Plot first 10 modes in a 2x5 grid
FE.multiplot(Vn.T, Vh, nrows=2, ncols=5, vmin=-0.1, vmax=0.1)
```

The first mode typically looks like the average solution; subsequent modes capture finer variations (e.g., differences between disks).

6.9 Projection onto the Reduced Basis

Any full-order solution $\mathbf{u} \in \mathbb{R}^{N_h}$ can be approximated by its orthogonal projection onto span($\mathbf{V}_n$). Since the basis is orthonormal, the projection coefficients are given by:

$$\hat{\mathbf{u}} = \mathbf{V}_n \mathbf{c}, \qquad \mathbf{c} = \mathbf{V}_n^\top \mathbf{u} \in \mathbb{R}^n.$$

In code:

```
# Take one test solution
u0 = U_test[0, :]       # shape (N_h,)
# Compute coefficients
c = Vn.T @ u0           # shape (n,)
# Reconstruct
u_reconstructed = Vn @ c
```

6.9.1 Example: Reconstructing a Solution Using Increasing Numbers of Modes

We can observe how the approximation improves as we add more modes:

```
modes_to_use = [1, 2, 3, 10]
reconstructions = []
for k in modes_to_use:
    c_k = V[:,:k].T @ u0
    u_k = V[:,:k] @ c_k
    reconstructions.append(u_k)

FE.multiplot(reconstructions, Vh, nrows=1, ncols=len(modes_to_use))
```

With only the first mode, all disks appear identical (because the first mode itself has symmetric disks). Adding the second and third modes allows the reconstruction to capture differences between individual disks.

6.10 Next Steps: Galerkin Projection of the PDE

Once we have the reduced basis $\mathbf{V}_n$, we can project the finite element system $\mathbf{A}(\boldsymbol{\mu})\mathbf{u} = \mathbf{F}(\boldsymbol{\mu})$ onto the reduced space. The reduced system is:

$$\mathbf{A}_n(\boldsymbol{\mu})\mathbf{c} = \mathbf{F}_n(\boldsymbol{\mu}),$$

where

$$\mathbf{A}_n = \mathbf{V}_n^\top \mathbf{A}(\boldsymbol{\mu})\mathbf{V}_n \in \mathbb{R}^{n\times n}, \qquad \mathbf{F}_n = \mathbf{V}_n^\top \mathbf{F}(\boldsymbol{\mu}) \in \mathbb{R}^n.$$

Solving this $n \times n$ system (with $n \approx 10$) instead of the $N_h \times N_h$ system gives a dramatic speedup. We will implement this fully in the next lab.

6.11 Important Notation Reminder

- In the lectures, the snapshot matrix is often defined as $\mathbf{S} \in \mathbb{R}^{N_h \times N_s}$ (snapshots as columns).
- In the labs (and in this document), we use the data-science convention: $\mathbf{U} \in \mathbb{R}^{N_s \times N_h}$ (snapshots as rows).
- The SVD is applied to the transpose $\mathbf{U}_{\text{train}}^\top$ to obtain the left singular vectors as columns of $\mathbf{V}$.

This difference only requires a transpose and does not change the mathematical properties of the POD basis.

6.12 Projection onto the POD Basis

Let $\mathbf{V}_n \in \mathbb{R}^{N_h \times n}$ be the matrix whose columns are the first n POD modes (orthonormal). For any full-order solution $\mathbf{u} \in \mathbb{R}^{N_h}$, its orthogonal projection onto the reduced space is:

$$\mathbf{u}_{\text{proj}} = \mathbf{V}_n \mathbf{c}, \qquad \mathbf{c} = \mathbf{V}_n^\top \mathbf{u} \in \mathbb{R}^n.$$

The coefficients $\mathbf{c}$ are the coordinates in the reduced basis. By construction, $\mathbf{u}_{\text{proj}}$ is the best approximation of $\mathbf{u}$ in the reduced space (in the Euclidean norm).

6.12.1 Example: Projection of a Single Test Solution

```
# u0 is a full-order solution (shape (N_h,))
c = Vn.T @ u0                  # coefficients
u_proj = Vn @ c                # reconstructed solution
```

The relative projection error is:

$$\varepsilon_{\text{rel}} = \frac{\|\mathbf{u}_0 - \mathbf{u}_{\text{proj}}\|_2}{\|\mathbf{u}_0\|_2}.$$

For a typical test case, this error is very small (e.g., 0.03%), meaning the reduced basis captures the solution almost exactly.

6.12.2 Batch Computation on the Entire Test Set

We have a test set $\mathbf{U}_{\text{test}} \in \mathbb{R}^{N_{\text{test}} \times N_h}$ (rows = samples). The projections of all test samples can be computed simultaneously using matrix multiplication:

```
C_test = U_test @ Vn                # shape (N_test, n)
U_proj_test = C_test @ Vn.T         # shape (N_test, N_h)
```

Then the relative error for each test sample is:

```
errors = np.linalg.norm(U_test - U_proj_test, axis=1) / np.linalg.norm(U_test, axis=1)
mean_rel_error = np.mean(errors)
```

In our example, the mean relative error on the test set is about 0.03%, confirming that the basis (with $n = 10$) is excellent.

6.13 From Projection to Galerkin Reduced Order Model (ROM)

The full-order finite element problem is:

$$\mathbf{A}(\boldsymbol{\mu})\,\mathbf{u}(\boldsymbol{\mu}) = \mathbf{F}(\boldsymbol{\mu}),$$

where $\mathbf{A}(\boldsymbol{\mu}) \in \mathbb{R}^{N_h \times N_h}$ and $\mathbf{F}(\boldsymbol{\mu}) \in \mathbb{R}^{N_h}$. The Galerkin ROM seeks an approximation $\mathbf{u}_n(\boldsymbol{\mu}) = \mathbf{V}_n \mathbf{c}(\boldsymbol{\mu})$ such that the residual is orthogonal to the reduced space:

$$\mathbf{V}_n^\top \big(\mathbf{A}(\boldsymbol{\mu})\mathbf{V}_n \mathbf{c}(\boldsymbol{\mu}) - \mathbf{F}(\boldsymbol{\mu})\big) = \mathbf{0}.$$

This yields the reduced system of size $n \times n$:

$$\underbrace{\big(\mathbf{V}_n^\top \mathbf{A}(\boldsymbol{\mu})\mathbf{V}_n\big)}_{\mathbf{A}_n(\boldsymbol{\mu})}\,\mathbf{c}(\boldsymbol{\mu}) = \underbrace{\mathbf{V}_n^\top \mathbf{F}(\boldsymbol{\mu})}_{\mathbf{F}_n(\boldsymbol{\mu})}.$$

6.13.1 Affine Parametrization: Key to Efficiency

If the parameter dependence is affine, i.e.,

$$\mathbf{A}(\boldsymbol{\mu}) = \mathbf{A}_0 + \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu}) \mathbf{A}_q,$$

then the reduced matrix can be preassembled offline:

$$\mathbf{A}_n(\boldsymbol{\mu}) = \underbrace{\mathbf{V}_n^\top \mathbf{A}_0 \mathbf{V}_n}_{\mathbf{A}_{n,0}} + \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu}) \underbrace{\mathbf{V}_n^\top \mathbf{A}_q \mathbf{V}_n}_{\mathbf{A}_{n,q}}.$$

The matrices $\mathbf{A}_{n,q}$ are independent of $\boldsymbol{\mu}$ and are computed once. The online stage then only requires forming a linear combination of these small matrices (cost $\mathcal{O}(Qn^2)$) and solving an $n \times n$ system. This is the key to real-time performance.

6.13.2 Affine Decomposition for the Cookie Problem

In the cookie problem, the conductivity is

$$\sigma(\boldsymbol{\mu}) = \sigma_{\text{out}} + \sum_{j=0}^{3} \mu_j^2 \, \mathbf{1}_{\Omega_j}.$$

The stiffness matrix (bilinear form) decomposes as:

$$\mathbf{A}(\boldsymbol{\mu}) = \mathbf{A}_{\text{out}} + \sum_{j=0}^{3} \mu_j^2 \, \mathbf{A}_j,$$

where $\mathbf{A}_{\text{out}}$ corresponds to σ_{out} integrated over the whole domain, and $\mathbf{A}_j$ corresponds to the diffusion restricted to disk j. The load vector $\mathbf{F}$ is parameter-independent (constant source $f = 1$). Thus the affine decomposition holds with $Q = 4$ and $\theta_j(\boldsymbol{\mu}) = \mu_j^2$.

6.14 Offline Preassembly of Reduced Matrices

```
# Assuming Vn is already built (POD basis)
A_out = ...   # full matrix from sigma_out (FE assembly)
A_j = [...]   # list of matrices for each disk

# Preassemble reduced matrices
A_out_rb = Vn.T @ A_out @ Vn          # shape (n, n)
A_j_rb   = [Vn.T @ Aj @ Vn for Aj in A_j]   # list of (n,n) matrices
F_rb     = Vn.T @ F                   # shape (n,)
```

6.15 Online Assembler and Solver

```
def assemble_rb(mu):
    """Assemble reduced matrix and RHS for parameter mu."""
    # mu is a 4-element array [mu0, mu1, mu2, mu3]
    A_rb = A_out_rb.copy()
    for j in range(4):
        A_rb += mu[j]**2 * A_j_rb[j]
```

```
    F_rb = F_rb   # constant
    return A_rb, F_rb

def solve_rb(mu):
    A_rb, F_rb = assemble_rb(mu)
    c = np.linalg.solve(A_rb, F_rb)       # coefficients (n,)
    u_rb = Vn @ c                         # reconstruct full field
    return u_rb
```

6.16 Validation: Compare ROM with Full FE Solution

Take a specific parameter, e.g., $\boldsymbol{\mu} = (0.2, 0.0, 1.0, 0.4)$.

```
mu_test = np.array([0.2, 0.0, 1.0, 0.4])
u_fem = solver_fem.solve(mu_test)   # full-order solution (N_h,)
u_rb  = solve_rb(mu_test)           # reduced-order solution
error = np.linalg.norm(u_fem - u_rb) / np.linalg.norm(u_fem)
print(f"Relative error: {error*100:.3f}%")
```

Typical error is around 0.3%, confirming the ROM is accurate.

6.17 Computational Speed Comparison

```
import time

# Full-order solve
t0 = time.time()
u_fem = solver_fem.solve(mu_test)
t_fem = time.time() - t0

# Reduced-order solve
t0 = time.time()
u_rb = solve_rb(mu_test)
t_rb = time.time() - t0

print(f"FEM time: {t_fem*1000:.2f} ms")
print(f"ROM time: {t_rb*1000:.3f} ms")
```

On the cookie problem with $N_h \approx 10\,000$ and $n = 10$:

- Full FEM: ≈ 890 ms per solve.
- ROM: ≈ 0.5 ms per solve (more than $1000\times$ speedup).

This enables real-time applications (e.g., parameter estimation, monitoring, or many-query scenarios).

6.18 Projection Coefficients vs. Galerkin Coefficients

Given a true solution $\mathbf{u}_{\text{fem}}(\boldsymbol{\mu})$, the *projection coefficients* are $\mathbf{c}_{\text{proj}} = \mathbf{V}_n^\top \mathbf{u}_{\text{fem}}$. The *Galerkin coefficients* $\mathbf{c}_{\text{rb}}$ are obtained by solving the reduced system. They are generally different because the Galerkin projection uses the PDE residual, not the exact solution. However, for a well-chosen basis, the difference is small. The error between $\mathbf{c}_{\text{proj}}$ and $\mathbf{c}_{\text{rb}}$ is another measure of the ROM's quality.

6.19 Conclusion

The reduced basis method with POD-Galerkin projection provides a dramatic speedup (orders of magnitude) while preserving high accuracy, provided the problem has an affine parameter dependence. For non-affine problems, additional techniques (e.g., empirical interpolation, hyper-reduction) are needed, but these are beyond the scope of this course.

Chapter 7

The Reduced Basis Method for Nonstationary Problems and Some Hints on Nonaffine Problems

So far we have discussed the reduced basis method. Before moving to new topics, let us recall the essential ingredients for the stationary linear case. We start from the usual full order model (FOM). Two main building blocks are needed:

1. A way to express reduced order solutions.
2. A way to determine the reduced vector $\mathbf{u}_n$ (the reduced coordinates).

7.1 Proper Orthogonal Decomposition (POD)

The first ingredient is typically addressed by Proper Orthogonal Decomposition. POD is based on the singular value decomposition (SVD) of a snapshot matrix. The snapshot matrix contains solutions of the full order model computed for selected parameter values. POD provides an *optimal* linear compression technique: given a set of snapshots, the SVD returns a basis that is optimal in the least-squares sense. The resulting basis is also *global*, i.e., we use the same basis functions (or vectors) to represent candidate solutions for all new parameter values.

Thus, the reduced approximation $\mathbf{u}_\mathrm{N}$ (originally of large dimension $\mathcal{N}$) is expressed as a linear combination of only $n \ll \mathcal{N}$ basis functions:

$$\mathbf{u}_\mathrm{N}(\mu) \approx \mathbf{V}\mathbf{u}_n(\mu),$$

where $\mathbf{V}$ is the matrix whose columns are the first n POD modes.

7.2 Galerkin Projection

The second ingredient is the determination of the reduced coordinates $\mathbf{u}_n(\mu)$. For a linear stationary problem, we insert the approximation into the full order model. The residual is

$$\mathbf{r}(\mu) = \mathbf{f}_\mathrm{N}(\mu) - \mathbf{A}_\mathrm{N}(\mu)\mathbf{V}\mathbf{u}_n(\mu).$$

We impose that this residual be orthogonal to the space spanned by the POD basis:

$$\mathbf{V}^T\mathbf{r}(\mu) = \mathbf{0}.$$

This Galerkin condition yields the reduced linear system

$$\mathbf{A}_n(\mu)\mathbf{u}_n(\mu) = \mathbf{f}_n(\mu),$$

with

$$\mathbf{A}_n(\mu) = \mathbf{V}^T \mathbf{A}_\mathrm{N}(\mu)\mathbf{V}, \qquad \mathbf{f}_n(\mu) = \mathbf{V}^T \mathbf{f}_\mathrm{N}(\mu).$$

Properties. The Galerkin projection is optimal in the sense that it gives the best approximation in the subspace (minimum residual). Moreover, it preserves the physics of the original problem because the residual is tested against the same basis that comes from the FOM.

7.3 Efficient Assembly: The Affine Case

To make the reduced order model (ROM) truly efficient, we must assemble $\mathbf{A}_n(\mu)$ and $\mathbf{f}_n(\mu)$ without repeatedly manipulating large $\mathcal{N}$-dimensional objects. This is possible when the parameter μ enters the problem in an *affine* way:

$$\mathbf{A}_\mathrm{N}(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu)\mathbf{A}_\mathrm{N}^q, \qquad \mathbf{f}_\mathrm{N}(\mu) = \sum_{q=1}^{Q_f} \Theta_f^q(\mu)\mathbf{f}_\mathrm{N}^q,$$

where the matrices $\mathbf{A}_\mathrm{N}^q$ and vectors $\mathbf{f}_\mathrm{N}^q$ are parameter-independent. Then the reduced operators become

$$\mathbf{A}_n(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu) \underbrace{\mathbf{V}^T\mathbf{A}_\mathrm{N}^q\mathbf{V}}_{\text{pre-computable}}, \qquad \mathbf{f}_n(\mu) = \sum_{q=1}^{Q_f} \Theta_f^q(\mu) \underbrace{\mathbf{V}^T\mathbf{f}_\mathrm{N}^q}_{\text{pre-computable}}.$$

All parameter-independent quantities can be computed once offline. For each new μ we only form linear combinations of small matrices/vectors, leading to huge computational savings.

7.4 Challenges Beyond the Stationary Linear Affine Setting

The workflow described above relies on several assumptions:

- The problem is **stationary** (no time dependence).
- The problem is **linear**.
- The parameter dependence is **affine**.

When one or more of these assumptions break down, the standard reduced basis method becomes less straightforward.

7.4.1 Non-Stationary Problems

If the problem is time-dependent (e.g., unsteady heat equation, wave equation), the reduced basis must also represent the temporal evolution. We will address this case today, still assuming linearity and affine parameter dependence.

7.4.2 Non-Affine Problems

When the parameter dependence is not affine, the matrices $\mathbf{A}_\mathrm{N}(\mu)$ and $\mathbf{f}_\mathrm{N}(\mu)$ cannot be split into a sum of separable terms. Consequently, the efficient offline/online decomposition is lost. The community has developed techniques to *approximate* the non-affine operators by affine expansions (e.g., the Empirical Interpolation Method, EIM). These methods attempt to restore the affine structure, but they introduce additional complexity.

7.4.3 Nonlinear Problems

For nonlinear problems, the residual is not a linear function of the unknown. Common solution strategies (e.g., Newton's method) linearize the problem at each iteration, leading to a sequence of linear systems. The Jacobian matrix and the right-hand side change at each Newton step, and they depend on the current iterate. Even if the original problem is affine in the parameters, the linearized problems are generally not affine, and the reduced basis approach would require repeated expensive operations. This motivates the use of less intrusive (or non-intrusive) methods, which will be discussed after Easter.

7.4.4 Combined Difficulties

Realistic applications often present multiple challenges simultaneously:

- Nonlinear and non-stationary (e.g., Navier–Stokes equations for fluids).
- Non-affine and non-stationary (e.g., advection–diffusion of a pollutant with a moving source).

For such problems, we need advanced strategies that preserve as many of the attractive building blocks as possible.

7.5 Hints on Nonaffine Problems

Even though a full treatment is beyond today's scope, we mention that for nonaffine problems one can attempt to *recover* an affine representation in an approximate manner. For instance, if the parameter dependence enters through a coefficient function $a(x;\mu)$, we may approximate it by

$$a(x;\mu) \approx \sum_{q=1}^{Q} \beta_q(\mu)\, \varphi_q(x),$$

where $\{\varphi_q\}$ are pre-selected basis functions (e.g., from snapshots of a). This is the essence of the Empirical Interpolation Method. Once such an approximation is available, the original nonaffine problem becomes affinely parametrized, and the efficient reduced basis procedure can be applied. However, the additional approximation error must be controlled.

7.6 Recap of the Stationary Linear Affine Case

So far we have discussed the reduced basis (RB) method for stationary, linear, affinely parametrized problems. Let us briefly recall the two essential ingredients:

1. **Express the reduced order solution.** This is done via Proper Orthogonal Decomposition (POD). POD takes a snapshot matrix (solutions of the full order model for selected parameter values) and, through the singular value decomposition (SVD), returns an optimal linear basis. The basis is *global*: the same basis functions are used for every parameter value. We write the approximation as

   $$\mathbf{u}_{\mathcal{N}}(\mu) \approx \mathbf{V}\mathbf{u}_n(\mu),$$

   where $\mathbf{V}$ is the matrix of the first n POD modes, $n \ll \mathcal{N}$.

2. **Find the reduced coordinates $\mathbf{u}_n(\mu)$.** Using a Galerkin projection, we impose that the residual be orthogonal to the columns of $\mathbf{V}$. This yields the reduced linear system

   $$\mathbf{A}_n(\mu)\mathbf{u}_n(\mu) = \mathbf{f}_n(\mu), \qquad \mathbf{A}_n(\mu) = \mathbf{V}^T\mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}, \quad \mathbf{f}_n(\mu) = \mathbf{V}^T\mathbf{f}_{\mathcal{N}}(\mu).$$

Efficient assembly (affine case). If the parameter dependence is affine,

$$\mathbf{A}_{\mathcal{N}}(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu)\mathbf{A}_{\mathcal{N}}^q, \qquad \mathbf{f}_{\mathcal{N}}(\mu) = \sum_{q=1}^{Q_f} \Theta_f^q(\mu)\mathbf{f}_{\mathcal{N}}^q,$$

then the reduced operators become

$$\mathbf{A}_n(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu) \underbrace{\mathbf{V}^T\mathbf{A}_{\mathcal{N}}^q\mathbf{V}}_{\text{pre-computable}}, \qquad \mathbf{f}_n(\mu) = \sum_{q=1}^{Q_f} \Theta_f^q(\mu) \underbrace{\mathbf{V}^T\mathbf{f}_{\mathcal{N}}^q}_{\text{pre-computable}}.$$

All parameter-independent quantities are computed once offline; for each new μ we only combine small matrices/vectors.

What if assumptions break? The difficulties begin when:

- The problem is **non-stationary** (time-dependent).
- The problem is **non-affine**.
- The problem is **nonlinear**.

Often these difficulties appear together (e.g., Navier–Stokes equations). After Easter we will explore alternative ways to combine the bases when assembling the reduced system becomes too expensive. Today, however, we focus on the first extension: *non-stationary problems*, while still assuming linearity and affinity.

7.7 Extension to Non-Stationary (Time-Dependent) Problems

We now consider a first-order time-dependent problem. (Second-order problems can be rewritten as first-order systems by introducing auxiliary variables.) A typical linear, affinely parametrized evolution problem reads:

$$\begin{cases} \dfrac{\partial u_{\mathcal{N}}}{\partial t}(t;\mu) + \mathbf{A}_{\mathcal{N}}(\mu)u_{\mathcal{N}}(t;\mu) = \mathbf{f}_{\mathcal{N}}(t;\mu), \quad t \in (0,T], \\ u_{\mathcal{N}}(0;\mu) = u_{\mathcal{N}}^0(\mu). \end{cases}$$

Here $\mathbf{A}_{\mathcal{N}}$ may depend on μ but not on t (for simplicity); the forcing term $\mathbf{f}_{\mathcal{N}}$ can depend on time. The initial condition may also depend on parameters. Examples include the heat equation (e.g., cookies in an oven) where the source or material properties change over time.

7.7.1 Full Order Discretization

At the full order level we already have a spatial discretization (finite elements, $\mathcal{N}$ degrees of freedom). We also introduce a time discretization, e.g., with constant time step $\Delta t = T/N_t$, where N_t is the number of time steps. A numerical scheme (Forward/Backward Euler, Crank–Nicolson, etc.) will produce a sequence of vectors:

$$\mathbf{u}_{\mathcal{N}}^k(\mu) \approx u_{\mathcal{N}}(t_k;\mu), \qquad t_k = k\Delta t,\ k = 0,\dots,N_t.$$

7.7.2 Recycling the Two Ingredients

We wish to reuse the same philosophy as in the stationary case:

1. **Basis construction via POD.** But now the snapshots are not just one vector per parameter; they are entire time trajectories.
2. **Determination of the reduced coordinates.** We will again apply a Galerkin projection, which will lead to a *reduced dynamical system* (a small system of ordinary differential equations or algebraic equations, depending on the time scheme).

7.7.3 POD for Time-Dependent Data

In the stationary case, with N_s parameter values, we built a snapshot matrix of size $\mathcal{N} \times N_s$. Now, for each parameter μ_j, we have $N_t + 1$ snapshots (the whole trajectory). If we collect snapshots for all parameters, the total number of columns becomes $N_s \times (N_t + 1)$. The snapshot matrix becomes

$$\mathbf{S} = \begin{bmatrix} \mathbf{u}_{\mathcal{N}}(t_0;\mu_1) & \cdots & \mathbf{u}_{\mathcal{N}}(t_{N_t};\mu_1) & \mathbf{u}_{\mathcal{N}}(t_0;\mu_2) & \cdots & \mathbf{u}_{\mathcal{N}}(t_{N_t};\mu_{N_s}) \end{bmatrix}.$$

Hope. Even though the data set is larger, we hope that the dynamics for different parameters are sufficiently similar (e.g., faster/slower versions of the same process). Then a single global basis $\mathbf{V}$ (of size $\mathcal{N} \times n$, $n \ll \mathcal{N}$) can approximate all trajectories for all parameters and all times:

$$\mathbf{u}_{\mathcal{N}}(t;\mu) \approx \mathbf{V}\mathbf{u}_n(t;\mu).$$

Practical issue. The matrix $\mathbf{S}$ can become huge: $N_s = 100$, $N_t = 1000$ gives 10^5 columns. While an SVD is theoretically possible, it may be computationally prohibitive. A pragmatic alternative is to perform POD in two stages:

1. For each parameter μ_j ($j = 1, \ldots, N_s$), compute a POD basis that compresses the *time trajectory* for that fixed parameter. This gives N_s local bases $\mathbf{V}_j$ (each of size $\mathcal{N} \times n_j$, with n_j small).

2. Then collect all these local bases (or the most important modes from each) and perform a second, global SVD to obtain a single basis $\mathbf{V}$ that works for all parameters and all times.

This "POD by chunks" approach is often used in practice. The resulting basis $\mathbf{V}$ is still time-independent and parameter-independent.

7.7.4 Reduced Dynamical System via Galerkin Projection

We insert the approximation $\mathbf{u}_{\mathcal{N}}(t;\mu) \approx \mathbf{V}\mathbf{u}_n(t;\mu)$ into the full order evolution equation and impose the Galerkin condition:

$$\mathbf{V}^T \left(\frac{\partial}{\partial t}(\mathbf{V}\mathbf{u}_n) + \mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}\mathbf{u}_n - \mathbf{f}_{\mathcal{N}}(t;\mu) \right) = \mathbf{0}.$$

Since $\mathbf{V}$ is constant in time, this becomes

$$\frac{d\mathbf{u}_n}{dt}(t;\mu) + \underbrace{\mathbf{V}^T\mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}}_{\mathbf{A}_n(\mu)}\,\mathbf{u}_n(t;\mu) = \underbrace{\mathbf{V}^T\mathbf{f}_{\mathcal{N}}(t;\mu)}_{\mathbf{f}_n(t;\mu)}.$$

Thus we obtain a small linear dynamical system of dimension n:

$$\boxed{\frac{d\mathbf{u}_n}{dt}(t;\mu) + \mathbf{A}_n(\mu)\,\mathbf{u}_n(t;\mu) = \mathbf{f}_n(t;\mu), \qquad \mathbf{u}_n(0;\mu) = \mathbf{V}^T\mathbf{u}^0_{\mathcal{N}}(\mu).}$$

Remarks.

- The reduced operators $\mathbf{A}_n(\mu)$ and $\mathbf{f}_n(t;\mu)$ are assembled using the same affine pre-computation strategy as in the stationary case, provided the problem is affinely parametrized.

- Any time discretization scheme (Backward Euler, etc.) can be applied directly to this reduced system. The computational cost per time step is now $O(n^3)$ (or less, if the system is solved efficiently), instead of $O(\mathcal{N}^3)$.

- The initial condition for the reduced system is simply the projection of the full initial condition onto the reduced space.

7.8 Outlook: Nonaffine and Nonlinear Problems

The above extension to time-dependent problems still relies on the affine assumption. What if the problem is not affine? What if it is nonlinear?

For nonaffine problems, one can try to *recover* an affine representation approximately, e.g., using the Empirical Interpolation Method (EIM). For nonlinear problems, a common approach is to linearize (Newton's method) and then apply RB to each linearized step. However, the Jacobian matrix changes at every Newton iteration and depends on the current solution, which makes the offline/online decomposition much more challenging. These difficulties will be addressed in later chapters, moving towards less intrusive or non-intrusive methods.

7.8.1 Two Strategies for Building a Global POD Basis from Time-Dependent Data

When the number of parameters N_s and the number of time steps N_t become large, performing a single SVD on the full snapshot matrix $\mathbf{S} \in \mathbb{R}^{\mathcal{N}\times(N_s(N_t+1))}$ may be computationally prohibitive. Two practical alternatives are commonly used:

1. **POD of PODs (two-step approach).**

   (a) For each parameter value μ_j ($j = 1, \ldots, N_s$), compute a POD basis $\mathbf{V}_j$ that optimally represents the time trajectory for that fixed parameter. Each $\mathbf{V}_j$ has size $\mathcal{N} \times n_j$ with $n_j \ll \mathcal{N}$.

   (b) Collect all these bases into a matrix $\widehat{\mathbf{V}} = [\mathbf{V}_1\ \mathbf{V}_2\ \cdots\ \mathbf{V}_{N_s}]$ and perform a second (global) SVD on $\widehat{\mathbf{V}}$. The first n left singular vectors of $\widehat{\mathbf{V}}$ form the final global basis $\mathbf{V}$.

   This approach first compresses each trajectory independently, then compresses across parameters. It is conceptually simple and often works well when the dynamics for different parameters are not extremely different.

2. **Progressive (incremental) POD.**

   (a) Start with an empty basis (or with the basis obtained from the first parameter).

   (b) For each new parameter μ_j, compute its trajectory snapshots and perform a POD that also takes into account the previously accumulated basis. This can be done, for example, by appending the new snapshots to the existing basis and then compressing via SVD.

   (c) After processing all parameters, the final basis $\mathbf{V}$ is obtained without an extra global SVD step.

   In practice, the two strategies are roughly equivalent. The first method (POD of PODs) is often simpler to implement and understand.

Optimality. The global POD basis obtained by collecting *all* snapshots (all parameters and all times) into a single snapshot matrix and then performing one SVD is optimal in the following sense: among all subspaces of dimension n, the subspace spanned by the first n left singular vectors minimizes the sum of squared projection errors over the entire data set. Formally, if $\{\sigma_i\}$ are the singular values of the full snapshot matrix, then for any orthonormal basis $\{\psi_i\}_{i=1}^n$ we have

$$\sum_{\text{all snapshots}} \|\text{snapshot} - \sum_{i=1}^{n} \langle \text{snapshot}, \psi_i \rangle \psi_i\|^2 = \sum_{i=n+1}^{N_{\text{snapshots}}} \sigma_i^2,$$

and this minimum is achieved by the POD basis. The two procedures described above (POD of PODs or progressive POD) are simply different computational routes to approximate this optimal basis without storing the entire huge snapshot matrix at once. They process the same data, only the order in which the SVD "sees" the snapshots differs.

7.8.2 Reduced Dynamical System via Galerkin Projection

We now derive the reduced system for a time-dependent problem, taking into account the presence of a mass matrix. The full order model (after spatial discretization) reads:

$$\mathbf{M}_{\mathcal{N}}\frac{d\mathbf{u}_{\mathcal{N}}}{dt}(t;\mu) + \mathbf{A}_{\mathcal{N}}(\mu)\mathbf{u}_{\mathcal{N}}(t;\mu) = \mathbf{f}_{\mathcal{N}}(t;\mu), \qquad \mathbf{u}_{\mathcal{N}}(0;\mu) = \mathbf{u}^0_{\mathcal{N}}(\mu),$$

where $\mathbf{M}_{\mathcal{N}}$ is the (symmetric positive definite) mass matrix. In many cases $\mathbf{M}_{\mathcal{N}}$ is the identity, but we keep it general.

Insert the reduced basis approximation

$$\mathbf{u}_{\mathcal{N}}(t;\mu) \approx \mathbf{V}\mathbf{u}_n(t;\mu),$$

where $\mathbf{V} \in \mathbb{R}^{\mathcal{N}\times n}$ is the global POD basis (constant in time and parameter). Define the residual

$$\mathbf{r}(t;\mu) = \mathbf{f}_{\mathcal{N}}(t;\mu) - \mathbf{M}_{\mathcal{N}}\mathbf{V}\frac{d\mathbf{u}_n}{dt}(t;\mu) - \mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}\mathbf{u}_n(t;\mu).$$

Galerkin orthogonality imposes

$$\mathbf{V}^T\mathbf{r}(t;\mu) = \mathbf{0}.$$

Since $\mathbf{V}$ does not depend on time, we obtain

$$\mathbf{V}^T\mathbf{M}_{\mathcal{N}}\mathbf{V}\frac{d\mathbf{u}_n}{dt}(t;\mu) + \mathbf{V}^T\mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}\mathbf{u}_n(t;\mu) = \mathbf{V}^T\mathbf{f}_{\mathcal{N}}(t;\mu).$$

Thus the reduced dynamical system is

$$\boxed{\mathbf{M}_n(\mu)\frac{d\mathbf{u}_n}{dt}(t;\mu) + \mathbf{A}_n(\mu)\mathbf{u}_n(t;\mu) = \mathbf{f}_n(t;\mu),}$$

with

$$\mathbf{M}_n(\mu) = \mathbf{V}^T\mathbf{M}_{\mathcal{N}}\mathbf{V}, \qquad \mathbf{A}_n(\mu) = \mathbf{V}^T\mathbf{A}_{\mathcal{N}}(\mu)\mathbf{V}, \qquad \mathbf{f}_n(t;\mu) = \mathbf{V}^T\mathbf{f}_{\mathcal{N}}(t;\mu).$$

Initial condition. The reduced initial condition is obtained by projecting the full initial datum onto the reduced space:

$$\mathbf{u}_n(0;\mu) = \mathbf{V}^T\mathbf{M}_{\mathcal{N}}\mathbf{u}^0_{\mathcal{N}}(\mu) \quad \text{(if } \mathbf{M}_{\mathcal{N}} \text{ is not identity, this is the } \mathbf{M}_{\mathcal{N}}\text{-orthogonal projection).}$$

If the POD basis is orthonormal with respect to the Euclidean inner product (and $\mathbf{M}_{\mathcal{N}} = \mathbf{I}$), then simply $\mathbf{u}_n(0;\mu) = \mathbf{V}^T\mathbf{u}^0_{\mathcal{N}}(\mu)$.

Remarks.

- The reduced system has dimension $n \ll \mathcal{N}$ and can be integrated in time using any standard scheme (Backward Euler, Crank–Nicolson, etc.) at a very low cost.
- If the original problem is affinely parametrized, the reduced operators $\mathbf{A}_n(\mu)$ and $\mathbf{f}_n(t;\mu)$ can be pre-computed offline using the same affine decomposition as in the stationary case. The reduced mass matrix $\mathbf{M}_n$ is constant (if $\mathbf{M}_{\mathcal{N}}$ does not depend on μ).
- The same approach extends naturally to second-order systems (e.g., structural dynamics) by rewriting them as first-order systems or by applying Galerkin projection directly to the second-order form.

7.9 Greedy Algorithms and Error Estimation (A Cultural Note)

An alternative to POD for constructing reduced bases is the *greedy algorithm*. The idea is to build the reduced space incrementally by selecting, at each step, the parameter value where the current reduced model yields the largest error (compared to the full order solution). One then adds the corresponding full order solution (or its orthogonal component) to the basis.

Why "greedy"? At each step we make a locally optimal choice: we pick the parameter that most reduces the error. We hope that by repeatedly making the best local improvement, we eventually obtain a globally good basis. This is analogous to building a sports team by always picking the strongest available player.

Illustration (1D parameter space). Suppose we have a reduced basis of dimension one (a single basis function). That basis will be excellent near the parameter value where the snapshot was taken, but its approximation quality deteriorates as we move away. The error (difference between the full order solution and the reduced approximation) as a function of the parameter μ is depicted by a curve with peaks. The first peak (the worst error) indicates where we should take the next snapshot. After adding that snapshot to the basis (now dimension two), the error reduces globally, and new peaks may appear elsewhere. Repeating this process, we eventually drive the error below a desired threshold over the whole parameter domain. Each peak corresponds to a parameter value where we have sampled a snapshot.

The main obstacle: how to obtain the error curve? To know where the error is large, we would need to compute the actual error, which in turn requires the full order solution — exactly what we want to avoid. Thus, a practical greedy algorithm requires an *a posteriori error estimator*: a computable quantity that approximates the true error without solving the full order problem. This estimator must be evaluated very quickly, because at each greedy step we may need to scan many candidate parameter values.

Historical context. About 10–15 years ago, for linear stationary (and sometimes linear time-dependent) problems, it was possible to derive rigorous and efficient error bounds. Greedy algorithms were successfully used to build reduced bases with the property that to obtain a basis of dimension n, one only needed to call the full order solver n times (plus the cost of the error estimator). However, for more complex problems (non-affine, nonlinear, or non-stationary with complex dynamics), obtaining such error estimators is extremely difficult, often impossible. Consequently, the reduced basis community has largely moved away from greedy sampling for such problems, although the method remains of historical and methodological interest.

7.10 Local Bases and the Bottleneck of Global Linear Techniques

A POD basis is *linear* (the approximation is a linear combination of basis functions) and *global* (the same basis is used for all parameter values and all times). One might ask: could we instead use different bases in different regions of the parameter space or at different time intervals? Yes, but then we need a criterion to decide when to switch bases. If we must choose the appropriate basis automatically, additional computations and checks are required. This is one of the bottlenecks of classical linear reduced order modeling techniques. In the second part of the course, we will explore alternatives that overcome these limitations.

7.11 Hints on Non-Affine Problems

So far we have assumed that the parameter dependence is affine, i.e.,

$$\mathbf{A}_{\mathcal{N}}(\mu) = \sum_{q=1}^{Q_a} \Theta_a^q(\mu)\mathbf{A}_{\mathcal{N}}^q, \qquad \mathbf{f}_{\mathcal{N}}(\mu) = \sum_{q=1}^{Q_f} \Theta_f^q(\mu)\mathbf{f}_{\mathcal{N}}^q.$$

This property is essential for the offline/online decomposition. But what if the problem is not affine? A typical example is an advection–diffusion problem with a source that moves around the domain, parameterized by its location (e.g., μ_3, μ_4) and shape (μ_5). Then the forcing vector $\mathbf{f}_{\mathcal{N}}(\mu)$ cannot be written as a linear combination of a few parameter-independent vectors with parameter-dependent coefficients. The same difficulty arises if the diffusion coefficient or the advection field depends non-affinely on the parameters.

7.11.1 Idea: Approximate the Non-Affine Object by an Affine Expansion

If we could restore affinity, i.e., approximate the non-affine term by a sum of separable functions, then we could reuse the efficient offline/online procedure. This leads to the concept of *hyper-reduction*: we need to reduce not only the solution but also the problem itself. One approach is to apply POD to the non-affine term itself.

For example, consider a parameter-dependent vector $\mathbf{g}(\mu)$ (e.g., the discretized source term). We can collect snapshots of $\mathbf{g}$ for different parameter values:

$$\mathbf{g}(\mu_1), \mathbf{g}(\mu_2), \ldots, \mathbf{g}(\mu_{N_s}).$$

Then we perform POD on these snapshots to obtain a basis $\mathbf{W}$ (of small dimension m) that approximates $\mathbf{g}(\mu)$:

$$\mathbf{g}(\mu) \approx \mathbf{W}\mathbf{g}_m(\mu),$$

where $\mathbf{g}_m(\mu)$ is the vector of reduced coefficients. Plugging this approximation into the full order model restores an affine structure, because now

$$\mathbf{f}_{\mathcal{N}}(\mu) \approx \sum_{i=1}^{m} (\mathbf{g}_m(\mu))_i \, \mathbf{w}_i,$$

with $\mathbf{w}_i$ the i-th column of $\mathbf{W}$. The reduced operators can then be pre-computed as $\mathbf{V}^T\mathbf{w}_i$ and so on.

What about nonlinear problems? In nonlinear problems, the operators depend on the solution itself. For instance, a Newton iteration linearizes the problem around the current iterate, leading to a sequence of linear systems whose Jacobian matrix changes at each step. Even if the original problem is affinely parametrized, the linearized systems are generally not affine in the parameters, and they also depend on the current solution. Hyper-reduction becomes even more involved. These difficulties motivate the less intrusive (or non-intrusive) methods that we will discuss after Easter.

7.12 Hyper-Reduction for Non-Affine Problems: Empirical Interpolation and Its Challenges

7.12.1 Restoring Affinity by Approximating the Non-Affine Term

Consider a non-affine parameter-dependent vector $\mathbf{g}(\mu)$ – for example, the discretized source term $\mathbf{f}_{\mathcal{N}}(\mu)$ in the full order model. We cannot write $\mathbf{g}(\mu)$ as a linear combination of a few parameter-independent vectors with parameter-dependent coefficients. However, we can try to *approximate* it by such a combination.

Step 1: Collect snapshots. Compute $\mathbf{g}(\mu)$ for a set of parameter values $\mu_1, \ldots, \mu_{N_s}$ and assemble the snapshot matrix

$$\mathbf{G} = \begin{bmatrix} \mathbf{g}(\mu_1) & \mathbf{g}(\mu_2) & \cdots & \mathbf{g}(\mu_{N_s}) \end{bmatrix}.$$

Step 2: Build a POD basis for g. Perform an SVD on $\mathbf{G}$ and retain the first m left singular vectors. Let $\mathbf{Q} \in \mathbb{R}^{\mathcal{N} \times m}$ be this basis (the notation $\mathbf{Q}$ is used in the slides; sometimes $\mathbf{\Psi}$). Then any $\mathbf{g}(\mu)$ can be approximated as

$$\mathbf{g}(\mu) \approx \mathbf{Q}\,\boldsymbol{\gamma}(\mu),$$

where $\boldsymbol{\gamma}(\mu) \in \mathbb{R}^m$ is a vector of coefficients. This is exactly analogous to approximating the solution $\mathbf{u}_{\mathcal{N}}(\mu) \approx \mathbf{V}\mathbf{u}_n(\mu)$.

7.12.2 Determining the Coefficients via Interpolation (Empirical Interpolation)

Unlike the solution, we have no differential equation to determine $\boldsymbol{\gamma}(\mu)$. The standard approach is *interpolation*: we require that the approximation $\mathbf{Q}\boldsymbol{\gamma}(\mu)$ matches the exact $\mathbf{g}(\mu)$ at a set of m selected points in the physical domain (or at m selected degrees of freedom). Let $\{\mathbf{y}_1, \ldots, \mathbf{y}_m\}$ be these interpolation points (each $\mathbf{y}_i$ is a spatial location, i.e., a row index in the discrete vector). The interpolation conditions are:

$$\big(\mathbf{Q}\boldsymbol{\gamma}(\mu)\big)\big|_{\mathbf{y}_i} = \mathbf{g}(\mu)\big|_{\mathbf{y}_i}, \qquad i = 1, \ldots, m.$$

In matrix form, if we define the *interpolation matrix* $\mathbf{P} \in \mathbb{R}^{m \times \mathcal{N}}$ that extracts the values at the m points, then

$$\mathbf{P}\mathbf{Q}\,\boldsymbol{\gamma}(\mu) = \mathbf{P}\mathbf{g}(\mu).$$

Assuming $\mathbf{PQ}$ is invertible (which can be ensured by a greedy selection of points, e.g., the Empirical Interpolation Method, EIM), we obtain

$$\boldsymbol{\gamma}(\mu) = (\mathbf{P}\mathbf{Q})^{-1}\mathbf{P}\mathbf{g}(\mu).$$

7.12.3 Hyper-Reduction: Two Levels of Reduction

Once we have the affine approximation

$$\mathbf{g}(\mu) \approx \mathbf{Q}\boldsymbol{\gamma}(\mu) = \sum_{j=1}^{m} \gamma_j(\mu)\,\mathbf{q}_j,$$

we can plug it into the full order model. For instance, if the non-affine term appears on the right-hand side, the reduced system becomes

$$\mathbf{A}_n(\mu)\mathbf{u}_n(\mu) = \mathbf{V}^T\mathbf{g}(\mu) \approx \sum_{j=1}^{m} \gamma_j(\mu) \underbrace{\mathbf{V}^T\mathbf{q}_j}_{\text{pre-computable}}.$$

Thus the online cost for each new μ includes:

- Evaluating $\boldsymbol{\gamma}(\mu)$: this requires solving an $m \times m$ linear system (or applying the pre-computed inverse matrix) and evaluating $\mathbf{P}\mathbf{g}(\mu)$ – the latter involves only m entries of the original full order vector, which is cheap if we can compute those entries without assembling the whole $\mathbf{g}(\mu)$.
- Then forming the reduced right-hand side as a linear combination of pre-computed vectors.

7.12.4 The Cost–Accuracy Trade-Off and Why Hyper-Reduction Is Often Impractical

The problem of large m. To accurately represent a complex non-affine function (e.g., a moving heat source or a spatially varying coefficient), the number of basis functions m needed may be large. In the worst case, m can be as large as $\mathcal{N}$ (the full order dimension), which defeats the purpose of reduction. Even when m is moderate (say, a few hundred), solving an $m \times m$ linear system for every new parameter μ can become the dominant cost – sometimes larger than solving the original reduced system of size n (where $n \ll m$). We end up paying more for the "preparatory" reduction than for the actual reduced order model.

Error accumulation. The approximation error introduced by representing $\mathbf{g}(\mu)$ with a low-rank basis $\mathbf{Q}$ will inevitably affect the final solution. The reduced basis method cannot "cure" inaccuracies in the data; they will propagate and possibly dominate. Thus we must choose m large enough to keep the interpolation error small, but that brings back the high cost.

Multiple non-affine terms. Realistic problems often have several non-affine ingredients: the diffusion tensor, the advection field, the source term, and possibly coefficients in the time derivative. Each of them would require its own basis $\mathbf{Q}_k$ and its own set of interpolation points, multiplying the overhead. For nonlinear problems, the Jacobian matrix in Newton's method changes at every iteration, and each linearization would demand a fresh hyper-reduction – a truly daunting task.

Why the community has moved away. For about a decade, empirical interpolation and its variants (e.g., DEIM, GNAT) were actively researched. However, the practical difficulties (choosing good interpolation points, controlling errors, high online cost for large m, and the lack of rigorous error bounds for complex problems) have led most practitioners to seek alternative paradigms. In many cases, the overhead of hyper-reduction outweighs its benefits, especially when the problem is only mildly non-affine or when the parameter space is low-dimensional.

7.13 Conclusion and Outlook

- For non-affine problems, one can *approximately restore affinity* by building a POD basis for the non-affine term and using interpolation (EIM) to determine the coefficients. This is called hyper-reduction.
- However, the cost of solving an $m \times m$ system at each parameter query, and the need for a sufficiently large m to control errors, often makes hyper-reduction as expensive as (or more expensive than) the original reduced order model.
- For this reason, the reduced basis community has largely abandoned hyper-reduction for complex applications and is instead exploring data-driven, non-intrusive, and scientific machine learning approaches.
- The slides of set number six provide further details on empirical interpolation, but we will not rely on these techniques in the remainder of the course. Instead, after Easter, we will introduce modern alternatives that overcome the limitations of classical linear reduction and hyper-reduction.

Chapter 8

Some Hints on Nonaffine Problems and Anover View on Projection-based Reduced Order Models

8.1 Overview and Scope of Today's Lecture

The goal of today's lecture is to provide a clear wrap-up of the material covered so far. We still need to discuss other classes of problems beyond those introduced last Friday. In particular, we focus on problems in which the dependence on the parameters is *non-affine*. Non-stationary problems were already treated in detail last Friday.

Regarding *nonlinear* problems, we will spend only a few words, since they are substantially more complex than non-affine ones. Importantly, the strategies required to tackle non-affine problems are essentially the same techniques that the reduced-order modelling community has employed until recently (a few years ago) to address nonlinear problems as well. Nonlinear problems continue to represent a major challenge.

Remark 8.1 (Slide sidebar)**.** *The content on the side of the slide is a compact summary of the first half of the semester. We highlight the key points in order to obtain a global overview of the topics we have developed.*

8.2 Full-Order Models and the Starting Point

We began the course by considering problems that become *very large* after discretization. Regardless of the chosen technique (finite elements, finite volumes, finite differences, spectral methods, etc.), the numerical solution of a linear stationary differential problem in two or three spatial dimensions typically leads to a linear algebraic system whose dimension N_h can easily reach millions of unknowns when high accuracy on fine meshes is required.

As a concrete example, think of a linear elasticity problem posed on a complex structure. All solutions of the *parametrized* problem do not generically span the entire high-dimensional finite-element space. Because the solutions share a common dependence on a (possibly vector-valued) parameter $\boldsymbol{\mu}$, they actually lie on a much lower-dimensional set, which we call the *solution manifold*:

$$\mathcal{M} = \left\{ \mathbf{u}_h(\boldsymbol{\mu}) \;\middle|\; \boldsymbol{\mu} \in \mathcal{P} \right\} \subset \mathbb{R}^{N_h}.$$

For simplicity, assume a scalar parameter $\mu \in \mathcal{P} \subset \mathbb{R}$. Then each solution $\mathbf{u}_h(\mu)$ can be viewed as a point lying on the manifold $\mathcal{M}$.

8.3 The Reduced-Basis Idea

The central idea is to exploit this low-dimensional structure:

1. Compute a set of *snapshots* (full-order solutions) *offline* for a carefully chosen set of parameter values $\{\mu_i\}_{i=1}^{N_s}$. The choice of sampling points and the number of snapshots was discussed in detail last week.

2. Use these snapshots to construct a *reduced basis space* $V_N \subset \mathbb{R}^{N_h}$, i.e., a low-dimensional linear subspace spanned by a small number $N \ll N_h$ of basis vectors. In matrix form we write

$$V = [\mathbf{v}_1, \ldots, \mathbf{v}_N] \in \mathbb{R}^{N_h \times N},$$

where the columns $\mathbf{v}_i$ are the selected basis functions (or POD modes, as we shall see later).

Instead of searching for the true solution along the (generally curved) black manifold, we now seek an approximation that lies inside the blue subspace V_N. The corresponding approximate solution traces a red curve that stays within V_N. The richer the basis (larger N), the closer the red curve approximates the true manifold.

8.4 The Two Pillars of the Reduced-Basis Method

Any reduced-basis procedure rests on two fundamental ingredients:

1. **Construction of the reduced space** V_N (offline phase).

2. **Generation of the reduced solution** for any new parameter value $\mu \notin \{\mu_i\}$ (online phase).

The key assumption underlying the whole approach is that every full-order solution can be represented (to a good accuracy) as a linear combination of the basis vectors:

$$\mathbf{u}_h(\boldsymbol{\mu}) \;\approx\; V\,\mathbf{u}_N(\boldsymbol{\mu}),$$

where $\mathbf{u}_N(\boldsymbol{\mu}) \in \mathbb{R}^N$ is the vector of reduced coefficients we must compute.

8.5 Galerkin Projection and the Reduced Linear System

Suppose the full-order (high-fidelity) problem reads

$$A_h(\boldsymbol{\mu})\,\mathbf{u}_h(\boldsymbol{\mu}) = \mathbf{f}_h(\boldsymbol{\mu}) \qquad \text{in } \mathbb{R}^{N_h}.$$

Substituting the reduced-basis ansatz and enforcing that the residual is orthogonal to the reduced space (Galerkin orthogonality) yields

$$V^T\big(A_h(\boldsymbol{\mu})\,V\,\mathbf{u}_N(\boldsymbol{\mu}) - \mathbf{f}_h(\boldsymbol{\mu})\big) = \mathbf{0}.$$

Rearrangement immediately gives the *reduced-order system*

$$\underbrace{\big(V^T A_h(\boldsymbol{\mu})\,V\big)}_{A_N(\boldsymbol{\mu})\in\mathbb{R}^{N\times N}}\mathbf{u}_N(\boldsymbol{\mu}) = \underbrace{V^T\mathbf{f}_h(\boldsymbol{\mu})}_{\mathbf{f}_N(\boldsymbol{\mu})\in\mathbb{R}^N}. \tag{8.1}$$

Equation (8.1) is a linear system of size $N \times N$ (independent of the huge dimension N_h) that must be solved for each new query parameter $\boldsymbol{\mu}$.

The matrix $A_N(\boldsymbol{\mu})$ is assembled from the full-order operator via the basis V, while the right-hand side $\mathbf{f}_N(\boldsymbol{\mu})$ is the projection of the full-order forcing term. This construction is precisely the *Galerkin projection* onto the reduced space.

We also discussed efficient assembly strategies (avoiding repeated expensive matrix–matrix products $V^T A_h V$) when the operators admit an *affine* parametric dependence. We will return to the non-affine case shortly.

8.6 Extension to Time-Dependent Problems

For non-stationary (time-dependent) problems the full-order model takes the more general form

$$M_h\,\dot{\mathbf{u}}_h(t,\boldsymbol{\mu}) + A_h(\boldsymbol{\mu})\,\mathbf{u}_h(t,\boldsymbol{\mu}) + \mathbf{F}_h\big(\mathbf{u}_h(t,\boldsymbol{\mu})\big) = \mathbf{f}_h(t,\boldsymbol{\mu}),$$

where M_h is the mass matrix associated with the time derivative. After reduction the same Galerkin procedure produces an analogous low-dimensional ODE system in the coefficients $\mathbf{u}_N(t,\boldsymbol{\mu})$.

8.7 Construction of the Reduced Basis: Proper Orthogonal Decomposition (POD)

A particularly powerful way to build the reduced space V_N is the *Proper Orthogonal Decomposition* (POD). Given a set of snapshots, POD furnishes the *optimal* orthonormal basis of dimension N that minimises the projection error onto the snapshot set (in the L^2 sense). This is exactly the mathematical justification for the linear-combination assumption that underpins the reduced-basis method.

8.8 Hyper-Reduction: Handling Non-Affine and Nonlinear Problems

So far we have assumed that the operators $A_h(\boldsymbol{\mu})$ and $\mathbf{f}_h(\boldsymbol{\mu})$ are *affine* in $\boldsymbol{\mu}$, i.e., they can be written as a sum of parameter-independent matrices/vectors weighted by functions of $\boldsymbol{\mu}$. When this assumption fails (non-affine dependence) or when the problem is genuinely nonlinear, the reduced system cannot be assembled efficiently.

Consider a nonlinear term $\mathbf{F}_h(\mathbf{u}_h)$. After reduction we obtain

$$V^T\mathbf{F}_h\big(V\,\mathbf{u}_N\big),$$

which still requires an expensive evaluation of the full-order nonlinear function at every time step / every new $\boldsymbol{\mu}$. The same difficulty arises for a non-affine linear operator $A_h(\boldsymbol{\mu})$ that cannot be pre-factorised.

The remedy is a family of techniques collectively called *hyper-reduction*. The idea is to approximate the troublesome (nonlinear or non-affine) operator itself by a separate low-dimensional expansion, e.g., via the Discrete Empirical Interpolation Method (DEIM) or its variants. In practice one builds an auxiliary basis for the nonlinear snapshots, then uses interpolation to evaluate the projected nonlinear term with a cost independent of N_h.

These hyper-reduction strategies were introduced at the end of last Friday's lecture for non-affine problems; exactly the same toolbox is employed (and was historically developed) to treat nonlinear problems.

8.8.1 Additional Challenges for Nonlinear Problems

That was the last brick we introduced last Friday. However, applying a technique such as the Discrete Empirical Interpolation Method (DEIM) becomes considerably more complicated for genuinely nonlinear problems. The vector being approximated now depends on the solution itself, which introduces a strong coupling that must be treated at every iteration or time step.

We will not go into further technical details here. If you are curious, the forthcoming slides contain some additional information, but the procedure quickly becomes very technical. For large-scale problems with strong nonlinearities, hyper-reduction can become unaffordable. This is precisely why roughly half of the course will be devoted to *non-intrusive* techniques. These methods try to overcome the difficulty by introducing alternative strategies for model construction and reuse, avoiding the expensive assembly of projected nonlinear terms altogether.

8.9 Offline–Online Decomposition for Linear Stationary Affine Problems

For the simplest class we have studied—linear stationary problems with *affine* parametric dependence—the reduced-basis method enjoys a perfect offline–online splitting:

- **Offline phase** (expensive, performed once):
  1. Generate snapshots $\mathbf{u}_h(\mu_i)$ for selected $\mu_i \in \mathcal{P}$.
  2. Construct the reduced basis matrix V (e.g., via POD).
  3. Precompute the parameter-independent high-fidelity matrices/vectors A_h^q and $\mathbf{f}_h^q$ (the "affine components").
  4. Project them once and for all:

$$A_N^q := V^T A_h^q V \in \mathbb{R}^{N\times N}, \qquad \mathbf{f}_N^q := V^T \mathbf{f}_h^q \in \mathbb{R}^N.$$

  (Here we assume a Galerkin method, so the test space equals the trial space: $W = V$.)

- **Online phase** (very inexpensive, for every new μ):
  1. Assemble the reduced system in $\mathcal{O}(QN^2)$ operations (independent of N_h):

$$A_N(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, A_N^q, \qquad \mathbf{f}_N(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, \mathbf{f}_N^q.$$

  2. Solve the small $N \times N$ system for $\mathbf{u}_N(\boldsymbol{\mu})$.
  3. Recover the high-fidelity solution (if needed for visualisation or post-processing):

$$\mathbf{u}_h(\boldsymbol{\mu}) \;\approx\; V\, \mathbf{u}_N(\boldsymbol{\mu}).$$

This workflow is exactly what we have developed over the past few weeks.

8.10 Mathematical Foundations: Singular Value Decomposition (SVD) and POD

The construction of the reduced space relies on the *Singular Value Decomposition* (SVD). Any matrix $A \in \mathbb{R}^{m\times n}$ admits the factorisation

$$A = U\,\Sigma\,V^T,$$

where U and V are orthogonal, and Σ is diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$ (ordered decreasingly). The singular values quantify the "energy" or information content carried by each mode.

The Eckart–Young–Mirsky theorem states that the best rank-k approximation of A (in the Frobenius or spectral norm) is obtained by truncating the SVD after the first k terms:

$$A_k = U_k\,\Sigma_k\,V_k^T.$$

The approximation error satisfies

$$\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2.$$

Proper Orthogonal Decomposition (POD) is simply the application of the SVD to the *snapshot matrix*

$$S = [\mathbf{u}_h(\mu_1), \ldots, \mathbf{u}_h(\mu_{N_s})] \in \mathbb{R}^{N_h\times N_s}.$$

The left singular vectors (columns of U) provide the optimal reduced basis V (the first N columns). The projection of any snapshot onto this basis is

$$\mathbf{u}_h(\mu_i) \;\approx\; VV^T\mathbf{u}_h(\mu_i),$$

and the total projection error over the snapshot set is exactly the sum of the squares of the neglected singular values. This is also known in the literature as the *method of snapshots*.

The POD basis is *optimal*: among all linear subspaces of dimension N, it minimises the mean-square projection error on the training data.

8.11 The Affine Assumption: The Third Pillar of Efficiency

A favourable situation occurs when the high-fidelity operators admit an affine parametric dependence:

$$A_h(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, A_h^q, \qquad \mathbf{f}_h(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, \mathbf{f}_h^q,$$

where the matrices A_h^q and vectors $\mathbf{f}_h^q$ are *parameter-independent*. Then the reduced quantities become

$$A_N(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, (V^T A_h^q V), \qquad \mathbf{f}_N(\boldsymbol{\mu}) = \sum_{q=1}^{Q} \theta_q(\boldsymbol{\mu})\, (V^T \mathbf{f}_h^q),$$

exactly as shown in the offline–online decomposition above. The small blocks $V^TA_h^qV$ and $V^T\mathbf{f}_h^q$ can be stored once and for all.

Together with (i) POD basis construction and (ii) Galerkin projection, the affine assumption forms the *third pillar* that makes the reduced-basis method computationally efficient. Without any one of these three ingredients, the online phase loses its independence from N_h and the method becomes far less attractive.

8.12 A Non-Affine Example and Hyper-Reduction for the Right-Hand Side

To illustrate a non-affine situation, consider the following advection–diffusion–reaction problem on the unit square:

$$-\mu_1 \Delta u + \mathbf{b}(\mu_2)\cdot\nabla u + u = f(\boldsymbol{\mu}) \quad \text{in } \Omega,$$

with homogeneous Dirichlet boundary conditions. Here:

- $\mu_1 > 0$: diffusion coefficient,
- $\mu_2 \in [0, 2\pi)$: angle defining the constant advection field $\mathbf{b}(\mu_2) = (\cos\mu_2, \sin\mu_2)$,
- μ_3, μ_4: coordinates of the centre of a Gaussian source,
- $\mu_5 > 0$: spreading parameter of the source

$$f(\boldsymbol{\mu}; x, y) = \exp\!\Big(-\frac{(x-\mu_3)^2 + (y-\mu_4)^2}{\mu_5^2}\Big).$$

The reaction term is parameter-independent, but the source term $f(\boldsymbol{\mu})$ is clearly non-affine because its location and width change with $\boldsymbol{\mu}$. Consequently, the right-hand side vector $\mathbf{f}_h(\boldsymbol{\mu})$ cannot be expressed as a short affine sum.

Since we can evaluate the source analytically, the remedy is to apply the same dimensionality-reduction idea used for the solution snapshots:

1. Sample the forcing term at selected parameter values and collect snapshots of $\mathbf{f}_h(\mu_i)$.

2. Compute the SVD of this forcing-snapshot matrix and obtain a low-dimensional basis Φ (again via POD).

3. Approximate $\mathbf{f}_h(\boldsymbol{\mu}) \approx \Phi\, \mathbf{c}(\boldsymbol{\mu})$ using interpolation (exactly the same spirit as DEIM).

This is analogous to classical polynomial interpolation. Instead of working directly with a complicated function, we seek an approximation that lives in a low-dimensional span of basis functions (monomials, or here the POD modes of the forcing). Once the basis Φ is built, we only need to determine a small number of coefficients $\mathbf{c}(\boldsymbol{\mu})$ at runtime—again at cost independent of N_h.

8.13 Classical Polynomial Interpolation as Motivation for Hyper-Reduction

The same idea that underlies hyper-reduction is already familiar from elementary polynomial interpolation in one dimension. Suppose we wish to represent a complicated scalar function $f(x)$ in a compact way as

$$P_m(f)(x) = \sum_{j=0}^{m} \gamma_j\, \rho_j(x),$$

where $\{\rho_j(x)\}_{j=0}^m$ is a known basis (e.g., the monomials $1, x, x^2, \ldots, x^m$) and the γ_j are unknown coefficients.

To determine the $m+1$ coefficients we impose $m+1$ interpolation conditions at carefully chosen points $\{x_0, x_1, \ldots, x_m\}$:

$$P_m(f)(x_i) = f(x_i), \qquad i = 0, \ldots, m.$$

This produces the linear system

$$B_m\, \boldsymbol{\gamma} = \mathbf{f}_m,$$

where

$$(B_m)_{i,j} = \rho_j(x_i), \qquad (\mathbf{f}_m)_i = f(x_i), \qquad \boldsymbol{\gamma} = (\gamma_0, \ldots, \gamma_m)^T.$$

Solving this $(m+1) \times (m+1)$ system yields the coefficients that allow us to evaluate the approximant everywhere.

In the reduced-basis context the situation is analogous, but we work in high-dimensional spaces (N_h degrees of freedom) and the functions also depend on a parameter $\boldsymbol{\mu}$. The basis $\{\boldsymbol{\rho}_j\}$ is no longer universal (monomials); instead it is constructed *empirically* from the data itself via POD. The interpolation points become a small set of carefully selected *degrees of freedom* (or mesh entities) in the original finite-element mesh.

8.14 The Discrete Empirical Interpolation Method (DEIM)

The algebraic counterpart of the interpolation procedure above is the *Discrete Empirical Interpolation Method* (DEIM), the most common hyper-reduction technique.

Let $\mathbf{g}(\boldsymbol{\mu}) \in \mathbb{R}^{N_h}$ be a generic vector (e.g., the assembled right-hand side or a nonlinear term) that we wish to approximate cheaply. We proceed in two steps:

1. **Basis construction (offline)**: Compute snapshots $\{\mathbf{g}(\mu_i)\}_{i=1}^{N_s}$ and apply POD to obtain an orthonormal basis matrix

   $$\Phi = [\boldsymbol{\rho}_1, \ldots, \boldsymbol{\rho}_M] \in \mathbb{R}^{N_h \times M},$$

   where $M \ll N_h$ is chosen according to the decay of the singular values.

2. **Interpolation (online)**: Approximate

   $$\mathbf{g}(\boldsymbol{\mu}) \;\approx\; \Phi\, \boldsymbol{\gamma}(\boldsymbol{\mu}).$$

The coefficients $\boldsymbol{\gamma}(\boldsymbol{\mu}) \in \mathbb{R}^M$ are determined by enforcing exact matching at M selected components (the so-called *magic points* or interpolation indices $\mathcal{I} \subset \{1, \ldots, N_h\}$):

$$\mathbf{P}^T \mathbf{g}(\boldsymbol{\mu}) = \mathbf{P}^T \Phi \, \boldsymbol{\gamma}(\boldsymbol{\mu}),$$

where $\mathbf{P} \in \mathbb{R}^{N_h \times M}$ is the selection matrix whose columns are the canonical basis vectors $\mathbf{e}_{i_k}$ for $i_k \in \mathcal{I}$. Solving the small $M \times M$ system

$$B \, \boldsymbol{\gamma}(\boldsymbol{\mu}) = \mathbf{g}_{\mathcal{I}}(\boldsymbol{\mu}), \qquad B = \mathbf{P}^T \Phi$$

yields the required coefficients. The matrix B is precomputed offline and is independent of $\boldsymbol{\mu}$.

The projected vector that enters the reduced system is then

$$V^T \mathbf{g}(\boldsymbol{\mu}) \;\approx\; V^T \Phi \, \boldsymbol{\gamma}(\boldsymbol{\mu}),$$

which can be evaluated at cost $\mathcal{O}(MN)$ (independent of N_h).

8.15 Greedy Selection of the Magic Points

The only missing ingredient is how to choose the M interpolation indices $\mathcal{I}$. A very effective strategy is the following greedy algorithm (the heart of DEIM):

1. Compute the POD basis Φ from the snapshots of $\mathbf{g}(\boldsymbol{\mu})$.
2. Initialise $\mathcal{I} = \emptyset$ and B as an empty matrix.
3. For $k = 1$ to M:
   (a) Let $\mathbf{r}(\boldsymbol{\mu})$ be the residual of the current $(k-1)$-term interpolant evaluated on the snapshot set.
   (b) Find the mesh index i_k that maximises $|\mathbf{r}(\boldsymbol{\mu})|$ (or the maximum over the training set).
   (c) Add i_k to $\mathcal{I}$ and augment B with the new column $\boldsymbol{\rho}_k$ restricted to the previous indices.

This procedure is computationally cheap (it only involves the small POD basis) and automatically selects the degrees of freedom that are most sensitive to parameter variations—exactly the locations where the approximation error would otherwise be largest.

8.16 Practical Considerations: Balancing Interpolation and Reduction Errors

Let N be the dimension of the solution reduced basis and M the number of DEIM terms (hyper-reduction dimension). The total error in the reduced solution is bounded by the sum of

- the *reduction error* (how well the POD basis approximates the solution manifold),
- the *interpolation error* (how well DEIM approximates the non-affine or nonlinear term).

If M is chosen too small, the interpolation error dominates and increasing N (adding more solution modes) cannot compensate for it. The error curves flatten or even deteriorate. Conversely, a very accurate hyper-reduction (M large) makes the online cost grow. A good practical rule is to choose M large enough that the interpolation error is negligible compared with the reduction error.

This balance is one of the crucial bottlenecks of hyper-reduction and explains why, for strongly nonlinear problems, the cost can become prohibitive.

8.17 Conclusion and Outlook: Where Do We Go from Here?

So far we have explored *linear* dimensionality-reduction techniques:

- Snapshots $\rightarrow$ POD basis (linear).
- Galerkin projection onto that basis.
- (Hyper-reduction via DEIM when needed.)

This toolbox (POD + Galerkin $\pm$ DEIM) is extremely powerful for many affine or mildly non-affine problems, but it has limitations:

- The POD basis may require a very large N (or may never be accurate enough) for problems with strong transport, sharp fronts, or highly varying parameter dependence.
- Hyper-reduction becomes expensive or even unaffordable for strong nonlinearities.

In the second half of the course we will keep the same mathematical rigour while exploring more powerful alternatives:

1. **Still linear reduction, different coefficient recovery**: Keep the POD basis but replace Galerkin projection by interpolation, regression, Gaussian processes, or neural networks (POD + NN).
2. **Nonlinear dimensionality reduction**: Replace POD altogether by (nonlinear) autoencoders. The encoder learns a low-dimensional latent space; the decoder reconstructs the high-fidelity solution. This opens the door to *deep-learning-based reduced-order models*.
3. **Time-dependent problems revisited**:
   - Treat time as an additional parameter (losing causality).
   - Or learn the *reduced dynamics* directly (neural ODEs, operator inference, etc.).

   The latter also connects naturally to system identification: given only time snapshots of a dynamical system (e.g., a pendulum), recover the underlying governing equations without knowing them a priori.

All these new bricks are built from the same LEGO blocks we already know (data generation from the finite-element code, linear algebra, projection ideas, interpolation). We will simply combine them in more flexible and powerful ways.

After the next lab we will have a dedicated crash-course on neural networks so that everyone is on the same page. Then we will put our hands on real implementations in the following labs.

Happy Easter, enjoy the spring break, and see you in about ten days!

Chapter 9

Lab 4 - Reduced basis (POD-Galerkin) for time-dependent problems

9.1 Introduction

Today we continue the first module on reduced basis methods, now addressing time-dependent problems. After this, we will move to data-driven methods (neural networks). A few organizational notes:

- Next Wednesday's lecture is skipped (you will receive an official notification).
- On Friday, Professor Mazzoni will hold a lab session introducing neural networks.
- The assignments for the first part will be presented at the end of today's lecture – a PDF with the code will be available tonight.

9.2 Time-Dependent Parametric PDEs

Consider a parabolic problem (e.g., unsteady heat equation) depending on parameters $\boldsymbol{\mu}$:

$$\begin{cases} \dfrac{\partial u}{\partial t}(t, x; \boldsymbol{\mu}) + \mathcal{A}(\boldsymbol{\mu}) u(t, x; \boldsymbol{\mu}) = f(t, x; \boldsymbol{\mu}), & (t, x) \in (0, T] \times \Omega, \\ u(0, x; \boldsymbol{\mu}) = u_0(x; \boldsymbol{\mu}), & x \in \Omega, \\ \text{boundary conditions.} \end{cases}$$

Here $\mathcal{A}(\boldsymbol{\mu})$ is a spatial differential operator depending on $\boldsymbol{\mu}$. Example: $\mathcal{A}(\boldsymbol{\mu}) = -\nabla \cdot (\sigma(\boldsymbol{\mu})\nabla)$, with parametrized conductivity σ.

9.2.1 Full-Order Discretization

Spatial discretization by finite elements leads to a system of ODEs:

$$\mathbf{M}\dot{\mathbf{u}}_h(t; \boldsymbol{\mu}) + \mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h(t; \boldsymbol{\mu}) = \mathbf{F}_h(t; \boldsymbol{\mu}), \qquad \mathbf{u}_h(0) = \mathbf{u}_{0,h},$$

where:

- $\mathbf{M}$ is the mass matrix (from $\int vu\, dx$),
- $\mathbf{A}_h(\boldsymbol{\mu})$ is the stiffness matrix,
- $\mathbf{F}_h$ is the load vector.

Time discretization (e.g., backward Euler) with time step Δt gives:

$$\frac{\mathbf{M}}{\Delta t}\mathbf{u}_h^{j+1} + \mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h^{j+1} = \frac{\mathbf{M}}{\Delta t}\mathbf{u}_h^{j} + \mathbf{F}_h^{j+1},$$

where $\mathbf{u}_h^j \approx \mathbf{u}_h(t_j)$.

9.2.2 Reduced Basis (POD-Galerkin) Formulation

We construct a POD basis $\mathbf{V} \in \mathbb{R}^{N_h \times n}$ from snapshots (solutions at various times and parameter values). The reduced solution is written as

$$\mathbf{u}_h(t;\boldsymbol{\mu}) \approx \mathbf{V}\mathbf{c}(t;\boldsymbol{\mu}), \quad \mathbf{c}(t) \in \mathbb{R}^n.$$

Projecting the full-order system onto the reduced space yields:

$$\mathbf{M}_n\dot{\mathbf{c}}(t) + \mathbf{A}_n(\boldsymbol{\mu})\mathbf{c}(t) = \mathbf{F}_n(t;\boldsymbol{\mu}), \qquad \mathbf{c}(0) = \mathbf{V}^\top\mathbf{u}_{0,h},$$

where

$$\mathbf{M}_n = \mathbf{V}^\top\mathbf{M}\mathbf{V}, \quad \mathbf{A}_n(\boldsymbol{\mu}) = \mathbf{V}^\top\mathbf{A}_h(\boldsymbol{\mu})\mathbf{V}, \quad \mathbf{F}_n = \mathbf{V}^\top\mathbf{F}_h.$$

If $\mathbf{A}_h(\boldsymbol{\mu})$ is affine in $\boldsymbol{\mu}$, the reduced matrices can be preassembled offline, enabling fast online solves.

Discretizing in time (backward Euler) leads to the reduced algebraic system:

$$\left(\frac{\mathbf{M}_n}{\Delta t} + \mathbf{A}_n(\boldsymbol{\mu})\right)\mathbf{c}^{j+1} = \frac{\mathbf{M}_n}{\Delta t}\mathbf{c}^{j} + \mathbf{F}_n^{j+1}.$$

9.3 Example: Heat Diffusion in a Three-Block Material

9.3.1 Problem Setup

We consider a 2D domain consisting of three blocks with different thermal conductivities:

- Center block: initially hot, rest cold.
- Conductivities are parameterized (three parameters, given on a logarithmic scale, e.g., -1 means 0.1).
- Insulating boundary conditions (no heat flux).

The full-order model is implemented using finite elements (space V_h with dimension N_h).

9.3.2 Full-Order Simulation

Given parameters $\boldsymbol{\mu} = (\mu_1, \mu_2, \mu_3)$ and an initial condition u_0, the solver computes a time series.

```
# Example: simulate for a given parameter configuration
mu = np.array([-1, 0, 1])           # log-scale conductivities
t_steps = 50
delta_t = 0.01
U_full = solver.solve(mu, t_steps, delta_t)
```

The output array `U_full` has shape (number of time steps $+ 1, N_h$).

Visualization:

```
FE.plot(U_full[0], Vh)       # initial condition (discontinuous jump)
FE.plot(U_full[-1], Vh)      # final temperature distribution
FE.animate(U_full, Vh)       # generate a GIF of the evolution
```

9.3.3 Precomputed Dataset

For the reduced basis construction, we are provided with a precomputed dataset:

- Parameter matrix $\boldsymbol{\mu} \in \mathbb{R}^{N_{\text{train}} \times 3}$,
- Solution tensor $\mathbf{U} \in \mathbb{R}^{N_{\text{train}} \times (N_t+1) \times N_h}$ (samples, time, space).

You can load it with:

```
# Provided code snippet (downloads data)
mu_train, U_train = load_dataset()
```

9.4 Exercise: POD Basis Construction and Projection Error

Your tasks:

1. **Understand the dataset:** How many training samples? How many time steps per sample? What is the spatial dimension N_h?
2. **Build the POD basis:** Flatten the time dimension for each sample? Or treat each time snapshot as an independent snapshot. Choose a strategy (e.g., collect all snapshots from all training simulations into a snapshot matrix $\mathbf{X} \in \mathbb{R}^{N_h \times N_{\text{snapshots}}}$).
3. **Perform SVD:** $\mathbf{X} = \mathbf{V}\boldsymbol{\Sigma}\mathbf{W}^\top$.
4. **Choose n:** Based on singular value decay and energy criterion.
5. **Compute projection errors:** For each test simulation (or using the training set via cross-validation), compute the relative projection error averaged over time and over samples:

$$\varepsilon_{\text{proj}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \max_j \frac{\|\mathbf{U}_{i,j,:} - \mathbf{V}\mathbf{V}^\top \mathbf{U}_{i,j,:}\|_2}{\|\mathbf{U}_{i,j,:}\|_2}.$$

   (Here $\mathbf{U}_{i,j,:}$ is the spatial vector at sample i and time step j.)
6. **Report:** Show the decay of singular values, the captured energy, and the projection error (average and maximum over the test set).

9.5 Next Steps

After completing the basis construction, we will:

- Assemble the reduced matrices $\mathbf{M}_n$, $\mathbf{A}_n(\boldsymbol{\mu})$ (exploiting affine decomposition),
- Implement the reduced time stepper,
- Compare the ROM solution with the full-order solution in terms of accuracy and speedup.

The assignment for the first part will be released tonight. It will guide you through these steps and prepare you for the exam.

9.6 Data Exploration

The provided dataset contains:

- $N_{\text{data}} = 100$ simulations (trajectories),
- Each trajectory has $N_t = 51$ time steps,
- Spatial finite element dimension N_h (e.g., degrees of freedom in the mesh).

We store these numbers for later use:

```
N_data, N_t, N_h = U.shape    # U: (100, 51, Nh)
```

The parameters $\boldsymbol{\mu}$ are also available; they have shape $(100, 3)$.

9.7 Train / Test Split

We split the 100 simulations into 50 training and 50 testing trajectories:

```
N_train = 50
U_train = U[:N_train]          # shape (50, 51, Nh)
U_test  = U[N_train:]          # shape (50, 51, Nh)
```

9.8 Constructing the Snapshot Matrix for POD

We want a matrix $\mathbf{X} \in \mathbb{R}^{N_h \times N_{\text{snapshots}}}$ where each column is a spatial snapshot (a single time instant from a simulation). The total number of snapshots is:

$$N_{\text{snap}} = N_{\text{train}} \cdot N_t.$$

We reshape the training tensor while preserving the order of dimensions (do not swap axes with reshape; use transpose if needed).

```
# Reshape: from (N_train, N_t, Nh) to (N_train * N_t, Nh)
U_train_flat = U_train.reshape(N_train * N_t, N_h)
# Then transpose to get Nh as first dimension
X = U_train_flat.T             # shape (Nh, N_train*N_t)
```

Important: Never swap dimensions with reshape – it corrupts the data layout. Use `transpose` to reorder axes.

9.9 Singular Value Decomposition (SVD)

We compute the thin SVD of $\mathbf{X}$:

```
from scipy.linalg import svd

V, s, Wt = svd(X, full_matrices=False)
```

Here:

- $\mathbf{V} \in \mathbb{R}^{N_h \times N_{\text{snap}}}$ contains the left singular vectors (POD modes) as columns,
- $\mathbf{s} \in \mathbb{R}^{N_{\text{snap}}}$ are the singular values (sorted descending),
- $\mathbf{W}^\top$ is not needed for the basis.

9.10 Choosing the Reduced Dimension n

Plot the singular values to see the decay:

```
import matplotlib.pyplot as plt

plt.figure()
plt.semilogy(s, 'o-')
plt.xlabel('Mode index')
plt.ylabel('Singular value')
plt.title('Singular value decay')
plt.grid(True)
```

Because the first singular value is often very large (data not centered), we may plot from index 1 onward:

```
plt.semilogy(s[1:100], 'o-')   # skip first, show first 100
```

From the plot, we choose $n = 30$ (the singular values become very small after that). The POD basis matrix is then:

```
n = 30
Vn = V[:, :n]        # shape (Nh, n)
```

9.10.1 Visualising the First Modes

```
# Plot first 4 modes
FE.multiplot(Vn[:, :4].T, Vh, nrows=1, ncols=4)
```

The first modes are symmetric; later modes become asymmetric and capture more complex features.

9.11 Projection Error on the Test Set

We project each test snapshot onto the reduced space and compute the relative error.

9.11.1 Build the test snapshot matrix

```
U_test_flat = U_test.reshape(N_test * N_t, N_h)   # shape (N_test*N_t, Nh)
X_test = U_test_flat.T                            # shape (Nh, N_test*N_t)
```

9.11.2 Project and reconstruct

```
# Project: coefficients = Vn^T * X_test    (shape: n, N_test*N_t)
coeff = Vn.T @ X_test
# Reconstruct: X_proj = Vn * coeff         (shape: Nh, N_test*N_t)
X_proj = Vn @ coeff
```

9.11.3 Compute relative errors

The shape of the difference is $(N_h, N_{\text{test}}N_t)$. We want one error per snapshot (each column). Use `np.linalg.norm` with axis=0 (the axis along the spatial dimension N_h):

```
abs_errors = np.linalg.norm(X_test - X_proj, axis=0)
norms = np.linalg.norm(X_test, axis=0)
rel_errors = abs_errors / norms
```

Now `rel_errors` is an array of length $N_{\text{test}}N_t$.

9.11.4 Aggregate over time and trajectories

We reshape back to (N_{test}, N_t) to separate trajectories:

```
rel_errors_mat = rel_errors.reshape(N_test, N_t)
```

For each trajectory, take the maximum relative error over time:

```
max_over_time = np.max(rel_errors_mat, axis=1)   # shape (N_test,)
```

Then average over the test trajectories:

```
avg_max_error = np.mean(max_over_time)
print(f"Average (over test simulations) of max-in-time relative error: {avg_max_error*100:
```

A value below 1% (e.g., 0.77%) indicates the basis is accurate.

9.12 Building the Reduced Solver

We now use the POD basis $\mathbf{V}_n$ to construct a reduced-order model for the time-dependent problem.

9.12.1 Precompute parameter-independent reduced matrices

- Reduced mass matrix: $\mathbf{M}_n = \mathbf{V}_n^\top \mathbf{M} \mathbf{V}_n$ (constant in time and parameters),
- If the stiffness matrix is affine: $\mathbf{A}_h(\boldsymbol{\mu}) = \mathbf{A}_0 + \sum_q \theta_q(\boldsymbol{\mu}) \mathbf{A}_q$, then precompute $\mathbf{A}_{n,q} = \mathbf{V}_n^\top \mathbf{A}_q \mathbf{V}_n$.

9.12.2 Time stepping (Backward Euler)

Given the current reduced state $\mathbf{c}^j$ (coefficients at time t_j), the next state $\mathbf{c}^{j+1}$ satisfies:

$$\left(\frac{\mathbf{M}_n}{\Delta t} + \mathbf{A}_n(\boldsymbol{\mu}) \right) \mathbf{c}^{j+1} = \frac{\mathbf{M}_n}{\Delta t} \mathbf{c}^j + \mathbf{F}_n^{j+1}.$$

If the forcing term $\mathbf{F}_n$ is zero (as in the example), we can implement a step as:

```
def rb_step(c_curr, delta_t, mu, M_n, A_n_affine_terms, theta_functions):
    """
    c_curr : current coefficients (n,)
    delta_t : time step
    mu : parameter vector
    M_n : reduced mass matrix (n,n)
    A_n_affine_terms : list of precomputed reduced matrices A_n_q
    theta_functions : list of functions that evaluate theta_q(mu)
    Returns c_next (n,)
    """
    # Assemble reduced stiffness matrix
    A_n = np.zeros_like(M_n)
    for theta, A_n_q in zip(theta_functions, A_n_affine_terms):
        A_n += theta(mu) * A_n_q
    # Left-hand side matrix
    LHS = M_n / delta_t + A_n
    # Right-hand side vector
    RHS = (M_n / delta_t) @ c_curr   # + F_n if present
    # Solve linear system
    c_next = np.linalg.solve(LHS, RHS)
    return c_next
```

9.12.3 Rollout

Starting from the initial condition $\mathbf{u}_0$, project to reduced coefficients:

$$\mathbf{c}^0 = \mathbf{V}_n^\top \mathbf{u}_0.$$

Then iterate the time stepping to obtain $\mathbf{c}^1, \mathbf{c}^2, \ldots$. Finally, reconstruct the full-field solutions:

$$\mathbf{u}_h^j = \mathbf{V}_n \mathbf{c}^j.$$

9.13 Summary of the Workflow

1. Load dataset (parameters and full-order solutions).
2. Split into training and test sets.
3. Build snapshot matrix $\mathbf{X}$ from training data (reshape and transpose correctly).
4. Compute SVD of $\mathbf{X}$.
5. Choose n based on singular value decay and projection error on test set.
6. Precompute reduced matrices (mass, affine stiffness terms).
7. Implement the reduced time stepper.
8. Validate against full-order solutions (speed and accuracy).

9.14 Time Stepping in the Reduced Space

We consider the backward Euler discretization of the reduced system:

$$\left(\frac{\mathbf{M}_n}{\Delta t} + \mathbf{A}_n(\boldsymbol{\mu})\right) \mathbf{c}^{j+1} = \frac{\mathbf{M}_n}{\Delta t} \mathbf{c}^j + \mathbf{F}_n^{j+1},$$

where $\mathbf{c}^j$ are the reduced coefficients at time t_j. In Python, one step can be implemented as:

```
def rb_step(c_curr, delta_t, mu, M_n, A_n_affine, theta_functions):
    # Assemble reduced stiffness matrix
    A_n = sum(theta(mu) * A_q for theta, A_q in zip(theta_functions, A_n_affine))
    LHS = M_n / delta_t + A_n
    RHS = (M_n / delta_t) @ c_curr   # + F_n if present
    c_next = np.linalg.solve(LHS, RHS)
    return c_next
```

9.14.1 Full Reduced Solver

To simulate the whole time interval, we start from the initial condition $\mathbf{u}_0$ (full space), project it to reduced coefficients, then iterate the time steps, and finally reconstruct the full-field solutions.

```
def rb_solver(mu, delta_t, n_steps, u0, Vn, M_n, A_n_affine, theta_functions):
    # Project initial condition
    c = Vn.T @ u0   # shape (n,)
    # List to store coefficients at all times
    c_list = [c]
    for _ in range(n_steps):
        c = rb_step(c, delta_t, mu, M_n, A_n_affine, theta_functions)
        c_list.append(c)
```

```
    # Convert list to array: shape (n_steps+1, n)
    C = np.stack(c_list, axis=0)
    # Reconstruct full fields: U = C @ Vn.T   -> shape (n_steps+1, Nh)
    U_rb = C @ Vn.T
    return U_rb
```

9.15 Exploiting Affine Parametrization

The reduced stiffness matrix $\mathbf{A}_n(\boldsymbol{\mu})$ is assembled from precomputed components:

$$\mathbf{A}_n(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{A}_h(\boldsymbol{\mu}) \mathbf{V}_n,$$

and if $\mathbf{A}_h(\boldsymbol{\mu})$ has an affine decomposition

$$\mathbf{A}_h(\boldsymbol{\mu}) = \sum_{q=0}^{Q} \theta_q(\boldsymbol{\mu}) \mathbf{A}_q,$$

then

$$\mathbf{A}_n(\boldsymbol{\mu}) = \sum_{q=0}^{Q} \theta_q(\boldsymbol{\mu}) \underbrace{\mathbf{V}_n^\top \mathbf{A}_q \mathbf{V}_n}_{\mathbf{A}_{n,q}}.$$

In the cookie problem (three-block material), the conductivity is piecewise constant, so:

$$\mathbf{A}_h(\boldsymbol{\mu}) = \sigma_0 \mathbf{A}_0 + \sigma_1 \mathbf{A}_1 + \sigma_2 \mathbf{A}_2,$$

where $\sigma_j = 10^{\mu_j}$ (logarithmic scale). We precompute

```
A_n_0 = Vn.T @ A0 @ Vn
A_n_1 = Vn.T @ A1 @ Vn
A_n_2 = Vn.T @ A2 @ Vn
```

and define $\theta_q(\boldsymbol{\mu}) = 10^{\mu_q}$.

9.15.1 Important Note on Efficiency

Because $\mathbf{A}_n(\boldsymbol{\mu})$ does not depend on time, it can be assembled once before the time loop and reused at each step, saving computation.

9.16 Verification and Debugging

When implementing the reduced solver, always compare the result with the full-order model for a few test parameters. If the reduced solution looks qualitatively correct but has a scaling error, check:

- The time step Δt is correctly included in the left-hand side and right-hand side.
- The affine coefficients $\theta_q(\boldsymbol{\mu})$ are computed correctly (e.g., 10^{μ_j} vs. μ_j).
- The initial condition is properly projected.

9.17 Assignments for the Exam

You must choose **one** of the following three assignments. You do not need to submit them now; they will be required a few days before your oral exam. During the exam, we will discuss your implementation and ask you to modify or extend it to test your understanding.

9.17.1 Assignment 1: Vector Field Problem (Stationary)

Description: The solution is a vector field (e.g., deformation of an object under load). The problem is stationary. Parameters: angle of the load, weight, material properties. The finite element code is given; you need to construct a reduced basis surrogate. The plot shows arrows (vector field) that can be used to deform the object.

Difficulty: Moderate – understanding how to handle vector-valued solutions and the associated finite element spaces.

9.17.2 Assignment 2: Chemical Mixer (Stationary, Non-homogeneous Dirichlet)

Description: A 2D domain with three rotating cylinders. The parameters are the rotational speeds (sign and magnitude). The PDE involves diffusion and convection. The boundary conditions are non-homogeneous Dirichlet conditions (prescribed values). You must handle these in the reduced basis context, using a lifting function or similar technique (discussed in lectures).

Difficulty: Higher – due to non-homogeneous Dirichlet conditions. The basis from homogeneous snapshots is not compliant; you need to adapt the formulation.

Visualization: The solution (e.g., concentration) is a scalar field; the cylinders rotate.

9.17.3 Assignment 3: Time-Dependent Schrödinger Equation (Time-Dependent)

Description: A quantum mechanics problem: an electron in a crystal under an electric field. The unknown is the complex wave function $\psi(x, t)$. The full-order model solves the Schrödinger equation. The reduced basis method is applied to the wave function. The probability density $|\psi|^2$ is plotted (real, non-negative). The parameters may include field strength, effective mass, etc.

Difficulty: Moderate – handling complex numbers in the reduced model. The code for computing probability from the wave function is provided.

Visualization: Animations of the probability density evolving in time (electron moving).

9.17.4 General Advice

- You are encouraged to collaborate, but make sure you understand every part of your implementation.
- Ask questions during lab sessions or by email – I will gladly help.
- All three assignments are doable in an afternoon once you understand the workflow.
- The exam will include a discussion of your code and possibly a live modification or extension.

9.18 Final Remarks

Today we completed the time-dependent reduced basis implementation. Next session (Friday) we will start the second part of the course: neural networks for scientific computing. The assignments for the first part are now available – choose one and start early to allow time for questions.

Chapter 10

An Introduction to Neural Networks from the Point of View of Functions' Approximation

10.1 Introduction: Why Data-Driven ROMs?

We now open the second part of the course, which focuses on **non-intrusive** (or *data-driven*) reduced order models (ROMs). To motivate this shift, let us review what we have accomplished so far.

Proper Orthogonal Decomposition (POD) provides an optimal linear dimensionality reduction technique. For a stationary problem, our high-fidelity solution $\mathbf{u}_h \in \mathbb{R}^{N_h}$ can be approximated as a linear combination of the POD basis vectors (columns of $\mathbf{V} \in \mathbb{R}^{N_h \times n}$):

$$\mathbf{u}_h \approx \mathbf{V}\mathbf{u}_n, \qquad \mathbf{u}_n \in \mathbb{R}^n.$$

So far, the reduced coordinates $\mathbf{u}_n$ have been obtained by solving a (possibly nonlinear) reduced system derived via Galerkin projection. However, assembling and solving such a reduced system can become computationally expensive or even infeasible for highly nonlinear problems.

A Direct Parameter-to-Coefficient Map

To avoid the Galerkin step, we seek a direct map from the parameter space to the reduced coefficients:

$$\boldsymbol{\pi} : \mathcal{P} \subset \mathbb{R}^p \to \mathbb{R}^n, \quad \boldsymbol{\mu} \mapsto \mathbf{u}_n.$$

The map $\boldsymbol{\pi}$ should be inexpensive to evaluate, even if its construction requires an offline investment. A natural candidate for approximating this map is a **neural network**. Methods that combine POD with a neural network for this purpose are often termed **POD-NN**. This will be the first methodology we introduce after reviewing the fundamentals of neural networks.

10.2 Beyond Linear Dimensionality Reduction: DL-ROMs

In many problems of interest, linear dimensionality reduction via POD becomes inadequate:

- The Kolmogorov n-width decays slowly, requiring a large reduced dimension n to achieve sufficient accuracy.
- The parameter-to-solution map exhibits complex features (e.g., propagating waves, sharp gradients) that linear subspaces cannot efficiently capture.

In such cases, we replace POD entirely with **nonlinear dimensionality reduction** techniques, leading to **Deep Learning based Reduced Order Models (DL-ROMs)**.

A DL-ROM consists of two primary components:

(i) A *parameter-to-latent* map $\mathbb{R}^p \to \mathbb{R}^\ell$, where ℓ (the latent dimension) can be significantly smaller than the POD dimension n (e.g., $\ell = 5$ vs. $n = 50$).

(ii) A *nonlinear dimensionality reduction* (and reconstruction) map between the high-fidelity space $\mathbb{R}^{N_h}$ and the latent space $\mathbb{R}^\ell$.

Because the transformations are no longer constrained to be linear, the solution manifold can be represented far more compactly.

Treatment of Time

For time-dependent problems, two strategies emerge:

- **Time as a parameter:** Augment the input space to $\mathbb{R}^{p+1}$ by including t alongside $\boldsymbol{\mu}$. This breaks temporal causality but allows querying the solution at arbitrary time instants without sequential time-stepping.
- **Latent dynamics:** Discover and solve a dynamical system directly in the latent space. Instead of approximating the full-order dynamics, we learn the evolution law in the low-dimensional latent coordinates, preserving the notion of proper time evolution.

10.3 From Polynomial Fitting to Neural Networks

Before diving into neural networks, we revisit classical function approximation tools that serve as a conceptual foundation.

10.3.1 Polynomial Interpolation

Given data pairs (x_i, y_i) for $i = 0, \dots, n$, we seek a polynomial $P_n(x)$ of degree n such that $P_n(x_i) = y_i$. The approximant is written as a linear combination of fixed basis functions (monomials):

$$P_n(x) = a_0 + a_1 x + \cdots + a_n x^n = \sum_{j=0}^{n} a_j \phi_j(x).$$

The coefficients $\{a_j\}$ are determined by solving an $(n+1) \times (n+1)$ linear system (Vandermonde).

Remark 10.1. *While exact at the data points, high-degree polynomial interpolation suffers from severe oscillations (Runge's phenomenon) and is impractical in high dimensions.*

10.3.2 Linear Least Squares Regression

When data are abundant ($N \gg m$), we prefer a low-degree polynomial model:

$$f_m(x; \boldsymbol{\theta}) = \theta_0 + \theta_1 x + \cdots + \theta_m x^m,$$

with $\boldsymbol{\theta} = (\theta_0, \dots, \theta_m) \in \mathbb{R}^{m+1}$. The optimal parameters are found by minimizing the sum of squared residuals:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=0}^{N} \big(y_i - f_m(x_i; \boldsymbol{\theta})\big)^2.$$

Because f_m is **linear in the parameters $\boldsymbol{\theta}$**, the minimizer is the solution of the **normal equations**:

$$(\mathbf{V}^\top \mathbf{V}) \boldsymbol{\theta}^* = \mathbf{V}^\top \mathbf{y},$$

where $\mathbf{V}_{ij} = \phi_j(x_i)$. This avoids iterative optimization and provides a unique solution (provided $\mathbf{V}^\top \mathbf{V}$ is invertible).

Limitations of Linear-in-Parameter Models

- The basis functions $\{\phi_j\}$ are fixed *a priori* and are not adapted to the data (unlike POD, where the basis is data-driven).
- The model is linear in the coefficients, restricting expressivity.
- Scaling to high-dimensional inputs (e.g., $\mathbb{R}^{100}$) is infeasible.

10.3.3 Towards Nonlinear Dependence on Parameters

Consider a scenario where the data suggest an oscillatory, damped behavior. A physically motivated model might be

$$f(t;\boldsymbol{\theta}) = A\, e^{-\lambda t}\cos(\omega t + \phi),$$

with parameters $\boldsymbol{\theta} = (A, \lambda, \omega, \phi)$. Here f is **nonlinear** in $\boldsymbol{\theta}$. Fitting such a model requires solving a *nonlinear least squares* problem, typically via iterative optimization (e.g., gradient descent, Gauss–Newton). While more computationally intensive, this approach offers vastly greater flexibility.

10.3.4 The Core Idea of Neural Networks

Neural networks take the idea of nonlinear parameter dependence to the extreme: they compose many simple nonlinear functions (layers) to produce a highly expressive, data-driven approximation that is *nonlinear both in the input and in the parameters*.

Instead of fixing basis functions *a priori*, we make them **adaptable** by parameterizing the transformations themselves. A neural network can be seen as a sequence of operations:

$$\mathbf{z}^{(1)} = \sigma^{(1)}\big(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\big), \quad \mathbf{z}^{(2)} = \sigma^{(2)}\big(\mathbf{W}^{(2)}\mathbf{z}^{(1)} + \mathbf{b}^{(2)}\big), \quad \dots$$

where:

- $\mathbf{W}^{(\ell)}$ are weight matrices,
- $\mathbf{b}^{(\ell)}$ are bias vectors,
- $\sigma^{(\ell)}$ are nonlinear *activation functions*.

The final output layer typically uses no activation (or the identity) for regression tasks.

This construction generalizes linear models: if all activations are linear, the entire network collapses to a single affine transformation. By introducing nonlinearities, we gain the ability to approximate highly complex maps from $\mathbb{R}^d$ to $\mathbb{R}^k$.

Remark 10.2 (Universal Approximation)**.** *Even a two-layer neural network with sufficiently many neurons can approximate any continuous function on a compact set arbitrarily well (universal approximation theorem). This is one of the key reasons for their widespread adoption.*

10.3.5 Training a Neural Network

Given a set of training data $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^{N_{\text{train}}}$, we define a loss function (e.g., mean squared error):

$$\mathcal{L}(\boldsymbol{\Theta}) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \|\mathbf{y}_i - \mathcal{NN}(\mathbf{x}_i;\boldsymbol{\Theta})\|^2,$$

where $\boldsymbol{\Theta}$ collects all weights and biases. Because $\mathcal{NN}$ is nonlinear in $\boldsymbol{\Theta}$, we must solve an optimization problem to find $\boldsymbol{\Theta}^*$.

The number of parameters $\boldsymbol{\Theta}$ can be enormous (hundreds of thousands or millions), making gradient-based methods the only practical choice. Computing the exact gradient over the entire dataset is expensive; hence we use **stochastic gradient descent (SGD)** and its variants (e.g., Adam), which estimate the gradient using small mini-batches.

The computation of gradients is efficiently performed via **backpropagation**, which leverages the chain rule and the layered structure of the network.

10.4 Autoencoders: Nonlinear Generalization of POD

POD provides an optimal linear compression and reconstruction:

$$\mathbf{u} \approx \mathbf{V}\mathbf{V}^\top \mathbf{u},$$

where $\mathbf{V} \in \mathbb{R}^{N_h \times n}$ has orthonormal columns. This can be seen as a two-stage process:

$$\begin{aligned} \text{Encoder:} \quad & \mathbf{u} \mapsto \mathbf{z} = \mathbf{V}^\top \mathbf{u} \in \mathbb{R}^n, \\ \text{Decoder:} \quad & \mathbf{z} \mapsto \tilde{\mathbf{u}} = \mathbf{V}\mathbf{z} \in \mathbb{R}^{N_h}. \end{aligned}$$

The matrix $\mathbf{V}$ is chosen to minimize the reconstruction error over a set of snapshots.

10.4.1 The Autoencoder Framework

An **autoencoder** replaces the linear maps with neural networks:

$$\begin{aligned} &\text{Encoder } \phi : \mathbb{R}^{N_h} \to \mathbb{R}^\ell, \\ &\text{Decoder } \psi : \mathbb{R}^\ell \to \mathbb{R}^{N_h}. \end{aligned}$$

The pair (ϕ, ψ) is trained to make the composition $\psi \circ \phi$ as close as possible to the identity map on the data distribution:

$$\mathcal{L}_{\text{AE}}(\phi, \psi) = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left\| \mathbf{u}_i - \psi\big(\phi(\mathbf{u}_i)\big) \right\|^2.$$

By introducing nonlinearities in ϕ and ψ, the autoencoder can discover a latent representation ℓ that is much more compact than the linear POD subspace of the same reconstruction quality.

Remark 10.3. *If we remove all nonlinear activations and use a single layer without bias, the autoencoder reduces exactly to POD:* $\phi(\mathbf{u}) = \mathbf{V}^\top \mathbf{u}$ *and* $\psi(\mathbf{z}) = \mathbf{V}\mathbf{z}$.

Autoencoders thus serve as a nonlinear dimensionality reduction tool, capable of capturing complex, low-dimensional manifolds embedded in high-dimensional data. In the context of ROMs, an autoencoder can replace POD entirely, providing a more expressive latent space for the subsequent dynamics learning or parameter-to-latent mapping.

Practical Implementation in PyTorch

In the upcoming lab session, we will implement these concepts using the **PyTorch** library. PyTorch provides high-level abstractions for defining network architectures, automatic differentiation (for backpropagation), and optimization algorithms. Once the architecture is defined, training reduces to calling built-in optimizers—much like we use built-in SVD solvers for POD without coding them from scratch.

Outlook

The following weeks will be devoted to:

- Formalizing the POD-NN and DL-ROM frameworks.
- Understanding the training process and hyperparameter tuning.
- Exploring latent dynamics discovery for time-dependent problems.
- Hands-on implementation in PyTorch.

Chapter 11

Lab 5 - Fundamentals of neural networks

11.1 Why Neural Networks?

Neural networks are an alternative to classical interpolation (polynomials, radial basis functions, etc.). They are particularly useful when:

- The input space is high-dimensional (e.g., images, complex data).
- Classical polynomial interpolation suffers from combinatorial explosion (e.g., in $\mathbb{R}^5$, a quadratic polynomial requires all pairs of variables).

Neural networks are very flexible and easy to implement even for high-dimensional data.

11.2 Basic Structure of a Neural Network

A neural network is built by composing **layers**. A typical layer performs:

$$\mathbf{y} = \rho(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where:

- $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input,
- $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix,
- $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector,
- $\rho : \mathbb{R} \to \mathbb{R}$ is a scalar activation function applied element-wise.

A network is a composition of several such layers. For example, a network approximating $f : \mathbb{R}^3 \to \mathbb{R}^4$ could have three layers:

$$\phi(\mathbf{x}) = \mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1(\mathbf{x}))\big),$$

with dimensions:

$$\begin{aligned}
&\mathbf{L}_1 : \mathbb{R}^3 \to \mathbb{R}^{N_1}, \\
&\mathbf{L}_2 : \mathbb{R}^{N_1} \to \mathbb{R}^{N_2}, \\
&\mathbf{L}_3 : \mathbb{R}^{N_2} \to \mathbb{R}^4.
\end{aligned}$$

11.2.1 Choosing Layer Dimensions

There is no fixed recipe; common heuristics:

- Avoid reducing dimension too early (unless you believe the input has low intrinsic dimensionality).
- Typically, start with a hidden dimension larger than or equal to the input dimension.
- The output layer often has no activation (identity) for regression tasks.

11.2.2 Activation Functions

The same activation ρ is usually used for all hidden layers. Common choices: ReLU, sigmoid, tanh, or even custom functions like x^2 (as in the example). The last layer typically uses the identity activation.

11.3 Training a Neural Network

A network ϕ is parameterized by all weights $\mathbf{W}^{(\ell)}$ and biases $\mathbf{b}^{(\ell)}$ for each layer ℓ. Given training data $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, we want:

$$\phi(\mathbf{x}_i) \approx \mathbf{y}_i.$$

This is achieved by minimizing the loss function, e.g., mean squared error:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|\phi(\mathbf{x}_i) - \mathbf{y}_i\|^2.$$

Optimization is performed using gradient-based methods (e.g., stochastic gradient descent, Adam). The gradients are computed automatically by automatic differentiation (backpropagation).

11.4 Implementing Neural Networks with PyTorch

The lab uses a custom wrapper around PyTorch, but the core concepts are directly transferable. PyTorch is a popular deep learning framework (originally from Facebook). The main data structure is a `torch.Tensor`, which is like a NumPy array but can be moved to GPU and supports automatic differentiation.

11.4.1 Creating a Layer

A dense (fully connected) layer is created with:

```
import torch
import torch.nn as nn

rho = lambda x: x**2          # custom activation
L = nn.Linear(3, 5)           # weights and bias are randomly initialized
```

Set a custom activation? In pure PyTorch you would define a custom module; here the lab's wrapper allows passing an activation function directly.

11.4.2 Weights and Biases

The weight matrix and bias vector can be accessed:

```
W = L.weight     # shape (5, 3)
b = L.bias       # shape (5,)
```

Initially they contain small random values.

11.4.3 Forward Pass

To compute the output for an input tensor, we call the layer as a function. However, note the device (CPU vs. GPU): if the input is on GPU, the layer must also be on GPU.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
L = L.to(device)

v = torch.tensor([1.0, 1.0, 0.0], device=device)   # shape (3,)
out = L(v)                                          # shape (5,)
```

The operation exactly matches the mathematical definition:

```
manual = rho(L.weight @ v + L.bias)
print(torch.allclose(out, manual))    # True
```

11.4.4 Batch Processing

PyTorch layers are designed to process multiple inputs simultaneously (batch mode). If we stack inputs as rows of a matrix $\mathbf{X} \in \mathbb{R}^{N_{\text{batch}} \times d_{\text{in}}}$, the layer computes:

$$\mathbf{Y} = \rho(\mathbf{X}\mathbf{W}^\top + \mathbf{b}),$$

where $\mathbf{Y} \in \mathbb{R}^{N_{\text{batch}} \times d_{\text{out}}}$.

Example:

```
X = torch.tensor([[1.0, 1.0, 0.0],
                  [0.0, 1.0, 1.0]], device=device)   # (2,3)
Y = L(X)                                              # (2,5)
```

11.4.5 Automatic Differentiation (Gradients)

Tensors that require gradients have the attribute `requires_grad=True`. By default, trainable parameters (weights, biases) have this set to `True`. Input data typically does not. PyTorch records the computation graph so that gradients can be computed later via backpropagation.

```
print(L.weight.requires_grad)   # True
print(v.requires_grad)          # False
```

The output of a computation involving trainable parameters will also track the gradient history:

```
out = L(v)
print(out.requires_grad)        # True
print(out.grad_fn)              # shows the operation that produced it (e.g., Power)
```

11.5 Device Management (CPU / GPU)

Neural networks benefit greatly from GPU acceleration. In Google Colab, you can select GPU as the runtime. When using GPU:

- Tensors and models must be explicitly moved to the GPU using `.to(device)`.
- All operations will then be executed on the GPU.
- If you forget to move a tensor, you get a device mismatch error.

Note: Prolonged heavy GPU usage may cause Colab to disconnect the session.

11.6 Next Steps

In the lab, we will:

1. Build a small neural network for a regression problem.
2. Train it using a dataset.
3. Compare with reduced basis methods.

Understanding the layer mechanics and automatic differentiation is essential for customizing models for scientific computing tasks.

11.7 From Reduced Basis to Neural Networks

Neural networks are an alternative to classical interpolation (polynomials, radial basis functions). They are particularly flexible for high-dimensional data (e.g., images, complex physical fields) where polynomial methods suffer from combinatorial explosion. The core idea is composition of affine maps with nonlinear activations.

11.7.1 Layer Definition

A dense (fully connected) layer is defined as:

$$\mathbf{y} = \rho(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where:

- $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ is the input,
- $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix,
- $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$ is the bias vector,
- $\rho : \mathbb{R} \to \mathbb{R}$ is a scalar activation function applied element-wise.

A neural network is a composition of several such layers. For example, a network mapping $\mathbb{R}^3 \to \mathbb{R}^4$ could be:

$$\phi(\mathbf{x}) = \mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1(\mathbf{x}))\big),$$

with dimensions:

$$\mathbf{L}_1 : \mathbb{R}^3 \to \mathbb{R}^{N_1}, \quad \mathbf{L}_2 : \mathbb{R}^{N_1} \to \mathbb{R}^{N_2}, \quad \mathbf{L}_3 : \mathbb{R}^{N_2} \to \mathbb{R}^4.$$

11.7.2 Choosing Layer Dimensions

There is no universal rule, but common heuristics:

- Do not reduce the dimension too early unless you believe the input has low intrinsic dimensionality.
- Hidden dimensions are often taken larger than or equal to the input dimension.
- The last layer usually has no activation (identity) for regression tasks.

11.7.3 Activation Functions

The same activation ρ is used for all hidden layers. Typical choices: ReLU, sigmoid, tanh, or even custom functions like x^2. The output layer typically uses identity.

11.8 Data Shape Conventions

When working with neural networks, data is usually stored as a 2D tensor: `(number_of_samples, input_dimension)`. For a scalar map $\mathbb{R} \to \mathbb{R}$ with 100 samples, the input tensor shape is $(100, 1)$ and output shape $(100, 1)$. This batch processing allows efficient computation.

11.9 Implementing a Network in PyTorch

The lab uses a wrapper around PyTorch, but the core concepts are directly from PyTorch.

11.9.1 Creating a Layer

```
import torch
import torch.nn as nn

rho = lambda x: x**2    # custom activation
L = nn.Linear(3, 5)     # weight matrix (5,3), bias (5)
```

To use a custom activation, the lab's wrapper provides a `Dense` layer:

```
L = Dense(3, 5, activation=rho)
```

11.9.2 Device Management (CPU/GPU)

PyTorch tensors and models can be moved to GPU. In Google Colab, choose GPU runtime. If input is on GPU, the model must also be on GPU.

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
L = L.to(device)
```

11.9.3 Forward Pass with Batching

Input tensors should have shape `(batch_size, input_dim)`. For a single input, reshape accordingly.

```
v = torch.tensor([[1.0, 1.0, 0.0]], device=device)    # shape (1,3)
out = L(v)                                             # shape (1,5)
```

PyTorch automatically applies the layer to each row in the batch.

11.9.4 Accessing Weights and Biases

```
W = L.weight    # shape (5,3)
b = L.bias      # shape (5,)
```

Initially random. The operation $L(v)$ is equivalent to:

```
manual = rho(v @ W.T + b)
```

11.9.5 Automatic Differentiation

Tensors that require gradients have `requires_grad=True` (e.g., weights, biases). Input data does not. PyTorch records the computational graph so that gradients can be computed via backpropagation.

11.10 Building a Network via Composition

Using the lab's `Dense` layers, a network is built as a sequence:

```
phi = Dense(3, 5, activation=rho) + \
      Dense(5, 8, activation=rho) + \
      Dense(8, 4, activation=None)   # last layer: identity
```

The whole network is then wrapped with `DFNN` to make it trainable and moved to the device.

11.11 Training a Neural Network

We illustrate with a 1D regression problem: $f : \mathbb{R} \to \mathbb{R}$ given noisy data points (x_i, y_i).

11.11.1 Data Preparation

Assume we have 100 samples stored as NumPy arrays. Convert to PyTorch tensors and reshape to $(100, 1)$:

```
import numpy as np
import torch

X_data = torch.tensor(X_numpy, dtype=torch.float32).reshape(-1, 1)
Y_data = torch.tensor(Y_numpy, dtype=torch.float32).reshape(-1, 1)
```

`reshape(-1,1)` automatically infers the number of rows (100) and sets the column dimension to 1.

11.11.2 Network Construction for 1D Problem

A simple three-layer network (input dimension 1, hidden dimension 5, output dimension 1):

```
rho = torch.relu   # or any activation
net = Dense(1, 5, activation=rho) + \
      Dense(5, 5, activation=rho) + \
      Dense(5, 1, activation=None)
dnn = DFNN(net).to(device)
```

11.11.3 Loss Function

We define the mean squared error (MSE) loss manually:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2,$$

where $\|\cdot\|$ is the Euclidean norm. In code, for batched predictions:

```
def mse_loss(y_true, y_pred):
    diff = y_true - y_pred                  # shape (N, q)
    squared = diff.pow(2)                   # element-wise square
    norm_sq = squared.sum(dim=-1)           # sum over output dimension -> shape (N,)
    return norm_sq.mean()                   # average over batch
```

Note: `sum(dim=-1)` sums over the last axis (the output dimension q). For scalar outputs ($q = 1$), it is still correct.

11.11.4 Training the Network

We split the data into training (e.g., 70 samples) and test (30 samples). The training procedure:

```
dnn.train(X_train, Y_train,
          n_epochs=50,
          batch_size=None,    # use full batch
          loss=mse_loss,
          optimizer='LBFGS')    # default
```

The optimizer can be changed (e.g., Adam) and additional parameters (learning rate) can be passed.

11.11.5 Initialization

PyTorch uses a default initialization. For better performance, one can apply Xavier/Glorot initialization:

```
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        nn.init.zeros_(m.bias)
dnn.apply(init_weights)
```

11.12 Optimizers and Batching

11.12.1 Full Batch vs. Mini-Batch

- **Full batch:** The loss is computed on the entire training set at each iteration. This gives a deterministic gradient but can be expensive and may get stuck in poor local minima.
- **Mini-batch:** The training set is split into small batches (e.g., 10 samples). The loss is computed and gradients are updated for each batch. This introduces stochasticity, often leading to better generalization.
- **Stochastic gradient descent (SGD):** Batch size = 1.

In the training call, set `batch_size` to an integer (e.g., 10). Then each epoch consists of multiple iterations (one per batch). The total number of iterations = (number of training samples / batch_size) $\times$ n_epochs.

11.12.2 Choice of Optimizer

- **LBFGS** (quasi-Newton method): Works well for small to medium problems, especially with full batch. It is computationally more expensive per iteration but often converges in fewer iterations. Not suitable for mini-batching (numerical issues).
- **Adam** (first-order method): Lightweight, works with mini-batching, and is the default for large-scale problems (e.g., deep learning on millions of samples). Requires tuning of learning rate (typically 10^{-3}). Needs more epochs.

For the 1D example, LBFGS with full batch and 50 epochs works well. Adam with mini-batches may require 5000+ epochs to reach similar accuracy.

11.12.3 Learning Rate and Epochs

When using Adam, specify the learning rate:

```
optimizer = Adam(lr=1e-3)
```

With mini-batching, more epochs are needed because each epoch processes the data in pieces, and the gradient directions are noisier.

11.12.4 Monitoring Training

During training, both training and test losses are reported. Typically the test loss is higher than the training loss. A well-trained network should produce a smooth approximation of the underlying function, as seen in the plot of predictions vs. ground truth.

11.13 Practical Recommendations

1. Start simple: small network (1-2 hidden layers, moderate hidden dimension).
2. Use full batch with LBFGS for small datasets (a few hundred samples).
3. If LBFGS fails to reduce the loss, switch to Adam with mini-batches.
4. When using Adam, increase the number of epochs (e.g., 1000–5000) and consider reducing the learning rate if the loss oscillates.
5. Always split data into training and test sets to monitor generalization.
6. For very large datasets (e.g., images, ¿10k samples), mini-batch Adam is the standard.

11.14 Conclusion

Neural networks provide a flexible framework for approximating functions from data. Understanding the layer mechanics, data shape conventions, device management, and training strategies (optimizer, batching, loss function) is essential for applying them to scientific computing problems, including reduced order modeling.

The next lab will apply these concepts to build surrogate models for parametrized PDEs, comparing neural networks with reduced basis methods.

11.15 Finite Elements in Python

11.15.1 Introduction

We discuss finite elements (FE) in Python. The implementation uses the `PINIX` package, but we employ a more user-friendly version from a GitHub package (developed during a PhD). This package also handles neural networks and reduced order modeling.

11.15.2 Mesh and Finite Element Space

A domain (e.g., rectangle) is defined and meshed:

```
import DRROMS.bases as FE

domain = FE.Rectangle((0,0), (1,1))
mesh = FE.mesh_domain(domain, step_size=0.5)
FE.plot(mesh)
```

The finite element space of continuous, piecewise linear functions (CG1) is:

```
Vh = FE.finite_element_space(mesh, continuous_galerkin=1)
```

11.15.3 Function – Vector Correspondence

A function $u(x) = x^2 + y^2$ can be interpolated:

```
u = lambda x: x[0]**2 + x[1]**2
uh = FE.interpolate(u, Vh)
```

The vector of nodal values is obtained via `uh.vector()`. Conversely, a vector of coefficients defines a function:

```
u_vec = np.array([...])    # length = dim(Vh)
uh = FE.function(u_vec, Vh)
```

11.15.4 PDE: Stationary Heat Equation

We solve $-\nabla \cdot (\sigma \nabla u) = f$ in Ω with $u = g$ on Γ_D and $\sigma \partial_n u = \phi$ on Γ_N.

The weak form: find $u \in H^1(\Omega)$ with $u = g$ on Γ_D such that

$$\int_\Omega \sigma \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx + \int_{\Gamma_N} \phi v \, ds \qquad \forall v \in H^1_0(\Omega).$$

In code, we define the bilinear form and linear form:

```
a = sigma_h * inner(grad(u), grad(v)) * dx
L = f * v * dx + phi * v * ds
```

Assembly yields $\mathbf{A}\mathbf{u} = \mathbf{F}$. Dirichlet conditions are applied via:

```
bc = FE.DirichletBC(Vh, g, boundary_function)
A, F = FE.apply_boundary_conditions(A, F, bc)
```

Solve with `scipy.sparse.linalg.spsolve`.

11.16 Reduced Basis Method (RBM)

11.16.1 Motivation for the Cookie Problem

We consider a plate with four disks of different conductivities. The conductivity is

$$\sigma(x; \boldsymbol{\mu}) = \sigma_{\text{out}} + \sum_{j=0}^{3} \mu_j^2 \mathbf{1}_{\Omega_j}(x),$$

with $\sigma_{\text{out}} = 0.001$ and $\boldsymbol{\mu} \in [0, 3]^4$. The goal is to build a fast surrogate model.

11.16.2 Snapshot Collection

We sample N_{samples} parameter vectors uniformly and compute the corresponding FE solutions $\mathbf{u}_i \in \mathbb{R}^{N_h}$. The snapshot matrix (samples $\times$ DoFs) is

$$\mathbf{U} \in \mathbb{R}^{N_{\text{samples}} \times N_h}, \qquad \mathbf{U}_{i,:} = \mathbf{u}_i^\top.$$

11.16.3 POD Basis

Using the training snapshots, we compute the SVD of $\mathbf{X} = \mathbf{U}_{\text{train}}^\top$:

$$\mathbf{X} = \mathbf{V} \boldsymbol{\Sigma} \mathbf{W}^\top.$$

The first n left singular vectors $\mathbf{V}_n \in \mathbb{R}^{N_h \times n}$ form an orthonormal reduced basis. Choose n from the decay of singular values or the energy criterion:

$$E(n) = \frac{\sum_{i=1}^{n} \sigma_i^2}{\sum_{i=1}^{N_{\text{train}}} \sigma_i^2}.$$

11.16.4 Galerkin Projection

The reduced system is

$$\mathbf{A}_n(\boldsymbol{\mu})\mathbf{c} = \mathbf{F}_n(\boldsymbol{\mu}), \qquad \mathbf{A}_n = \mathbf{V}_n^\top \mathbf{A}(\boldsymbol{\mu})\mathbf{V}_n,\ \mathbf{F}_n = \mathbf{V}_n^\top \mathbf{F}(\boldsymbol{\mu}).$$

If $\mathbf{A}(\boldsymbol{\mu})$ is affine in $\boldsymbol{\mu}$, preassemble $\mathbf{A}_{n,q} = \mathbf{V}_n^\top \mathbf{A}_q \mathbf{V}_n$ and assemble online:

$$\mathbf{A}_n(\boldsymbol{\mu}) = \sum_q \theta_q(\boldsymbol{\mu})\mathbf{A}_{n,q}.$$

11.16.5 Time-Dependent Problems

The full-order system is

$$\mathbf{M}\dot{\mathbf{u}}_h + \mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h = \mathbf{F}_h.$$

Backward Euler gives:

$$\left(\frac{\mathbf{M}}{\Delta t} + \mathbf{A}_h(\boldsymbol{\mu})\right)\mathbf{u}_h^{j+1} = \frac{\mathbf{M}}{\Delta t}\mathbf{u}_h^j + \mathbf{F}_h^{j+1}.$$

Projecting onto $\mathbf{V}_n$ yields the reduced scheme:

$$\left(\frac{\mathbf{M}_n}{\Delta t} + \mathbf{A}_n(\boldsymbol{\mu})\right)\mathbf{c}^{j+1} = \frac{\mathbf{M}_n}{\Delta t}\mathbf{c}^j + \mathbf{F}_n^{j+1},$$

with $\mathbf{M}_n = \mathbf{V}_n^\top \mathbf{M}\mathbf{V}_n$. The affine decomposition (if available) is exploited again.

11.17 Neural Networks for Surrogate Modeling

11.17.1 Basic Layer

A dense layer:

$$\mathbf{y} = \rho(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

where ρ is a scalar activation (applied element-wise). A network is a composition of several layers.

11.17.2 PyTorch Implementation

The lab uses a wrapper around PyTorch. Create a layer:

```
L = Dense(d_in, d_out, activation=rho)
```

Move to GPU:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
L = L.to(device)
```

Forward pass with batching: input shape $(N_{\text{batch}}, d_{\text{in}})$, output shape $(N_{\text{batch}}, d_{\text{out}})$.

11.17.3 Training

Define a loss function (e.g., MSE):

```
def mse_loss(y_true, y_pred):
    diff = y_true - y_pred
    return diff.pow(2).sum(dim=-1).mean()
```

Split data into training and test sets. Use `dnn.train(...)` with options:

- `optimizer='LBFGS'` (full batch) or `'Adam'` (mini-batch).

- `batch_size` for mini-batching.
- `n_epochs` (iterations over the whole dataset).

For validation (to avoid overfitting), pass a validation set; the code returns the network state with best validation loss.

11.17.4 Activation Functions

Avoid polynomial activations (violate universal approximation). Good choices:

- ReLU: $\rho(x) = \max(0, x)$,
- Leaky ReLU,
- GeLU (smooth, differentiable).

In code: `activation='gelu'`.

11.17.5 Saving and Loading

```
dnn.save('model.zip')
dnn = DFNN.load('model.zip')
```

11.17.6 High-Dimensional Inputs: Convolutional Layers

For image-like inputs (e.g., 51×51), dense layers become very large. Convolutional layers use small filters with shared weights, drastically reducing the number of parameters. They are not required for the reduced basis part but are introduced as an option for the image regression exercise.

11.18 Assignments (Exam Preparation)

Three assignments are available; choose one.

11.18.1 Assignment 1: Vector Field (Stationary)

Solve a linear elasticity problem (vector solution). Construct a reduced basis for the displacement field. Parameters: load angle, weight, material properties.

11.18.2 Assignment 2: Chemical Mixer (Stationary, Non-homogeneous Dirichlet)

A 2D domain with three rotating cylinders. The boundary condition is non-homogeneous Dirichlet. Implement a lifting function to handle the boundary data in the reduced basis context.

11.18.3 Assignment 3: Time-Dependent Schrödinger Equation

A quantum electron in a crystal with an electric field. The unknown is the complex wave function. Build a reduced model for the wave function; plot the probability density.

11.18.4 Exam Procedure

Submit the assignment a few days before the oral exam. During the exam, the code is discussed, and questions about modifications or extensions are asked.

11.19 Concluding Remarks

These notes cover the essential steps: finite element discretization, POD–Galerkin reduction (both stationary and time-dependent), and neural network surrogates. Understanding the affine parametrization, the handling of boundary conditions, and the training of networks is crucial for the assignments and the exam.

Chapter 12

Non-intrusive ROMs for Parametrized Problems through Proper Orthogonal Decomposition & Neural Networks (POD+NN)

12.1 From Neural Networks to PoDNN

Today we introduce the *PoDNN* method, which is one of the first examples in which we combine *proper orthogonal decomposition* (POD) with *neural networks*. This is a typical *gray-box reduced-order model*: neural networks are used only for a specific task, while the reduction step still relies on a linear, physics-based technique such as POD.

Before discussing PoDNN, we briefly review some basic facts about neural networks. In the last few lectures, we gave a practical introduction to neural networks and saw how to implement and train them in Python. At this point, it is useful to understand what happens "behind the scenes" during the training process.

12.1.1 Neural networks: a formal viewpoint

We consider, for the moment, *fully connected neural networks*, meaning that each neuron in a given layer is connected to every neuron in the next layer. Other architectures will be introduced later.

From a formal point of view, a neural network is a composition of several layers. Each layer is an affine map followed by a nonlinear activation function. More precisely, if x denotes the input to a layer, then the output is of the form

$$x \mapsto \sigma(Wx + b),$$

where:

- W is a matrix of weights,
- b is a vector of biases,
- σ is a nonlinear activation function.

A common example is the *rectified linear unit* (ReLU), defined by

$$\sigma(x) = \max\{0, x\}.$$

Other activation functions are also possible.

A neural network with L layers is therefore a composition of L maps of this type:

$$\mathcal{N}(x) = \varphi_L \circ \varphi_{L-1} \circ \cdots \circ \varphi_1(x),$$

where each φ_ℓ is an affine transformation followed by a nonlinear activation.

If the activation functions were linear, then the whole network would still be linear, since a composition of linear maps is again linear. Thus, the nonlinear activation is essential: it is what makes neural networks genuinely expressive.

12.1.2 Why training a neural network is difficult

The number of layers and neurons is a priori unknown and must be chosen by the user. Before discussing that issue, let us interpret a neural network as a function approximator.

Recall what happens in polynomial regression. Given data points

$$\{(x_i, y_i)\}_{i=1}^n,$$

we look for a polynomial p_m of degree m that approximates the data in a least-squares sense:

$$\min_{p_m} \sum_{i=1}^{n} \big|y_i - p_m(x_i)\big|^2.$$

If the model is linear in its unknown coefficients, then this becomes a linear least-squares problem. For polynomials, for instance, one can solve the problem by a linear system.

More generally, if a model has the form

$$f(x; w) = \sum_{j=1}^{m} w_j \phi_j(x),$$

where the ϕ_j are fixed basis functions (monomials, Fourier modes, wavelets, and so on), then the dependence on the coefficients $w = (w_1, \ldots, w_m)$ is linear. In that case, the loss function is quadratic in the unknowns, and the minimization problem is relatively easy.

Neural networks are different. The model is nonlinear in the parameters because of the composition of several layers and nonlinear activation functions. As a result, the training problem becomes a genuine nonlinear optimization problem. This makes it much harder to solve.

There are two main difficulties:

1. **High dimensionality of the parameter space.** A neural network may contain a very large number of weights and biases. The optimization problem is therefore posed in a high-dimensional space.

2. **Nonconvexity.** Since the network depends nonlinearly on the parameters, the loss function is typically nonconvex. Hence, it may have many local minima, and the result of training may depend on the initialization.

12.1.3 Gradient-based training

To train a neural network, one typically minimizes a loss function. Let us denote by

$$L(w)$$

the loss, where w collects all the weights and biases of the network.

Given an initial guess $w^{(0)}$, gradient descent generates a sequence

$$w^{(k+1)} = w^{(k)} - \alpha_k \nabla L\big(w^{(k)}\big),$$

where α_k is the learning rate.

The idea is classical: the gradient $\nabla L(w)$ points in the direction of steepest increase of the loss, so moving in the opposite direction should decrease it. This is the same intuition used in classical optimization and in numerical linear algebra when searching for descent directions.

However, in neural network training there is an important issue: the gradient is a very large vector, since the number of parameters is large. Moreover, computing it can be expensive because the loss depends on all layers and all data points. This is why training neural networks is a challenging optimization problem.

12.1.4 Transition to PoDNN

With this background in mind, we are now ready to introduce the PoDNN method. The main idea is to use POD for dimensionality reduction and neural networks only for a specific part of the approximation pipeline. This makes PoDNN a reduced-order model that combines a classical physics-based reduction technique with a data-driven component.

12.1.5 Stochastic gradient descent

To reduce the computational cost of training, one typically uses a stochastic version of gradient descent. Instead of evaluating the gradient of the loss using all the training samples at each step, one computes it using either a single sample or, more commonly, a small *mini-batch*.

More precisely, if the loss has the form

$$L(w) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(w),$$

then in stochastic gradient descent (SGD) one replaces the full gradient

$$\nabla L(w) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(w)$$

with an approximation computed on a randomly selected subset of the data.

This random subset is typically refreshed at each iteration. The advantage is that the cost of each update is much smaller than in full gradient descent. The disadvantage is that the loss curve is no longer smooth: the stochastic nature of the updates introduces oscillations and fluctuations.

An *epoch* is completed when all the training samples have been used once. After one epoch, one starts again with a new random order of the data.

In practice, more sophisticated first-order methods are often used. A popular example is the *Adam* algorithm, which can be viewed as an adaptive stochastic optimization method combining features of SGD and momentum-type corrections. These methods are widely used in machine learning libraries and are standard tools for neural network training.

12.1.6 Underfitting, overfitting, and validation

The choice of the model complexity is crucial. Consider again, for simplicity, the one-dimensional case. If the model is too simple, it may fail to capture the structure of the data. This is the case of *underfitting*. In that case, the bias is large, while the variance is small.

On the other hand, if the model is too flexible, it may follow the training data too closely and start capturing noise. This is the case of *overfitting*. In that case, the bias is small, but the variance is large.

This phenomenon is already familiar in polynomial regression. Given a family of polynomials of increasing degree, one may obtain:

- too low a degree $\Rightarrow$ underfitting,
- too high a degree $\Rightarrow$ overfitting.

A standard strategy to prevent overfitting is to split the available data into different sets:

- a *training set*, used to fit the model;
- a *validation set*, used to tune the hyperparameters and monitor generalization;
- optionally, a *test set*, used only at the end to assess the final performance.

The validation error is monitored during training in order to choose the best architecture or the best hyperparameters, such as the number of layers, the number of neurons, or the learning rate.

12.1.7 How should one choose a neural network architecture?

A natural question is the following: how many layers and how many neurons should one use?

This is still a difficult problem in practice. In general, one chooses the architecture empirically and then adjusts it by comparing the performance on the validation set. The goal is to find a good balance between:

- enough expressive power to fit the data;
- enough regularity to avoid overfitting.

Thus, model selection remains an important part of neural network design.

12.1.8 Classical approximation results

It is worth stressing that the theoretical study of neural networks is an active and long-standing research area. Some fundamental approximation results date back to the 1980s and 1990s.

The first key result is the *universal approximation theorem*, which states that neural networks with a suitable nonlinearity can approximate continuous functions arbitrarily well.

Theorem 12.1 (Universal approximation, informal statement)**.** *Let $K \subset \mathbb{R}^d$ be compact and let*

$$f : K \to \mathbb{R}$$

be continuous. Then, for every $\varepsilon > 0$, there exists a neural network $\mathcal{N}$ such that

$$\|f - \mathcal{N}\|_{L^\infty(K)} < \varepsilon.$$

This result is conceptually similar to approximation theorems for polynomials. The key point is that neural networks can approximate very general functions, although the theorem does not tell us how to construct an efficient architecture in practice.

A more general class of approximation results shows that the same conclusion remains valid for many activation functions, provided they are not polynomials.

12.1.9 Quantitative approximation results

More recent theory gives quantitative estimates on the size of the neural network needed to reach a prescribed accuracy. Roughly speaking, if a target function is sufficiently smooth, then one can construct a network whose size depends explicitly on the desired tolerance ε.

These results show that:

- the required number of layers may grow with $\log(1/\varepsilon)$,
- the number of active parameters must also increase as the accuracy requirement becomes more stringent,
- the regularity of the target function plays an important role.

This is useful because it gives a more precise answer to the question: *How large should the network be to achieve a given accuracy?*

The main message is that neural networks are not only practical tools, but also objects with a rich mathematical theory behind them. This theoretical viewpoint helps explain why deep learning works so well in many applications.

12.1.10 Transition to the next part

We now have the main ingredients needed to move on to the reduced-order setting. In the next part, we will combine POD with neural networks, leading to the PoDNN method.

12.1.11 The PoDNN idea

We now close the mathematical detour and turn to the *POD-NN* method.

Recall that, given a parameter $\mu \in \mathbb{R}^p$, the POD approximation of the high-fidelity solution $u_h(\mu)$ is written as

$$u_h(\mu) \approx V u_N(\mu),$$

where:

- $V \in \mathbb{R}^{N_h \times N}$ is the matrix whose columns are the POD basis functions,
- $u_N(\mu) \in \mathbb{R}^N$ is the vector of reduced coordinates.

The reduced coefficients are usually obtained by projection:

$$u_N(\mu) = V^T u_h(\mu),$$

provided that the POD basis is orthonormal.

This is the key point: in the standard reduced-basis setting, computing the reduced solution still requires the evaluation of the high-fidelity solution or, more generally, the assembly and solution of a parameter-dependent reduced system. This is the intrusive part of the method.

The goal of POD-NN is to replace the map

$$\mu \mapsto u_N(\mu)$$

with a non-intrusive surrogate based on a neural network.

12.1.12 Learning the reduced coordinates

We now view the reduced coefficient map as a function

$$u_N : \mathbb{R}^p \to \mathbb{R}^N.$$

The idea is to train a neural network

$$\pi(\,\cdot\,; w) : \mathbb{R}^p \to \mathbb{R}^N$$

so that

$$\pi(\mu; w) \approx u_N(\mu) = V^T u_h(\mu).$$

Given training data $\{\mu_i\}_{i=1}^{n_s}$ and the corresponding snapshots

$$u_h(\mu_i), \qquad i = 1, \dots, n_s,$$

we compute the projected coefficients

$$u_N(\mu_i) = V^T u_h(\mu_i),$$

and train the network by minimizing the loss

$$\mathcal{L}(w) = \frac{1}{n_s} \sum_{i=1}^{n_s} \left\| \pi(\mu_i; w) - V^T u_h(\mu_i) \right\|^2.$$

After training, we freeze the network and denote the trained model by $\widehat{\pi}$. The POD-NN approximation is then given by

$$u_{\text{POD-NN}}(\mu) = V\,\widehat{\pi}(\mu).$$

Thus, for a new parameter value μ, the reduced solution is obtained directly from the network, without solving a reduced Galerkin problem.

12.1.13 Why this is useful

The main advantage is that the online stage becomes extremely cheap. Once the network has been trained, evaluating $\widehat{\pi}(\mu)$ is fast, and the approximation of the full solution is recovered by a simple multiplication with the POD basis.

This is especially attractive when:

- the reduced basis is effective, so that $N \ll N_h$;
- the parameter dimension p is moderate;
- one wants to avoid the assembly and solution of parameter-dependent reduced systems.

In that sense, POD-NN combines:

- the physical interpretability of POD,
- the flexibility of neural networks,
- and a very efficient online evaluation stage.

12.1.14 Time-dependent problems

The same idea extends naturally to time-dependent problems. In that case, the high-fidelity solution depends on both time and parameters:

$$u_h = u_h(t, \mu).$$

The POD basis is still built offline and remains independent of time and parameters, while the neural network is trained to approximate the coefficient map

$$(t, \mu) \mapsto V^T u_h(t, \mu).$$

Accordingly, one trains a network

$$\pi(\,\cdot\,; w) : \mathbb{R}^{p+1} \to \mathbb{R}^N$$

such that

$$\pi(t, \mu; w) \approx V^T u_h(t, \mu).$$

The loss function now uses snapshots collected at different times and parameter values:

$$\mathcal{L}(w) = \frac{1}{n_s} \sum_{i=1}^{n_s} \left\| \pi(t_i, \mu_i; w) - V^T u_h(t_i, \mu_i) \right\|^2.$$

Once trained, the surrogate provides an approximation of the solution at any time and parameter without solving a reduced dynamical system step by step. This is a major difference with standard time-dependent reduced basis methods, where one still has to advance the reduced system in time.

12.1.15 Summary

The POD-NN strategy can be summarized as follows:

1. build a POD basis from high-fidelity snapshots;
2. compute the reduced coefficients by projection offline;
3. train a neural network to learn the map from parameters (and possibly time) to reduced coefficients;
4. reconstruct the approximate solution by combining the POD basis with the trained network.

The method is therefore non-intrusive in the online phase and very efficient once training is completed.

12.1.16 Next step

If POD is no longer a good approximation tool, one can go one step further and replace the linear POD encoder with a nonlinear neural-network-based encoder, for instance an autoencoder. This leads to deep-learning-based reduced-order models, which we will discuss next.

Chapter 13

Lab 6 - The POD-NN method

13.1 Introduction

Today we return to model order reduction and explore a hybrid approach: **POD-NN** (Proper Orthogonal Decomposition + Neural Network). The idea is to replace the Galerkin projection step of the reduced basis method with a neural network that learns the map from parameters to POD coefficients directly.

13.1.1 Recap of Standard Reduced Basis (RB) Method

We consider a parametric stationary PDE:

$$\mathcal{A}(\boldsymbol{\mu})u(\boldsymbol{\mu}) = f(\boldsymbol{\mu}), \qquad \boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p.$$

After finite element discretization, we obtain a high-dimensional system:

$$\mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h(\boldsymbol{\mu}) = \mathbf{F}_h(\boldsymbol{\mu}), \qquad \mathbf{u}_h \in \mathbb{R}^{N_h}.$$

The POD basis $\mathbf{V}_n \in \mathbb{R}^{N_h \times n}$ (columns orthonormal) defines a reduced space. Any solution is approximated as

$$\mathbf{u}_h(\boldsymbol{\mu}) \approx \mathbf{V}_n \mathbf{c}(\boldsymbol{\mu}), \qquad \mathbf{c}(\boldsymbol{\mu}) \in \mathbb{R}^n.$$

The **optimal coefficients** (best approximation in the reduced space) are given by orthogonal projection:

$$\mathbf{c}_{\text{proj}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{u}_h(\boldsymbol{\mu}).$$

In the standard RB method, the coefficients are obtained by solving the projected (Galerkin) system:

$$\mathbf{V}_n^\top \mathbf{A}_h(\boldsymbol{\mu})\mathbf{V}_n \, \mathbf{c}_{\text{RB}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{F}_h(\boldsymbol{\mu}).$$

In general, $\mathbf{c}_{\text{RB}}(\boldsymbol{\mu}) \neq \mathbf{c}_{\text{proj}}(\boldsymbol{\mu})$, but error estimates exist.

13.1.2 Motivation for POD-NN

The RB method requires:

- Affine parameter dependence of $\mathbf{A}_h(\boldsymbol{\mu})$ and $\mathbf{F}_h(\boldsymbol{\mu})$ to enable efficient online assembly.
- The PDE to be linear (or treat nonlinearities with hyper-reduction techniques, which are complex).

When these conditions are not met (e.g., nonlinear PDEs, non-affine parameters), the RB approach becomes impractical.

POD-NN bypasses these difficulties:

- We directly learn the map $\boldsymbol{\mu} \mapsto \mathbf{c}_{\text{proj}}(\boldsymbol{\mu})$ using a neural network.

- No need to assemble or project the PDE operators.
- The network is trained on a dataset of parameter–solution pairs.

Moreover, because we learn the *optimal* coefficients (the projection), the POD-NN can potentially outperform standard RB in accuracy.

13.2 Test Case: A Nonlinear Stationary Heat Problem

We consider a 2D rod-like object with five parameters (see figure). The PDE is:

$$-\nabla \cdot (\kappa(u)\nabla u) + \alpha u^4 = f \quad \text{in } \Omega,$$

with Dirichlet boundary conditions on three inner circles (temperatures T_1, T_2, T_3) and homogeneous Neumann elsewhere. The parameters are:

- T_1, T_2, T_3 – prescribed temperatures (boundary conditions),
- f – uniform volumetric heat source (given on logarithmic scale),
- α – radiation coefficient (logarithmic scale).

The thermal conductivity $\kappa(u)$ may depend nonlinearly on u (not detailed here). The term αu^4 models radiative heat loss.

The FE discretization yields $N_h \approx 4000$ degrees of freedom. The problem is **nonlinear** and **non-affine** in parameters, making standard RB difficult.

13.3 Dataset and POD Decomposition

A precomputed dataset is provided:

```
mu, U = load_dataset()    # mu: (N_samples, 5), U: (N_samples, N_h)
```

The first few solutions can be visualised using `FE.plot`.

13.3.1 Step 1: Compute POD Basis

Split the data into training and test sets (e.g., 80% / 20%). Build the snapshot matrix $\mathbf{X} = \mathbf{U}_{\text{train}}^\top \in \mathbb{R}^{N_h \times N_{\text{train}}}$. Perform SVD:

```
from scipy.linalg import svd

X = U_train.T
V, s, Wt = svd(X, full_matrices=False)
```

Here $\mathbf{V} \in \mathbb{R}^{N_h \times N_{\text{train}}}$ contains the left singular vectors (POD modes) as columns.

13.3.2 Step 2: Choose the Reduced Dimension n

Plot the singular values (use a logarithmic scale). Look for a decay elbow. Also compute the cumulative energy:

$$E(n) = \frac{\sum_{i=1}^{n} s_i^2}{\sum_{i=1}^{N_{\text{train}}} s_i^2}.$$

Select n such that $E(n) \geq 0.999$ (99.9%). Let $\mathbf{V}_n = \mathbf{V}(:, 1:n)$.

13.3.3 Step 3: Validate the Basis via Projection Error

Compute the projection of all test solutions onto the reduced space and evaluate the relative error:

```
coeff_test = Vn.T @ U_test.T       # shape (n, N_test)
U_proj = (Vn @ coeff_test).T       # shape (N_test, N_h)
rel_errors = np.linalg.norm(U_test - U_proj, axis=1) / np.linalg.norm(U_test, axis=1)
mean_rel_err = np.mean(rel_errors)
print(f"Mean projection error: {mean_rel_err*100:.3f}%")
```

If the error is small (e.g., $< 1\%$), the basis is adequate.

13.4 Step 4: Train the Neural Network (POD-NN)

We now want a neural network $\Phi : \mathbb{R}^p \to \mathbb{R}^n$ that approximates $\mathbf{c}_{\text{proj}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{u}_h(\boldsymbol{\mu})$.

13.4.1 Prepare Training Data

For each training parameter $\boldsymbol{\mu}_i$, compute the projection coefficients:

```
coeff_train = Vn.T @ U_train.T   # shape (n, N_train)
```

We have inputs $\boldsymbol{\mu}_i$ (size $p = 5$) and outputs $\mathbf{c}_i$ (size n).

13.4.2 Network Architecture

A simple fully connected network (e.g., 3 hidden layers with 50 neurons each, ReLU activations, no activation at output) is a good start. In the lab's wrapper:

```
net = Dense(5, 50, activation='relu') + \
      Dense(50, 50, activation='relu') + \
      Dense(50, 50, activation='relu') + \
      Dense(50, n, activation=None)
dnn = DFNN(net).to(device)
```

13.4.3 Loss Function and Training

Use the mean squared error (MSE) loss:

$$\mathcal{L} = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \|\Phi(\boldsymbol{\mu}_i) - \mathbf{c}_i\|^2.$$

Training options:

- Optimizer: LBFGS (full batch) for small datasets, or Adam with mini-batches for larger ones.
- Validation split: use a portion of the training set (e.g., 20%) to monitor overfitting.
- Early stopping: the training routine can return the model with best validation loss.

13.4.4 Online Prediction

Once trained, for a new parameter $\boldsymbol{\mu}^*$, we obtain coefficients $\hat{\mathbf{c}} = \Phi(\boldsymbol{\mu}^*)$ and reconstruct the full solution:

$$\hat{\mathbf{u}}_h = \mathbf{V}_n \hat{\mathbf{c}}.$$

13.5 Comparison with Standard RB (Where Applicable)

In problems where standard RB is feasible (linear, affine), POD-NN may still be useful:

- It may yield lower errors because it learns the optimal projection coefficients rather than the Galerkin ones.
- However, it requires a sufficiently large and representative training set.
- The online cost is extremely low (a few matrix multiplications in the network).

For nonlinear or non-affine problems, POD-NN is often the only practical model reduction approach without heavy hyper-reduction.

13.6 Exercises for the Lab

1. Load the dataset, split into train/test.
2. Compute the SVD of the training snapshot matrix.
3. Plot the singular values and choose n (e.g., where energy $> 99.9\%$).
4. Compute the projection error on the test set – verify that the basis is good.
5. Build and train a neural network to map $\boldsymbol{\mu}$ to the POD coefficients.
6. Evaluate the POD-NN error on the test set and compare with the projection error.
7. (Optional) Try different network architectures and activations (ReLU, GeLU) to improve performance.

13.7 Conclusion

POD-NN combines the dimension reduction power of POD with the flexibility of neural networks. It is particularly attractive for nonlinear or non-affine parametric PDEs where classical reduced basis methods fail or become overly complex. The same idea can be extended to time-dependent problems by learning the map from parameters to the entire time series of coefficients (or by learning a time stepper).

13.8 Introduction

In this lab we combine Proper Orthogonal Decomposition (POD) with Neural Networks (NN) to build a surrogate model for a nonlinear parametric PDE. This approach is useful when standard reduced basis (RB) methods are difficult to apply due to nonlinearities or non-affine parameter dependence.

13.9 Recap: Standard Reduced Basis (RB) Method

Consider a parametric PDE in stationary form:

$$\mathcal{A}(\boldsymbol{\mu})u(\boldsymbol{\mu}) = f(\boldsymbol{\mu}), \qquad \boldsymbol{\mu} \in \mathcal{P} \subset \mathbb{R}^p.$$

After finite element discretization we obtain a large linear system:

$$\mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h(\boldsymbol{\mu}) = \mathbf{F}_h(\boldsymbol{\mu}), \qquad \mathbf{u}_h \in \mathbb{R}^{N_h}.$$

Given a POD basis $\mathbf{V}_n \in \mathbb{R}^{N_h \times n}$ (orthonormal columns), the reduced basis approximation is

$$\mathbf{u}_h(\boldsymbol{\mu}) \approx \mathbf{V}_n \mathbf{c}(\boldsymbol{\mu}), \quad \mathbf{c}(\boldsymbol{\mu}) \in \mathbb{R}^n.$$

The optimal coefficients (best approximation in the reduced space) are obtained by orthogonal projection:

$$\mathbf{c}_{\text{proj}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{u}_h(\boldsymbol{\mu}).$$

In standard RB, the coefficients are found by solving the Galerkin projected system:

$$\mathbf{V}_n^\top \mathbf{A}_h(\boldsymbol{\mu}) \mathbf{V}_n \, \mathbf{c}_{\text{RB}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{F}_h(\boldsymbol{\mu}).$$

This requires:

- Affine parameter dependence of $\mathbf{A}_h(\boldsymbol{\mu})$ and $\mathbf{F}_h(\boldsymbol{\mu})$ for efficient online assembly,
- The PDE to be linear (or hyper-reduction techniques for nonlinearities).

13.9.1 Motivation for POD-NN

When the PDE is nonlinear or the parameter dependence is non-affine, the standard RB approach becomes difficult. The **POD-NN** method bypasses these issues:

- We directly learn the map $\boldsymbol{\mu} \mapsto \mathbf{c}_{\text{proj}}(\boldsymbol{\mu})$ using a neural network.
- No projection of operators is needed – only a dataset of parameter–solution pairs.
- The network learns the optimal coefficients, which can lead to better accuracy than Galerkin RB.

13.10 Test Case: Nonlinear Stationary Heat Equation on a Rod

The geometry is a 2D rod with five parameters (see figure). The PDE is:

$$-\nabla \cdot (\kappa(u) \nabla u) + \alpha u^4 = f \quad \text{in } \Omega,$$

with Dirichlet conditions on three inner circles (temperatures T_1, T_2, T_3) and homogeneous Neumann elsewhere. The parameters are:

- T_1, T_2, T_3 – boundary temperatures,
- f – uniform heat source (log scale),
- α – radiation coefficient (log scale).

The problem is **nonlinear** (due to $\kappa(u)$ and the u^4 term) and **non-affine** in parameters, making standard RB impractical.

13.11 Dataset and POD Decomposition

A precomputed dataset is provided:

```
mu, U = load_dataset()   # mu: (500, 5), U: (500, N_h) with N_h \approx 4000
```

13.11.1 Split into Training and Test

```
N_total = U.shape[0]
N_train = N_total // 2   # 250
U_train = U[:N_train, :]
mu_train = mu[:N_train, :]
U_test  = U[N_train:, :]
mu_test  = mu[N_train:, :]
```

13.11.2 Compute the POD Basis

The snapshot matrix (following the mathematical convention) is $\mathbf{X} = \mathbf{U}_{\text{train}}^{\top} \in \mathbb{R}^{N_h \times N_{\text{train}}}$.

```
from scipy.linalg import svd

X = U_train.T
V, s, Wt = svd(X, full_matrices=False)
```

Here $\mathbf{V} \in \mathbb{R}^{N_h \times N_{\text{train}}}$ contains the left singular vectors (POD modes) as columns.

13.11.3 Choosing the Reduced Dimension n

Plot the singular values (skip the first for better scaling):

```
import matplotlib.pyplot as plt
plt.semilogy(s[1:], 'o-')
plt.xlabel('Mode index')
plt.ylabel('Singular value')
plt.grid(True)
```

A sharp decay indicates that POD is suitable. For this problem, $n = 5$ captures most of the energy. The POD matrix is:

```
n = 5
Vn = V[:, :n]    # shape (N_h, n)
```

13.11.4 Projection Error on Training and Test

The projection of all solutions onto the reduced space is computed as:

```
U_train_proj = U_train @ Vn @ Vn.T    # shape (N_train, N_h)
```

The relative error (Euclidean norm) per sample:

```
rel_err_train = np.linalg.norm(U_train - U_train_proj, axis=1) / np.linalg.norm(U_train, a
mean_rel_err_train = np.mean(rel_err_train)
print(f"Projection error (train): {mean_rel_err_train*100:.3f}%")
```

For the test set:

```
U_test_proj = U_test @ Vn @ Vn.T
rel_err_test = np.linalg.norm(U_test - U_test_proj, axis=1) / np.linalg.norm(U_test, axis=
mean_rel_err_test = np.mean(rel_err_test)
```

Typical values: $\approx 0.2\%$ on training, slightly higher on test.

13.12 Training the Neural Network (POD-NN)

We now build a neural network $\Phi : \mathbb{R}^p \to \mathbb{R}^n$ that maps parameters to the POD coefficients $\mathbf{c}_{\text{proj}}(\boldsymbol{\mu})$.

13.12.1 Prepare the Coefficient Dataset

For each training parameter, compute the coefficients:

```
C_train = U_train @ Vn    # shape (N_train, n)
```

Similarly for test:

```
C_test = U_test @ Vn
```

13.12.2 Design the Network Architecture

A simple fully connected network with 3 hidden layers (example):

```
import torch
from DRROMS.nn import Dense, DFNN

p = mu.shape[1]    # 5
n = 5
net = Dense(p, 10, activation='gelu') + \
      Dense(10, 10, activation='gelu') + \
      Dense(10, n, activation=None)
dnn = DFNN(net).to(device)
```

Important: Use a non-polynomial activation (e.g., GeLU, ReLU) to satisfy universal approximation. Avoid polynomial activations like x^2.

13.12.3 Training

Use mean squared error (MSE) loss. The lab wrapper allows:

```
dnn.train(mu_train, C_train,
          n_epochs=500,
          batch_size=None,          # full batch
          loss='mse_euclidean',
          optimizer='LBFGS',
          validation_split=0.2)     # 20% of training used for validation
```

The validation split helps avoid overfitting; the best model (lowest validation loss) is kept.

13.13 Evaluating the POD-NN Model

After training, evaluate on the test set:

```
C_pred = dnn.predict(mu_test)                    # shape (N_test, n)
U_pred = C_pred @ Vn.T                           # reconstruct full field
rel_err_nn = np.linalg.norm(U_test - U_pred, axis=1) / np.linalg.norm(U_test, axis=1)
mean_rel_err_nn = np.mean(rel_err_nn)
print(f"POD-NN test error: {mean_rel_err_nn*100:.3f}%")
```

Compare with the projection error (which is the optimal achievable with the given basis). The POD-NN error is usually slightly larger but still very small if the network is well trained.

13.14 Discussion

13.14.1 When to Use POD-NN vs. Standard RB

- **Standard RB**: Linear PDE, affine parameter dependence, small n (e.g., $n < 50$). Provides rigorous error bounds.
- **POD-NN**: Nonlinear or non-affine problems, large parameter space, or when implementation of hyper-reduction is too complex. Requires sufficient training data.

13.14.2 Advantages of POD-NN

- No need to assemble or project the PDE operators – only a dataset is required.

- The network learns the optimal projection coefficients, potentially giving better accuracy than Galerkin RB.
- Online evaluation is extremely fast (simple feed-forward through the network).

13.14.3 Practical Tips

- Always check the singular value decay before attempting POD-NN. If the decay is slow, POD may not be effective.
- Use validation split to monitor overfitting.
- Start with a small network and increase complexity only if needed.
- For the loss function, use the MSE in Euclidean norm (the lab's ''mse_euclidean'').

13.15 Exercises

1. Load the dataset, split into train/test, compute the POD basis, and choose n based on the singular values.
2. Compute the projection error on the test set – verify that the basis captures the solution manifold well.
3. Build and train a neural network to map parameters to POD coefficients. Experiment with different architectures (hidden layers, neurons, activations).
4. Compare the POD-NN test error with the projection error. Discuss possible reasons for any gap.
5. (Optional) Try to improve the network by tuning hyperparameters (learning rate, batch size, epochs, optimizers).

13.16 Conclusion

The POD-NN method combines the dimension reduction of POD with the flexibility of neural networks. It is particularly well-suited for nonlinear or non-affine parametric PDEs where classical reduced basis methods become cumbersome. The steps are:

1. Generate or load a dataset of parameter–solution pairs.
2. Compute the POD basis and truncate.
3. Project the dataset onto the POD coefficients.
4. Train a neural network to map parameters to coefficients.
5. Use the trained network for fast online predictions.

This approach is the focus of the current lab and will be part of the assignments.

13.17 Finite Elements in Python

13.17.1 Introduction

We discuss finite elements (FE) in Python using the `PINIX` package, but we employ a more user-friendly version from a custom GitHub package. The same package also handles reduced order modeling and neural networks.

13.17.2 Mesh and Finite Element Space

Define a domain and generate a mesh:

```
import DRROMS.bases as FE

domain = FE.Rectangle((0,0), (1,1))
mesh = FE.mesh_domain(domain, step_size=0.5)
FE.plot(mesh)
```

Build a continuous Galerkin space of degree 1 (piecewise linear):

```
Vh = FE.finite_element_space(mesh, continuous_galerkin=1)
```

13.17.3 Function–Vector Correspondence

Interpolate a function $u(x, y) = x^2 + y^2$:

```
u = lambda x: x[0]**2 + x[1]**2
uh = FE.interpolate(u, Vh)
```

Get the vector of nodal values: `uh.vector()`. Reconstruct a function from a vector:

```
u_vec = np.array([...])    # length = dim(Vh)
uh = FE.function(u_vec, Vh)
```

13.17.4 PDE: Stationary Heat Equation

We solve $-\nabla \cdot (\sigma \nabla u) = f$ with Dirichlet and Neumann conditions. Weak form:

$$\int_\Omega \sigma \nabla u \cdot \nabla v \, dx = \int_\Omega f v \, dx + \int_{\Gamma_N} \phi v \, ds.$$

Assembly and solution:

```
a = sigma_h * inner(grad(u), grad(v)) * dx
L = f * v * dx + phi * v * ds
A = FE.assemble(a, Vh)
F = FE.assemble(L, Vh)
bc = FE.DirichletBC(Vh, g, boundary_function)
A, F = FE.apply_boundary_conditions(A, F, bc)
u_vec = spsolve(A, F)
uh = FE.function(u_vec, Vh)
```

13.18 Reduced Basis Method (RBM)

13.18.1 The Cookie Problem

A plate with four disks. Conductivity:

$$\sigma(x; \boldsymbol{\mu}) = \sigma_{\text{out}} + \sum_{j=0}^{3} \mu_j^2 \mathbf{1}_{\Omega_j}(x), \quad \sigma_{\text{out}} = 0.001, \ \boldsymbol{\mu} \in [0, 3]^4.$$

We collect snapshots $\mathbf{u}_h(\boldsymbol{\mu}_i)$ and build a POD basis $\mathbf{V}_n$.

13.18.2 POD via SVD

Snapshot matrix (samples × DoFs) $\mathbf{U} \in \mathbb{R}^{N_s \times N_h}$. For training $\mathbf{U}_{\text{train}}$, set $\mathbf{X} = \mathbf{U}_{\text{train}}^\top$ and compute

$$\mathbf{X} = \mathbf{V}\boldsymbol{\Sigma}\mathbf{W}^\top.$$

Choose n from singular value decay or energy criterion:

$$E(n) = \frac{\sum_{i=1}^{n} \sigma_i^2}{\sum_{i=1}^{N_{\text{train}}} \sigma_i^2}.$$

13.18.3 Galerkin Projection

Reduced system: $\mathbf{V}_n^\top \mathbf{A}_h(\boldsymbol{\mu})\mathbf{V}_n\, \mathbf{c} = \mathbf{V}_n^\top \mathbf{F}_h(\boldsymbol{\mu})$. For affine parameter dependence, preassemble $\mathbf{V}_n^\top \mathbf{A}_q \mathbf{V}_n$.

13.18.4 Time-Dependent Problems

Full order: $\mathbf{M}\dot{\mathbf{u}}_h + \mathbf{A}_h(\boldsymbol{\mu})\mathbf{u}_h = \mathbf{F}_h$. Backward Euler:

$$\left(\frac{\mathbf{M}}{\Delta t} + \mathbf{A}_h\right)\mathbf{u}_h^{j+1} = \frac{\mathbf{M}}{\Delta t}\mathbf{u}_h^{j} + \mathbf{F}_h^{j+1}.$$

Reduced: same with $\mathbf{M}_n = \mathbf{V}_n^\top \mathbf{M}\mathbf{V}_n$, $\mathbf{A}_n = \mathbf{V}_n^\top \mathbf{A}_h \mathbf{V}_n$, $\mathbf{F}_n = \mathbf{V}_n^\top \mathbf{F}_h$.

13.19 Neural Networks

13.19.1 Dense Layer

$$\mathbf{y} = \rho(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

with activation ρ (e.g., ReLU, GeLU). Avoid polynomial activations (violate universal approximation).

13.19.2 PyTorch Implementation

Create a layer:

```
L = Dense(d_in, d_out, activation='gelu')
```

Move to GPU:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
L = L.to(device)
```

Forward pass with batching: input $(N_{\text{batch}}, d_{\text{in}})$.

13.19.3 Training a Network

Define MSE loss:

```
def mse_loss(y_true, y_pred):
    diff = y_true - y_pred
    return diff.pow(2).sum(dim=-1).mean()
```

Use `dnn.train(...)` with optimizer ('LBFGS' or 'Adam'), batch size, epochs, validation split.

13.20 POD-NN: Hybrid Approach

13.20.1 Motivation

For nonlinear or non-affine parametric PDEs, standard RB is difficult. POD-NN learns the map $\boldsymbol{\mu} \mapsto \mathbf{c}_{\text{proj}}(\boldsymbol{\mu}) = \mathbf{V}_n^\top \mathbf{u}_h(\boldsymbol{\mu})$ using a neural network.

13.20.2 Steps

1. Load or generate dataset $\{(\boldsymbol{\mu}_i, \mathbf{u}_i)\}$.
2. Split into train/test. Compute POD basis $\mathbf{V}_n$ from training snapshots.
3. Project training solutions to coefficients $\mathbf{c}_i = \mathbf{V}_n^\top \mathbf{u}_i$ (using proper shape handling).
4. Train a neural network Φ to map $\boldsymbol{\mu}_i$ to $\mathbf{c}_i$.
5. For a new parameter, predict coefficients $\hat{\mathbf{c}} = \Phi(\boldsymbol{\mu}^*)$ and reconstruct $\hat{\mathbf{u}} = \mathbf{V}_n \hat{\mathbf{c}}$.

13.20.3 Shape Considerations

In code, $\mathbf{U}$ is $(N_{\text{samples}}, N_h)$. Then:

```
C = U @ Vn          # coefficients, shape (N_samples, n)
U_proj = C @ Vn.T   # projection, shape (N_samples, N_h)
```

For a single solution $\mathbf{u}$ (shape $(N_h,)$): coefficients $\mathbf{c} = \mathbf{V}_n^\top \mathbf{u}$.

13.20.4 Error Decomposition

For any solution $\mathbf{u}$ and network prediction $\hat{\mathbf{c}}$, let $\mathbf{c}_{\text{proj}} = \mathbf{V}_n^\top \mathbf{u}$, $\hat{\mathbf{u}} = \mathbf{V}_n \hat{\mathbf{c}}$. Then

$$\|\mathbf{u} - \hat{\mathbf{u}}\|^2 = \underbrace{\|\mathbf{u} - \mathbf{V}_n \mathbf{c}_{\text{proj}}\|^2}_{\text{projection error}} + \underbrace{\|\mathbf{c}_{\text{proj}} - \hat{\mathbf{c}}\|^2}_{\text{coefficient error}}.$$

This holds because $\mathbf{u} - \mathbf{V}_n \mathbf{c}_{\text{proj}}$ is orthogonal to the column space of $\mathbf{V}_n$ and $\mathbf{V}_n(\mathbf{c}_{\text{proj}} - \hat{\mathbf{c}})$ lies in that space. The norm of the latter equals $\|\mathbf{c}_{\text{proj}} - \hat{\mathbf{c}}\|$ due to orthonormality.

13.20.5 Practical Training

Example for the nonlinear rod problem (5 parameters, $N_h \approx 4000$):

```
p = 5; n = 5   # choose n via SVD
net = Dense(p, 10, activation='gelu') + \
      Dense(10, 10, activation='gelu') + \
      Dense(10, n, activation=None)
dnn = DFNN(net).to(device)
dnn.train(mu_train, C_train,
          n_epochs=500,
          loss='mse_euclidean',
          optimizer='LBFGS',
          validation_split=0.2)
```

After training, evaluate:

```
C_pred = dnn.predict(mu_test)
U_pred = C_pred @ Vn.T
rel_err = np.linalg.norm(U_test - U_pred, axis=1) / np.linalg.norm(U_test, axis=1)
mean_err = np.mean(rel_err)
```

Typical test errors are around 0.3%–1%.

13.20.6 Hyperparameter Tuning

- Increase n: reduces projection error but makes coefficient learning harder.
- Network depth/width: start simple, increase if training error is high.
- Optimizer: LBFGS for small datasets (full batch), Adam for larger with mini-batching.
- Validation split prevents overfitting.

13.21 Assignments (Exam Preparation)

Three assignments are available (choose one). They cover vector fields, non-homogeneous Dirichlet conditions, and the time-dependent Schrödinger equation. Submit the code a few days before the oral exam; during the exam we will discuss your implementation and ask for modifications.

13.22 Conclusion

These notes cover the full pipeline: finite element discretization, POD–Galerkin reduction (stationary and time-dependent), neural network training, and the hybrid POD-NN method. The error decomposition provides a clear guide for balancing basis size and network capacity. The techniques are directly applicable to the assignments and to real-world parametric PDEs where classical reduced basis methods are infeasible.

Chapter 14

Deep learning-based reduced order models (DL-ROM & POD-DL-ROM)

14.1 Deep-Learning Reduced-Order Models (DL-ROMs)

The goal of this lecture is to introduce *deep-learning reduced-order models* (DL-ROMs), a data-driven approach to reduced-order modeling, in the same spirit as the POD-NN strategy discussed previously.

14.1.1 From POD to DL-ROMs

The philosophy is still purely data-driven: the physics is not enforced through the model construction, but is only contained in the available snapshots. For time-dependent problems, the basic formulation treats time as a parameter, rather than as a true dynamical variable. This is a limitation of the approach, and it will be addressed later by introducing more advanced formulations.

Why move beyond POD? In some cases, the reduced basis obtained from POD is not sufficiently low-dimensional. In other words, the POD representation may still require a relatively large number of modes.

14.1.2 POD as a Linear Compression Method

Let u^h denote a high-fidelity snapshot. In POD, we approximate it as

$$u^h \approx V u^N,$$

where:

- V is the POD basis matrix,
- u^N is the reduced-coordinate representation.

The reduced coordinates are obtained by projection:

$$u^N = V^T u^h,$$

and therefore the reconstructed field is

$$u^h \approx V V^T u^h.$$

POD finds the best linear subspace of prescribed dimension by minimizing the projection error over the snapshot set. Equivalently, it provides the optimal low-rank approximation in the sense of the singular value decomposition (SVD).

14.1.3 POD-NN and Its Limitation

In the POD-NN approach, the POD basis is first computed offline. Then a neural network is trained to learn the map from the parameters to the reduced coordinates:

$$\mu \mapsto V^T u^h(\mu).$$

This is a two-stage procedure:

1. compute the POD basis from the data;
2. train a neural network to predict the reduced coefficients.

However, this approach still relies on a linear reduced space, since the POD basis is fixed once and for all.

14.1.4 Autoencoders as a Nonlinear Generalization of POD

Autoencoders generalize the POD idea by learning a nonlinear compression and reconstruction map.

- The *encoder* compresses high-dimensional data into a low-dimensional latent space.
- The *decoder* reconstructs the original field from the latent variables.

Let

$$\phi : \mathbb{R}^{N_h} \to \mathbb{R}^n, \qquad \psi : \mathbb{R}^n \to \mathbb{R}^{N_h},$$

with $n \ll N_h$, where:

- ϕ is the encoder,
- ψ is the decoder.

Given a snapshot $u^h(\mu_i)$, the autoencoder aims at minimizing the reconstruction error:

$$\psi(\phi(u^h(\mu_i))) \approx u^h(\mu_i).$$

A typical loss function is

$$\mathcal{L}_{\text{AE}} = \sum_i \left\| u^h(\mu_i) - \psi(\phi(u^h(\mu_i))) \right\|^2.$$

Unlike POD, this is a true optimization problem: the weights and biases of the neural networks must be learned.

14.1.5 The DL-ROM Idea

The DL-ROM framework combines:

- a nonlinear reduction map based on an autoencoder;
- a neural network that maps the input parameters to the latent space.

More precisely, one learns three maps:

$$\phi, \qquad \psi, \qquad \Phi,$$

where:

- ϕ is the encoder,
- ψ is the decoder,
- Φ maps the parameters to the latent space.

Thus, for a parameter value μ, the model predicts

$$\Phi(\mu) \approx \phi\big(u^h(\mu)\big),$$

and the reconstructed solution is obtained as

$$\hat{u}^h(\mu) = \psi(\Phi(\mu)).$$

14.1.6 Training and Inference

During training, one has access to snapshots of the form

$$\left\{\mu_i, u^h(\mu_i)\right\}_{i=1}^{N_s},$$

possibly extended to time-dependent data

$$\left\{(\mu_i, t_j),\, u^h(\mu_i, t_j)\right\}_{i,j}.$$

The training objective is to make the latent representation produced by Φ match the one produced by the encoder ϕ:

$$\Phi(\mu_i) \approx \phi\big(u^h(\mu_i)\big).$$

At the same time, the decoder must reconstruct the original snapshot:

$$\psi(\phi(u^h(\mu_i))) \approx u^h(\mu_i).$$

Therefore, the full training process can be formulated through a single loss function that learns the encoder, the decoder, and the parameter-to-latent map jointly.

14.1.7 Main Message

The key difference with POD is that DL-ROMs do not restrict the reduced representation to be linear. This increased flexibility may capture features that linear subspaces cannot represent efficiently, at the cost of a more complex training procedure.

14.1.8 Training the DL-ROM

Once the map from the input parameters to the encoded snapshot has been learned, the three neural networks in the architecture,

$$\phi, \qquad \psi, \qquad \Phi,$$

can be trained jointly by minimizing a loss function with respect to all their weights and biases.

Here:

- ϕ is the encoder, which compresses the solution;
- ψ is the decoder, which reconstructs the solution;
- Φ is the parameter-to-latent map.

The latent dimension n is a crucial design choice. It is an hyperparameter of the model, together with the number of layers, the number of neurons, and the overall network architecture.

14.1.9 Joint Learning Strategy

The purpose of the architecture is to learn a hidden representation of the data. The encoder ϕ compresses the high-dimensional snapshot into a low-dimensional latent vector, while the decoder ψ reconstructs the original field from that compressed representation.

The training objective is to minimize a loss function of the form

$$\mathcal{L} = \mathcal{L}(\phi, \psi, \Phi),$$

with respect to all the parameters of the three networks.

This approach can be implemented in different ways:

1. **Joint training**: all networks are trained simultaneously with a single loss function.
2. **Sequential training**: first the autoencoder (ϕ, ψ) is trained, and then Φ is learned on the resulting latent space.

For simplicity, the basic version discussed here relies on a single loss function that combines all the required terms.

14.1.10 Training Stage and Testing Stage

During training, the model is fitted on a set of snapshots

$$\left\{(\mu_i, u^h(\mu_i))\right\}_{i=1}^{N_s},$$

where μ_i denotes a parameter instance and $u^h(\mu_i)$ the corresponding high-fidelity solution.

The goal is to learn a map such that

$$\Phi(\mu_i) \approx \phi\big(u^h(\mu_i)\big),$$

and simultaneously

$$\psi\big(\phi\big(u^h(\mu_i)\big)\big) \approx u^h(\mu_i).$$

Once the training is completed, the weights and biases of the networks are fixed. At inference time, given a new parameter value μ, one simply evaluates

$$\hat{u}^h(\mu) = \psi(\Phi(\mu)).$$

This is computationally cheap: after training, the model only requires the evaluation of two neural networks.

14.1.11 Advantages and Limitations

The main advantage of the DL-ROM approach is its flexibility. Compared with classical reduced-order models, it can represent nonlinear structures that linear subspaces may fail to capture.

However, this comes at a price:

- the model needs training data;
- the training may require significant computational resources;
- the neural networks do not automatically enforce the governing physics;
- time is still treated as a parameter, so the causal structure of dynamical systems is not preserved.

Therefore, the model is highly expressive, but it is only *a posteriori* accurate, not necessarily *a priori* physically consistent.

14.1.12 Physics-Informed Extensions

To address the lack of physical constraints, one can enrich the loss function with additional terms derived from the governing equations.

For instance, if the solution must satisfy constraints such as incompressibility, one may add penalties involving derivatives of the learned fields. This leads to a physics-informed version of the architecture.

14.1.13 Choice of the Latent Dimension

A major practical question is how to choose the latent dimension n.

Unlike POD, where the singular values provide a natural criterion for truncation, the latent dimension in a DL-ROM has no direct physical interpretation. Nevertheless, for some simple stationary linear problems, theoretical results show that n can be related to the number of parameters p.

In particular, one may expect

$$p \leq n \leq 2p + 1$$

for certain classes of problems.

This gives at least a rough guideline, although it does not apply universally.

14.1.14 Convolutional Architectures

The encoder and decoder may also be implemented using convolutional layers. This is especially useful when the solution fields can be interpreted as images or spatially structured data.

Convolutional filters act locally, for example by averaging information over small neighborhoods. Repeated application of such filters produces a compressed representation that retains the main features of the solution while discarding fine-scale details.

This is often an effective way to reduce the dimensionality in high-dimensional problems.

14.1.15 Motivation for the POD-DL-ROM Architecture

As the method was tested on increasingly large datasets and more complex geometries, training the original DL-ROM became increasingly difficult. The main issue was not the raw computational power, but rather the growth of the input dimension N_h.

Indeed, if the dimension of the high-fidelity space becomes very large, the neural network must learn a mapping from a very high-dimensional space to a very low-dimensional latent space. This typically requires many layers, many weights and biases, and consequently a difficult optimization problem. In practice, the training may require a large number of epochs before the loss function starts to decrease significantly.

14.1.16 A Hybrid Idea: POD Followed by a DL-ROM

This difficulty suggested returning to the POD lesson.

The idea is to use POD as a first compression step, and then apply the DL-ROM machinery on the already reduced representation. In this way, the dimensionality is reduced in two stages:

1. a first linear reduction by POD;
2. a second nonlinear reduction by the autoencoder.

This leads to the *POD-DL-ROM* architecture.

14.1.17 The POD-DL-ROM Pipeline

Let u^h denote a high-fidelity snapshot. The POD projection is first applied:

$$u^h \mapsto V^T u^h \in \mathbb{R}^n,$$

where $V \in \mathbb{R}^{N_h \times n}$ is the POD basis matrix and $n \ll N_h$.

Then the autoencoder is trained on the POD-reduced data:

$$\phi : \mathbb{R}^n \to \mathbb{R}^{n_\ell}, \qquad \psi : \mathbb{R}^{n_\ell} \to \mathbb{R}^n, \qquad \Phi : \mathbb{R}^p \to \mathbb{R}^{n_\ell},$$

where:

- ϕ is the encoder,
- ψ is the decoder,
- Φ maps the parameters to the latent space,
- n_ℓ is the latent dimension.

The full architecture can be summarized as

$$u^h(\mu) \longmapsto V^T u^h(\mu) \longmapsto \phi\big(V^T u^h(\mu)\big),$$

while the prediction stage is

$$\hat{u}^h(\mu) = V\, \psi(\Phi(\mu)).$$

14.1.18 Loss Function

The loss function is built on the POD-projected snapshots. For each training sample $u^h(\mu_i)$, the encoder receives the reduced quantity $V^T u^h(\mu_i)$ rather than the full snapshot.

A typical training objective is therefore

$$\mathcal{L} = \sum_i \left\| V^T u^h(\mu_i) - \psi\big(\phi\big(V^T u^h(\mu_i)\big)\big) \right\|^2 + \sum_i \left\| \Phi(\mu_i) - \phi\big(V^T u^h(\mu_i)\big) \right\|^2 .$$

The key point is that POD is not the final reduced representation anymore: it is only a preprocessing step that makes the subsequent neural-network training much easier.

14.1.19 Why This Helps

The POD space must still be sufficiently expressive. If too few POD modes are retained, then the neural network can only learn from a poor approximation of the data. A neural network cannot recover information that has already been lost.

On the other hand, the POD dimension can still be much larger than the final latent dimension. This makes the optimization problem more manageable while preserving enough information for the nonlinear stage.

14.1.20 Application: Cardiac Electrophysiology

One of the motivating test cases is a problem in cardiac electrophysiology.

From the numerical viewpoint, the solution depends on a small number of parameters, for instance:

- the size of a damaged region,
- a stimulation time,
- a recovery-related tissue parameter.

Even with only a few parameters, the solution may exhibit a very rich variability. In such a case, a single global linear basis is often not enough to represent all regimes accurately. This is precisely where the POD-DL-ROM strategy becomes useful.

14.1.21 Why Classical Reduction Becomes Difficult

If one follows the classical reduced-order modeling route, one must also address:

- the nonlinear nature of the problem,
- the time dependence,
- the hyperreduction step,
- the need to handle all relevant physical variables.

In the cardiac example, however, one is often interested mainly in the electric potential, while the other variables are not necessarily included in the reduced model. This makes the problem even more challenging for classical projection-based ROMs.

The POD-DL-ROM approach was introduced precisely to improve the reduction capability while keeping the method computationally efficient.

14.1.22 Why an Autoencoder?

At this point, one can notice a useful analogy: POD already behaves like an autoencoder. Indeed, POD starts from a high-dimensional vector u^h, compresses it through a projection, and then reconstructs it back in the original space. This naturally suggests replacing the linear compression-reconstruction mechanism with a nonlinear autoencoder.

The architecture then consists of:

- an *encoder* ϕ;
- a *decoder* ψ;
- a deep feedforward neural network Φ mapping parameters to the latent space.

Thus, the prediction stage is obtained by composing Φ and ψ:

$$\hat{u}^h(\mu) = \psi(\Phi(\mu)).$$

14.1.23 Training Strategy

The model is trained in two stages:

1. first, the autoencoder is trained to learn a nonlinear dimensionality reduction;
2. then, the map from the parameters to the latent representation is learned.

In the architecture, the three sets of weights are:

- θ_e for the encoder;
- θ_d for the decoder;
- θ_{df} for the deep feedforward network Φ.

The final prediction is obtained by feeding the parameters to Φ and then passing the result through the decoder ψ.

14.1.24 A Simple One-Dimensional Example

To illustrate the method, consider a one-dimensional wave propagation problem along a cable. The system depends on a parameter such as the electrical conductivity and exhibits nonlinear wave behavior.

For a latent dimension equal to 2, the DL-ROM is able to approximate the full-order model with an error of order 10^{-3}. This shows that the latent space can be very small while still producing accurate predictions.

14.1.25 A Two-Parameter Benchmark

Another test problem involves a domain with a localized region of low conductivity, whose position is controlled by two parameters μ_1 and μ_2. A third parameter is time, so the effective parameter dimension is three.

This is a more challenging case because the solution exhibits strong variability across the parameter domain. A global POD basis is not always sufficiently compact: in the example discussed, a POD approximation required 87 basis functions, whereas the POD-DL-ROM architecture reduced the latent space to dimension 3.

14.1.26 Computational Performance

The main advantage of the POD-DL-ROM architecture is the large speedup in the online stage.

For the benchmark problems discussed:

- the full-order model required much larger computational effort;
- the POD-based model was already faster, but still expensive;
- the POD-DL-ROM achieved predictions in fractions of a second.

This speedup remains essentially constant at inference time: once trained, the neural network can be queried for any new parameter value and any new time without repeating the expensive full-order simulation.

14.1.27 Application to Cardiac Electrophysiology

The method was successfully applied to cardiac electrophysiology problems. In these simulations, the goal was to reproduce the propagation of the electrical activation in cardiac tissue.

For such cases:

- the full-order model could require several hours;
- the POD-DL-ROM could produce predictions in less than one second.

This made it possible to obtain results faster than real time, which is particularly attractive in biomedical applications.

14.1.28 Application to Fluid Dynamics

The same strategy was then applied to fluid flow problems, such as the classical flow around a cylinder.

In this benchmark:

- the Reynolds number is the main parameter;
- the POD-DL-ROM again provides a substantial speedup over the full-order solver.

The method was also tested on more complex geometries, such as cerebral aneurysms. In that case, the geometry is fixed, while the inflow profile depends on a small number of parameters related to the cardiac cycle.

The result is a very large acceleration with respect to finite elements, while maintaining a very small reconstruction error.

14.1.29 Fluid-Structure Interaction

The approach was further extended to fluid-structure interaction problems.

A typical benchmark consists of:

- a rigid obstacle in the flow;
- an elastic beam in the wake of the obstacle.

The model must capture:

- the fluid velocity;
- the fluid pressure;
- the structural displacement.

Even in this coupled setting, the POD-DL-ROM architecture remains effective.

14.1.30 MEMS Applications

The same framework was also applied to microelectromechanical systems (MEMS). These are small structures that often exhibit nonlinear oscillatory behavior.

In this case, one is typically interested in the frequency response surface, which describes how the amplitude of the response varies with the forcing frequency. Computing this surface with a full-order model may require an enormous number of simulations, whereas the POD-DL-ROM can drastically reduce the cost.

14.1.31 Main Message

The numerical examples show that POD-DL-ROMs are effective across very different physical problems:

- wave propagation,
- cardiac electrophysiology,
- fluid dynamics,
- fluid-structure interaction,
- MEMS.

The key advantage is that the method combines:

- a strong reduction of dimensionality;
- a nonlinear approximation power;
- a very fast online evaluation stage.

Chapter 15

Autoencoder-based DL-ROMs

15.1 From POD-NN to Deep Learning ROMs for Time-Dependent Problems

We previously discussed the POD-NN method for a stationary setting. Today we extend the discussion to **DL-ROM** (deep learning-based reduced order models) applied to a time-dependent problem.

- The choice between POD-NN and DL-ROM is not dictated by stationarity or time dependence alone; both can be used in either regime. The decisive factors will be addressed later.

15.2 Example: 2D Navier–Stokes Flow Past a Cylinder

15.2.1 Physical Problem

We consider the incompressible Navier–Stokes equations in two dimensions:

- Unknowns: velocity field $\mathbf{V}$ and pressure Q.
- Quantity of interest: the scalar **speed** $|\mathbf{V}|$ (magnitude of velocity).
- Domain: a rectangular channel with a circular obstacle (cylinder) that obstructs the flow.
- Inlet flow from the left, moving to the right.

Depending on the fluid parameters (density ρ and viscosity ε), the flow may be laminar (straight) or may develop a periodic **vortex shedding** (a limit cycle). The dataset captures this behaviour.

15.2.2 Available Dataset

- **No full-order model code** is provided; we work directly from simulation data.
- Data tensor `U`: shape (n_s, n_t, N_h) where

$$
\begin{aligned}
n_s &= 41 \quad \text{(number of simulations)},\\
n_t &= 141 \quad \text{(time snapshots per simulation, uniform grid from } t = 0 \text{ to } t = 3.5),\\
N_h &\approx 6000 \quad \text{(spatial degrees of freedom, P2 finite elements for the velocity field)}.
\end{aligned}
$$

- Each simulation corresponds to a different pair (ρ_i, ε_i), with ε given on a logarithmic scale.
- The tensor can be viewed as an $(n_s \times n_t)$ matrix of vectors, where entry (i, j) is the spatial solution vector $\mathbf{u}(\boldsymbol{\mu}_i, t_j) \in \mathbb{R}^{N_h}$.

15.3 Why Not an Intrusive Reduced Basis Method?

Building a reduced basis (RB) surrogate for this problem presents several difficulties:

1. **Nonlinearity:** The Navier–Stokes equations are nonlinear in $\mathbf{V}$, preventing simple pre-assembly and projection of matrices.
2. **Missing fields:** An RB method would require data for both the velocity $\mathbf{V}$ and the pressure Q to construct separate reduced bases, but we only have the speed $|\mathbf{V}|$.
3. **Full-order code requirement:** We do not possess the discretised PDE code, making an intrusive approach impossible.

Therefore, a purely **data-driven** surrogate (such as a deep neural network) becomes the natural choice.

15.4 Data Preparation: Mapping (μ, t) to Snapshots

15.4.1 Conceptual Framework

We aim to learn a map

$$(\boldsymbol{\mu}, t) \in \mathbb{R}^{p+1} \longmapsto \mathbf{u}(\boldsymbol{\mu}, t) \in \mathbb{R}^{N_h},$$

where $p = 2$ (the two physical parameters). The network training requires paired input–output samples:

$$\big(\tilde{\boldsymbol{\mu}}_k,\ \mathbf{u}_k\big), \qquad k = 1, \ldots, n_s\, n_t,$$

with $\tilde{\boldsymbol{\mu}}_k$ encoding the combination of parameters and time for a single snapshot.

15.4.2 Constructing the Input Tensor $\tilde{\boldsymbol{\mu}}$

The original data tensor `U` has shape $(n_s,\, n_t,\, N_h)$. We build a corresponding tensor `mu_tilde` of shape $(n_s,\, n_t,\, p+1)$ such that for each simulation i and time index j,

$$\texttt{mu_tilde}[i, j, :] = \big(\rho_i,\ \varepsilon_i,\ t_j\big).$$

The implementation proceeds as follows:

1. Unpack dimensions: `ns, nt, Nh = U.shape`, `p = mu.shape[1]` (here $p = 2$).
2. Create the time grid: `t = torch.linspace(0, 3.5, nt)`.
3. Initialise the tensor: `mu_tilde = torch.zeros(ns, nt, p+1)`.
4. Loop over simulations ($i = 0, \ldots,$`ns-1`):
   - For every time index j, set the first two components to the constant parameters of simulation i:

     ```
     mu_tilde[i, :, 0] = mu[i,0]   # density
     mu_tilde[i, :, 1] = mu[i,1]   # viscosity (log scale)
     ```

   - The third component receives the time values:

     ```
     mu_tilde[i, :, 2] = t
     ```

The resulting 3D tensor `mu_tilde` now aligns exactly with `U`: each spatial snapshot at index (i, j) is paired with the corresponding parameter–time vector.

15.4.3 Verification

Printing `mu_tilde` (e.g., `print(mu_tilde.shape)`) confirms that we have a three-dimensional tensor of the expected size (41, 141, 3), ready for use in training a deep neural network surrogate.

15.5 Data Preparation (Continued)

15.5.1 Verification of the Input Tensor

Printing `mu_tilde` reveals its structure:

- The first block corresponds to simulation $i = 0$: time varies across the 141 snapshots while the two parameters (ρ_0, ε_0) remain fixed.
- Changing the simulation index i switches to a new parameter pair, and the time coordinate restarts from $t = 0$.
- This yields a one-to-one correspondence between each spatial snapshot and its generating conditions.

15.5.2 Reshaping to Standard Machine-Learning Format

Neural networks operate most naturally on 2D tensors of shape (batch size, feature dimension).

- **Input:** The tensor `mu_tilde` of shape $(n_s,\ n_t,\ p+1)$ is flattened to $(n_s n_t,\ p+1)$.
- **Output:** The snapshot tensor `U` of shape $(n_s,\ n_t,\ N_h)$ is flattened to $(n_s n_t,\ N_h)$.

Thus the total number of training samples is $B = n_s \cdot n_t \approx 5781$, each comprising a parameter–time vector $\tilde{\boldsymbol{\mu}} \in \mathbb{R}^{p+1}$ and a spatial field $\mathbf{u} \in \mathbb{R}^{N_h}$. The reshaping is performed via `mu_tilde.reshape(-1, p+1)` and `U.reshape(-1, Nh)`.

15.6 Toward a Surrogate: Singular Value Analysis

Before building any surrogate, one examines the decay of the singular values (POD) of the snapshot matrix.

$$\mathbf{U}_{\text{train}} \in \mathbb{R}^{N_h \times B_{\text{train}}}$$

- The singular values decay slowly; there is **no sharp elbow**.
- Retaining around 40–50 POD modes may still be acceptable, but the lack of a rapid drop suggests that a **nonlinear compression** (autoencoder) can achieve much smaller latent dimensions than linear POD.

This observation motivates the transition from POD-based approaches to deep autoencoders.

15.7 Autoencoder for Nonlinear Dimensionality Reduction

15.7.1 Architecture

We replace the linear POD basis with a trainable autoencoder composed of:

$$\text{Encoder } \Psi' : \mathbb{R}^{N_h} \to \mathbb{R}^n, \qquad \text{Decoder } \Psi : \mathbb{R}^n \to \mathbb{R}^{N_h},$$

where $n \ll N_h$ is the **latent dimension**. The composition $\Psi \circ \Psi'$ approximates the identity on the data manifold.

15.7.2 Training Objective

Given the snapshot data $\{\mathbf{u}_i\}_{i=1}^{B}$, we minimise the reconstruction loss

$$\frac{1}{B}\sum_{i=1}^{B}\left\|\mathbf{u}_i - \Psi(\Psi'(\mathbf{u}_i))\right\|^2.$$

Norm choice: The discrete $L^2(\Omega)$ norm should be preferred over the Euclidean norm because it accounts for the non-uniform density of nodes in the finite-element mesh (more points near the cylinder). The code provides an `L2_error` function that computes the relative L^2 error in space and time:

$$\text{error} = \frac{\|\mathbf{u}_{\text{pred}} - \mathbf{u}_{\text{true}}\|_{L^2(L^2(\Omega))}}{\|\mathbf{u}_{\text{true}}\|_{L^2(L^2(\Omega))}} \times 100\%.$$

15.7.3 Implementation Details

- **Encoder network:** Input N_h, a sequence of hidden layers (dimensions and activations of choice), output n. No activation on the final layer.
- **Decoder network:** Input n, hidden layers, output N_h. No final activation.
- The full model `Autoencoder` is the composition `decoder(encoder(x))`.
- The architecture is moved to the GPU for training.
- Loss function: `MSELoss` (mean squared error); the error function for monitoring is the relative L^2 error described above.
- Hyperparameters: number of epochs, optimizer (e.g., Adam), batch size, learning rate. Default choices can be tuned after an initial run.

15.8 Choice of Activation Functions

Standard activations considered include:

- ReLU: $\max(0, x)$,
- LeakyReLU: $\max(x, \alpha x)$ with $0 < \alpha < 1$,
- GELU (Gaussian Error Linear Unit).

For autoencoders, **invertible** activations can prevent information loss through the nonlinearity. A family of invertible functions is given by the parametric LeakyReLU:

$$\rho(x) = \text{ReLU}(x) - \alpha\,\text{ReLU}(-x).$$

When $\alpha = 1$, this recovers the identity; for $0 < \alpha < 1$ it is a leaky ReLU, which is invertible (unlike ReLU or GELU). It is therefore advantageous to use such activations, possibly with different slopes α_e for the encoder and α_d for the decoder.

- $\rho_e(x) = \text{ReLU}(x) - \alpha_e\,\text{ReLU}(-x)$,
- $\rho_d(x) = \text{ReLU}(x) - \alpha_d\,\text{ReLU}(-x)$.

This choice allows the network to learn a more faithful compression without intrinsic loss from the activation function itself.

15.9 Autoencoder Design: Practical Guidelines

15.9.1 Choice of LeakyReLU Slopes for Encoder and Decoder

The parametric LeakyReLU activation $\rho(x) = \mathrm{ReLU}(x) - \alpha\,\mathrm{ReLU}(-x)$ can be employed with different slopes for the encoder and decoder.

- **Encoder (ρ_e):** Large α_e (e.g., $\alpha_e = 10$) is preferred. Since the encoder compresses information, a large slope helps keep encoded vectors distinguishable and prevents them from collapsing to near-zero values.
- **Decoder (ρ_d):** Small α_d (e.g., $\alpha_d = 0.1$) is used to promote stability during reconstruction. A small slope makes the decoder less sensitive to variations in the latent variables.

Notice that with $\alpha_e = 10$ and $\alpha_d = 0.1$, the decoder activation is precisely the inverse of the encoder activation. While not mandatory, this inverse pairing often yields well-conditioned autoencoders. These choices are heuristic "cards to play" when performance is unsatisfactory, not rigid rules.

15.9.2 Network Depth and Symmetry

The encoder and decoder do not need to be symmetric ("butterfly" structure). Intuitively, reconstructing a high-dimensional field from a low-dimensional code (decoding) is harder than compressing it (encoding). Therefore the decoder is typically given greater representational capacity:

- The encoder may contain fewer layers.
- The decoder may contain at least one extra hidden layer, making it deeper and more expressive.

In the example, a latent dimension of $n = 7$ is chosen and training is performed for 100 epochs (further fine-tuning with L-BFGS followed by Adam is used in the provided solution to reach even lower errors).

15.10 On the Minimal Latent Dimension

15.10.1 Theoretical Bounds

For a parameter-to-solution map $\boldsymbol{\mu} \mapsto u(\boldsymbol{\mu})$ that is **Lipschitz continuous**, there exists a smallest latent dimension n^* such that an autoencoder can achieve arbitrarily small reconstruction error on the solution manifold.

- **Lower bound:** $n^* \geq \tilde{p}$, where $\tilde{p}$ is the total number of intrinsic parameters (physical parameters plus possibly time). For a time-dependent problem with p parameters, $\tilde{p} = p + 1$; for a stationary one, $\tilde{p} = p$. **Hence one cannot compress with fewer than $\tilde{p}$ numbers without losing information.**
- **Upper bound:** $n^* \leq 2\tilde{p} + 1$. The $+1$ is required only for very pathological (non-typical) solution manifolds; in most practical cases $n^* = 2\tilde{p}$ is sufficient.

For the Navier–Stokes example, $p = 2$ and time adds a third dimension, so $\tilde{p} = 3$. The lower bound is 3, the upper bound is 7 (or 6 omitting the pathological $+1$). The code example uses $n = 7$.

15.10.2 Comparison with Linear POD

At $n = 7$, the linear POD reconstruction already achieves an average relative L^2 error of around 4%. The autoencoder yields $\approx 3.6\%$ with basic training—an improvement that can be made more substantial with additional optimisation. The slow singular value decay originally indicated that a nonlinear approach could beat POD, which is confirmed albeit modestly here.

15.11 Geometric Intuition for the Upper Bound

The necessity of $n \leq 2\tilde{p}$ (and the occasional $+1$) can be understood from the geometry of the solution manifold.

Case $p = 1$ (no time). The solution set traces a **curve** in the function space $\mathbb{R}^{N_h}$ as the single parameter μ varies.

- If the curve is simple (no self-intersections), a latent dimension $n = 1$ suffices: a 1D manifold can be continuously mapped to and from a line via an autoencoder (the encoder "unrolls" the curve, the decoder maps it back).
- **Loop scenario:** Suppose the solution for $\mu = A$ is identical to the solution for $\mu = B$ (e.g., a wave that enters and leaves the domain, with zero signal at both extremes). The curve then forms a closed loop (a circle). A 1D latent space (a line) cannot faithfully represent a circle without forcing two distinct solutions to share the same code. To recover both solutions after decoding, the latent space must have dimension at least 2, i.e., $n = 2 = 2p$. This justifies the upper bound $2p$.

The additional $+1$ accounts for extremely degenerate, non-typical manifolds (e.g., loops with extra twists) where $2p$ dimensions still fail; those cases are rarely encountered in practice.

15.12 Next Steps

The second network of the DL-ROM (the map from $(\boldsymbol{\mu}, t)$ to latent variables) will be introduced in the next lab, together with further training tricks and the complete surrogate pipeline. Future discussions will also address challenges that arise when the parameter dimension p is large.

Chapter 16

Data-driven dynamical systems (DMD & SINDy)

16.0.1 Driven Dynamical Systems

A central motivation for this line of research is the treatment of *driven dynamical systems*. The main goals are the following:

1. make reduced-order models aware of the time dependence of the solution;
2. move from state observations to the discovery of the underlying dynamics;
3. use a reduced latent representation in order to forecast future states efficiently.

In the classical reduced-order setting, time is still treated as a parameter. This is true both for POD-based schemes and for POD-NN or DL-ROM constructions: time is simply fed into a network as an input variable, rather than being handled as the genuine evolution variable of a dynamical system.

16.0.2 From State Observations to Dynamics Discovery

The usual forward problem is the following: given an initial condition and a dynamical law, one advances the system in time using a numerical integrator such as Euler or Runge–Kutta.

Here, instead, the viewpoint is different. One observes a sequence of states and asks:

> What dynamical system could have generated these observations?

This is a problem of *discovering the dynamics from state data*. It is especially relevant when one collects snapshots of a high-dimensional system, for instance a fluid or a fluid–structure interaction problem, and wants to predict what happens at future times.

16.0.3 Latent Dynamics Discovery

Suppose that the solution $u(t)$ is sampled at times $t_1, \dots, t_m$, and that the resulting snapshots are collected in a matrix. The idea is to compress the data into a lower-dimensional representation, for example by means of an encoder

$$\phi : \mathbb{R}^{N_h} \to \mathbb{R}^n,$$

and to denote by $z(t)$ the encoded state:

$$z(t) = \phi(u(t)).$$

If the original state $u(t)$ follows a dynamical system, then it is natural to expect that the latent variable $z(t)$ evolves according to some reduced dynamics as well:

$$\dot{z}(t) = g(t, z(t)),$$

for some unknown reduced vector field g.

The key issue is that, once the encoding map ϕ is nonlinear, discovering the reduced dynamics becomes much more difficult. If ϕ were linear, as in POD, one could recover the reduced equations more directly. With nonlinear maps, the latent evolution is much harder to characterize.

16.0.4 Why the Latent Dynamics Matter

If the objective is only interpolation within the range of observed data, then a purely data-driven network may be sufficient. However, if the objective is *forecasting* at future times, then one needs a representation that respects the time evolution of the system.

Neural networks are typically very effective inside the range of the training data, but extrapolation in time is much harder. This is why it is advantageous to learn not only a reduced representation, but also the dynamics in that reduced space.

16.0.5 Reduced-Time Integration in Latent Space

Once the latent dynamics are known, the forecast procedure becomes very efficient.

1. encode the state into the latent variable z;
2. integrate the reduced dynamical system in time using a standard ODE solver;
3. decode the latent solution back to the full space.

Since the latent dimension n is small, the reduced dynamical system is cheap to integrate. This preserves the causal structure of the original problem: time is no longer treated as a mere input parameter, but as the variable governing the evolution of the latent state.

16.0.6 Connection with POD

This idea is closely related to POD.

Let X denote the snapshot matrix. If we compute its singular value decomposition,

$$X = U\Sigma V^T,$$

then the columns of U provide a basis for the solution space. A state can then be written as

$$u(t) \approx \sum_{i=1}^{r} a_i(t)\varphi_i,$$

where $\{\varphi_i\}_{i=1}^r$ are the leading POD modes and $a_i(t)$ are the reduced coordinates.

Equivalently,

$$u(t) \approx V_r a(t),$$

where V_r collects the retained POD modes and $a(t)$ is the reduced vector of coefficients.

In this viewpoint, the main task becomes the discovery of the evolution law for the reduced coefficients:

$$a(t) \longmapsto a'(t).$$

Once the coefficients are known, the full solution is reconstructed by expansion on the reduced basis.

16.0.7 Main Message

The overarching idea is therefore the following: given high-dimensional snapshot data generated by a dynamical system, one wants to

- compress the data into a reduced representation;
- discover the evolution law in that reduced space;
- use the reduced dynamics to forecast future states efficiently.

This is the bridge between classical reduced-order modeling and latent-dynamics learning.

16.0.8 Learning the Reduced Dynamics

Once the high-dimensional state has been compressed, the next goal is to learn how the reduced variables evolve in time.

Let $a(t)$ denote the reduced coordinates of the solution. The central question is now:

$$\dot{a}(t) = F(a(t))\,?$$

In other words, we want to identify the dynamics of the latent or reduced state.

16.0.9 Option 1: Black-Box Neural Dynamics

A first possibility is to use a neural network as a black-box approximation of the time evolution operator.

Given the reduced state at one or more previous time instances, the network is trained to predict the next state:

$$a(t_k) \mapsto a(t_{k+1}).$$

More generally, one may use a neural network to approximate the unknown vector field:

$$\dot{a}(t) = G(a(t)),$$

where G is learned directly from data.

This strategy is very flexible and can include time explicitly as an input. However, it has two important limitations:

- the forecast quality may deteriorate when predicting far into the future;
- the learned model is often difficult to interpret physically.

16.0.10 Reduced Dynamics as a Forecasting Problem

The idea is to first perform dimensionality reduction, and then learn the dynamics in the reduced space.

1. compress the full-order state into a reduced vector $a(t)$;
2. learn the evolution law of $a(t)$;
3. use the learned dynamics to forecast future states.

This is advantageous because the reduced system is much smaller than the full-order one, so time integration becomes much cheaper.

16.0.11 Option 2: Linear Reduced Dynamics and DMD

A second possibility is to look for a linear approximation of the reduced dynamics.

In this case, one seeks a linear operator L such that

$$\dot{a}(t) \approx La(t).$$

This is the spirit of *Dynamic Mode Decomposition* (DMD): find the best linear dynamical system capable of reproducing the observed data.

The continuous-time viewpoint can be related to the discrete evolution

$$a_{k+1} \approx Aa_k,$$

where A is a linear propagator advancing the reduced state from one time step to the next.

16.0.12 Discrete Formulation of DMD

Assume that snapshots are collected in the matrices

$$X = \begin{bmatrix} a_1 & a_2 & \cdots & a_{m-1} \end{bmatrix}, \qquad X' = \begin{bmatrix} a_2 & a_3 & \cdots & a_m \end{bmatrix}.$$

The goal is to find the best matrix A such that

$$X' \approx AX.$$

This is a linear regression problem. The optimal matrix is typically obtained through a least-squares formulation:

$$A = X'X^{\dagger},$$

where $X^{\dagger}$ is the pseudoinverse of X.

16.0.13 Computation via the SVD

Since X is generally rectangular, its inverse does not exist. Instead, one computes a thin singular value decomposition:

$$X = U\Sigma V^T.$$

Then the pseudoinverse is given by

$$X^{\dagger} = V\Sigma^{-1}U^T,$$

so that

$$A = X'V\Sigma^{-1}U^T.$$

This provides a purely data-driven approximation of the reduced dynamics.

16.0.14 Why DMD is Useful

DMD is appealing because it gives a linear reduced model that is simple to interpret and easy to integrate in time.

If the underlying system is approximately linear, then DMD can be very effective. For linear systems with constant coefficients, the dynamics can be expressed in terms of eigenvalues and eigenvectors, leading to solutions of exponential type.

However, DMD is also limited by its linearity assumption. If the true dynamics are strongly nonlinear, a purely linear surrogate may be too restrictive.

16.0.15 Sparse Identification of Nonlinear Dynamics

A more expressive alternative is to search for a sparse nonlinear model.

One assumes that the reduced dynamics can be written as

$$\dot{a}(t) = \Theta(a(t))\,\xi,$$

where:

- $\Theta(a)$ is a dictionary of candidate nonlinear functions,
- ξ is a vector of coefficients to be identified.

The dictionary may include:

- polynomials,
- trigonometric functions,
- user-defined nonlinear terms.

The key idea is to enforce sparsity, so that only a few terms in the dictionary remain active. This leads to the *sparse identification of nonlinear dynamics* (SINDy) framework.

16.0.16 Interpretation of the Sparse Model

The sparse regression problem is typically solved by least squares with thresholding:

- first, solve a regression problem;
- then remove coefficients that are sufficiently small;
- repeat until only the relevant terms remain.

This makes the model more interpretable than a black-box neural network, while still allowing nonlinear dynamics.

16.0.17 Connection with Reduced-Order Modeling

The reduced-order modeling pipeline can therefore be summarized as follows:

1. reduce the state dimension, using POD or an autoencoder;
2. learn the reduced dynamics;
3. reconstruct the full solution from the reduced evolution.

In this sense, DMD and SINDy can be combined with POD, or even with autoencoders, leading to hybrid methods such as:

$$\text{POD} + \text{DMD}, \qquad \text{POD} + \text{SINDy}, \qquad \text{Autoencoder} + \text{DMD}, \qquad \text{Autoencoder} + \text{SINDy}.$$

16.0.18 Main Message

The reduced-dynamics viewpoint is crucial when one wants not only to compress data, but also to predict future states in a way that respects time evolution.

The main options are:

- a black-box neural network for the reduced dynamics;
- a linear operator identified by DMD;
- a sparse nonlinear model identified by SINDy.

Each option balances flexibility, interpretability, and long-time forecasting ability in a different way.

16.0.19 Practical Implementation of DMD

In practice, the DMD algorithm is usually implemented by retaining only the first r modes. These are the most energetic modes, identified by the decay of the singular values.

The main advantage is that one does not need to work with the full matrix A, which may be very large. Instead, one considers a reduced matrix obtained by projection onto the dominant POD subspace.

Let U_r denote the matrix collecting the first r left singular vectors. Then one works with the reduced operator

$$A_r = U_r^T A U_r,$$

which is small enough to be handled efficiently.

The eigenvalues and eigenvectors of the reduced matrix provide the relevant dynamical information. This is possible because U_r is orthogonal, so the projected system preserves the key spectral features of the original one.

16.0.20 Connection with POD-DMD

The idea behind POD-DMD is the following:

1. start from the snapshot data;
2. compute the SVD and identify the dominant POD modes;
3. project the dynamics onto the reduced POD subspace;
4. apply DMD on the projected system.

In this way, POD is used to reduce the dimensionality, while DMD is used as a surrogate for the time evolution.

16.0.21 Beyond Linear Dynamics

A purely linear model is often too restrictive. Many systems are better described by a combination of linear and nonlinear effects.

In that case, one may write the reduced dynamics in the form

$$\dot{a}(t) = La(t) + \mathcal{N}(a(t)),$$

where:

- L is the linear part, often obtained by projection;
- $\mathcal{N}$ is the nonlinear part.

The idea is then to approximate only the nonlinear contribution with a data-driven technique such as DMD or a related model.

16.0.22 Sparse Identification of Nonlinear Dynamics

Another option is to use sparse regression to identify the reduced dynamics.

Assume that the latent or reduced state $a(t)$ satisfies

$$\dot{a}(t) = f(a(t)).$$

We collect snapshots of the state and approximate the time derivatives by numerical differentiation:

$$\dot{a}(t_1), \dot{a}(t_2), \ldots, \dot{a}(t_m).$$

Then we build a dictionary of candidate functions

$$\Theta(a) = \big[\theta_1(a), \theta_2(a), \ldots, \theta_b(a)\big],$$

where the functions may be:

- polynomials,
- trigonometric functions,
- user-defined nonlinear terms.

The reduced dynamics is then written as

$$\dot{a}(t) \approx \Theta(a(t))\,\xi,$$

where ξ is a vector of unknown coefficients.

16.0.23 Sparse Regression

The coefficients are obtained by solving a regularized least-squares problem:

$$\min_{\xi} \left\| \dot{A} - \Theta(A)\xi \right\|^2 + \lambda \|\xi\|_1.$$

The ℓ_1 penalty promotes sparsity, so that only a few terms in the dictionary remain active. This is the key idea behind the *sparse identification of nonlinear dynamics* (SINDy).

A practical implementation often proceeds by:

1. solving a least-squares problem;
2. removing small coefficients;
3. repeating the procedure until convergence.

16.0.24 Interpretation

The advantage of SINDy is that it provides a sparse and interpretable model. Instead of fitting a fully black-box dynamical system, one identifies only the few relevant terms that explain the observed behavior.

This is particularly attractive when one suspects that the true dynamics is governed by a small number of nonlinear mechanisms.

16.0.25 Example: The Lorenz System

The original work of Brunton, Proctor, and Kutz considered the Lorenz system as a benchmark.

The Lorenz system is a three-dimensional nonlinear dynamical system with state (x, y, z) and parameters such as σ, ρ, and β. Even though it is simple in dimension, it exhibits strongly nonlinear and sensitive behavior.

For this example, one chooses a polynomial library up to a given order, for instance up to fifth order, and then performs sparse regression to recover the governing equations.

This example shows that, if the library is chosen appropriately, SINDy can recover a compact and interpretable dynamical model directly from data.

16.0.26 Main Message

The reduced-dynamics problem can therefore be addressed in several ways:

- by DMD, if one wants a linear reduced evolution;
- by POD-DMD, if one first reduces the dimension and then learns the dynamics;
- by SINDy, if one wants a sparse nonlinear representation.

The common theme is to learn a low-dimensional evolution law from snapshots, while keeping the model as simple and interpretable as possible.

16.0.27 Sparse Identification of Nonlinear Dynamics: Interpretation of the Regression

Let us look more carefully at the structure of SINDy. The data matrix contains the time evolution of the state components, for instance

$$x(t), \qquad y(t), \qquad z(t),$$

together with all the nonlinear functions included in the library.

For example, if one wants to describe the evolution of $\dot{x}$, one writes it as a linear combination of candidate functions such as

$$x,\ y,\ z,\ x^2,\ xy,\ xz,\ \ldots$$

evaluated over all the available time instances.

The same procedure is repeated for all components of the state:

$$\dot{a}(t) = \Theta(a(t))\,\xi,$$

where:

- $a(t)$ is the state vector;
- $\Theta(a)$ is the library of candidate functions;
- ξ is the vector of coefficients to be identified.

16.0.28 Sparse Regression and Coefficient Selection

The regression is regularized so that only a few coefficients remain nonzero. In this way, the identified model is sparse:

$$\min_{\xi} \left\| \dot{A} - \Theta(A)\xi \right\|^2 + \lambda \|\xi\|_1.$$

For the Lorenz system, the resulting coefficient vectors ξ_1, ξ_2, and ξ_3 recover the dominant nonlinear terms of the dynamics, provided that the chosen library is rich enough.

The key point is that the identified system is not necessarily exactly the original one; rather, it is the best sparse approximation consistent with the observed data and the selected dictionary of functions.

16.0.29 Autoencoder + SINDy

A more recent idea is to combine SINDy with an autoencoder.

In this framework, the data are first mapped to a latent space by an encoder

$$\phi : x \mapsto z,$$

and then the sparse dynamics are learned in the latent variable z.

Thus, one seeks a reduced model of the form

$$\dot{z} = \Theta(z)\,\xi,$$

together with a decoder

$$\psi : z \mapsto \hat{x},$$

so that the reconstructed state satisfies

$$\hat{x} = \psi(z).$$

The aim is to learn simultaneously:

1. a nonlinear compression map;
2. a sparse latent dynamical system;
3. a reconstruction map back to the original space.

16.0.30 Joint Loss Function

The training can be formulated through a single loss function containing several terms.

One term enforces sparsity in the latent dynamics:

$$\|\dot{z} - \Theta(z)\,\xi\|^2 + \lambda \|\xi\|_1.$$

Another term enforces reconstruction accuracy:

$$\|x - \psi(\phi(x))\|^2.$$

A further term may compare the observed time derivative in the original space with the decoded latent dynamics, using the chain rule. Indeed, if

$$z = \phi(x),$$

then

$$\dot{z} = D\phi(x)\,\dot{x},$$

so the latent dynamics can be related to the observed dynamics through the Jacobian of the encoder.

16.0.31 Why This Matters

These formulations are useful because they allow one to:

- discover dynamics directly from data;
- obtain interpretable sparse models;
- combine dimensionality reduction with equation discovery;
- handle high-dimensional observations such as images or spatial fields.

This is especially attractive when the data are collected from systems that are already highly nonlinear or very high-dimensional, such as reaction-diffusion equations, pendulum data from images, or spatially distributed wave phenomena.

16.0.32 Examples

For a pendulum observed through images, the input data are high-dimensional because they consist of pixels. Nevertheless, the underlying dynamics are low-dimensional, and the autoencoder + SINDy framework can recover a compact reduced model.

Similarly, for a reaction-diffusion system or a spiral wave problem, the observed fields live in a large-dimensional space, but the latent dynamics may be much simpler.

16.0.33 Main Message

The main idea is that the sparse identification of nonlinear dynamics can be applied either:

- directly in a low-dimensional space, if the state is already reduced;
- after a preliminary dimensionality reduction by POD;
- jointly with an autoencoder, by learning both the latent space and the latent dynamics.

This provides a flexible framework for equation discovery from data.

Chapter 17

High-dimensional parameter spaces (PCA-net & mesh-informed layers)

17.1 DL-ROM: Learning the Parameter-to-Latent Map

17.1.1 Recap of the Current State

We are dealing with the time-dependent flow past a cylinder, parameterised by density ρ and viscosity ε.

- An autoencoder (Ψ', Ψ) has been trained to compress and reconstruct the spatial snapshots, achieving a relative L^2 error of $\approx 3\%$ (compared to 4% with linear POD at the same latent dimension $n = 7$).
- For every snapshot $\mathbf{u}(\boldsymbol{\mu}, t)$ we now have a corresponding latent vector $\boldsymbol{\nu} = \Psi'(\mathbf{u}) \in \mathbb{R}^n$.
- The missing ingredient is the map
$$\phi : (\boldsymbol{\mu}, t) \mapsto \boldsymbol{\nu},$$
which, given a new parameter $\boldsymbol{\mu}$ and time t, predicts the latent representation of the solution. The full surrogate is then DL-ROM$(\boldsymbol{\mu}, t) = \Psi(\phi(\boldsymbol{\mu}, t))$.

17.1.2 Constructing the Training Set for ϕ

We already have the input tensor `mu_tilde` of shape $(B, p+1)$ where $B = n_s n_t$ and $p+1 = 3$ (two parameters plus time). We need the corresponding output latent vectors.

1. **Freeze the autoencoder**: Both encoder and decoder are set to evaluation mode and their parameters are frozen (no gradient computation).
2. **Generate latent data**: Apply the encoder to all snapshots,
$$\texttt{nu} = \texttt{encoder}(\mathbf{U}) \quad \text{with shape } (B, n).$$
Note: `encoder` is already linked to the autoencoder object, so freezing the autoencoder automatically freezes the encoder.
3. The training pairs for ϕ are $\{(\tilde{\boldsymbol{\mu}}_i, \boldsymbol{\nu}_i)\}_{i=1}^{B}$.

17.1.3 Network Architecture and Training

The network ϕ is a standard feed-forward neural network:

$$\mathbb{R}^{p+1} \longrightarrow \text{hidden layers} \longrightarrow \mathbb{R}^n.$$

- Loss function: Mean Squared Error (MSE) in Euclidean norm on the latent space.

- Training is performed for a fixed number of epochs, using the GPU.
- The absolute value of the loss may appear large initially; this is not necessarily alarming because the scale and geometry of the latent space are determined by the encoder and are not known *a priori*.

17.1.4 Assembling the Final DROM Surrogate

The surrogate is simply the composition of the decoder with ϕ:

$$\mathrm{DROM}(\boldsymbol{\mu}, t) = \Psi\big(\phi(\boldsymbol{\mu}, t)\big).$$

It can be implemented as a function or a combined `nn.Module`. Given a batch of $(\boldsymbol{\mu}, t)$ inputs, it returns the full-order field reconstructions.

17.2 Evaluation and Performance

17.2.1 Choice of Error Metric

For time-dependent problems, a meaningful relative error must account for the space-time nature of the solution, especially because the initial condition may be zero (causing division by zero if relative errors are computed per snapshot). The correct metric is the relative L^2 error in space-time, already defined as:

$$\mathrm{error}(\mathbf{u}_{\mathrm{pred}}, \mathbf{u}_{\mathrm{true}}) = \frac{\|\mathbf{u}_{\mathrm{pred}} - \mathbf{u}_{\mathrm{true}}\|_{L^2(0,T;L^2(\Omega))}}{\|\mathbf{u}_{\mathrm{true}}\|_{L^2(0,T;L^2(\Omega))}} \times 100\%.$$

17.2.2 Test Results

The test set consists of 21 unseen parameter configurations, each with the full time history (roughly 3000 snapshots total).

- The DROM surrogate achieves a mean relative space-time error of approximately **6.8–6.9%**.
- This error can be decomposed: $\approx 3\%$ from the autoencoder reconstruction and $\approx 3\%$ from the approximation of ϕ.
- **Speed-up:** Evaluating the surrogate for all 21 simulations takes **milliseconds**, whereas a full FEM solver would require 3–4 minutes per simulation.

17.3 Improving Accuracy: Fourier Embedding for Time

17.3.1 Motivation

The flow exhibits a nearly periodic vortex-shedding behaviour after an initial transient. Treating time merely as an additional scalar parameter discards its inherent structure. A common trick is to preprocess the time coordinate with a **Fourier embedding** before feeding it into the network.

17.3.2 Implementation Idea

Given a time value t, instead of passing only t, we augment the input vector with sine and cosine of t at multiple frequencies:

$$t \longmapsto \big[\, t,\ \sin(\omega_1 t),\ \cos(\omega_1 t),\ \sin(\omega_2 t),\ \cos(\omega_2 t),\ \ldots,\ \sin(\omega_K t),\ \cos(\omega_K t) \,\big].$$

The frequency scale is a hyperparameter (e.g., $\omega_k = k \cdot \omega_0$). The parameters (ρ, ε) are kept unchanged.

- This is **not a learnable layer**; it is a fixed, deterministic transformation applied to the time coordinate before the first linear layer.

- The idea is to help the network discover periodic patterns more easily, as it now has access to explicit periodic basis functions.
- If the signal is not periodic, the network can simply ignore the Fourier features.
- The number of frequencies K controls the capacity of the embedding. Larger K increases input dimension and thus the number of parameters in the first hidden layer. As always, the choice is constrained by the amount of training data (to avoid overfitting).

17.3.3 Implementation Sketch

Define a `FourierLayer` module that takes the original input vector, keeps the non-time components unchanged, and applies the Fourier expansion to a specified time index.

```
class FourierLayer(nn.Module):
    def __init__(self, in_dim, time_idx, num_freq):
        ...
    def forward(self, x):
        # x[:, time_idx] is the time coordinate
        # Append sin/cos terms for each frequency
        ...
```

The expanded tensor is then passed to the rest of the network. Validation performance guides the choice of K and the frequency range.

17.4 High-Dimensional Parameter Field: Permeability Mapping

17.4.1 Problem Statement

We consider a stationary diffusion-type equation on a domain $\Omega \subset \mathbb{R}^2$,

$$-\nabla \cdot \big(\mu(\mathbf{x})\, \nabla u(\mathbf{x})\big) = f(\mathbf{x}), \qquad \mathbf{x} \in \Omega,$$

with appropriate boundary conditions and a normalisation condition (e.g., $\int_\Omega u = 0$) to fix the constant.

- The source term f is a sum of three Dirac delta functions, representing three fixed point sources on the domain.
- The parameter $\mu(\mathbf{x})$ is the **permeability field** of the porous medium; it varies spatially and is defined at every point of the domain.
- The solution $u(\mathbf{x})$ is the pressure field.

Thus both the input (parameter) and the output (solution) are **spatial fields**, not low-dimensional vectors.

17.4.2 Discretisation and High Dimensionality

After finite-element discretisation on a mesh with N_h degrees of freedom, the continuous fields become vectors:

$$\boldsymbol{\mu} \in \mathbb{R}^{N_h}, \qquad \mathbf{u} \in \mathbb{R}^{N_h}.$$

Crucially, the number of parameters equals the mesh dimension: $p = N_h \approx 9\,500$. This is a **high-dimensional parameter space**, unlike the previous low-dimensional (ρ, ε) case.

17.4.3 Dataset

- **1000 random simulations** are available, each obtained by sampling a permeability field $\boldsymbol{\mu}_i$ and solving the PDE to obtain $\mathbf{u}_i$.
- Input–output pairs: $\{(\boldsymbol{\mu}_i, \mathbf{u}_i)\}_{i=1}^{1000}$.
- The goal is to build a surrogate model that approximates the map $\boldsymbol{\mu} \mapsto \mathbf{u}$.

17.4.4 Why Reduced Basis Fails

The weak formulation of the problem involves the bilinear form

$$a(u, v; \mu) = \int_\Omega \mu(\mathbf{x})\, \nabla u(\mathbf{x}) \cdot \nabla v(\mathbf{x})\, \mathrm{d}\mathbf{x}.$$

If μ were a scalar parameter (or a sum of a few scalar parameters), one could write

$$a(u, v; \mu) = \sum_{q=1}^{Q} \theta_q(\mu)\, a_q(u, v)$$

with parameter-independent forms a_q. However, when μ is an arbitrary spatial field, **it cannot be pulled out of the integral**; the matrix depends on the whole field and cannot be preassembled offline. Classical reduced basis methods, which rely on such affine parameter dependence, are therefore not straightforwardly applicable (hyper-reduction techniques would be required).

17.4.5 Singular Value Analysis of the Solution Manifold

As a first step, we perform a POD on the set of solution snapshots to assess their complexity.

- The singular value decay is not extremely sharp, but not prohibitively slow.
- Truncating at $n = 15$ POD modes yields a projection (reconstruction) error of approximately 2% (relative L^2 in space, averaged over the test set).
- For an exploratory analysis, a 2% error is often acceptable — especially when the permeability itself carries significant uncertainty.

Thus, a linear POD representation of the **output** field with $n = 15$ is a reasonable first choice.

17.4.6 Next Step: POD-NN with High-Dimensional Input

With a low-dimensional representation of u via POD coefficients $\mathbf{c} \in \mathbb{R}^n$, we still face the challenge of mapping the **high-dimensional input** $\boldsymbol{\mu} \in \mathbb{R}^{N_h}$ to these coefficients. A plain fully connected network from $\mathbb{R}^{N_h}$ to $\mathbb{R}^n$ would have an enormous number of parameters and would likely fail without proper regularisation or compression. This motivates the need for architectures that can handle high-dimensional inputs, which will be discussed in the remainder of the lab.

17.5 Addressing High-Dimensional Inputs: The PCINet Architecture

17.5.1 Failure of Plain PODNN

The first attempt to build a surrogate uses a POD-NN: a fully connected network ϕ mapping the high-dimensional permeability field $\boldsymbol{\mu} \in \mathbb{R}^{N_h}$ to the POD coefficients $\mathbf{c} \in \mathbb{R}^n$ of the pressure field.

$$\phi : \mathbb{R}^{N_h} \to \mathbb{R}^n.$$

The network is trained by minimising the MSE between predicted and true POD coefficients.

Training observations:

- After moderate training (a few hundred epochs), the test error remains high ($\approx 16\%$) while the training error drops to $\approx 5\%$.
- Plotting the error history (first 10 epochs omitted due to huge initial values) shows the training error continuing to decrease while the test error stagnates and eventually increases — a clear sign of **overfitting**.

- The network has over **1.2 million trainable parameters**. The vast majority come from the first layer:

$$\text{First layer: } \mathbb{R}^{N_h} \to \mathbb{R}^{131}, \qquad N_h \approx 9\,500 \implies 9\,500 \times 131 > 1.2 \times 10^6 \text{ parameters.}$$

Root cause: The input dimension is huge, making the first layer extremely wide. The number of training samples (1000) is insufficient to constrain so many parameters, leading to poor generalisation.

17.5.2 Possible Remedies

Two families of strategies exist to handle high-dimensional inputs:

1. **Clever network design:** Exploit structure in the input, e.g., use convolutional layers for grid-like data. However, the mesh here is unstructured, so standard CNNs do not apply directly. Specialised geometric deep learning methods (graph neural networks, mesh convolutions) would be needed.
2. **Dimensionality reduction of the input:** Apply a compression technique (e.g., POD) to the permeability field $\boldsymbol{\mu}$ itself, reducing its dimension from N_h to a moderate $M \ll N_h$. The network then learns a map between the reduced parameter coefficients and the solution coefficients.

We pursue the second approach, which leads to the PCINet surrogate.

17.5.3 PCINet: Principal Component Interaction Network

The idea is symmetric to what is done for the output: compress both the parameter field and the solution field using their respective POD bases.

$$\boldsymbol{\mu} \approx \mathbf{Z}\mathbf{z}, \qquad \mathbf{u} \approx \mathbf{V}\mathbf{c},$$

where

- $\mathbf{Z} \in \mathbb{R}^{N_h \times M}$ is the POD basis for the permeability snapshots,
- $\mathbf{z} \in \mathbb{R}^M$ are the corresponding reduced coefficients (M modes),
- $\mathbf{V} \in \mathbb{R}^{N_h \times n}$ is the POD basis for the pressure snapshots,
- $\mathbf{c} \in \mathbb{R}^n$ are the pressure coefficients (n modes, e.g., $n = 15$).

The surrogate model ϕ_{PCI} then learns the map

$$\phi_{\text{PCI}} : \mathbb{R}^M \to \mathbb{R}^n, \qquad \mathbf{z} \mapsto \mathbf{c}.$$

Full reconstruction is obtained by

$$\mathbf{u}^{\text{pred}} = \mathbf{V}\,\phi_{\text{PCI}}(\mathbf{Z}^\mathsf{T}\boldsymbol{\mu}).$$

This architecture was introduced under the name PCINet (Bhattacharya et al., Caltech, 2019) and later evolved into the concept of *neural operators*.

17.5.4 Implementation and Comparison

To assess whether PCINet outperforms plain PODNN, we keep the same latent dimension for the parameters as the size of the first hidden layer in the previous network, i.e., $M = 131$.

1. Perform SVD on the matrix of permeability snapshots (shape $N_h \times N_{\text{train}}$) to obtain $\mathbf{Z}$ and truncate to $M = 131$ modes.
2. Project all $\boldsymbol{\mu}$ to coefficients $\mathbf{z} = \mathbf{Z}^\mathsf{T}\boldsymbol{\mu}$.

3. Define a simple feed-forward network:

$$\mathbb{R}^M \xrightarrow{\text{Linear, GELU}} \mathbb{R}^{50} \xrightarrow{\text{Linear}} \mathbb{R}^n,$$

with no activation on the final layer.

4. Train on the pairs $(\mathbf{z}_i, \mathbf{c}_i)$ for the same number of epochs (e.g., 100) and with the same loss (MSE).

Resulting parameter count: The first layer now has size $131 \times 50 \approx 6\,550$ parameters, and the total network size is drastically reduced (roughly 7 000 parameters), completely eliminating the over-parameterisation issue.

The test error of this PCINet surrogate is then compared with the earlier plain PODNN to evaluate whether input compression resolves the generalisation gap.

17.6 PCINet Performance and Further Refinements

17.6.1 Evaluating PCINet on Test Data

The PCINet surrogate is constructed as

$$\mathbf{u}^{\text{pred}}(\boldsymbol{\mu}) = \mathbf{V}\, \phi_{\text{PCI}}\big(\mathbf{Z}^{\mathsf{T}}\boldsymbol{\mu}\big),$$

where $\phi_{\text{PCI}} : \mathbb{R}^{131} \to \mathbb{R}^n$ is the small feed-forward network trained on the reduced coefficients $(\mathbf{z}, \mathbf{c})$.

- The relative L^2 test error (space-time style, averaged over test samples) is **6–7%**.
- This is a dramatic improvement over the plain PODNN ($\approx 16\%$), obtained with essentially the same hidden-layer architecture, training epochs, and optimisation settings.
- The training and test losses now decrease together, indicating that the overfitting problem has been resolved by drastically reducing the input dimension.

17.6.2 Why POD Compression of the Input May Be Suboptimal

Strategy B (PCINet) compresses the permeability field $\boldsymbol{\mu}$ via POD, which finds a basis that reconstructs $\boldsymbol{\mu}$ *globally* well. However, the network does not need a faithful reconstruction of $\boldsymbol{\mu}$ everywhere; it only needs to extract the **features of $\boldsymbol{\mu}$ that influence the pressure field** u.

- For example, errors in the permeability near the sources may impact the pressure much more than errors in far-away regions.
- A POD basis for $\boldsymbol{\mu}$ is unaware of this importance weighting; it treats all spatial locations equally.

This suggests that a compression strategy guided by the final task might yield an even better surrogate. One such strategy is to use **sparse, mesh-informed** architectures.

17.7 Strategy A: Sparse Mesh-Informed Layers (Geometric Deep Learning)

17.7.1 Intuition: Limiting the Receptive Field

In a standard dense (linear) layer, each output neuron is connected to *all* input nodes, i.e., the whole mesh. This yields a fully populated weight matrix with $N_h \times M$ parameters (dominating the parameter count).

- We may instead let each output neuron only "see" a local neighbourhood of the input mesh, mimicking convolutional layers on structured grids.
- Physically, a neuron placed at a spatial location should aggregate information only from nearby points, because the elliptic PDE exhibits a local dependence on the coefficient.

This leads to the concept of a **mesh-informed layer** (also called a local or geometric layer).

17.7.2 Constructing a Mesh-Informed Layer

1. **Auxiliary coarse mesh:** A new mesh with $M = 131$ vertices is created; each vertex will correspond to one output neuron.
2. **Neuron placement:** The M neurons are located at the vertices of the coarse mesh.
3. **Support radius r:** A neuron collects information only from the input nodes that lie within a ball of radius r centred at its vertex.
4. The resulting weight matrix is **sparse**: many entries are forced to be zero, because only geometrically close pairs are connected.

The implementation uses a `mesh_informed_layer` class that replaces the first `Linear` layer.

```
layer = MeshInformedLayer(vertices_fine, vertices_coarse, support=0.2)
```

If r is large (e.g., covering the whole domain), the layer becomes dense again. As r decreases, the layer becomes sparser, dramatically reducing the number of trainable parameters and forcing local feature extraction.

17.7.3 Parameter Count and Sparsity

Using the auxiliary mesh with $M = 131$ and a moderate support, the number of parameters in the first layer drops from ≈ 1.25 million to roughly **one quarter** of that, solving the over-parameterisation issue without any dimensionality reduction of the input. A plot of sparsity versus support radius shows a smooth transition: very small r yields $\approx 100\%$ sparsity (and possibly too little information flow); increasing r makes the layer denser.

17.7.4 Full Model and Results

The rest of the architecture remains unchanged:

$$\mathbb{R}^{N_h} \xrightarrow{\text{MeshInformed}} \mathbb{R}^{131} \xrightarrow{\text{GELU}} \mathbb{R}^{50} \xrightarrow{\text{Linear}} \mathbb{R}^{n},$$

trained exactly like the plain PODNN (MSE loss, same optimiser, same number of epochs).

- The test relative L^2 error drops to **2–3%**, which is even better than PCINet ($\approx 6\%$) and far superior to the dense PODNN.
- This demonstrates that a task-adapted, sparse geometric layer can outperform a generic global compression when the input is a spatial field.

17.8 Visualising the Distribution of Errors: Box Plots

In addition to average errors, it is instructive to examine how individual test samples behave. A **box plot** (box-and-whisker plot) summarises the distribution of the per-sample errors.

- The **box** represents the interquartile range (IQR): the middle 50% of the errors.
- The line inside the box is the median.
- The **whiskers** extend to the most extreme non-outlier points.
- Outliers are plotted individually.

For the three methods, the box plots reveal:

- **PCINet:** errors spread between $\approx 2\%$ and 12%, with the central half roughly in the 2–6% range.

- **Mesh-informed layer:** all errors are tightly concentrated below 4–5%, not just a lower mean but also much smaller variance, indicating a uniformly reliable surrogate.

This confirms that the sparse geometric architecture yields not only better average accuracy but also greater robustness across the test set.

Chapter 18

Time extrapolation and latent dynamics for data-driven ROMs via DMD & SINDy

18.1 Extrapolation in Time: System Identification from Surrogate Data

18.1.1 Motivation: Limitations of Data-Driven Surrogates

We have built data-driven surrogates (POD-NN, DL-ROM) for parameterised time-dependent problems. These are trained on snapshot data coming from simulations on a Cartesian product $\mathcal{P} \times [0, T]$, where $\mathcal{P}$ is the parameter space and T the final time.

- **Parameter extrapolation:** If a new parameter $\boldsymbol{\mu}$ lies far from the training set, the surrogate is not guaranteed to produce physically meaningful results; it is essentially an interpolation in the combined input space.
- **Time extrapolation:** Asking the surrogate to predict beyond T (i.e. extend the time horizon) is equally unreliable if done by merely querying the trained network for $t > T$.

However, for a *fixed* parameter $\boldsymbol{\mu}_0$, the surrogate can generate a sequence of snapshots (a "movie") that captures the system's evolution on $[0, T]$. We may then attempt to **extract the underlying dynamical law** from this sequence and use it to step forward in time beyond T. This process is often termed **distillation**: the surrogate, trained on data, has an implicit dynamics; we distil it into an explicit dynamical model.

18.1.2 Distillation: Using DMD/SINDy on ROM Output

- Fix a parameter $\boldsymbol{\mu}_0$ and use the ROM to produce a discrete trajectory $\{\mathbf{u}_k\}_{k=0}^{N_t}$ (or its latent representation).
- Apply a system identification technique (e.g. Dynamic Mode Decomposition – DMD, or Sparse Identification of Nonlinear Dynamics – SINDy) to learn a dynamical model $\mathbf{u}_{k+1} = \mathbf{f}(\mathbf{u}_k)$ from these data.
- Use the learned model to advance the state beyond the original time window.

This approach exploits the fact that the ROM, while purely data-driven, encodes the system's temporal behaviour. Note that classical DMD/SINDy are typically *non-parametric*: they assume a single trajectory, not a family parameterised by $\boldsymbol{\mu}$. Hence we focus on a single fixed parameter.

18.1.3 Comparison with Reduced Basis Methods

In an intrusive reduced basis method, the time dynamics is explicitly given by the projected PDE operators; extrapolation in time is possible, albeit possibly degrading due to basis truncation errors. In data-driven surrogates no such dynamical model is available, making the distillation step necessary if one wishes to go beyond the training time interval.

18.2 Learning Dynamics from Data: A Simple Harmonic Oscillator

To introduce DMD, we consider a low-dimensional example with no spatial dependence.

18.2.1 True System

A damped harmonic oscillator in phase-space variables $z = (x,\ v)^\mathsf{T}$ (position and velocity):

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{pmatrix} x \\ v \end{pmatrix} = \begin{pmatrix} v \\ -4x - 0.1\,v \end{pmatrix},$$

i.e. $\dot{z} = Bz$ with a matrix B encoding the linear dynamics. The true trajectory is obtained by integrating this ODE from an initial condition $(x_0, v_0) = (0, 1)$, yielding $N_t = 1000$ equally spaced snapshots over $[0, T]$. The phase portrait spirals into the origin due to the damping.

18.2.2 Assumed Partial Data

We pretend that only the first *half* of the trajectory is observed ($N_{\text{train}} = N_t/2$ snapshots). The task is to learn a model from this partial data that can predict the remaining half.

18.2.3 Linear Dynamics Assumption (DMD Ansatz)

DMD assumes a discrete-time linear model

$$z_{k+1} = A\, z_k,$$

where $A \in \mathbb{R}^{2\times 2}$ is an unknown matrix. (Even if the true continuous system is linear, after time discretisation the map from z_k to z_{k+1} is indeed linear for a fixed time step Δt.)

18.2.4 Least-Squares Estimation of A

From the training snapshots $\{z_0, z_1, \ldots, z_{N_{\text{train}}-1}\}$ we form the matrices

$$Z_{\text{now}} = \begin{bmatrix} z_0 & z_1 & \cdots & z_{N_{\text{train}}-2} \end{bmatrix}^\mathsf{T}, \qquad Z_{\text{next}} = \begin{bmatrix} z_1 & z_2 & \cdots & z_{N_{\text{train}}-1} \end{bmatrix}^\mathsf{T}.$$

The relation $Z_{\text{next}} \approx A\, Z_{\text{now}}$ (where each row is transposed, so careful with dimensions) is solved in the least-squares sense:

$$A = \left(Z_{\text{now}}^{+} Z_{\text{next}}\right)^\mathsf{T},$$

which can be computed via `np.linalg.lstsq` with appropriate transposes.

18.2.5 Eigenvalue Decomposition of A

While one could simply iterate $z_{k+1} = Az_k$, it is instructive to diagonalise A:

$$A = \Phi \Lambda \Phi^{-1}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \lambda_2).$$

The eigenvalues are complex conjugates with magnitude $|\lambda| < 1$, reflecting the damped oscillation. The discrete-time evolution becomes

$$z_k = \Phi \Lambda^k \Phi^{-1} z_0,$$

providing a clean framework to advance the state arbitrarily far into the future.

18.2.6 From Simple Example to DMD on PDE Data

This toy problem illustrates the core idea of DMD: given a sequence of snapshots, assume a linear evolution operator, estimate it via least squares, and use its spectral decomposition for prediction. For high-dimensional systems (like discretised PDEs), DMD additionally couples this with a preliminary dimensionality reduction (POD), leading to the standard DMD algorithm. We will next apply this to the ROM-generated movie for the flow past a cylinder.

18.3 Distillation: Extracting Dynamics from a Trained Surrogate

18.3.1 Motivation and Problem Setup

Data-driven surrogates (POD-NN, DL-ROM) are constructed by interpolating training data over the Cartesian product of parameter space $\mathcal{P}$ and time interval $[0, T]$.

- They cannot reliably **extrapolate in time** beyond T — attempting to evaluate the surrogate for $t > T$ typically yields physically meaningless, diverging solutions.
- However, for a *fixed* parameter $\boldsymbol{\mu}_0$, the surrogate can generate a trajectory (a "movie") on $[0, T]$ that captures the system's true dynamics implicitly.
- The idea of **distillation** is to apply system identification techniques (e.g. Dynamic Mode Decomposition — DMD, Sparse Identification of Nonlinear Dynamics — SINDy) to this trajectory to learn an explicit dynamical model, which can then be used to advance the solution beyond T.

This approach is non-parametric (restricted to a single $\boldsymbol{\mu}_0$) because classical DMD/SINDy operate on a single time series, not a parameterised family.

18.4 Dynamic Mode Decomposition: Linear Dynamics from Data

18.4.1 A Toy Example: Damped Harmonic Oscillator

Consider a two-dimensional linear system in phase space $\mathbf{z} = (x,\ v)^\mathsf{T}$:

$$\dot{\mathbf{z}} = \mathbf{B}\,\mathbf{z}, \qquad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ -4 & -0.1 \end{pmatrix}.$$

We generate a discrete trajectory $\{\mathbf{z}_k\}_{k=0}^{N_t-1}$ with $N_t = 1000$ equally spaced snapshots, and pretend that only the first half ($k = 0, \ldots, N_t/2 - 1$) is observed. The goal is to learn a model from the observed data that can predict the second half.

18.4.2 Linear Discrete-Time Model

DMD assumes a linear map between consecutive snapshots:

$$\mathbf{z}_{k+1} = \mathbf{A}\,\mathbf{z}_k, \qquad \mathbf{A} \in \mathbb{R}^{2\times 2}.$$

Even though the original ODE is linear, after time discretisation the map from $\mathbf{z}_k$ to $\mathbf{z}_{k+1}$ is indeed linear (for a fixed time step Δt). The matrix $\mathbf{A}$ is unknown and must be estimated from data.

18.4.3 Least-Squares Estimation of A

Construct the data matrices

$$\mathbf{Z}_{\text{now}} = \begin{bmatrix} \mathbf{z}_0 & \mathbf{z}_1 & \cdots & \mathbf{z}_{N-2} \end{bmatrix}^\mathsf{T}, \qquad \mathbf{Z}_{\text{next}} = \begin{bmatrix} \mathbf{z}_1 & \mathbf{z}_2 & \cdots & \mathbf{z}_{N-1} \end{bmatrix}^\mathsf{T},$$

where $N = N_t/2$ is the number of training snapshots. The relation $\mathbf{Z}_{\text{next}} \approx \mathbf{A}\,\mathbf{Z}_{\text{now}}$ is solved in the least-squares sense:

$$\mathbf{A} = \big(\mathbf{Z}_{\text{now}}^{+}\mathbf{Z}_{\text{next}}\big)^{\mathsf{T}},$$

using `np.linalg.lstsq` (with appropriate transposes). The resulting $\mathbf{A}$ is a 2×2 matrix that best fits the observed transitions.

18.4.4 Prediction via Eigenvalue Decomposition

To advance the state many steps efficiently, diagonalise $\mathbf{A}$:

$$\mathbf{A} = \mathbf{\Phi}\,\mathbf{\Lambda}\,\mathbf{\Phi}^{-1}, \qquad \mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \lambda_2).$$

The eigenvalues are complex conjugates with $|\lambda_i| < 1$ (damping). Given an initial condition $\mathbf{z}_0$, the state at time k is

$$\mathbf{z}_k = \mathbf{\Phi}\,\mathbf{\Lambda}^k\,\mathbf{\Phi}^{-1}\mathbf{z}_0.$$

Equivalently, expand $\mathbf{z}_0$ in the eigenbasis: $\mathbf{z}_0 = \mathbf{\Phi}\,\mathbf{c}$, then

$$\mathbf{z}_k = \mathbf{\Phi}\big(\boldsymbol{\lambda}^k \odot \mathbf{c}\big),$$

where $\odot$ is the element-wise (Hadamard) product and $\boldsymbol{\lambda} = (\lambda_1, \lambda_2)^{\mathsf{T}}$.

- The coefficient vector $\mathbf{c}$ is found by solving the linear system $\mathbf{\Phi}\,\mathbf{c} = \mathbf{z}_0$ (since $\mathbf{\Phi}$ is square and invertible).
- Although eigenvalues and eigenvectors are complex, the resulting trajectory $\mathbf{z}_k$ is real; numerical round-off can be discarded by taking `np.real`.

Starting from the last observed state $\mathbf{z}_{N-1}$ and evolving for the remaining N steps perfectly reproduces the unseen test trajectory.

18.4.5 SVD-Based Formulation and Truncation

A mathematically equivalent formulation operates on the singular value decomposition (SVD) of $\mathbf{Z}_{\text{now}}$:

$$\mathbf{Z}_{\text{now}} = \mathbf{U}\,\mathbf{\Sigma}\,\mathbf{V}^{\mathsf{T}}.$$

One can show that the matrix $\tilde{\mathbf{A}} = \mathbf{U}^{\mathsf{T}}\mathbf{Z}_{\text{next}}\mathbf{V}\mathbf{\Sigma}^{-1}$ has the same non-zero eigenvalues as $\mathbf{A}$, with eigenvectors related by a rotation.

- In the 2×2 full-rank case this yields identical results.
- The advantage: when the state dimension is large (e.g. after discretising a PDE), we can **truncate** the SVD, keeping only r modes ($r \ll N_h$). Then $\tilde{\mathbf{A}}$ is $r \times r$, and the dynamics is evolved in the low-dimensional subspace.

This truncated SVD formulation is the essence of the standard DMD algorithm.

18.5 DMD on the DL-ROM Surrogate for the Navier–Stokes Problem

18.5.1 Problem Setup

We return to the flow past a cylinder. A DL-ROM surrogate (autoencoder + parameter-to-latent network ϕ) has been trained on the time interval $t \in [0, 3.5]$. For a fixed parameter $\boldsymbol{\mu}_0$, the surrogate produces a trajectory $\{\mathbf{u}_k\}_{k=0}^{N_t}$ of spatial snapshots (dimension $N_h \approx 6000$).

- Evaluating the surrogate directly for $t > 3.5$ leads to a catastrophic blow-up; the model is purely interpolatory and has no built-in physical stability.
- We will use the "reliable" part of the surrogate trajectory ($0 \le t \le 3.5$) as training data for DMD, then use the learned DMD model to extrapolate to $t = 7.0$.

18.5.2 Implementation Steps

1. **Prepare data matrices:** From the surrogate trajectory, form $\mathbf{U}_{\text{now}}$ and $\mathbf{U}_{\text{next}}$ as before, using snapshots from 0 to $T = 3.5$.

2. **Truncated SVD:** Compute the SVD of $\mathbf{U}_{\text{now}}$, retain r modes (e.g. $r = 10$).

$$\mathbf{U}_{\text{now}} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^{\mathsf{T}}.$$

3. **Reduced dynamics matrix:** Form

$$\tilde{\mathbf{A}} = \mathbf{U}_r^{\mathsf{T}} \mathbf{U}_{\text{next}} \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \in \mathbb{R}^{r \times r}.$$

4. **Eigen decomposition:** Diagonalise $\tilde{\mathbf{A}} = \mathbf{W} \mathbf{\Lambda} \mathbf{W}^{-1}$.

5. **Initial condition in reduced space:** The initial snapshot for extrapolation is the last reliable snapshot $\mathbf{u}_{N-1}$ (at $t = 3.5$). Find its reduced coefficients $\mathbf{c}$ by *least squares* (since $\mathbf{U}_r$ is rectangular):

$$\mathbf{c} = \arg\min_{\mathbf{c}} \|\mathbf{U}_r \mathbf{c} - \mathbf{u}_{N-1}\|_2^2 \quad \Longrightarrow \quad \mathbf{c} = \mathbf{U}_r^{+} \mathbf{u}_{N-1}.$$

   This is equivalent to a projection onto the POD basis.

6. **Time extrapolation:** For $k = 0, 1, 2, \ldots$, compute

$$\mathbf{u}_{N-1+k}^{\text{pred}} = \mathbf{U}_r \, \mathbf{W} \big(\boldsymbol{\lambda}^k \odot \mathbf{c} \big),$$

   where $\boldsymbol{\lambda}$ are the diagonal entries of $\mathbf{\Lambda}$ (eigenvalues of $\tilde{\mathbf{A}}$).

18.5.3 Expected Outcome

The DMD-extrapolated trajectory remains bounded and physically plausible well beyond $T = 3.5$, in stark contrast to the direct DL-ROM evaluation which explodes. The quality depends on the richness of the training trajectory and the chosen rank r. This demonstrates that distilling the implicit dynamics of a data-driven surrogate with DMD enables robust time extrapolation without requiring access to the full-order model or intrusive projection.

18.6 Truncated SVD and DMD Evolution

18.6.1 Truncation of the SVD

Given the data matrices $\mathbf{U}_{\text{now}}$ and $\mathbf{U}_{\text{next}}$, we compute the singular value decomposition of $\mathbf{U}_{\text{now}}$:

$$\mathbf{U}_{\text{now}} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^{\mathsf{T}}.$$

To obtain r DMD modes (e.g. $r = 10$), we truncate all three factors:

- $\mathbf{U}_r = \mathbf{U}[:, : r]$ (keep first r columns)
- $\mathbf{\Sigma}_r = \mathbf{\Sigma}[: r, : r]$ (leading $r \times r$ submatrix)
- $\mathbf{V}_r = \mathbf{V}[: r, :]$ (keep first r rows of $\mathbf{V}^{\mathsf{T}}$, i.e. first r columns of $\mathbf{V}$)

This ensures that the reduced dynamics matrix

$$\tilde{\mathbf{A}} = \mathbf{U}_r^{\mathsf{T}} \mathbf{U}_{\text{next}} \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$$

has size $r \times r$, yielding r eigenvalues and eigenvectors.

18.6.2 DMD-based Evolution

The evolution function for the truncated DMD is adapted from the full-rank case. To obtain the initial coefficient vector $\mathbf{c}$ we must now solve a least-squares problem because $\mathbf{U}_r$ is rectangular:

$$\mathbf{c} = \arg\min_{\mathbf{c}} \|\mathbf{U}_r\mathbf{c} - \mathbf{u}_0\|_2^2 \quad \Longrightarrow \quad \mathbf{c} = \mathbf{U}_r^+\mathbf{u}_0,$$

using `np.linalg.lstsq`. The subsequent time stepping is

$$\mathbf{u}_k^{\text{pred}} = \mathbf{U}_r\,\mathbf{W}\big(\boldsymbol{\lambda}^k \odot \mathbf{c}\big),$$

where $\mathbf{W}$ and $\boldsymbol{\lambda}$ are the eigenvectors and eigenvalues of $\tilde{\mathbf{A}}$. The predicted snapshots are stacked along the time axis to form the extrapolated trajectory.

18.7 Issue of Growing Modes: Non-Autonomous Bias

The DMD spectrum for the flow past the cylinder shows eigenvalues with magnitude slightly larger than 1, indicating a growing mode. In the predicted movie, the flow strength increases unphysically over time.

- The DMD assumes an **autonomous** dynamical system $\mathbf{u}_{k+1} = \mathbf{A}\mathbf{u}_k$, i.e. the evolution law does not depend explicitly on time.
- The original Navier–Stokes problem, however, is **non-autonomous** during the start-up phase: the inlet velocity ramps up from zero to its nominal value over a finite time interval.
- If the DMD is trained on the whole trajectory (including the transient), it tries to capture the increasing inflow as an intrinsic growth mode, leading to a spuriously growing prediction.

Remedy: train the DMD only on the portion of the trajectory that has reached a (statistically) steady regime (limit cycle). This removes the artificial growth and yields a stable autonomous model.

18.8 Motivation for Latent Space Dynamics

The DMD was applied to the full-order snapshots ($N_h \approx 6000$). However, the autoencoder already provides a low-dimensional latent representation ($n = 7$). It would be computationally attractive to learn the dynamics directly in the latent space.

- If the original dynamics for $\mathbf{u}$ is linear (or even nonlinear), the encoded dynamics for the latent variable $\mathbf{z} = \Psi'(\mathbf{u})$ is **nonlinear** in general:

  $$\dot{\mathbf{z}} = \frac{\partial \Psi'}{\partial \mathbf{u}} \cdot \dot{\mathbf{u}} = \mathbf{J}_{\Psi'}(\mathbf{u})\,\mathbf{f}(\mathbf{u}) \approx \mathbf{J}_{\Psi'}(\Psi(\mathbf{z}))\,\mathbf{f}(\Psi(\mathbf{z})),$$

  where $\mathbf{J}_{\Psi'}$ is the Jacobian of the encoder.
- Consequently, even if $\mathbf{f}$ is linear, the latent dynamics involves products of nonlinear functions (the decoder Ψ and the Jacobian), making it inherently nonlinear.
- Standard linear DMD would therefore fail in the latent space. We need a method that can handle **nonlinear** dynamics on a low-dimensional state.

18.9 Nonlinear Dynamics Example: DMD Failure

To illustrate the limitation of linear DMD, we modify the harmonic oscillator by introducing a nonlinear term in the acceleration:

$$\ddot{x} = -4x - 0.1\,\dot{x} + \varepsilon\, x\,\dot{x}.$$

The resulting phase portrait is qualitatively similar to the linear oscillator but slightly deformed. When we train a linear DMD on the first half of the trajectory and attempt to predict the second half, the prediction initially tracks the true trajectory but quickly diverges in both phase and amplitude, failing to converge to the correct equilibrium. This demonstrates that a linear model is insufficient to capture the dynamics of even this mildly nonlinear system.

18.10 SINDy: Sparse Identification of Nonlinear Dynamics

To overcome the limitations of linear DMD, we employ **SINDy** (Sparse Identification of Nonlinear Dynamics), introduced by Brunton, Proctor, and Kutz. The core idea is to fit the dynamics to a **sparse** combination of candidate nonlinear functions from a user-defined library.

18.10.1 Mathematical Formulation

In continuous time, we assume the dynamics is governed by

$$\dot{\mathbf{z}} = \mathbf{f}(\mathbf{z}),$$

and we seek to identify $\mathbf{f}$ from data. A library of candidate functions $\boldsymbol{\Theta}(\mathbf{z})$ is constructed, e.g. for a two-dimensional state $\mathbf{z} = [z_1, z_2]^\mathsf{T}$:

$$\boldsymbol{\Theta}(\mathbf{z}) = \big[1,\ z_1,\ z_2,\ z_1^2,\ z_1 z_2,\ z_2^2,\ \ldots\big].$$

We then approximate

$$\dot{\mathbf{Z}} \approx \boldsymbol{\Theta}(\mathbf{Z})\,\boldsymbol{\Xi},$$

where $\dot{\mathbf{Z}}$ contains the measured (or numerically estimated) time derivatives, $\boldsymbol{\Theta}(\mathbf{Z})$ is the library evaluated on all snapshots, and $\boldsymbol{\Xi}$ is a sparse coefficient matrix. Sparsity is promoted via regularisation (e.g. sequentially thresholded least squares or L^1 penalty), so that only a few active terms are selected.

18.10.2 PySINDy Package

The `pysindy` Python package provides a ready-to-use implementation. The user specifies:

- the library of candidate functions (polynomial, trigonometric, etc.);
- the optimisation routine (e.g. STLSQ – Sequentially Thresholded Least Squares);
- and the differentiation method (e.g. finite difference to estimate $\dot{\mathbf{z}}$ from data).

The result is a compact, interpretable dynamical model that can be integrated forward in time.

18.10.3 Application to Latent Space Dynamics

In the context of the DL-ROM surrogate for Navier–Stokes, we will apply SINDy not to the full-order snapshots, but to the **latent trajectory** $\{\mathbf{z}_k\}$ produced by the encoder. Because the latent space dimension is already small ($n = 7$), no truncation is needed. SINDy will identify a sparse nonlinear ODE governing $\mathbf{z}$, which can then be used to extrapolate the latent state beyond the training time window, and subsequently decoded back to the full physical field.

18.11 SINDy: Sparse Identification of Nonlinear Dynamics

18.11.1 Extending Linear DMD to Nonlinear Dynamics

While DMD fits a linear model $\dot{\mathbf{z}} = \mathbf{A}\mathbf{z}$, many systems (including the encoded latent dynamics of a DL-ROM) are inherently nonlinear. SINDy generalises this by assuming the dynamics can be expressed as a **sparse**

combination of candidate nonlinear functions from a pre-defined library. For a state $\mathbf{z} \in \mathbb{R}^d$ (e.g. $d = 2$ for a harmonic oscillator), we construct a library $\boldsymbol{\Theta}(\mathbf{z})$ containing functions such as

$$\boldsymbol{\Theta}(\mathbf{z}) = \big[\, 1,\ z_1,\ z_2,\ z_1^2,\ z_1 z_2,\ z_2^2,\ z_1^3,\ z_1^2 z_2,\ \ldots \big].$$

We then seek a sparse coefficient matrix $\boldsymbol{\Xi}$ such that

$$\dot{\mathbf{Z}} \approx \boldsymbol{\Theta}(\mathbf{Z})\, \boldsymbol{\Xi},$$

with $\dot{\mathbf{Z}}$ obtained via numerical differentiation of the observed trajectory $\mathbf{Z}$.

18.11.2 Application to a Nonlinear Oscillator

Consider the harmonically damped oscillator with an additional nonlinear term:

$$\ddot{x} = -4x - 0.1\,\dot{x} + \varepsilon\, x\dot{x}.$$

The true continuous dynamics is nonlinear. When DMD is applied, the linear model fails to capture the long-term evolution. SINDy, on the other hand, can identify the correct structure.

Using the `pysindy` package, we define:

- **Library:** polynomial terms up to degree 3, including cross-terms.
- **Optimiser:** Sequentially Thresholded Least Squares (STLSQ) with a sparsity-promoting threshold.
- **Derivatives:** centred finite differences to estimate $\dot{\mathbf{z}}$ from $\mathbf{z}(t)$.

The resulting model returns a sparse set of coefficients. For the chosen threshold, it identifies

$$\dot{z}_1 = z_2, \qquad \dot{z}_2 = -4z_1 - 0.05z_2 - 0.001z_1^2 + 1.0\, z_1 z_2 + 0.007\, z_1 z_2^2,$$

which closely matches the true dynamics (the extraneous z_1^2 and $z_1 z_2^2$ terms have very small coefficients). By adjusting the threshold, these spurious terms can be eliminated, recovering the exact governing equations. The SINDy model, when integrated forward, reproduces the unseen part of the trajectory with high fidelity, demonstrating its ability to identify nonlinear dynamics from data.

18.12 SINDy on the Latent Space of a DL-ROM

18.12.1 Motivation

In the context of the Navier–Stokes flow past a cylinder, the DL-ROM provides a low-dimensional latent representation $\mathbf{z}(t) \in \mathbb{R}^n$ ($n = 7$) of the flow snapshots. Evolving the latent state directly would be computationally cheap, but the latent dynamics is nonlinear due to the autoencoder's nonlinear activation functions. We therefore apply SINDy to discover a sparse ODE governing $\mathbf{z}(t)$.

To avoid the non-autonomous transient (inflow ramp-up), we discard the first portion of the latent trajectory and retain only the part where the flow has reached a statistically steady limit cycle. This ensures that the identified dynamics is autonomous and can be safely extrapolated in time.

18.12.2 Data Preparation

From the DL-ROM surrogate at a fixed parameter $\boldsymbol{\mu}_0$, we generate the latent trajectory $\mathbf{Z}_{\text{train}} \in \mathbb{R}^{91\times 7}$ (snapshots from $t \approx 1.4$ to 3.5). The time derivative $\dot{\mathbf{Z}}_{\text{train}}$ is estimated via centered finite differences. Care is taken with axis conventions: time is on the rows, states on columns.

18.12.3 Choice of the Library

A naive polynomial library up to moderate degree suffers from a combinatorial explosion (for $d = 7$, the number of monomials grows rapidly), leading to a dense, uninterpretable model and overfitting.

Inspired by the architecture of the autoencoder, which uses LeakyReLU-type activations, a custom library is constructed from functions that mimic these nonlinearities:

$$\phi_1(z) = \max(z, 0), \qquad \phi_2(z) = \max(-z, 0).$$

By including these (and possibly their polynomial combinations), the library remains manageable and aligns with the inductive bias of the encoder/decoder, resulting in a sparse model that captures the essential dynamics.

18.12.4 Training and Results

We fit a SINDy model with this custom library and a moderate sparsity threshold. The identified latent ODE is integrated forward from the last training state ($t = 3.5$) to $t = 7$. The extrapolated latent trajectory is then decoded back to the physical space via the decoder Ψ:

$$\mathbf{u}^{\text{pred}}(t) = \Psi\big(\mathbf{z}^{\text{SINDy}}(t)\big).$$

The combined movie (DL-ROM snapshots for $0 \le t \le 3.5$ and SINDy-extrapolated snapshots for $t > 3.5$) shows a physically plausible continuation of the vortex shedding, without the artificial blow-up or collapse that would result from directly querying the DL-ROM outside its training horizon.

18.12.5 Discussion

- **Interpretability vs. robustness:** The SINDy model provides an interpretable ODE, but its performance is sensitive to the choice of library and threshold. For higher latent dimensions, finding a suitable sparse representation becomes challenging.
- **Computational cost:** Evolving the SINDy ODE is slower than a single DL-ROM forward pass, but still orders of magnitude faster than the full-order Navier–Stokes solver.
- **Comparison with DMD:** While DMD on the full-order snapshots also produced plausible extrapolation, SINDy on the latent space leverages the autoencoder's compression and naturally handles nonlinearities, making it a more principled approach for surrogate-based time extrapolation.

18.13 Outlook

The combination of DL-ROM surrogates with system identification techniques (DMD, SINDy) enables data-driven time extrapolation without requiring intrusive projection or full-order solvers. The next step is the introduction of the **SHRED** architecture, a further development that integrates these ideas and will be presented in the following lecture. The final lab will also cover practical assignments and course wrap-up.

Chapter 19

Shallow recurrent decoder networks for reduced order modeling of parametrized systems from sensor data (SHRED-ROMs)

19.1 Motivation: learning solutions from sensor data

The goal of this part of the course is to introduce an estimation strategy that uses *sensor measurements* as inputs to a model.

Up to now, the setting we have considered is the classical one in reduced-order modeling: one prescribes a parameter vector describing the scenario of interest, and the model returns the corresponding solution. This viewpoint is very natural when the input is a set of physical parameters such as the Reynolds number, the angle of attack, or other coefficients describing the configuration.

However, in many applications one does not have access only to parameters. Instead, one may observe the system through a set of sensors that measure quantities such as velocity, pressure, temperature, or other physical fields at selected points in space and time. This suggests a different learning problem: rather than mapping parameters to solutions, we would like to map *sensor readings* to the solution of the underlying dynamical system.

19.1.1 From parameters to measurements

The classical reduced-order workflow can be summarized as follows:

- collect a set of solutions for different parameter values;
- store solution snapshots;
- train a model that learns the map from parameters to solutions.

In the new setting, the available information is not a parameter vector but a stream of measurements produced by sensors. The objective is then to infer the state of the system from these observations, possibly in real time.

19.2 Classical motivation: separation of variables

To understand the idea behind this approach, it is useful to recall the classical method of separation of variables for linear partial differential equations.

Suppose that the unknown solution depends on space and time,

$$u = u(x, t).$$

A standard ansatz is to write the solution as a product of two functions,

$$u(x, t) = X(x)\, T(t),$$

where X depends only on space and T depends only on time.

This transformation reduces the original problem into two simpler problems:

- one equation for the temporal factor $T(t)$;
- one equation for the spatial factor $X(x)$.

Typically, this leads to eigenvalue problems of the form

$$\mathcal{L}X = \lambda X,$$

where $\mathcal{L}$ is the spatial operator. Depending on the operator, the eigenfunctions may be exponential, sinusoidal, or cosinusoidal functions.

19.2.1 Reconstruction from the initial condition

Once the separated solutions are obtained, the general solution is expressed as a linear superposition:

$$u(x, t) = \sum_n a_n\, X_n(x)\, T_n(t).$$

The coefficients a_n are determined by the initial condition:

$$u(x, 0) = u_0(x).$$

Indeed, setting $t = 0$ gives

$$u_0(x) = \sum_n a_n\, X_n(x),$$

so the initial condition provides the information needed to recover the unknown coefficients.

In many classical settings, these coefficients are computed by projecting the initial datum onto the spatial eigenfunctions:

$$a_n = \int_\Omega u_0(x)\, X_n(x)\, dx,$$

up to normalization constants.

19.2.2 Observing the solution at sensor locations

A similar idea applies if, instead of knowing the initial condition everywhere in space, we know the solution at a fixed sensor location x_s over time. In that case,

$$u(x_s, t_k) = \sum_n a_n\, X_n(x_s)\, T_n(t_k),$$

where t_k denotes a measurement time.

Thus, sensor data can also be used to infer the coefficients a_n. The same reasoning extends to moving sensors, where the measurement location becomes time-dependent:

$$x = x_s(t).$$

This is important because, in practice, sensors need not be fixed.

19.3 From separation of variables to SHRED

The key idea of the lecture is to keep the intuition behind separation of variables, but generalize it to a much broader setting.

The classical method works for linear partial differential equations. Here, instead, we want to address a generic dynamical system and build a learning architecture that can exploit sensor measurements directly.

19.3.1 The learning problem

The problem can be divided into two parts:

1. **Temporal prediction.** Given the history of a sensor signal, predict its future evolution.
2. **Spatial reconstruction.** Use the sensor measurements to reconstruct the full state of the system in the entire domain.

The first task is a time-series forecasting problem. For example, if we know the temperature measured in Milan over the past year, we may try to predict its future evolution from that history alone.

The second task is a map from a low-dimensional observation space, whose dimension is the number of sensors, to the full high-dimensional state space.

19.3.2 Why the encoder is no longer needed

In earlier reduced-order learning methods, an encoder was used to compress high-dimensional solution snapshots into a low-dimensional latent space. This was necessary because the input data lived in the full state space.

Here, the situation is different. The input is already low-dimensional, because it consists of sensor readings. Therefore, there is no need for an encoder whose role is dimensionality reduction.

What we need instead is a model that:

- takes sensor measurements as input;
- evolves them in time;
- reconstructs the full solution in the physical domain.

19.3.3 Role of the recurrent decoder

This is the motivation for the *recurrent decoder* architecture, also referred to in the course as *SHRED*. The idea is to combine:

- a recurrent component, to process the temporal evolution of sensor data;
- a decoder, to map the resulting latent representation back to the full spatial field.

In this framework, the sensors provide the input signal, the recurrent network captures the time dependence, and the decoder reconstructs the solution over the whole domain.

19.4 Summary of the main intuition

The entire story can be summarized as follows:

- classical separation of variables expresses a solution as a sum of spatial and temporal modes;
- the coefficients of these modes can be recovered from initial data or from sensor measurements;
- sensor data suggest a natural learning problem where the input is already low-dimensional;
- therefore, instead of using an encoder, we use a recurrent decoder to forecast sensor signals and reconstruct the full state.

19.5 Time-windowed sensor data and SHRED

19.5.1 Chunking the time series

In practice, a time series is often divided into small windows. At each step, the model receives a fixed number of past observations and uses them to predict a new window in the future. In this way, the future evolution is inferred by looking only at a finite history of previous chunks.

The idea is particularly useful when dealing with complicated dynamics. In the original formulation, this approach was introduced mainly for time forecasting, not yet in the reduced-order modeling setting with varying parameters. The goal was simply to predict the future of a given scenario from observed data.

19.5.2 From forecasting to state reconstruction

The setting becomes more interesting when one wants to generalize not only in time, but also across different scenarios. In that case, one may have snapshots collected across several parameter values and several time instants.

The original shallow recurrent decoder (SHRED) architecture was introduced precisely in this spirit. During training, the network receives the sensor measurements collected over time, divided into chunks. These chunks are passed through the model in order to learn how to forecast the future evolution of the observed signal.

19.5.3 The role of the recurrent network

A key component of SHRED is a recurrent neural network, for instance an LSTM (*Long Short-Term Memory*) network. Such architectures are designed to process sequences and are therefore well suited for time-series prediction.

The input to the recurrent network consists of the past sensor measurements over a fixed number of lags. From this information, the network produces a latent representation that is then passed to the decoder.

19.5.4 Decoder-based reconstruction

The decoder takes the output of the recurrent network and reconstructs the quantity of interest in the full spatial domain. In this way, the model learns how to map sensor data to the full field.

This is similar in spirit to the DL-ROM setting discussed earlier, where an encoder, a latent network, and a decoder were combined together. Here, however, the input is already low-dimensional because it consists of sensor readings, so the encoder is no longer needed. The architecture is therefore simpler: the recurrent block processes the temporal information, and the decoder reconstructs the field.

19.6 Training objective

During training, the parameters of the neural network are determined by minimizing a loss function. The most common choice is a mean-square error between the true data and the reconstructed output.

If u_k denotes the target quantity at time window k and $\hat{u}_k$ the prediction produced by the network, then the training objective can be written as

$$\mathcal{L} = \sum_k \|u_k - \hat{u}_k\|^2.$$

The prediction $\hat{u}_k$ is obtained by feeding the sensor data from the previous L time lags into the recurrent block and then passing the result through the decoder.

The purpose of the training is to learn the map

$$\text{sensor history} \longmapsto \text{full field}.$$

Once trained, the model can be used online: as new sensor measurements arrive, they are passed through the network to produce new predictions.

19.7 Remarks on dynamical constraints

One could in principle try to enforce the structure of the underlying dynamical system directly in the architecture. In some cases this may be useful, but SHRED is already flexible enough that such constraints are not always necessary. The trade-off is that the model becomes less interpretable, because one no longer explicitly identifies the dynamical law satisfied by the latent variables.

This is one of the differences with classical model reduction methods such as POD-based reduced-order models, where the structure of the approximation is more transparent.

19.8 POD-based variants

In some variants, the decoder output is not directly the full field, but rather the coefficients of a reduced representation. These coefficients are then expanded on a POD basis.

If Ψ denotes the matrix of POD modes, the reconstructed state can be written as

$$u \approx \Psi a,$$

where a is the coefficient vector produced by the network.

This corresponds to a hybrid architecture, in which the neural network predicts reduced coefficients and POD is used as a final lifting step. The loss can then be computed either in the full space or after projection onto the POD subspace.

19.8.1 Interpretation

This type of construction is useful when the full state is very high-dimensional, because the POD basis provides an additional compression of the output. In that sense, the network learns only the reduced coordinates, while the physical reconstruction is delegated to the POD basis.

19.9 Examples and intuition

The architecture has been tested on examples with moving sensors and time-dependent trajectories. A typical case is a system with one or more sensors moving through the domain while measuring the quantity of interest.

Another example is the shallow water equations, where the relevant quantities may include the water height and the velocity field. In such settings, the SHRED architecture can successfully reconstruct the evolution of the system from sparse measurements.

19.10 Extension to multiple scenarios

The same logic can be extended to parametric problems. Suppose now that the solution depends not only on time, but also on a parameter vector μ describing the scenario.

Then the coefficients, the basis functions, and the initial data may all depend on μ. The same separation-of-variables intuition still applies, but the formulas now involve parameter-dependent quantities.

For example, if the initial condition depends on μ, then the coefficients of the expansion also depend on μ. If the sensor locations are fixed, or even moving in time, the corresponding measurements may still be used to identify these coefficients.

19.10.1 Learning from parametric sensor data

Rather than carrying out all these calculations analytically, one can train a neural architecture to learn the relation between sensor measurements and the solution. This allows us to handle cases in which the dependence on μ is too complicated to treat by hand.

In this way, the same general philosophy remains valid:

- use sensor data as input;
- learn the temporal evolution;
- reconstruct the full solution;
- extend the method to parameter-dependent and moving-sensor settings.

The main message is that sensor-based learning provides a practical way to close the problem, even in situations where the classical analytical derivation would be too cumbersome or impossible to perform explicitly.

19.11 A pedagogical example: a three-dimensional dynamical system

To make the idea more intuitive, let us consider a simple three-dimensional dynamical system depending on a parameter μ. This example has no fluid or structural interpretation; it is only meant to illustrate the mechanism behind the method.

19.11.1 Classical solution via eigen-decomposition

Consider a linear system of the form

$$\dot{u}(t) = Au(t),$$

where $A \in \mathbb{R}^{3\times 3}$.

As discussed earlier, the solution can be written as a superposition of three fundamental modes:

$$u(t) = c_1\phi_1 e^{\lambda_1 t} + c_2\phi_2 e^{\lambda_2 t} + c_3\phi_3 e^{\lambda_3 t},$$

where:

- $\lambda_1, \lambda_2, \lambda_3$ are the eigenvalues of A;
- ϕ_1, ϕ_2, ϕ_3 are the associated eigenvectors;
- c_1, c_2, c_3 are the coefficients determined by the initial condition.

In the particular example discussed in class, the three eigenvalues are 2, -1, and 1.

19.11.2 Recovering the coefficients from the initial condition

If the full initial state $u(0)$ is known, then the coefficients c_1, c_2, c_3 can be found by imposing

$$u(0) = c_1\phi_1 + c_2\phi_2 + c_3\phi_3.$$

Since this gives three equations for three unknowns, the system is fully determined.

19.11.3 Recovering the coefficients from sensor data

Instead of knowing the whole initial state, we may only observe one component of the system at a fixed sensor location, and at a few time instants. This gives a different way to determine the same coefficients.

If the sensor measurements are sufficiently informative, then the observed data can be inserted into the explicit solution formula, and the unknown coefficients can again be recovered.

Thus, in this simple setting, there are two alternative ways to identify the solution:

1. using the full initial condition;
2. using sparse sensor measurements over time.

19.12 Parametric setting and time-windowed sensor data

We now move to the general case considered by SHRED, where both the state and the measurements may depend on a parameter μ.

19.12.1 Training data

For each parameter value μ_i, we observe a sequence of sensor measurements over time. These measurements are split into chunks, or windows, of length L.

For each scenario μ_i, we therefore have a collection of training pairs of the form

$$\big(\text{sensor history, future target}\big).$$

The data are organized as follows:

- N_p parameter scenarios are considered;
- for each scenario, sensor data are available over multiple time steps;
- each time series is divided into chunks of fixed length;
- the network learns from all trajectories simultaneously.

19.12.2 Offline and online phases

The method is split into two stages:

Offline stage. The network is trained on a dataset containing:

- sensor measurements;
- full-state snapshots;
- multiple parameter values;
- multiple time windows.

Online stage. Once trained, the model receives new sensor measurements for an unseen parameter value μ and predicts the corresponding state or future evolution.

19.13 Training objective

The loss function is built by comparing the network output with the reference snapshots.

If u_i^k denotes the true state corresponding to scenario μ_i and time window k, and $\hat{u}_i^k$ the reconstructed state, then the training objective has the form

$$\mathcal{L} = \sum_{i=1}^{N_p} \sum_{k} \left\| u_i^k - \hat{u}_i^k \right\|^2 .$$

The prediction $\hat{u}_i^k$ is obtained by:

1. feeding the past sensor chunks into the time-dependent network;
2. passing its output through the decoder;
3. comparing the reconstructed field with the reference data.

19.14 Why this is easier than autoencoder-based approaches

This architecture is simpler than a standard encoder-decoder model.

In an autoencoder-based reduced-order model, one must learn both:

- the map from the full state to a latent space;
- the map from the latent space back to the full state.

Here, the sensors already provide a low-dimensional input, so the encoder is not needed. The network only has to learn:

- how to process the time history of the sensors;
- how to lift the result back to the full state space.

This makes the training problem more robust and easier to handle.

19.14.1 Shallow recurrent decoder

The resulting model is called a *shallow recurrent decoder*:

- *recurrent*, because it processes time-dependent sequences;
- *decoder*, because it reconstructs the full field from the internal representation;
- *shallow*, because the architecture is intentionally compact.

19.15 Numerical examples

19.15.1 Unsteady cylinder flow

A first example is a fluid-flow simulation around a cylinder with no parameters, only time dependence.

The method is trained using three sensors placed in the domain. After training, the network is able to reconstruct the flow field and produce accurate predictions. The results show that the reconstructed trajectory follows the reference data closely, and the error remains small.

19.15.2 A one-dimensional parametric problem

A second example is a more challenging one-dimensional problem with parameter dependence and nontrivial operators.

In this case, the initial condition changes with the parameter, and the solution profile may vary substantially from one scenario to another. The method is tested on different initial conditions, and the SHRED reconstruction remains accurate.

19.16 Robustness with respect to sensors and chunk size

The experiments also show that performance depends on:

- the number of sensors;
- the location of the sensors;
- the size of the time chunks used as input.

As expected, the method becomes more reliable when enough sensors are available and when the input windows contain sufficient temporal information.

19.16.1 Main takeaway

The main message is that, once a suitable amount of sensor information is available, the shallow recurrent decoder can successfully reconstruct the state of the system and forecast its evolution, both in time-dependent and parametric settings.

19.17 Numerical experiments: fixed and moving sensors

19.17.1 Sensor placement and robustness to noise

In the first set of experiments, the sensors are fixed in space and the effect of measurement noise is investigated. The goal is to understand how robust the method is when the observed data become increasingly corrupted.

The tracer is a transported quantity, possibly with some diffusion, whose evolution is driven by the underlying flow. The objective is to learn the behavior of this tracer from sparse sensor observations, for different scenarios of the system.

19.17.2 Fixed versus moving sensors

Two representative cases are considered:

- a configuration with three fixed sensors;
- a configuration with a single moving sensor.

In both cases, the reconstruction error remains small, showing that the architecture can cope well with sparse observations. Increasing the number of sensors does not necessarily lead to a dramatic improvement once the measurement set is already sufficiently informative.

19.17.3 Parametric setting

The method is also tested in a parametric context, where the system depends on several physical parameters, such as:

- the inflow intensity;
- the angle of attack;
- small variations in the geometry of the domain.

The experiments indicate that the approach remains effective even when the problem includes parameter dependence. The information coming from the sensors and the full-field snapshots collected during training is enough to learn the map from sensor data to the solution, even when the dependence on the parameters is not explicitly used in the architecture.

19.18 Why this is useful

One of the strengths of the method is that it does not require the parameter vector μ to appear explicitly as an input. This is different from approaches such as POD-DL-ROM or other parametric reduced-order methods, where the parameter vector is part of the network input.

Here, the sensor measurements contain sufficient information to infer the state of the system, and in some cases even to identify the corresponding parameter regime.

19.19 Need for a time-prediction network

At this point, one part of the architecture is still missing: we know how the decoder works, but we still need a network that processes the temporal sequence of sensor measurements.

The goal is to learn how to forecast future values of a quantity from a sequence of past observations. This is a sequence-to-sequence forecasting problem.

19.20 Recurrent neural networks for time forecasting

19.20.1 Motivation

Suppose we observe a scalar quantity over time, or more generally a sequence of sensor measurements. The task is to predict future values from past data.

A recurrent neural network is designed exactly for this purpose. It builds a latent representation that summarizes the past history of the input sequence.

19.20.2 Basic recurrent architecture

Let x_t denote the current measurement and h_t the hidden state, i.e., a compressed representation of the past observations. A recurrent update has the form

$$h_t = \sigma\big(W_h h_{t-1} + W_x x_t + b\big),$$

where:

- W_h and W_x are trainable weights;
- b is a bias term;
- σ (or another activation function) provides the nonlinearity.

The hidden state h_t combines:

- the current input x_t;
- the information stored from the previous time steps.

This memory effect is what makes recurrent networks suitable for time-dependent problems.

19.20.3 Prediction

Once the hidden state has been computed, it can be used to predict the next value:

$$x_{t+1} = \mathcal{G}(h_t).$$

More generally, one may forecast multiple future steps, not only one-step-ahead predictions.

19.21 Training of recurrent models

The network is trained by comparing the predicted sequence with the observed data. A typical loss function is a mean-square error of the form

$$\mathcal{L} = \sum_t \|x_t - \hat{x}_t\|^2,$$

or, for multi-step forecasting, a similar loss over the entire predicted horizon.

The training procedure learns the weights and biases so that the reconstructed trajectory matches the observed time series as closely as possible.

19.22 Variants of recurrent architectures

The basic recurrent neural network can be enhanced by more advanced architectures, such as:

- **LSTM** (*Long Short-Term Memory*);
- **GRU** (*Gated Recurrent Unit*).

These models are designed to better preserve long-term information and avoid some of the limitations of simple recurrent networks.

19.23 Connection with the global picture

At this point, we can summarize the evolution of the course as follows:

- we started from classical reduced basis methods;
- then we discussed POD-based reduced-order models;
- then DL-ROM and POD-DL-ROM architectures;
- then latent-dynamics models such as DMD and related approaches;
- finally, we moved to sensor-based learning with SHRED-like architectures.

The main novelty of the last part is that the input is no longer a parameter vector or a full snapshot, but sensor data collected over time. This richer information makes it possible to reconstruct and forecast the full state of the system.

19.24 Closing remarks

The story is not finished. The methods discussed so far do not cover every possible class of problems, and further generalizations are still possible.

The next step will be to revisit these ideas in a more applied setting and discuss additional open questions and ongoing research directions.

Chapter 20

Recovering from sensors with SHRED based ROMs

20.1 Assignments for the Second Part of the Course

20.1.1 General Guidelines

Three exercises are proposed, from which you can choose one (or more, but only one needs to be presented at the oral exam). The main differences from the first series of assignments are:

- You will work exclusively with **data-driven ROMs**: `PODNN` and `DL-ROM`. The choice of which surrogate to employ is up to you.
- You will be given the data (snapshots) directly; no full-order model code is provided. You must construct the surrogate, split data into training/test sets, evaluate its accuracy, and use it to answer a specific question.
- You need to be able to **justify your methodological choices**: why a data-driven approach instead of an intrusive reduced basis method, and why PODNN or DL-ROM (or both) for the specific problem.

20.1.2 The Three Test Cases

1. **Stationary Darcy flow.** A 2D simplified hydrogeological model governed by an elliptic PDE. The parameter is a porosity coefficient. The output is a pressure field, post-processed to visualise the flow. The question involves parameter inference or optimisation.
2. **Time-dependent damage propagation.** Models the evolution of damage inside a mechanical component. The parameters are material properties and the initial location of the damage. You are asked to **extrapolate in time** the damage evolution using the surrogate and possibly distillation techniques (DMD/SINDy).
3. **Time-dependent linear elasticity (sidewalk deformation).** A square domain consisting of two materials is compressed from the top by a force (mimicking a footstep). The parameters include layer thicknesses. You must build a surrogate and then **infer an unknown thickness** from a given surface deformation measurement, using the surrogate for fast inversion.

The notebooks are to be sent at least one day before the oral exam.

20.2 SHRED: Sensor-Based Surrogate Modelling

20.2.1 Motivation and Core Idea

In many real-world applications, the parameters governing a system are unknown or cannot be measured directly, but a few **sensors** placed in the domain provide time series of some observable quantities (pressure, velocity, etc.). The goal of SHRED (SHallow REcurrent Decoder) is to reconstruct the full spatio-temporal field from the history of these few sensor measurements, without ever knowing the underlying parameters.

- Unlike PODNN/DL-ROM, no parameter vector $\boldsymbol{\mu}$ is fed as input. The sensor signals *contain* the parameter information implicitly.
- This makes the approach suitable for black-box systems, experimental data, or scenarios where a parametrisation is not available.

20.2.2 Architecture

The model consists of two main components:

1. **Recurrent neural network (RNN):** typically an LSTM (Long Short-Term Memory), which processes the time series of sensor readings. The input at each time step is the concatenated sensor values; the RNN updates a hidden state that summarises the temporal history.
2. **Shallow decoder:** a small feed-forward network that maps the final hidden state of the RNN to the full spatial field at the corresponding time instant.

Optionally, the output can be further compressed using **POD**:

- Compute a POD basis from the snapshot matrix and train the decoder to predict only the POD coefficients (e.g. 500 modes) instead of the full field. The reconstruction is then $\mathbf{u}(t) = \mathbf{V}\mathbf{c}(t)$.
- This drastically reduces the size of the output layer and accelerates training, while retaining most of the energy even if the singular value decay is slow.

20.2.3 Training and Generalisation

Training uses a dataset of sensor histories and corresponding full-field snapshots, which may come from multiple parameter configurations (different Reynolds numbers, geometries, etc.). All data are merged and the model is trained with a reconstruction loss (e.g. MSE) on the spatial field (or on the POD coefficients). Once trained, the model can generalise to **unseen parameters**: given a new sensor time series (from a new experiment or simulation), it produces the full field in quasi-real-time. The sensor data automatically capture the parameter signature.

Additionally, one can append the parameter itself as an auxiliary output of the decoder, turning the model into a joint state and parameter estimator, even for time-varying parameters.

20.2.4 Examples

- **Rotating planet:** With three sensors measuring the water height, SHRED reconstructs the water height and velocity fields over the entire globe.
- **Oscillating cylinder (experimental video):** From only three pixels (treated as sensors) of the video frames, the model reconstructs the full flow field and generalises to unseen time intervals and configurations.
- **Parametric Navier–Stokes:** Given a few pressure/velocity sensors inside the domain, the model reconstructs the full velocity field channel-by-channel for varying Reynolds numbers and angles of attack, without being told the parameter values.

20.2.5 Connection with Separation of Variables

For linear (or linearised) PDEs with a parametric dependence, the solution can often be expressed as

$$u(x,t;\mu) = \sum_k c_k(t;\mu)\,\phi_k(x),$$

where $\phi_k(x)$ are eigenfunctions of the spatial operator and $c_k(t;\mu)$ are time- and parameter-dependent coefficients. A few well-placed sensors can, in principle, determine these coefficients. SHRED can be seen as a data-driven method that learns both the spatial basis (or the decoding map) and the time-stepping of the coefficients directly from sensor measurements, without requiring explicit knowledge of the PDE.

20.3 SHRED Tutorial: Drug Diffusion Model

20.3.1 Problem Description

We consider a three-compartment pharmacokinetic model describing the evolution of a drug administered to a patient. The state vector $\mathbf{x}(t) = (x_1, x_2, x_3)^\mathsf{T}$ represents the amount of drug in the central blood compartment and in two peripheral tissue compartments (highly and poorly perfused). The dynamics is governed by the linear ODE system

$$\dot{\mathbf{x}} = \mathbf{A}(\mu)\,\mathbf{x}, \qquad \mathbf{x}(0) = (1,\,0,\,0)^\mathsf{T},$$

where the parameter μ is the **elimination rate** (varied in $[0,1]$). The matrix $\mathbf{A}(\mu)$ encodes transfer rates between compartments and elimination. The problem admits an explicit solution as a sum of exponentials involving the eigenvalues and eigenvectors of $\mathbf{A}$.

We generate synthetic data by solving the ODE for N_μ equally spaced values of μ, producing a snapshot tensor of shape (parameters $\times$ time steps $\times$ 3). The training set is split into training/validation (80/20), and a separate test set with unseen μ is kept.

20.3.2 Observability and Sensor Choice

To apply SHRED, we place a single sensor measuring the drug concentration in the blood (first component of $\mathbf{x}$). For linear ODE systems, the observability matrix

$$\mathcal{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \end{bmatrix}, \qquad \mathbf{C} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix},$$

must have full rank to recover the full state from the output $y = \mathbf{Cx}$. This condition holds in our case. The intuition extends to PDEs: sensors should be placed where information from all relevant modes can be captured; random placements work well provided they avoid regions where the measured field is identically zero (e.g., on boundaries with Dirichlet conditions). If the spatial domain consists of disconnected dynamics, at least one sensor per region is needed.

20.3.3 Data Preprocessing for LSTM

The sensor time series (here a single channel) is preprocessed by:

- **Padding:** appending zeros to the beginning of the sequence so that the first input window aligns with the first time step.
- **Lagging (windowing):** splitting the padded time series into overlapping subsequences of fixed window length L. Each subsequence becomes one input sample to the LSTM, and the corresponding target is the full state vector at the final time step of that window (or the full trajectory, depending on design).

A custom `Torch Dataset` class encapsulates the logic to retrieve input–output pairs for training.

20.3.4 Model Architecture and Training

The SHRED model consists of:

- An LSTM layer processing the windowed sensor history (input size = number of sensors).
- A shallow decoder (a few fully connected layers) mapping the final LSTM hidden state to the full state vector (or to its POD coefficients in higher-dimensional problems).

Training uses the MSE loss between predicted and true states (or coefficients). After training on the merged data from multiple μ values, the model generalises to **new elimination rates** using only the corresponding sensor measurements; no parameter value is fed to the network. The decoder output can also be augmented to estimate μ as an auxiliary task, enabling simultaneous state and parameter identification.

20.4 SHRED Model Implementation

20.4.1 LSTM and Decoder Initialization

The SHRED model is implemented as a PyTorch `nn.Module` with two main components:

- An **LSTM** layer that processes the windowed sensor history.
- A **shallow decoder** that maps the final LSTM hidden state to the full-order state or POD coefficients.

In the constructor, the LSTM is defined with:

- `input_size` = number of sensors.
- `hidden_size` = dimension of the LSTM's hidden state (a hyperparameter).
- `num_layers` = number of stacked LSTM layers.
- `batch_first = True` to have the batch dimension as the first axis in the input tensor (shape: `[batch, seq_len, input_size]`).

The decoder is built dynamically from a list `decoder_sizes` that specifies the dimensions of all linear layers. The list is augmented by prepending `hidden_size` (the output of the LSTM) and appending `output_size` (the final target dimension). Hence

$$\texttt{decoder_sizes} = [\texttt{hidden_size}, d_1, d_2, \ldots, \texttt{output_size}].$$

A loop over consecutive pairs creates linear layers `Linear(in_features, out_features)`, each followed (except for the last layer) by a `ReLU` activation and optionally a `Dropout` layer. The absence of a nonlinearity on the final layer avoids restricting the output range, a common practice also seen in autoencoder decoders.

20.4.2 Forward Pass

The `forward(x)` method receives a batch of sensor windows `x` of shape `[batch, seq_len, n_sensors]`.

1. **Initialize LSTM states**: The hidden state $\mathbf{h}_0$ and cell state $\mathbf{c}_0$ are initialized to zero tensors of appropriate size.
2. **LSTM processing**: The input `x` and the initial states are passed through the LSTM, which returns an output sequence (only the last time step's hidden state is typically used, depending on the architecture) and the final hidden and cell states $(\mathbf{h}_T, \mathbf{c}_T)$.
3. **Decoder**: The final hidden state $\mathbf{h}_T$ (or a slice of it) is fed into the decoder network, producing the predicted full state or POD coefficients.

Optionally, if the decoder is also trained to predict the parameter $\boldsymbol{\mu}$, the output layer's size is increased accordingly, and the loss function combines reconstruction and parameter estimation errors.

20.4.3 Understanding the LSTM Recurrence

An LSTM maintains a hidden state $\mathbf{h}_t$ and a cell state $\mathbf{c}_t$ that summarise the information from all previous sensor readings up to time t. At each time step, it receives the current input $\mathbf{x}_t$ (the sensor values) and the previous states $(\mathbf{h}_{t-1}, \mathbf{c}_{t-1})$ and updates them via gated mechanisms. By unrolling over the window length, the final hidden state $\mathbf{h}_T$ captures the entire temporal context, which the decoder then lifts to the spatial domain. This split mirrors the separation of variables: the LSTM learns the temporal dynamics of the (latent) coefficients, and the decoder reconstructs the spatial field.

20.4.4 Training and Regularization

- **Dropout**: Applied between hidden layers of the decoder with a specified probability (e.g., 0.2) to randomly deactivate neurons during training, reducing overfitting.
- **Loss**: Mean squared error between the predicted and true full states (or POD coefficients), averaged over the batch.
- **Freezing**: After training, the model can be set to evaluation mode and its weights frozen via `model.eval()` and `torch.no_grad()`.

With this architecture, the SHRED model learns to map a short history of sensor measurements to the complete state, generalizing to unseen parameter values without ever being told the parameter.

20.5 SHRED Model: LSTM–Decoder Forward Pass Details

20.5.1 Role of LSTM Hidden States

The LSTM maintains two vectors that evolve over time: the hidden state $\mathbf{h}_t$ and the cell state $\mathbf{c}_t$. At each time step, they are updated using the current sensor observation $\mathbf{x}_t$ and the previous states ($\mathbf{h}_{t-1}$, $\mathbf{c}_{t-1}$).

- After processing the whole window of length L, the final hidden state $\mathbf{h}_L$ contains a summary of the entire sensor history.
- This vector is then used as the input to the shallow decoder to reconstruct the full-order state (or its POD coefficients).
- Alternatively, one could simply concatenate all sensor readings into one large vector and feed it to a feed-forward network, but the LSTM architecture better captures temporal dependencies and helps mitigate vanishing gradients, which often plague plain RNNs.

20.5.2 Computational Details in PyTorch

In the `forward` method:

1. The input `x` has shape `[batch, seq_len, n_sensors]`.
2. The initial hidden state $\mathbf{h}_0$ and cell state $\mathbf{c}_0$ are initialised as zero tensors (moved to GPU if available).
3. The LSTM returns three quantities: the output sequence (hidden states at every time step), the final hidden state $\mathbf{h}_L$, and the final cell state $\mathbf{c}_L$.
4. Only the final hidden state $\mathbf{h}_L$ is passed to the decoder; the other outputs are discarded.
5. The decoder is applied layer-by-layer via a simple loop, and the final linear layer (without activation) produces the prediction.

20.6 Training and Evaluation

20.6.1 Training Loop

- **Batch size:** a typical value is 64; each training step uses a mini-batch of short sensor windows.
- **Loss function:** mean squared error (MSE) between the predicted output (full state or POD coefficients) and the ground truth.
- **Optimiser:** Adam.
- **Early stopping:** a patience parameter (e.g., 10 epochs) monitors the validation loss; if it does not improve for a set number of epochs, training terminates.
- **Device:** all tensors are moved to GPU if available (Colab provides GPU support); the small size of the problem also permits fast CPU training.

20.6.2 Extracting Synthetic Sensor Data

Given the snapshot tensor `train` of shape $[N_\mu,\ N_t,\ 3]$, the sensor measurement is obtained by selecting only the first component (the drug concentration in the blood):

$$\texttt{sensor_train} = \texttt{train}[:, :, 0] \quad \in \mathbb{R}^{N_\mu \times N_t}.$$

The same procedure is applied to validation and test sets. This mimics a real experiment where only the blood concentration is monitored over time.

20.6.3 Applying Padding and Windowing

The function `pad_and_lag` processes the raw sensor time series:

- It prepends zeros to the beginning so that the first window starts at $t = 0$.
- It then extracts overlapping sub-sequences of a specified window length L (the lag).
- For the full-state snapshots, the same function is used with lag 1 only to align the dimensions without building a history.

The resulting windows are wrapped into a `TimeseriesDataset` class that provides indexed access to input–output pairs.

20.6.4 Model Initialisation and Training

The SHRED model is instantiated with:

- `input_size` = number of sensors (here 1).
- `output_size` = dimension of the state (here 3).
- Hyperparameters: hidden size, number of LSTM layers, decoder layer sizes, dropout rate.

After training, the loss curves show rapid convergence. Evaluating on the test set, the predicted trajectories (dashed) perfectly overlay the true ones (solid lines) for unseen elimination rates, demonstrating successful generalisation without any parameter input.

20.7 Extension to a High-Dimensional Problem: Double-Gyre Flow

20.7.1 Problem Description

The double-gyre flow is a 2D time-dependent velocity field, frequently used as a testbed for Lagrangian coherent structures. It is described by an analytical stream function containing two counter-rotating vortices whose oscillation frequency and amplitude are governed by a scalar parameter μ. Discretising on a 50×50 grid yields $N_h \approx 1250$ spatial degrees of freedom per velocity component. The parameter space is sampled to generate a dataset of parametric trajectories.

20.7.2 Output Compression via POD

To handle the high dimension, the output is compressed using Proper Orthogonal Decomposition. The singular value decay is extremely fast: four POD modes capture essentially all the energy for the x-velocity component, and similarly four modes for the y-velocity component. The state tensor is projected onto these bases, yielding a total of 8 POD coefficients per time step. Thus the decoder of SHRED only needs to predict these 8 coefficients; the full field is then recovered by multiplying with the POD basis.

20.7.3 Sensors and Training

Three random spatial points are selected as sensors (their coordinates are fixed during the simulation). For each new parameter, the sensor values are obtained by evaluating the solution at those locations. The procedure of padding, windowing, and training is identical to the drug diffusion case. The SHRED model now outputs the 8 POD coefficients, and training converges quickly. On test parameters, the full 2D velocity fields are faithfully reconstructed from only three scalar sensor time series.

20.8 Fourier DROM: Alternative Compression Strategy

Instead of projecting the solution snapshots onto a POD basis, one can use a Fourier basis. The Fast Fourier Transform (FFT) is applied to each snapshot, and a subset of low-frequency Fourier coefficients is retained (discarding high frequencies). This yields a compact frequency-domain representation. The SHRED decoder is then trained to predict these Fourier coefficients from the LSTM hidden state. This variant, termed **Fourier DROM**, offers an alternative to POD that may be advantageous when the solution is dominated by a few global oscillatory modes. The rest of the training pipeline remains unchanged.

20.9 Concluding Remarks

- SHRED demonstrates that a very small number of sensors, when their time history is processed by an LSTM, can reconstruct the full state of a parametric dynamical system, even for previously unseen parameters.
- The approach avoids both intrusive model reduction and the need for a known parameterisation.
- Through output compression (POD, Fourier) it scales to high-dimensional problems like discretised PDEs.
- Extensions include joint state-parameter estimation by appending the parameter to the decoder output and training with an augmented loss.
- Further examples (Kuramoto–Sivashinsky, Navier–Stokes) confirm the method's versatility.

Chapter 21

Our Path Review and Conclusion

21.1 The Broader Role of Reduced Order Modeling in Engineering

Reduced order modeling (ROM) plays a key role in several engineering tasks that require repeated, fast simulations:

- **Simulation:** Rapid evaluation of parameterised physical systems.
- **Design and control:** Optimising shapes, products, or steering systems toward desired behaviours – all demanding many queries of the underlying model.
- **Data assimilation:** Merging physics-based models with observations (e.g., filtering, uncertainty quantification).
- **Digital twins:** Real-time model updating and control require a reduced representation to close the loop.

While the course focused mainly on simulation, the techniques are directly transferable to these other domains.

21.2 Recap: Classical Reduced Basis Methods and Their Limits

The classical ROM pipeline consists of two steps:

1. **Manifold learning:** Compute a low-dimensional basis via Proper Orthogonal Decomposition (POD) of snapshot data.
2. **Dynamics reduction:** Project the governing equations onto that basis (Galerkin projection) to obtain a system of ODEs for the reduced coordinates.

For a general nonlinear time-dependent problem

$$\mathbf{M}\dot{\mathbf{u}} = \mathbf{F}(\mathbf{u}),$$

this yields a reduced dynamics for the coefficients $\mathbf{c}(t)$.

Obstacles encountered:

- **Slow singular value decay:** For convection-dominated problems (e.g., flow past a cylinder), the Kolmogorov n-width is large, so many POD modes are needed to capture the solution manifold.
- **Nonlinearity and non-affine parameter dependence:** Pre-assembling parameter-independent matrices is impossible; hyper-reduction techniques (EIM, DEIM) become necessary but can introduce additional complexity.

- **Multiphysics / multiparticle systems:** Each field or particle needs its own reduced basis, making the classical approach cumbersome.

These limitations motivated the exploration of **data-driven** surrogates.

21.3 Entering the World of Neural Networks: POD-NN

To avoid the intricacies of intrusive projection while retaining the POD basis, the first data-driven approach introduced was **POD-NN**:

$$\mathbf{c} = \phi(\boldsymbol{\mu}, t; \boldsymbol{\theta}),$$

where ϕ is a neural network trained to map the parameters and time directly to the POD coefficients, bypassing the Galerkin step entirely.

- The spatial basis is still extracted via POD, preserving physical interpretability.
- The network learns the mapping from $(\boldsymbol{\mu}, t)$ to the coefficient vector $\mathbf{c}$ from snapshot data.

This strategy decouples the manifold learning (still linear via POD) from the coefficient prediction (now nonlinear, handled by the network), offering a flexible framework that can handle problems where the classical projection fails.

21.4 An Industrial Application: Centrifugal Pump Efficiency

As an example of POD-NN in practice, consider a simplified model of a **centrifugal pump**. The goal was to predict the pump's efficiency curve as a function of the flow rate (the only varying parameter, the fluid being fixed).

- Snapshots of the flow field were generated for different flow rates.
- A POD basis was extracted, and a neural network was trained to map the flow rate to the POD coefficients of the efficiency-related quantities.
- The surrogate then provided rapid evaluations of the efficiency for any flow rate in the range, demonstrating the viability of the POD-NN approach for a problem of engineering interest.

This case study highlights how data-driven ROMs can be tailored to specific performance metrics with minimal intrusive code.

21.5 The Broader Role of Reduced Order Modeling in Engineering

Reduced order modeling (ROM) plays a key role in numerous engineering tasks that require rapid, repeated simulations:

- **Simulation**: fast evaluation of parameterised systems.
- **Design and optimal control**: optimizing shapes, products, or steering systems toward desired states.
- **Data assimilation and filtering**: merging physics-based models with observations.
- **Digital twins**: real-time model updating and control.

All these applications share the need for a cheap but accurate surrogate of a high-fidelity model.

21.6 Recap: Classical Reduced Basis Methods and Their Limits

The classical intrusive ROM pipeline consists of:

1. **Manifold learning**: extract a low-dimensional basis via Proper Orthogonal Decomposition (POD) of snapshot data.

2. **Dynamics reduction**: project the governing equations onto that basis (Galerkin projection) to obtain a reduced system for the coefficients $\mathbf{c}(t)$.

For a general nonlinear time-dependent problem $\mathbf{M}\dot{\mathbf{u}} = \mathbf{F}(\mathbf{u})$, this yields a reduced-order ODE.

Limitations.

- **Slow Kolmogorov n-width decay**: for convection-dominated or hyperbolic problems, many POD modes are needed to capture the solution manifold accurately.

- **Non-affine parameter dependence**: when the parameter appears inside nonlinearities, pre-assembling parameter-independent matrices is impossible; hyper-reduction (EIM, DEIM) becomes necessary but adds complexity.

- **Multiphysics / multiparticle systems**: each field or particle requires its own reduced basis, making the approach cumbersome.

These limitations motivated the shift toward data-driven surrogates.

21.7 Data-Driven Surrogates: From POD-NN to DL-ROM

21.7.1 POD-NN

To bypass the intrusive Galerkin step, the first data-driven extension is **POD-NN**:

$$\mathbf{c} = \phi(\boldsymbol{\mu}, t; \boldsymbol{\theta}),$$

where ϕ is a neural network that maps parameters and time directly to the POD coefficients. The spatial field is then reconstructed as $\mathbf{u} \approx \mathbf{V}\,\phi(\boldsymbol{\mu}, t)$. The POD basis remains, preserving physical interpretability, while the network learns the parameter-to-coefficient mapping from data.

21.7.2 DL-ROM and POD-DL-ROM

The next step replaces the linear POD basis with a nonlinear autoencoder (Ψ', Ψ). The **DL-ROM** learns jointly the encoder, decoder, and a parameter-to-latent network ϕ, all from data. To combine the best of both worlds, **POD-DL-ROM** uses POD to first compress the high-dimensional output to a moderate number of coefficients, and then applies a DL-ROM strategy on those coefficients, drastically reducing the decoder output size while retaining the benefits of nonlinear reduction.

21.7.3 Industrial Example: Centrifugal Pump Efficiency

A master's thesis applied POD-NN to a simplified centrifugal pump model. With the fluid fixed, the flow rate was the only varying parameter. Snapshots of velocity and pressure were collected, a POD basis extracted, and a neural network trained to predict the POD coefficients from the flow rate. This surrogate allowed rapid evaluation of the pump's efficiency curve across the operating range, demonstrating the practical utility of POD-NN for an engineering design task.

21.8 Injecting Physics: Physics-Informed ROMs

In fully data-driven surrogates, the governing equations are only present implicitly through the training data. To improve extrapolation and guarantee physical consistency, one can add the residuals of the governing equations (and boundary conditions) as penalty terms in the loss function. This borrows ideas from *Physics-Informed Neural Networks* (PINNs).

Effect on extrapolation.

- **Parameter extrapolation**: A POD-DL-ROM trained on a limited parameter range exhibits growing error when queried outside that range. Adding the PDE residual to the loss (physics-informed POD-DL-ROM) significantly reduces the extrapolation error.
- **Time extrapolation**: Similarly, when the model is asked to predict beyond the training time horizon, the physics-informed variant maintains a much lower and bounded error compared to the purely data-driven one, which eventually diverges.

Thus, physics-informed training acts as a regulariser, compensating for the lack of data in unseen regions by enforcing the known differential equation.

21.9 Extension to Optimal Control

Optimal control aims to find a control action $\mathbf{u}(t)$ that steers the system state $\mathbf{y}(t)$ as close as possible to a desired target. Two control paradigms exist:

- **Open-loop**: the control is precomputed as a function of time (and parameters).
- **Closed-loop**: the control depends on the current state, $\mathbf{u} = \mathbf{u}(\mathbf{y})$, requiring a feedback map.

21.9.1 Learning Controlled Solutions

If one has access to snapshots of optimal state–control pairs for various scenarios (parameters), the same surrogate philosophy can be applied:

- Use separate (or shared) autoencoders to reduce both state and control to low-dimensional latent representations.
- Train networks to map the parameter (and possibly time) to the latent state and latent control simultaneously.

This allows rapid synthesis of near-optimal control actions in new scenarios without solving the full optimal control problem online.

21.9.2 Closing the Loop in Latent Space

For closed-loop control, the goal is to learn a bidirectional mapping between the reduced state $\mathbf{z}_y$ and the reduced control $\mathbf{z}_u$. By working entirely in the latent space, one avoids expensive high-dimensional feedback computations. Data for this mapping can be extracted from optimal trajectories (state–control snapshots) or even from uncontrolled simulations combined with optimality conditions. This represents a promising direction for real-time, low-dimensional feedback control of complex systems.

21.10 Concluding Remarks

The course journeyed from classical reduced basis methods—limited by slow Kolmogorov n-width and intrusive projections—to modern data-driven surrogates. By marrying reduced order modeling with neural networks, we obtained flexible tools (POD-NN, DL-ROM, POD-DL-ROM) that can handle nonlinear, non-affine

problems without ever requiring a fully intrusive ROM code. Further enhancements like physics-informed training and latent-space control widen the applicability toward robust extrapolation and real-time decision-making. The story does not end here: model discovery, multi-fidelity fusion, and integration with digital twins are active research frontiers where ROMs and scientific machine learning continue to converge.

21.11 The Broader Role of Reduced Order Modeling in Engineering

Reduced order modeling (ROM) is not limited to simulation. It underpins many engineering tasks where repeated, fast evaluations of a physical system are essential:

- **Simulation**: rapid parameter sweeps and many-query scenarios.
- **Design and optimal control**: shape optimisation, trajectory steering, model predictive control.
- **Data assimilation and filtering**: merging physics-based models with observations (e.g., in mechatronics).
- **Digital twins**: real-time model updating and decision support require a compact, responsive model.

In all these contexts, the ability to simulate the system many times, quickly and accurately, is the common denominator.

21.12 Recap: Classical Reduced Basis Methods and Their Limitations

The classical intrusive ROM pipeline consists of two steps:

1. **Manifold learning**: extract a low-dimensional basis via Proper Orthogonal Decomposition (POD) of snapshot data.
2. **Dynamics reduction**: project the governing equations onto that basis (Galerkin projection) to obtain a reduced system of ODEs for the coefficients $\mathbf{c}(t)$.

For a general nonlinear time-dependent problem $\mathbf{M}\dot{\mathbf{u}} = \mathbf{F}(\mathbf{u})$, this yields a reduced-order dynamical system.

Key difficulties encountered:

- **Slow Kolmogorov n-width decay**: for convection-dominated or hyperbolic problems, many POD modes are needed to accurately capture the solution manifold.
- **Non-affine parameter dependence**: when parameters appear inside nonlinearities, pre-assembling parameter-independent matrices is impossible; hyper-reduction (EIM/DEIM) becomes necessary but adds complexity.
- **Multiphysics and multiparticle systems**: each field or particle requires its own reduced basis, making the classical approach cumbersome.

These limitations motivated the exploration of **data-driven surrogates**.

21.13 Data-Driven Surrogates: From POD-NN to DL-ROM

21.13.1 POD-NN

The first non-intrusive extension is **POD-NN**: instead of solving a Galerkin system, a neural network ϕ directly maps parameters and time to the POD coefficients,

$$\mathbf{c} = \phi(\boldsymbol{\mu}, t; \boldsymbol{\theta}), \qquad \mathbf{u} \approx \mathbf{V}\,\phi(\boldsymbol{\mu}, t).$$

The POD basis remains, preserving physical interpretability, while the network learns the parameter-to-coefficient mapping from data.

21.13.2 DL-ROM and POD-DL-ROM

To overcome the limitations of a linear trial manifold, the **DL-ROM** replaces the POD basis with a nonlinear autoencoder (Ψ', Ψ). The **POD-DL-ROM** combines the two ideas: first use POD to compress the high-dimensional output to a moderate number of coefficients, then apply a DL-ROM strategy on those coefficients. This drastically reduces the decoder output size while retaining the benefits of nonlinear reduction.

21.13.3 Industrial Example: Centrifugal Pump Efficiency

As a practical application, a master's thesis built a POD-NN surrogate for a simplified centrifugal pump. With the fluid fixed, only the flow rate varied. Snapshots of velocity and pressure were collected, a POD basis extracted, and a neural network trained to map the flow rate to the POD coefficients. The surrogate enabled rapid evaluation of the pump's efficiency curve across the operating range, demonstrating the potential of POD-NN for performance prediction in engineering design.

21.14 Injecting Physics: Physics-Informed ROMs

In purely data-driven surrogates, the governing equations only appear indirectly through the training data. To improve extrapolation and enforce physical consistency, one can add the residuals of the governing equations (and boundary conditions) as penalty terms in the loss function, borrowing ideas from *Physics-Informed Neural Networks* (PINNs).

- **Parameter extrapolation**: a POD-DL-ROM trained on a limited parameter interval shows growing error outside that interval. Adding PDE residuals during training (physics-informed POD-DL-ROM) significantly reduces the extrapolation error.

- **Time extrapolation**: similarly, when predicting beyond the training time horizon, the physics-informed variant maintains a bounded error, while the purely data-driven version eventually diverges.

Physics-informed training thus acts as a regulariser that compensates for missing data by enforcing known physical laws.

21.15 Extension to Optimal Control

Optimal control aims to find a control action $\mathbf{u}(t)$ that steers the state $\mathbf{y}(t)$ toward a desired target. Two paradigms exist:

- **Open-loop**: the control is precomputed as a function of time (and parameters).

- **Closed-loop**: the control depends on the current state, $\mathbf{u} = \mathbf{u}(\mathbf{y})$, requiring a feedback map.

If optimal state–control snapshots are available for various scenarios, the surrogate philosophy extends naturally:

- Use separate (or shared) autoencoders to reduce both state and control to low-dimensional latent representations.

- Train networks to map the parameter to the latent state and control simultaneously.

For closed-loop control, the goal becomes learning a bidirectional mapping between the reduced state $\mathbf{z}_y$ and the reduced control $\mathbf{z}_u$ entirely in the latent space, avoiding expensive high-dimensional feedback computations. This opens the door to real-time low-dimensional feedback control of complex systems.

21.16 Time Extrapolation via Latent Dynamics Discovery

An alternative to direct time prediction with a neural network is to learn the **latent dynamics** of the reduced state. Given an autoencoder (Ψ', Ψ), one can apply system identification techniques (e.g., SINDy) to the encoded trajectory $\mathbf{z}(t) = \Psi'(\mathbf{u}(t))$ to discover a low-dimensional ODE

$$\dot{\mathbf{z}} = \mathbf{f}(\mathbf{z}).$$

This ODE can then be integrated far beyond the training time window and decoded to obtain the full physical field. For the Navier–Stokes flow past a cylinder, this strategy allowed extrapolation up to 50% beyond the training time horizon, a significant improvement over naive neural network extrapolation.

21.17 Handling Uncertainty: When Parameters Are Unknown

In real-world applications, some parameters may be unknown or hidden. For instance, in an elasticity problem of an object falling on a two-material plate, the mass and size may be known, but the orientation of the material interface (angle θ) is not. Different θ values produce different deformations for the same known inputs. A surrogate must therefore be **probabilistic**: given the known parameters, it should generate a distribution of plausible solutions.

This can be achieved by coupling POD with a **conditional normalizing flow**—a generative tool that maps a simple noise distribution to the distribution of POD coefficients conditioned on the known parameters. The result is a stochastic surrogate that, for the same input, produces multiple realistic solutions, capturing the uncertainty due to hidden variables. Such techniques are the same underlying technology used in image generation (e.g., DALL-E, Stable Diffusion), adapted to physical problems.

21.18 Geometry Adaptation

When the computational domain changes (e.g., shape optimization), the POD basis must adapt. Recent work uses neural networks to predict how the basis vectors should deform with the geometry, still grounding the reduction on a POD-like linear superposition but making the basis geometry-aware. This preserves the simplicity and interpretability of linear methods while extending their applicability to variable-geometry problems.

21.19 Concluding Reflections: Blending Simplicity and Power

The course has traced a path from classical, intrusive reduced basis methods to modern data-driven surrogates that incorporate neural networks, autoencoders, and generative models. A common thread is the attempt to retain well-understood, provable components—like POD—while leveraging the flexibility of deep learning to overcome classical bottlenecks (nonlinear manifolds, non-affine parameters, unknown dynamics, etc.).

This modular philosophy is motivated by two considerations:

- **Trustworthiness**: In engineering, predictions must be reliable. Using interpretable building blocks (e.g., POD, least squares, sparse regression) alongside neural networks helps maintain a degree of understanding and control over the model.

- **Modularity**: A clear separation of tasks (compression, parameter mapping, dynamics learning) allows individual components to be replaced or improved without discarding the entire framework.

While powerful tools like transformers or deep generative models are not yet fully understood mathematically, combining them with principled reduction techniques offers a promising route toward accurate, efficient, and ultimately trustworthy reduced order models for the engineering problems of today and tomorrow.

21.20 Stochastic and Generative Reduced Order Models

21.20.1 Motivation: Uncertainty in Data and Models

Real-world data are often corrupted by noise, and models may be misspecified or rely on unknown parameters. A deterministic surrogate that yields a single prediction for a given input cannot capture the resulting ambiguity. This motivates **generative** ROMs that produce a distribution of possible solutions, reflecting both aleatoric (data) and epistemic (model) uncertainty.

21.20.2 Latent Dynamics with Random Coefficients

Instead of learning a deterministic ODE for the reduced coordinates, one can model the latent coefficients as **random variables**. For example, the reduced dynamics can be expressed as

$$\dot{\mathbf{z}} = \mathbf{f}(\mathbf{z}; \boldsymbol{\xi}), \qquad \boldsymbol{\xi} \sim p(\boldsymbol{\xi}),$$

where $\boldsymbol{\xi}$ are latent random variables. Every time a sample of $\boldsymbol{\xi}$ is drawn, a different trajectory of the reduced state is generated. After decoding, this yields an ensemble of plausible full-order solutions, accounting for the uncertainty present in the training data or in the model itself.

- The approach can be implemented with a POD basis and a conditional normalizing flow, or with an autoencoder whose latent space is endowed with a probabilistic prior (e.g., variational autoencoder).
- It provides not only a mean prediction but also confidence intervals, which is crucial for risk-informed decision making in engineering.

21.21 Multifidelity Reduced Order Modeling

21.21.1 The Multifidelity Paradigm

In many practical scenarios, multiple computational models of the same physical system are available, differing in accuracy and cost:

- **Low-fidelity (LF)**: coarse mesh, simplified physics, cheap but inaccurate.
- **High-fidelity (HF)**: fine mesh, complete physics, accurate but expensive.

The idea of **multifidelity ROM** is to use the expensive HF model only a few times to learn a correction map from LF to HF solutions, and then to query the cheap LF model and apply the correction, thereby obtaining HF-like accuracy at LF cost.

21.21.2 POD-Based Multifidelity Correction

A practical strategy exploits POD to represent all solutions (both LF and HF) on a common basis.

1. Generate a few HF simulations (for a limited time window or parameter range) and many LF simulations (covering a broader range).
2. Compute a POD basis from the HF snapshots. For the LF data, first interpolate the snapshots onto the fine mesh, then project onto the same basis.
3. Obtain time-dependent POD coefficients for both fidelity levels: $\mathbf{c}^{\mathrm{LF}}(t)$ and $\mathbf{c}^{\mathrm{HF}}(t)$.
4. Train a neural network (e.g., LSTM or feed-forward) to map the LF coefficient trajectory to the HF coefficient trajectory.

During deployment, one runs the cheap LF solver for a new scenario, projects the results onto the POD basis, and uses the trained network to predict the corrected (HF-like) coefficients. The full HF solution is then reconstructed as $\mathbf{u} \approx \mathbf{V}\,\hat{\mathbf{c}}^{\mathrm{HF}}$.

21.21.3 Example: Navier–Stokes Flow Past a Cylinder

- Two fidelity levels: LF mesh with 24 000 dofs, HF mesh with 73 000 dofs; Reynolds number varies.
- LF simulation is run for 18 s, HF only for 12 s (training window).
- A neural network maps the 10-dimensional LF POD coefficients to the HF ones, trained on the common time interval.
- In the extrapolation region (12–18 s), the corrected (multi-fidelity) trajectory (black) closely follows the (unknown) HF reference (orange), while the raw LF solution (green) is visibly inaccurate.
- The computational cost of the multi-fidelity prediction is essentially that of the LF solver, yet the error is comparable to the HF model.

21.21.4 Progressive / Multi-Level Fidelity

The framework can be extended to more than two levels, each level contributing additional information:

- **Level 0**: only parameters and time (no spatial information) → rough prediction.
- **Level 1**: add integral quantities (lift, drag) → improved prediction.
- **Level 2**: add sensor measurements on the outflow boundary.
- **Level 3**: add partial snapshots in a subdomain (e.g., near the obstacle).

Each level is handled by a separate neural network module that learns the correction with respect to the previous level. The final prediction is the sum of the contributions from all levels. Results show a monotonic decrease in error as more levels are fused, demonstrating the flexibility of the modular, learning-based ROM framework.

21.22 Conclusions and Outlook

The course journeyed from classical intrusive reduced basis methods—limited by slow Kolmogorov n-width decay and non-affine parameter dependence—to modern data-driven surrogates. By blending dimensionality reduction (POD, autoencoders) with neural networks (feed-forward, LSTM, SINDy, normalizing flows), we obtained a rich toolbox that can:

- Build non-intrusive, highly accurate surrogates for complex parametric PDEs.
- Extrapolate in time by distilling latent dynamics.
- Incorporate physical knowledge through physics-informed training.
- Handle uncertainty via generative models.
- Fuse multiple fidelity levels to achieve high accuracy at low cost.

Crucially, the modular philosophy—retaining well-understood components like POD while leveraging deep learning for the difficult parts—enhances trustworthiness and adaptability. These techniques are not limited to simulation; they extend naturally to optimal control, design, data assimilation, and digital twins.

This is only the beginning. As an engineer, you now have a set of building blocks that can be combined, adapted, and extended to tackle a wide variety of problems. The field is rapidly evolving, but a solid grasp of the fundamentals—both of numerical methods and of machine learning—will enable you to critically evaluate, trust, and innovate upon these tools in your future career.